

國立臺北科技大學
人工智慧科技碩士學位
學人

碩士
論

文
強化學習應用於機器狗之行為規劃策略

研究生：汪品儀





國立臺北科技大學

人工智慧科技碩士學位學程

碩士學位論文

強化學習應用於機器狗之行為規劃策略

Reinforcement Learning for Behavior Planning
Strategies in Spot Robot dog

研究生：汪品儀

指導教授：姚立德 博士

中華民國一百一十四年五月

「學位論文口試委員會審定書」掃描檔

審定書填寫方式以系所規定為準，但檢附在電子論文內的掃描檔須具備以下條件：

1. 含指導教授、口試委員及系所主管的完整簽名。
2. 口試委員人數正確，碩士口試委員至少 3 人、博士口試委員至少 5 人。
3. 若此頁有論文題目，題目應和書背、封面、書名頁、摘要頁的題目相符。
4. 此頁有無浮水印皆可。

摘要

關鍵詞：強化學習、行為規劃、機器狗、模塊化、即時性、泛化性

本論文探討了移動機器人行為規劃中的關鍵挑戰，提出了一種以強化學習（RL）為核心的解決方案，針對目標抵達、障礙物避讓、姿態控制及硬體限制提供了系統化設計方法。利用模塊化神經網路結構，將動態障礙物的預測與策略決策分離，結合 RNN 模型來預測障礙物動態資訊，並透過 RL 策略生成最優行動計畫。此外，針對實際應用中的資訊延遲問題，簡化了對障礙物類別的依賴，改為基於點雲資訊的統一處理方式，大幅提升了系統的即時性與穩定性。

這一系統設計提供了明確的訓練流程，包括模塊化設計、數據管理與高效測試，使得開發者能快速定位並解決設計中的瓶頸。同時，本論文還對動態場景中的障礙物應對、樓梯場景的姿態調整及實際部署中的策略優化進行了詳細探討，為開發者提供了高效構建強化學習系統的實用參考。

ABSTRACT

Keywords: Reinforcement Learning, Behavior Planning, Robot Dogs, Modularity, Real-Time Performance, Generalization

This thesis addresses key challenges in behavior planning for mobile robots and proposes a reinforcement learning (RL)-based solution. The methodology focuses on systematic design for goal-reaching, obstacle avoidance, posture control, and hardware constraints. By utilizing a modular neural network structure, it separates dynamic obstacle prediction from strategy decisions. The framework integrates RNN models to predict obstacle dynamics and generates optimal action plans via RL strategies. Moreover, the study tackles the issue of information latency in practical applications by simplifying obstacle classification and adopting unified processing based on point cloud data, significantly enhancing system responsiveness and stability.

This design provides a well-defined training process, including modular design, data management, and efficient testing, enabling developers to swiftly identify and address bottlenecks in system development. Additionally, the thesis elaborates on strategies for handling dynamic obstacles, posture adjustments in stair scenarios, and optimization in real-world deployments, offering practical guidance for efficiently building reinforcement learning systems.

誌謝

在這項研究的完成過程中，我得到了許多人的支持與幫助，在此表達我最誠摯的感謝。首先，特別感謝我的指導教授**姚立德**教授。教授為我提供了充裕的開發時間，讓我能在反覆試驗中找到最佳的解決方案，順利完成這項研究。您的指導不僅在技術上帶給我突破，還提供了寶貴的研究資源和實踐機會，並鍛鍊了我的實作能力。感謝先鋒材料科技**陳勝標**學長，無償提供波士頓動力之四足機器人作為實驗平台，並在機器維修期間主動協助處理相關費用，讓我得以順利進行系統開發與測試。感謝麻省理工學院城市科學實驗室，讓我有機會拓展國際視野，進一步深化我對智慧系統整合的理解。感謝我的父母，給予我無條件的信任和 support。在我感到疲憊時，總是關心我的狀況，並為我準備宵夜。你們的支持是我克服每一個挑戰的動力來源。感謝學弟**吳秉宸**，無償提供我使用 NVIDIA GeForce RTX 4080 顯示卡進行訓練和分析。這使我能夠在強化學習的研究中，全面測試不同情況下的數據，讓論文得以取得現有的成果。感謝同學**藍昶泰**、**蔡宗綦**、**蔡易倫**，陪伴我一起在實驗室努力。我們共同面對困難、分享經驗，這段合作的時光讓研究過程更加充實。感謝學弟**呂易騰**，與我一同測試系統，並協助處理各種硬體問題，你的幫助使研究得以順利推進。最後，感謝所有在這段旅途中支持和幫助過我的人。正是因為有了你們的陪伴和鼓勵，我才能夠專注於研究，克服種種挑戰，取得今天的成果。這份感激之情，將成為我未來不斷前行的動力。

汪品儀 謹誌

國立臺北科技大學 人工智慧科技碩士

中華民國 114 年 5 月

目錄

摘要.....	iii
ABSTRACT.....	iv
誌謝.....	v
表目錄.....	x
圖目錄.....	xii
第一章 緒論.....	1
1.1 研究背景與動機.....	1
1.2 泛化能力的重要性.....	2
1.3 使用強化學習的原因.....	4
1.4 論文規劃.....	6
1.5 論文貢獻.....	7
2 第二章 系統設計.....	9
2.1 系統架構.....	9
2.2 行為規劃系統的核心能力.....	10
2.3 障礙物系統實踐.....	11
2.4 目的地系統實踐.....	13
2.5 系統分析.....	16
2.5.1 系統的泛化性.....	16
2.5.2 效能與泛化性的平衡.....	17
3 第三章 RL 行為規劃策略.....	18
3.1 RL 基本結構.....	18
3.2 RL 環境中的物體.....	19
3.3 Agent 的設計.....	22
3.3.1 Agent 的動作.....	22

3.3.2 RL 環境中的 Agent.....	27
3.4 Reward 的設計	29
3.4.1 Reward 的目的	30
3.4.2 Single Agent 的分析	31
3.4.3 Multi Agent RL 的分析	39
3.5 Multi RL 的分析.....	47
3.5.1 RL 測試環境	48
3.5.2 Multi-Environment RL	50
3.5.3 Multi-Task RL.....	51
3.6 融合能力的方式.....	55
3.6.1 修改 Action 的 Reward	55
3.6.2 障礙物的改變姿態控制.....	57
3.6.3 修改 Spot 的能力上限	59
3.6.4 融合能力的結論.....	60
3.6.5 本論文融合能力的方法.....	63
3.7 Multi-Phase RL.....	64
3.7.1 適合 Multi Phase RL 的任務	65
3.7.2 調整 RL 環境參數優化能力	68
3.7.3 訓練能力的順序.....	70
3.7.4 本論文實際 Reward 的設計策略	71
3.8 Agent 的訓練策略	74
3.8.1 Agent 的折扣因子	74
3.8.2 Agent Action 的採樣	76
3.8.3 Agent 行為調整	77
3.8.4 Action 數量的選擇	80
3.9 State 的設計.....	82

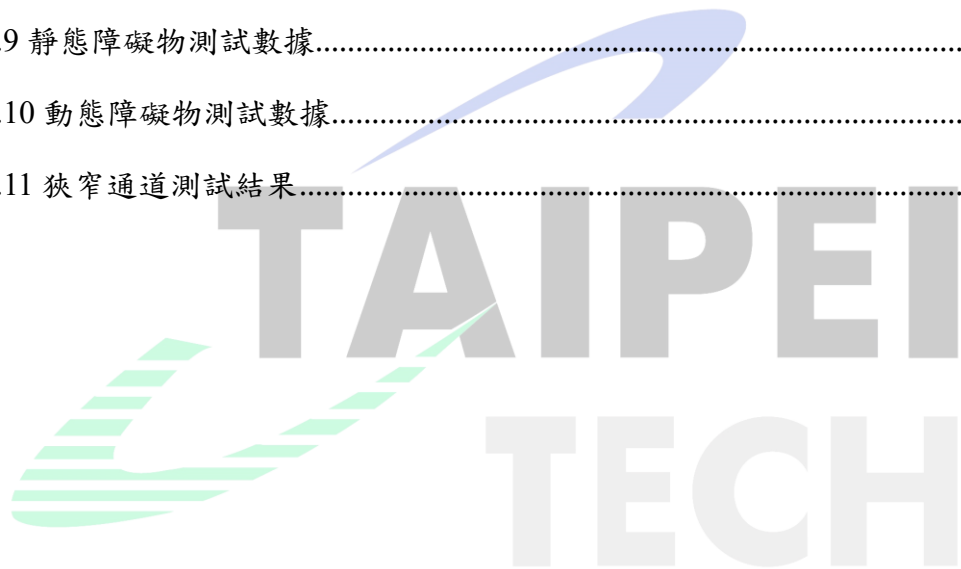
3.9.1 State 撰寫方式的重要性.....	83
3.9.2 State 資訊長度的影響.....	84
3.9.3 State 加噪音.....	84
3.9.4 State 設計結論.....	85
3.9.5 Spot Agent 的 State.....	86
3.9.6 Goal Agent 的 State	90
3.9.7 Obs Agent 的 State.....	92
4 第四章 RL Model 的分析	93
4.1 Model 的分析	93
4.2 演算法的修改.....	97
4.2.1 PPO 演算法	97
4.2.2 A2C 演算法	98
4.2.3 演算法改進方法.....	99
4.3 資訊延遲.....	102
4.3.1 資訊延遲的影響.....	102
4.3.2 RNN Model 的嘗試.....	104
4.3.3 模塊化神經網路.....	106
4.3.4 RL Model 的設計結論	108
5 第五章 實驗測試.....	110
5.1 RL 優化方式	111
5.1.1 Spot Agent 訓練數據.....	112
5.1.2 Goal Agent 訓練數據	116
5.1.3 Obs Agent 訓練數據.....	117
5.2 RL 微觀測試	118
5.3 RL 狀態分析	119
5.4 RL 行為分析	120

5.5 RL 完整測試	122
5.6 ROS 系統測試.....	137
5.6.1 測試流程.....	137
5.6.2 ROS 抵達目的地測試.....	138
5.6.3 ROS 避障能力測試.....	141
5.6.4 ROS 模擬數據測試.....	144
5.6.5 系統更新頻率.....	152
5.7 實際測試.....	153
5.7.1 測試環境與設備.....	153
5.7.2 實際測試.....	156
5.7.3 Sim2Real 挑戰與延遲補償方法.....	170
5.7.4 實際測試結果討論.....	176
6 第六章 結論與未來展望.....	178
6.1 結論.....	178
6.2 未來展望.....	179
參考文獻.....	180

表目錄

表 3.1 同一種 Agent 能否合作表.....	41
表 3.2 不同互動關係策略對 Spot 訓練效果的影響	42
表 3.3 不同 Agent 能否合作表.....	44
表 3.4 測試環境的參數.....	49
表 3.5 Multi Agent 和 Multi Environment 的比較	51
表 3.6 隨機與固定地圖的效果對比（測試 1）	52
表 3.7 隨機與固定地圖的效果對比（測試 2）	52
表 3.8 隨機 Agent 起始位置測試.....	53
表 3.9 Multi Phase RL 有效性測試	66
表 3.10 Multi Phase 任務策略測試	67
表 3.11 RL 環境長度訓練效果測試.....	68
表 3.12 不同能力得因果關係.....	70
表 3.13 RL 環境長度 90 秒	75
表 3.14 RL 環境長度 150 秒	75
表 3.15 SAC entropy 重要性測試.....	78
表 3.16 初始狀態重要性測試.....	79
表 3.17 Action 數量測試.....	80
表 3.18 State 撰寫方式測試.....	83
表 3.19 State 資訊長度影響測試.....	84
表 3.20 State 資訊抗噪測試.....	85
表 4.1 不同網路測試效果.....	95
表 4.2 Merge Model 效果測試.....	95
表 4.3 分裂架構效果測試.....	96
表 4.4 網絡寬度測試.....	97

表 4.5 RNN 模塊測試.....	108
表 5.1 空曠的地形能力測試.....	126
表 5.2 狹窄的地形能力測試.....	127
表 5.3 動態障礙物多的地方能力測試.....	129
表 5.4 靜態障礙物多的地方能力測試.....	131
表 5.5 上樓梯的能力測試.....	132
表 5.6 下樓梯能力測試.....	133
表 5.7 不能碰撞測試.....	135
表 5.8 空曠地形測試數據.....	158
表 5.9 靜態障礙物測試數據.....	159
表 5.10 動態障礙物測試數據.....	161
表 5.11 狹窄通道測試結果.....	163



圖目錄

圖 1.1 Autoware 系統流程圖	1
圖 1.2 CARLA[3]模擬自駕車虛擬環境	3
圖 1.3 未來三年內各領域 AI 商用化趨勢	5
圖 2.1 系統更新速度.....	9
圖 2.2 模塊化系統.....	10
圖 2.3 Tesla 自駕車未識別異常障礙物發生碰撞	11
圖 2.4 靜態障礙物轉換.....	13
圖 2.5 bdd_simple_1k 汽車可行駛區域訓練數據.....	13
圖 2.6 Cartographer 建圖	14
圖 2.7 導航系統.....	14
圖 2.8 機器狗設定目的地.....	15
圖 3.1 RL 基本結構	18
圖 3.2 模擬 RL 訓練場景	21
圖 3.3 模擬 RL 實際場景	22
圖 3.4 真實 Spot 運動能力	23
圖 3.5 Spot 在 RL 環境中的設定	27
圖 3.6 Spot 只會右轉就能減少碰撞	40
圖 3.7 Spot 對抗實驗	40
圖 3.8 期望中 Spot 在合作關係的策略	42
圖 3.9 實際 Spot 在合作關係的策略	43
圖 3.10 實際 Spot 在對抗關係的策略	43
圖 3.11 實際 Spot 在無關關係的策略	44
圖 3.12 其中八個 RL 測試環境	49
圖 3.13 隨機和固定地圖策略的碰撞率.....	52

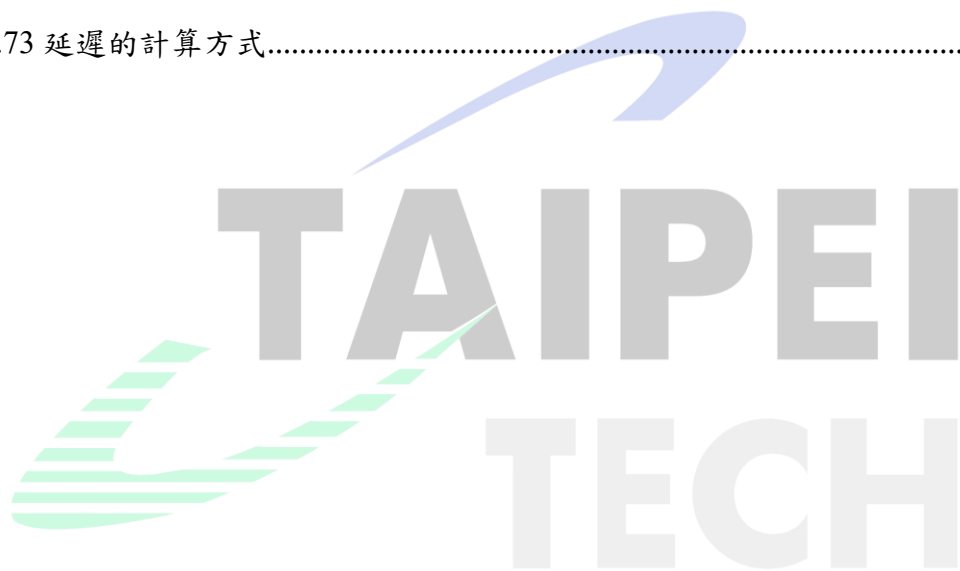
圖 3.14 Spot 一開始的碰撞機率	54
圖 3.15 Spot Model 損毀.....	54
圖 3.16 Spot 一開始的碰撞機率	55
圖 3.17 修改不同 Action 的 Reward，Spot 的速度分布圖	56
圖 3.18 Spot 的姿態控制策略	57
圖 3.19 改變 Agent 的行為喜好.....	58
圖 3.20 計算 Spot 碰撞的 Reward.....	58
圖 3.21 改變 Spot 的能力上限	59
圖 3.22 修改 Agent 的能力上限，Spot 的速度分布圖	60
圖 3.23 Spot 面對障礙物的移動策略	61
圖 3.24 抵達目的地能力對姿態控制的影響.....	62
圖 3.25 姿態控制能力對於抵達目的地能力的影響.....	62
圖 3.26 單一階段 RL 訓練的學習	65
圖 3.27 Spot 的最佳路徑	77
圖 3.28 A_{acc_x} 分佈機率.....	80
圖 3.29 A_{acc_y} 分佈機率	81
圖 3.30 A_{ω} 分佈機率	81
圖 3.31 7 個 Action 的速度分布圖.....	81
圖 3.32 11 個 Action 的速度分布圖.....	82
圖 3.33 19 個 Action 的速度分布圖.....	82
圖 3.34 Spot 的 $Sweep_{angle}$	88
圖 3.35 Spot 的 $Sweep_{Map-gradient}$	89
圖 3.36 Boston Dynamics Spot 的感測器視角.....	89
圖 3.37 實際的 $Sweep_{Map-gradient}$	90
圖 3.38 Goal Agent 的 $Sweep_{Map-gradient}$	91
圖 4.1 常見的 A2C Model.....	93

圖 4.2 Policy Large Model.....	94
圖 4.3 Critic Large Model.....	94
圖 4.4 Small Model.....	94
圖 4.5 Merge Model.....	95
圖 4.6 Split Model.....	96
圖 4.7 Thin Model.....	96
圖 4.8 障礙物擬合示意圖.....	103
圖 4.9 系統流程圖的變化.....	104
圖 4.10 S_{Obs} 轉 $S_{Sweep_{angle}}$ 示意圖	104
圖 4.11 RNN RL Model.....	105
圖 4.12 第一和第二階段訓練數據.....	105
圖 4.13 第三階段訓練數據.....	105
圖 4.14 Module RNN Model	106
圖 4.15 Module RNN Model 梯度更新	107
圖 4.16 本論文 RL Model 任務分工	108
圖 5.1 RL 系統研發流程	110
圖 5.2 實際研發流程.....	110
圖 5.3 存在期望值和行為期望值.....	112
圖 5.4 上樓梯的期望值.....	112
圖 5.5 Spot 碰撞動態和靜態障礙物的期望值	113
圖 5.6 Spot 執行完的存活率和 Action Reward 的期望值.....	113
圖 5.7 Spot 的 $V_{Spot-Goal}$ 和環境生成時 Spot 的死亡率。	113
圖 5.8 全部訓練和 Spot 訓練時 Goal Agent 剩餘的機率	114
圖 5.9 在 Goal 訓練時 Goal Agent 剩餘的機率	114
圖 5.10 總體的 Spot Reward.....	115
圖 5.11 Spot Loss.....	115

圖 5.12 Spot module entropy	116
圖 5.13 Goal 存在期望值和移動行為的期望值	116
圖 5.14 Goal 的 V_{Stair} 和 V_{RStair}	116
圖 5.15 Goal 的 Reward 和 Loss	117
圖 5.16 Goal module entropy	117
圖 5.17 Obs 的 Reward 和 Loss	117
圖 5.18 Obs module entropy	118
圖 5.19 Spot Agent 在短期訓練階段的效率	118
圖 5.20 Spot 訓練期間 Goal 剩餘比例的變化	119
圖 5.21 Spot Phase 3 樓梯 Reward	119
圖 5.22 RL 環境可視化	120
圖 5.23 $p_{\psi}(d\phi)$ 的差別	121
圖 5.24 可視化訓練階段的成果	121
圖 5.25 Wandb 版本管理	122
圖 5.26 RL 測試環境	123
圖 5.27 Goal Agent 移動測試，需要抵達目的地的能力	123
圖 5.28 Goal Agent 移動測試，需要上下樓梯的能力	124
圖 5.29 Goal Agent 移動測試，固定的 Goal Agent	124
圖 5.30 空曠的地形測試 Case 1 和 Case 2	125
圖 5.31 空曠的地形測試 Case 3 和 Case 4	125
圖 5.32 狹窄的地形測試 Case 1 和 Case 2	126
圖 5.33 狹窄的地形測試 Case 3 和 Case 4	127
圖 5.34 狹窄的地形測試，Case 4 測試失敗	128
圖 5.35 動態障礙物測試 Case 1 和 Case 2	128
圖 5.36 動態障礙物測試 Case 3 和 Case 4	129
圖 5.37 靜態障礙物測試 Case 1 和 Case 2	130

圖 5.38 靜態障礙物測試 Case 3 和 Case 4	130
圖 5.39 上樓梯能力測試 Case 1 和 Case 2	131
圖 5.40 上樓梯能力測試 Case 3 和 Case 4	132
圖 5.41 下樓梯能力測試 Case 1 和 Case 2	133
圖 5.42 下樓梯能力測試 Case 3 和 Case 4	133
圖 5.43 不能碰撞測試 Case 1 和 Case 2	134
圖 5.44 不能碰撞測試 Case 3 和 Case 4	135
圖 5.45 數據值域比對	137
圖 5.46 設定 Spot 的目的地	139
圖 5.47 測試 Spot 抵達前方的目的地	139
圖 5.48 測試 Spot 抵達左邊的目的地	139
圖 5.49 測試 Spot 抵達右邊的目的地	140
圖 5.50 測試 Spot 抵達後面的目的地	140
圖 5.51 Spot 穿越眾多障礙物，抵達目的地	142
圖 5.52 Spot 抵達目的地後，仍舊可以躲避障礙物	144
圖 5.53 Gazebo 的 Cheetah 機器狗	145
圖 5.54 一般地形測試環境	145
圖 5.55 一般地形測試	149
圖 5.56 樓梯測試環境	149
圖 5.57 15 度樓梯測試	151
圖 5.58 45 度樓梯測試	152
圖 5.59 速度控制曲線（y 軸單位：公尺/秒，x 軸： dt ）	153
圖 5.60 NVIDIA Jetson AGX Orin	155
圖 5.61 測試場域點位	156
圖 5.62 測試場域	157
圖 5.63 空曠地形測試軌跡	157

圖 5.64 靜態障礙物測試場域.....	158
圖 5.65 靜態障礙物測試軌跡.....	159
圖 5.66 動態障礙物測試場域.....	160
圖 5.67 動態障礙物測試軌跡.....	160
圖 5.68 狹窄通道測試場域點位.....	161
圖 5.69 狹窄通道測試場域.....	162
圖 5.70 狹窄通道測試軌跡.....	162
圖 5.71 多動態障礙物測試.....	169
圖 5.72 多動態障礙物測試軌跡.....	169
圖 5.73 延遲的計算方式.....	172



第一章 緒論

1.1 研究背景與動機

本論文的研究目的是要讓 Boston Dynamics 的 Spot 機器狗能夠適應各種複雜的環境變化，避開不同的障礙物並準確到達指定位置。為了做到這點，本論文參考了 Autoware[1]系統的建置，尤其是在行為控制的模組，有將此系統設計的重點納入本論文的系統架構設計的考量中，如圖 1.1 所示。

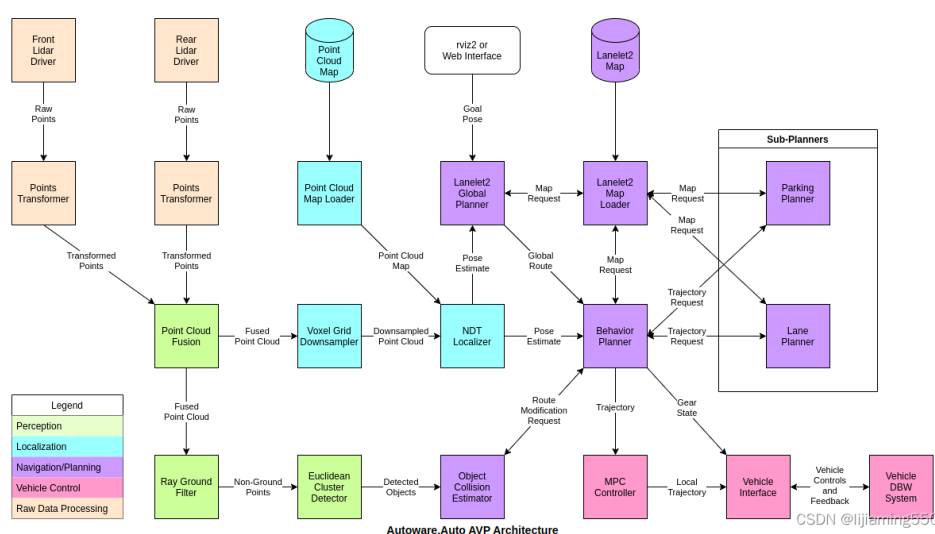


圖 1.1 Autoware 系統流程圖

Autoware 系統是個開放源碼的自駕車技術平台，它整合了多種模組來執行自動導航、避障、定位與控制等功能。在這個系統中，控制模組分為以下兩個主要部分，分別對應到了不同的性能要求。

全局規劃（Lanelet2 Global Planner）：

負責在地圖數據中對車輛從當前位置到目標地點的最優行進路徑進行計算。此過程是根據整體視角來規劃的，因而對實時性的要求相對較低，對幀率（fps）的需求不高。

行為規劃 (Behavior Planner)：

與全局規劃相比，行為規劃更注重實時性和動態性，需要結合即時的障礙物檢測數據來進行快速且靈活的局部路徑生成。這部分的系統對幀率的要求非常高，因為它需要對路徑進行快速更新以應對動態變化的環境。

本論文研究的重點在行為規劃的模塊，需要專注於提高避障策略的實時反應能力和準確性，以確保 Spot 機器狗能夠在行進中穩定高效地避開障礙物，並且在保證動態避障的同時，還能準確控制自己的姿態，以符合硬體限制和保持穩定性。透過這一系列的研究和開發，我們期望 Spot 機器狗在自主移動過程中能夠具備更高的智慧化和適應性。

1.2 泛化能力的重要性

自駕車領域利用原始資訊(例如 3D 激光雷達)進行避障始終是一大挑戰。以往即使是特斯拉這樣的先驅公司，也曾在特定情況下遭遇撞擊卡車的困難[2]，這凸顯出原本的技術在解決該問題上的局限性，也可以看到真實數據與 AI 訓練之間落差帶來的影響。

另外有些使用虛擬環境訓練的自動駕駛策略，會在模擬的城市街道上學習，這些街道設計有各種交通標誌、行人和其他車輛，如圖 1.2 所示。這樣的模擬環境提供了一個相對受控且變量已知的學習場景，其中包括行人行為模式、車輛動態和天氣條件等因素，使得策略模型能夠透過反覆嘗試與錯誤來學習如何安全駕駛。



圖 1.2 CARLA[3]模擬自駕車虛擬環境

然而當將這些模型部署到真實世界時，它們往往會遇到一系列在模擬環境中未曾遭遇過的挑戰。真實世界的行人行為可能遠比模擬環境中設定的要不可預測，天氣條件（如雨、雪、霧）可能嚴重影響能見度，而且真實街道的環境比模擬場景要複雜得多。這些真實世界的變數增加了，使得策略模型在真實世界中的表現可能大不如在模擬環境中的理想狀態。

另外在模擬環境中，光線的呈現都是標準化的，使得模型能夠輕易識別。但在真實世界中，紅燈可能因太陽直射、髒污或被樹葉遮擋而難以識別。若 AI 模型未能學會如何處理這種視覺上的不確定性，則可能無法在真實世界中作出正確的反應。

這種真實數據與 AI 訓練之間落差，會導致意外的發生。為了縮小這一差距，AI 模型需要能夠理解並適應真實世界中的不確定性和多變性，這就要求模型具備更強的泛化能力，以確保在多樣化的真實環境中依然能保持良好的性能表現。

1.3 使用強化學習的原因

強化學習和其他 AI 的差別

如論文[4]強化學習（Reinforcement Learning, RL）與其他 AI 訓練的主要差異在於訓練資料的重要性。對於一般 AI 訓練來說，每一筆訓練資料的重要程度相同，模型的目標是找到能讓每筆資料在平均情況下，達到最符合 Loss function 設計的最小值。然而，強化學習的情況並不一樣。RL 模型的目標是找到能最大化效能的策略，因此每筆訓練資料的重要性會隨著訓練過程的進展而有所不同。

雖然 RL 在單次訓練中，每筆訓練資料在單一時刻是同等重要的，但由於模型會改變其輸出結果，策略的變化會進一步影響資料收集的分佈，進而使每筆資料的重要性變得不一致。這一差異使得 RL 訓練能夠更有效地最優化策略。

世界上著名的 RL 訓練項目包括 AlphaGo[5]和 ChatGPT[6]。AlphaGo 通過 RL 訓練在複雜的棋盤環境中實現了超越人類的決策能力。類似的 ChatGPT 也利用 RL 來調整其生成回應的策略，從而提供更加符合上下文和人類直覺的答案。

目前，大多數車廠在行為規劃中的避障問題上使用基於 CNN 的演算法。這些方法通常先使用 CNN 識別出需要避開的障礙物，接著再進行避障策略的計算。然而，這一過程通常並未考慮障礙物與車體之間的距離或事件的重要性，只是單純地識別物件，導致整個過程繁瑣且不夠高效。

相較之下，人類面對障礙物時，往往能夠立即避開，而不需要經過複雜的識別過程。因此，本研究提倡使用 RL 來訓練行為規劃策略，通過 RL 訓練系統即時決定障礙物的重要性，並根據實時的資訊進行姿態控制，實現更自然、更直觀的避障行為。因此，本論文通過 RL 訓練提出了一種更加符合人類直覺的行為規劃策略，旨在克服現有技術的缺陷，提供更高效的解決方案。

RL 需要解決的問題

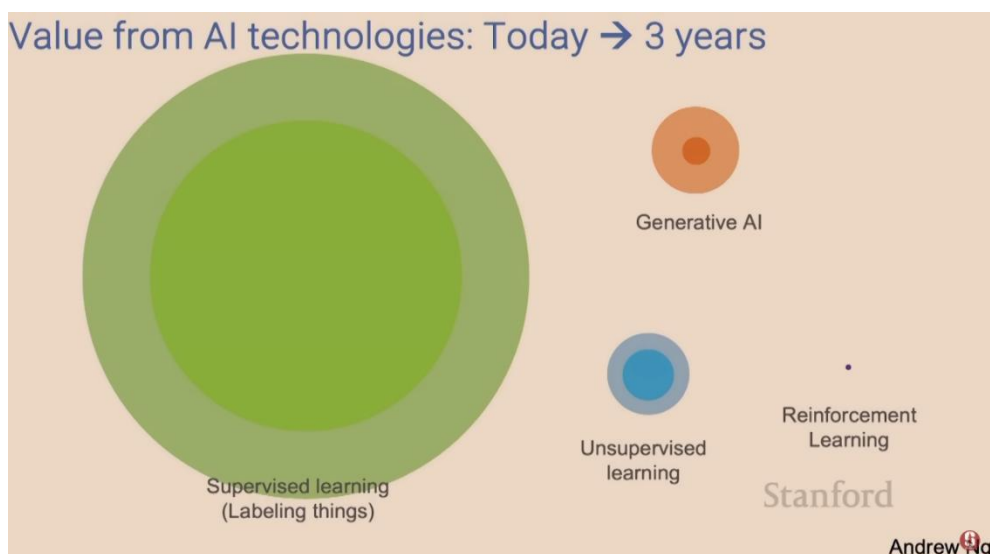


圖 1.3 未來三年內各領域 AI 商用化趨勢

Stanford 的 Andrew Ng (吳恩達) 教授 2023 年 7 月 26 日的演講中[7]，主要探討了未來三年內各行業領域 AI 商用化的發展趨勢，如圖 1.3 所示。雖然演講核心並非強化學習，但其中的觀點揭示了 RL 在商業應用上面臨的挑戰。

本論文針對強化學習在發展中所面臨的三大關鍵問題進行分析：

撰寫客製化 RL 環境的困難：

開發專屬於特定任務或應用場景的 RL 環境需要耗費大量人力物力，並非每個企業都有資源投入。

開發 RL 的時間成本：

RL 的開發與調試過程相對較長，尤其是模型需要長時間收集足夠數據進行策略學習，這對於商業應用來說是一個挑戰。

解決真實數據與 RL 環境的落差：

模擬環境中訓練的 RL 策略在應用於真實場景時，往往會面臨 Sim2Real 問題，即模擬環境與實際環境的差異會影響訓練模型的表現。

1.4 論文規劃

1. 緒論

本章節闡述本論文研究的背景與動機，從機器人運動控制系統的角度出發，說明行為規劃策略在機器人系統中的重要性。現有技術的主要瓶頸在於系統的泛化能力不足，導致必須依賴更高的運算性能來提升策略的正確率。然而，這種方式無法有效應對真實環境中的複雜變化，也難以從根本上解決問題。為克服這一挑戰，本論文採用了強化學習技術，並分析其能突破其他 AI 技術瓶頸的原因。強化學習透過與環境交互學習有效的策略，展現出較強的適應能力，特別是在面對未見過的情境時。本論文將聚焦於 RL 訓練過程中的關鍵挑戰，提出相應的解決方案，並驗證這些方法的實用性與有效性。

2. 系統設計

本章節介紹運動控制系統的整體架構設計。為了有效處理控制問題，系統根據頻率需求進行模塊化設計，將問題分解為不同的模塊，各自負責特定任務，並以協調方式整合為完整系統。

在降低系統運算負擔的同時，本論文分析了 RL 行為規劃策略的角色與可行性，明確了 RL 策略在系統中需要實現的目標，並說明如何確保整體系統的穩定性與高效性。

3. RL 行為規劃策略

本章節討論 RL 行為規劃策略的設計方法與實現細節。針對 RL 訓練過程中遇到的關鍵問題，本論文將逐一分析問題，提出解決方案，並通過實驗驗證其有效性。此外，本論文還將比較不同技術在 RL 訓練中的優劣，說明各技術在解決具體問題時的適用性與局限性，提供設計策略的參考。

4. RL Model 的分析

本章聚焦於 RL 策略所採用的神經網路模型，探討其架構設計與模組分工，說明各模塊對整體表現的貢獻。進一步地，本論文究導入 RNN 強化模型對

歷史資訊的記憶能力。此外，本章將針對模型訓練與推論效能進行量化分析，以驗證所提模型在決策準確率、收斂速度與資源消耗等多面向的優化成果。

5. 實驗測試

本章節闡述 RL 訓練流程及實驗結果。如何從 RL 的訓練參數修改到最終在機器狗上的測試，系統性介紹如何在最短時間內找出可優化的問題，並進行有效改進。實驗內容包括 RL 行為規劃策略的微觀測試、狀態分析與完整測試。再到系統的模擬測試，最後才是實際測試。

6. 結論和未來展望

本章節總結研究成果，說明強化學習的應用價值，並分析本論文方法的貢獻與局限性。針對未來可能的研究方向提出建議。

藉由本論文的討論與實驗結果，希望為資源有限的開發者提供一個實用的強化學習設計與應用指南，幫助提升強化學習在實際任務中的應用成功率。

1.5 論文貢獻

本論文深入探討基於強化學習的行為規劃策略，提出了一系列創新解決方案，論文的主要貢獻可總結為以下五點。

1. 機器人移動系統

本論文分析了機器人移動的問題，依據資訊的更新頻率，將問題切分為不同模塊分別處理。這種設計使機器人僅依靠自身感測器即可進行即時的姿態控制，同時提升了系統的穩定性與處理速度。

2. 行為規劃策略的效能

現有 RL 研究在行為規劃策略方面取得了一定進展，但在真實場景中的效能仍然有限。例如論文[9]和[10]探討了路徑規劃問題，但在動態環境中缺乏靈活性。而論文[11]可以有有效的規劃路徑，但並不包含避障能力，也沒有實際場域的測試，由此可知對於未知場域的泛化能力有限。本論文利用強化學習設計的行為規劃策略，除了能夠解決以上問題外，還突破了傳統平面避障的

限制，成功實現機器狗在高低起伏環境中的行為規劃控制，顯著提升了策略的適應性與效能，為行為規劃策略提供了實用解決方案。

3. RL 獎勵函數設計

本論文提出了一個適用於 Multi-Task 和 Multi Agent 的強化學習的獎勵設計框架。通過細化不同階段的設計目標，逐步確定每個 Agent 的目標與環境的交互的方式，並為不同的目標設計相應的獎勵函數，以提升行為規劃策略的效能。

4. RL 融合能力

為了解決行為規劃策略同時需要多種能力的挑戰（如抵達目的地、避開障礙物、姿態控制和上下樓梯），本論文結合 Reward 設計、Multi-Phase RL 與 RL 環境設計，使策略模型能夠同時考慮多種能力需求，實現適應多種條件的最優策略。

5. RL 模型架構的設計

本論文詳細分析了 A2C 模型的設計方式，通過實驗驗證了不同架構設計對效能的影響，並對效果背後的原因進行解釋。最終基於實驗結果，提出了模型設計與選擇的實踐建議。

第二章 系統設計

2.1 系統架構

以機器人系統為出發點，根據執行速度，可以將全局規劃系統與行為規劃系統區分開來。為了使整個行為規劃系統具備穩定效能，我們應減少系統的依賴條件，使機器人在僅依賴行為規劃系統的情況下，仍能夠維持基本的執行能力，從而確保系統的穩定性。此外，全局規劃系統與行為規劃系統之間必須具備良好的溝通能力，僅需少量指令便能進行交互，並且全局規劃系統只需處理相對於計算時間來說不太緊急的資訊。

在機器人控制層面，由於馬達控制參數會隨場景需求發生變化，行為規劃系統應避免直接接管馬達細節操作。馬達控制應由機器人的基礎控制模塊負責，根據回授系統調整旋轉角度與力度。行為規劃系統則聚焦於規劃移動速度與身體姿態，專注於**速度控制**，從而實現不同層次間的職能分離與協同增效。

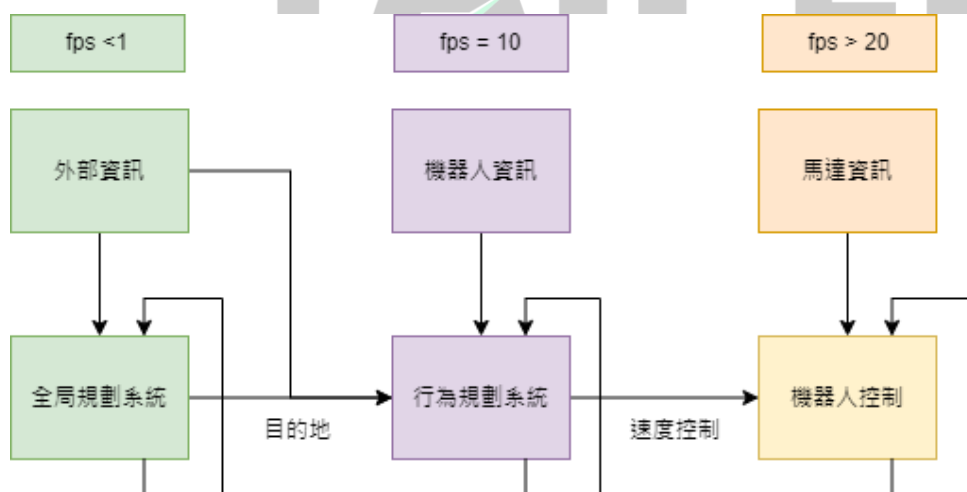


圖 2.1 系統更新速度

如圖 2.1 所示，系統中出現了一個有趣的現象，即更新頻率低的系統需要控制更新頻率高的系統，而更新頻率越高的系統所需的輸入資訊和運算量就應該越少。因此，為了實現本系統的行為規劃系統，我們需要降低不同模塊間的依賴性，

並設計合理的模塊間通訊方式。

為了降低全局規劃系統與行為規劃系統之間的資訊傳遞量，本論文將其溝通內容限定為目的地位置，這樣僅需傳遞兩個數值的資訊。此外，為了減少目的地資訊所需的更新頻率，本系統將目的地資訊設定在 Odometry 坐標系中，並在資料傳輸至行為規劃系統需要使用時，再將其轉換為機器人的座標系。這樣既能降低全局規劃系統的更新速度，也能降低傳輸延遲。

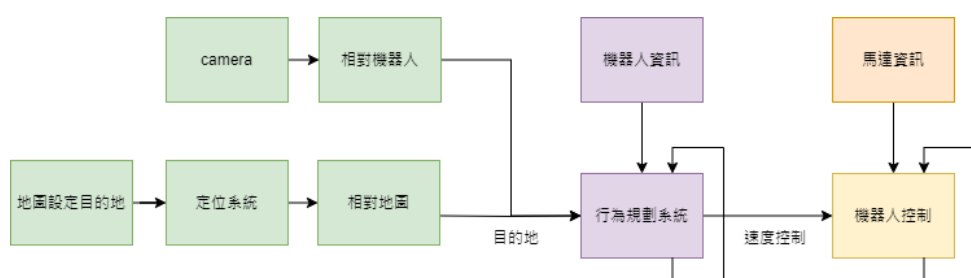


圖 2.2 模塊化系統

如圖 2.2 所示，我們可以將移動機器人的任務分為兩類：一類是基於地圖的長遠路徑規劃，另一類是基於機器人感測器所獲得的即時資訊來判斷目的地。無論使用哪種方式來傳遞目的地資訊，最終都會將其轉換至 Odometry 坐標系中。雖然 Odometry 坐標系會有累積誤差，但通過每秒更新一次全局規劃系統，可以消除 Odometry 坐標系與目的地之間的誤差。通過這兩種類型的任務，Spot 機器狗能夠即時響應並適應各種環境需求，無論是事先規劃好的行動路徑，還是突發情況下都能做出即時反應。

2.2 行為規劃系統的核心能力

Spot 機器狗的行為規劃系統的主要目標是使其能夠簡單地控制 Boston Dynamics 開發的硬體。使用者只需輸入目的地指令，系統便能自動計算出適合硬體限制的行為策略，確保機器狗能夠安全抵達目的地。為達成這一目標，行為規劃系統需要具備以下幾項關鍵能力：

1. 抵達目的地的能力：能夠準確計算出從當前位置移動至指定目的地的運動控

制。

2. 避開障礙物的能力：能夠及時識別並避開路徑上的障礙物。
3. 上下樓梯的能力：處理高低起伏的地形，如樓梯等。
4. 姿態控制的能力：在移動過程中保持機器狗的姿態，符合正常的定義。
5. 應對未知狀況的能力：能夠適應環境中的各種未知情景。
6. 運算速度快：保證系統的實時響應速度足夠快。

行為規劃系統需包含的關鍵資訊有：機器人的自身狀態 (Speed)、目的地資訊 (Goal)、可見的障礙物資訊 (Obs) 以及地圖資訊 (Sweep)，這些共同構成了系統的狀態定義：

$$S_{Spot} = \{Speed, Goal, Obs, Sweep_{angle}, Sweep_{Map-gradient}\}$$

2.3 障礙物系統實踐

在自動駕駛系統（例如 Tesla）的障礙物處理架構中，通常需先進行物件特徵提取，才能辨識障礙物並進行避障[8]。此方式雖然具備語意辨識能力，但當系統遇到未曾見過的物件時，由於無法有效提取其特徵，容易導致錯誤判斷甚至碰撞，如圖 2.3 所示。



圖 2.3 Tesla 自駕車未識別異常障礙物發生碰撞

為解決上述問題並提升整體系統的穩定性與實用性，本論文提出一種以點雲資料為基礎的障礙物資訊表示方式。該方式不依賴物件分類或語意辨識流程，而是直接將感測器（如 3D 雷達或深度相機）所取得的點雲資料，透過鳥瞰壓縮處理，轉換為 $S_{SpotSweep}$ 資訊格式，作為強化學習模型的輸入。此作法具備以下三項優勢：

1. 省略語意辨識步驟，降低系統依賴性與誤判風險：不需分類物件，只根據距離資訊即可進行避障與規劃。
2. 提升泛化能力：即使遇到從未出現過的障礙物，也能透過其幾何輪廓與距離直接做出反應。
3. 減少運算流程：無需經過深度學習特徵提取（如 CNN 或 YOLO），可加快處理速度，降低資源消耗。

相較於傳統依賴語意辨識的方式，本論文強調資料結構簡化與策略穩定性設計，使系統即使在無法正確識別物件的場景中，仍能維持安全與有效的決策能力，降低因模組失效所造成的整體風險，亦更適用於即時應用環境。

此外，根據論文[12]的實驗顯示，過度追求模擬資料的保真度（例如影像中的光影細節）反而會增加訓練難度，對強化學習表現並無幫助。實驗指出，使用結構化、簡化的真實資料格式，比使用高保真輸入更能提升泛化效果與學習效率。

基於上述考量，本論文在訓練階段也沒有使用實際感測器數據作為輸入，而是直接在模擬環境中產生 $S_{SpotSweep}$ 格式的障礙物資訊作為學習輸入。此一設計不僅簡化訓練流程，也有效減少訓練與實際部署間的資訊落差。

實際操作時， $S_{SpotSweep}$ 資訊可由多種感測來源轉換獲得，本文實作中採用了 3D 激光雷達與深度相機作為主要輸入。不同感測器的資料經過整合後，透過鳥瞰視角進行壓縮處理，如圖 2.4 所示。圖中最後一張為轉換後的 $S_{SpotSweep}$ 資訊，其內容即為由點雲資料壓縮所得的各方向距離資訊，可直接輸入至行為規劃系統，作為決策依據。

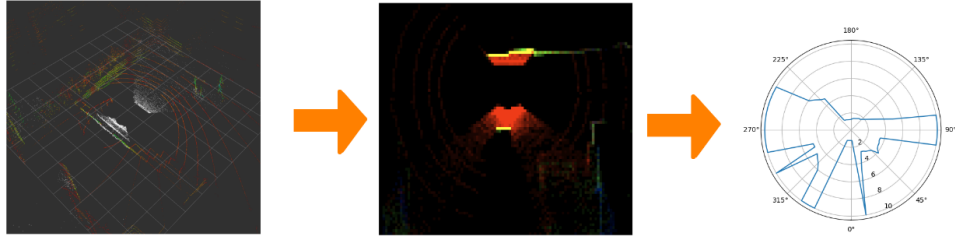


圖 2.4 靜態障礙物轉換

儘管本論文並未使用 RGB 影像進行障礙物偵測，我們仍展示一種替代方式：例如使用如 bdd_simple_1k[13] 資料集，如圖 2.5 所示，搭配影像辨識模型（如 YOLO），訓練出可行駛區域後轉換為鳥瞰圖，再進一步轉為 $S_{SpotSweep}$ 格式，適用於本論文提出的行為規劃架構。

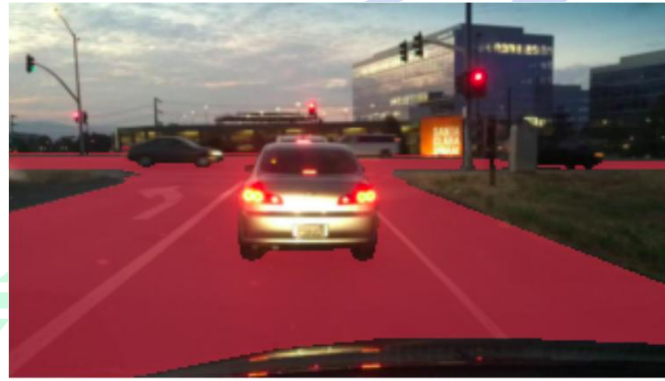


圖 2.5 bdd_simple_1k 汽車可行駛區域訓練數據

值得注意的是，在 RL 訓練過程中，本論文並沒有實際使用 3D 激光雷達、深度相機或 RGB 影像資料，而是透過模擬環境生成 $S_{SpotSweep}$ 和 $S_{SpotObs}$ 的數據。這意味著訓練過程中的 $S_{SpotSweep}$ 資訊與實際執行過程中的 $S_{SpotSweep}$ 會存在一定差異。然而，經過簡化的資料結構使得訓練數據更加精簡，從而減少了訓練與實際執行之間的資訊落差，提升了系統在真實環境中的應用效果。

2.4 目的地系統實踐

地圖座標系設定目的地系統

本論文對於「地圖設定目的地系統」的定義是：使用者在地圖上選定任一座標，機器人則需要到達該座標對應的現實位置。因此，我們首先需要生成一張地圖，讓使用者與系統對位置的認知一致。本論文使用 **Cartographer**[14]套件來建立地圖，如圖 2.6 所示。

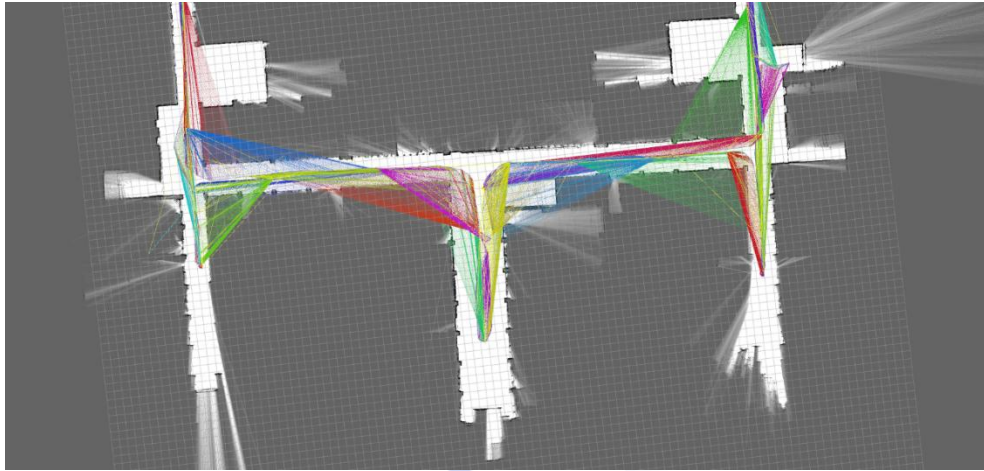


圖 2.6 Cartographer 建圖

有了地圖之後，需要基於地圖來設定機器人可行走的路徑。此系統的主要目標是讓機器人能夠知道如何抵達較遠的目的地，並選擇最優化的路徑。因此，我們必須記錄路徑中較為重要的座標、座標的類型以及座標之間的距離，如圖 2.7 所示。

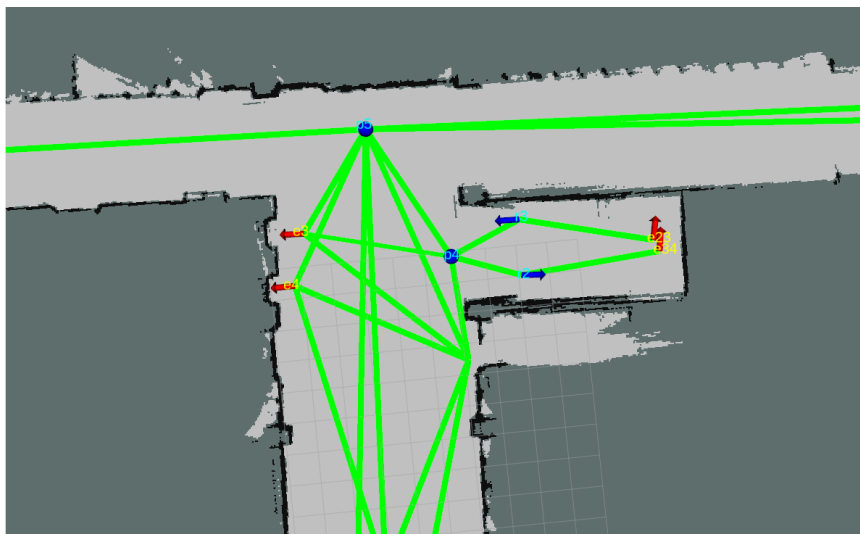


圖 2.7 導航系統

座標的類型包含以下幾種：

- p：普通中繼點
- r：房間
- e：可跨樓層移動的中繼點

當地圖路徑資訊生成後，系統便能開始執行「地圖設定目的地」的任務。每次當使用者修改目的地時，系統會根據 Cartographer 的定位系統確定機器人當前的位置，並使用廣度優先搜索法找出最短的可行路徑。此外，系統會以每秒更新一次的頻率，判斷機器人是否到達最近的中繼點，並決定是否切換至下一個要抵達的中繼點。行為規劃系統會依照系統的運行頻率，將最近的中繼點轉換為機器人座標系中的目標位置，從而引導機器人朝正確的方向前進。

機器人座標系設定目的地系統

本論文對於「機器人設定目的地系統」的定義是：使用者為機器人設定一個目標，機器人則根據當前的感測器數據，判斷該目標的位置，並前往該位置。

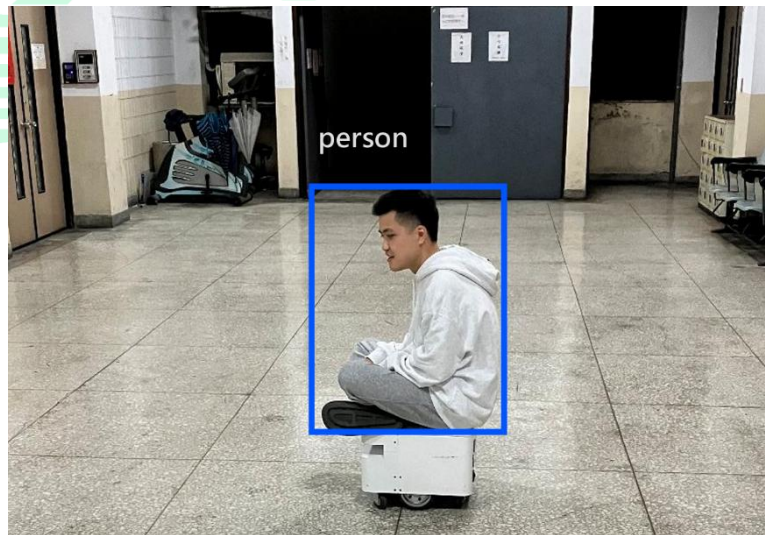


圖 2.8 機器狗設定目的地

如圖 2.8 所示，本論文使用機器狗的相機來獲取 RGB 影像，並利用 YOLO[15]演算法來檢測行人的位置。根據行人的位置，系統會決定機器人的該

前往的目標位置。由於影像中的位置信息是基於機器人的座標系，影像判斷目的地的系統是每秒更新一次，速度太慢會導致位置判斷的誤差。因此，本論文將行人的位置資訊轉換至 Odometry 座標系，以確保機器人在移動過程中，不會改對行人位置的評估發生改變。此外，行為規劃系統會依照運行頻率，持續將行人的位置轉換至相對於機器人的座標系中的目標位置，從而引導機器人朝正確的方向前進。

2.5 系統分析

2.5.1 系統的泛化性

系統的泛化性指的是一個系統能夠適應並成功完成各種不同任務的能力。一個具有高泛化性的系統能夠使用同一套流程來應對所有任務類型。目前，最成功的例子之一是 OpenAI 的 ChatGPT，它能夠利用文字回答各式各樣的問題。

模塊化在提升系統泛化性中的作用：

眾多研究指出，模塊化是提高系統泛化性的關鍵。例如，論文[16]和[17]都提到了如何利用 NLP 的 Transformer 模型，通過零樣本學習方式轉換任務，從而直接控制機器手臂。此外論文[18]的研究展示了如何通過模塊化的強化學習模型來訓練多任務，證明了模塊化系統在不同系統組件中的實施都能顯著提升泛化能力。

論文[19]進一步提出，通過分別訓練任務和控制模塊，並在最終將不同的任務和控制模塊進行任意組合的測試，這顯示了經過模塊重組後的系統能夠展現出優異的泛化性，即使是從未一起訓練過的 AI 模型組合也能夠理解任務並控制機器人。

論文[20]通過使用多模塊結構，在多個機器人上訓練一個模型，最終實現了將移動能力泛化到未曾訓練過的機器人上的目標。

這些研究表明，要實現更高的系統泛化性，採用模塊化設計至關重要。無論是在模型的架構上，還是在處理輸入輸出資訊的過程中實施模塊化，都被證明是促進泛化性的有效選擇。

2.5.2 效能與泛化性的平衡

本論文在系統設計，將移動控制的目標根據控制頻率區分模塊，同時謹慎考量系統的效能。這不僅是分配模型的計算資源，還涉及系統的即時反應能力。這些措施確保了在擴展系統能力的同時，系統效能不會受到影響。

在實際應用中，本論文的 RL 行為規劃系統模型僅有 1.5M 個參數（相較於 yolov4-tiny 的 6M 個參數），這使得模型在大多數的嵌入式開發板的 CPU 上都能達到 20 個 fps 的運行速度，而且本論文的 RL 模型在 3 個 fps 以上時就能確保達到訓練時的效果。因此，運算效能不會成為本系統的瓶頸。



第三章 RL 行為規劃策略

本章節將針對 RL 的技術難點，詳細說明如何訓練出具備如此繁雜能力的 Spot Agent。此外，將會深入探討本論文針對此任務所使用的 RL 訓練每個部分的想法與結論，並從頭構建出一個 RL 的行為規劃策略時所需要的思考重點，設計過程，實驗設計和結論總結。

3.1 RL 基本結構

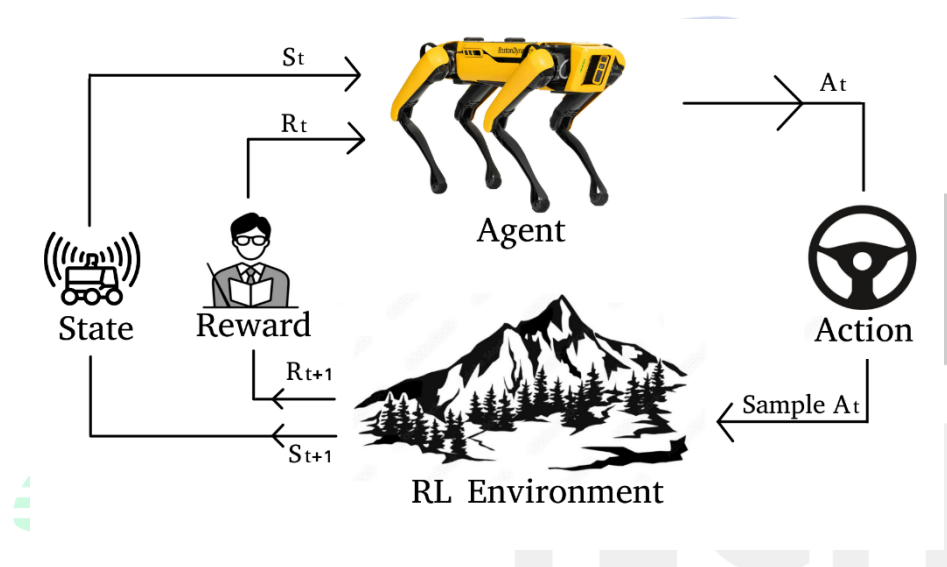


圖 3.1 RL 基本結構

Agent

Agent 是 RL 環境中的決策者和行動者，負責感知環境狀態並採取行動。

Reward

在 RL 環境中根據 Agent 的動作給予的反饋，圖 3.1 以 R_t 和 R_{t+1} 表示，它可以是正面的獎勵或負面的懲罰，用於指導 Agent 學習。

State

RL 環境在某個時間點的可見資訊，圖 3.1 以 S_t 表示，它包含了 Agent 感知到的所有信息，例如雷達傳感器、自身位置等。

Action

Agent 所能採取的行動，例如：往前走，左轉右轉。

RL 環境

Agent 所處的周圍環境，圖 3.1 以山脈和樹林示意，它會根據 Agent 的 Action 判斷觸發的事件，並根據事件好壞給予反饋的 Reward。

3.2 RL 環境中的物體

目前具備抵達目的地，避開障礙物，上下樓梯，姿態控制和即時應對未知狀況能力的論文和系統都非常少見。為了讓系統能更好地應對未知狀況，本論文提出了使用移動的 Goal 和 Obs Agent，來模擬真實世界可能出現的目標點與障礙物。這種方法通過多個 Agent 的移動豐富了 Spot 在不同情景學習策略的多樣性。

此外，本論文採用了隨機生成新地圖的方式進行訓練，使 Spot 能夠根據當下接收到周圍的資訊來判斷周圍的地形，從而選擇最合適的行為策略，進一步增強了其在多變環境中的適應能力。

在 RL 任務目標的設計至關重要，這會影響在 RL 環境中的每一個 Agent 的行為，本論文的目的是在於訓練 Spot 的行為規劃策略，因此在 RL 環境中的每一個 Agent 都是為了 Spot 完成任務目標而存在的。

Spot Agent

Spot 作為核心 Agent，其設計需符合 Boston Dynamics Spot 的實際運動模型。

在 Spot 的行動空間包括 x 和 y 方向的加速度以及角速度。為了提供更精細的控制，每個 Action 都根據其上下限被劃分成多個線性層級。Spot 的運動模型考慮了其硬體限制，例如最大速度和加速度，以確保訓練結果的真實性和可行性。

Goal Agent

Goal 的行為如何影響到 Spot?

Goal Agent 的設置是為了模仿真實世界中 Spot 機器狗可能遇到的目標或任務。通過設計一個可移動的目標 Agent 來增加訓練的複雜性，促進 Spot 學習多樣化的技能，如避開牆壁、有效上下樓梯及瀏覽狹窄區域等。此動態的 Goal Agent 的目標是通過引導 Spot 至特定的學習挑戰區域（如牆壁後方、樓梯或狹縫之間），迫使其習得相應的避障技巧並能順利抵達目的地。

為何 Goal 也是 Agent?

Goal 設計成 Agent 是受到生成對抗網絡(GAN)的啟發。在 GAN 中，生成器(Generator)學習產生逼真的數據，而辨別器(Discriminator)學習區分真實數據和生成數據。類似 Spot Agent 可以被視為 Generator，產生最好的運動控制策略。Goal Agent 可以當成 Discriminator，分辨 Spot Agent 哪個角度追不到 Goal 或是會導致 Spot Agent 撞到牆，讓 Goal 生成 Spot 需要學習的挑戰，生成 Spot 能力較差的任務。

Obstacle Agent

Obs Agent(動態障礙物)的目的是為了增加 Spot 能有針對動態障礙物避障的能力，它們的行為模式旨在模擬真實世界中可能遇到的各種動態障礙物，例如行人、車輛、也有可能是石頭等。設計 Obs 為 Agent 除了模擬動態障礙物，也是為了訓練 Spot 在面對真實世界不規則移動的動態障礙物時，該執行的策略，例如球可能會突然飛過來撞 Spot。所以將 Obs 設計為 Agent 可以讓 Spot 學習如何應對這種動態性，提高其在真實世界中的適應力。

為了增加訓練的多樣性，Obs 的移動方式加入了部份隨機行為，以增加 Spot 訓練的多樣性，並避免 Spot 學習到過度依賴某種特定障礙物的行為模式。

RL 環境設計

在 RL 環境中，本論文設計了邊界和地形，包括牆壁、樓梯等靜態障礙物。提供了 Spot 在訓練過程中需要應對的不同挑戰，同時也決定了 Spot 在最終訓練中所能達到的能力上限。本論文將現實環境中的地形大致分為兩類：

1. 無法跨越的地形：這些地形對 Spot 的硬體來說過高或無法穿越，在 RL 環境中將這些障礙物設計為牆壁。
2. 可以跨越的地形：這些地形是 Spot 能夠通過硬體規劃和姿態調整來應對的，在 RL 環境中設計為樓梯。

此種設計的目的是希望能將現實中的每一類障礙物轉換為 RL 環境中的模擬障礙物，從而使 Spot 在實際場景中具備對應的處理能力。為了讓 Spot 在不同情境下具備高泛化性，本論文採用了定期更新 RL 環境的策略。在 Spot 完成一百次訓練後，會隨機更改 Agent 的初始位置及 RL 環境中地形的配置，並調整障礙物的大小和位置。這樣的設計可以確保 Spot 在不同地形和場景中都能夠適應變化，並根據障礙物的大小及距離調整行為規劃的策略。

泛化性的重要性

RL 的訓練雖然與一般 AI 訓練方式有所不同，可以控制訓練資料的重要性，這也意味著如何收集資料變得非常關鍵。如果收集資料（與 RL 環境的互動）的分布與真實世界有很大的落差，即使訓練效果再好，到了實際執行時，效果也難以達到預期。

許多 RL 的訓練策略會遇到類似圖 3.2 的問題。其中 Spot 希望能通過不斷訓練，成功抵達目的地並避開障礙物。訓練完成後，Spot 就能夠達到圖中藍色路徑的效果。

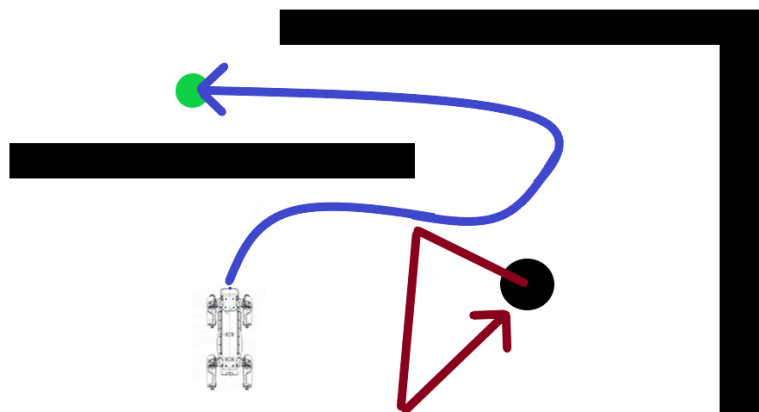


圖 3.2 模擬 RL 訓練場景

然而，實際情況中，障礙物和目的地的位置與訓練時的場景不同，如圖 3.3 所示。現實裡 Spot 使用訓練時學到的策略（藍色路徑），卻既無法抵達目的地，還可能撞到障礙物。

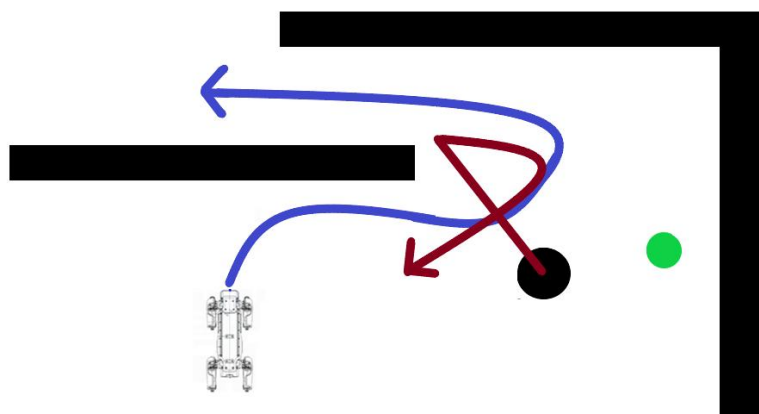


圖 3.3 模擬 RL 實際場景

這種情況對於 RL 訓練來說是常見的問題，因為我們無法完全預測實際情況會發生什麼變化。因此，本論文希望 RL 訓練出來的策略具備足夠的泛化性，使得訓練策略能夠應用於真實世界中。

為了達到這一目標，本論文採用了 Multi-Task RL 的訓練方法，讓訓練場景經常更換。同時，使用 Multi-Agent RL，將現實中可能移動的物體都設計為 Agent，使得 Spot Agent 能夠在變化的環境中學習並採用真實有效的策略。

3.3 Agent 的設計

3.3.1 Agent 的動作

Action(動作)是 Agent 與環境交互的基本方式，通過這些交互，Agent 才能訓練哪種動作策略能獲取最高的 Reward。此小節介紹我們如何設計 Action，並解釋每一個 Agent 的 Action 設計邏輯和方法。

Spot Agent 的 Action

Boston Dynamics 的 Spot 機器狗採用了「速度控制」來操控其行為，而本論文則使用「加速度控制」來設計 RL 中的 Spot Agent 操控策略。相較於速度控制，加速度控制提供了更精細的運動調整，使機器人在力量控制上更加精確，並能實現更接近自然的行為規劃策略。

雖然本論文的 Spot Agent 使用加速度控制來調整其運動狀態，但在實際操作中仍須將其轉換回 Boston Dynamics Spot 所需的基本速度控制模式。該機器狗的速度限制如圖 3.4 所示。

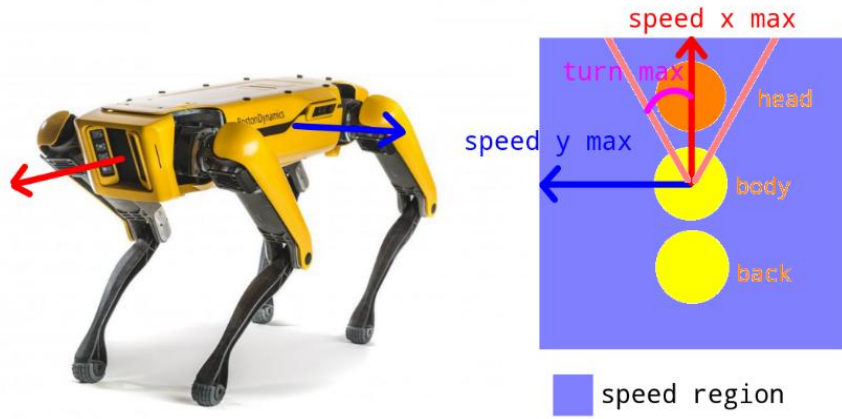


圖 3.4 真實 Spot 運動能力

Boston Dynamics Spot 的速度(v) 限制，不論速度方向其上限皆為 1 m/s，且角速度(ω)的上限 45 度/s，如(3.1)式所示。

$$\begin{aligned} \max(v_x) &= 1\text{m/s} & \min(v_x) &= -1\text{m/s} \\ \max(v_y) &= 1\text{m/s} & \min(v_y) &= -1\text{m/s} \\ \max(\omega) &= 0.25\pi \text{ rad/s} \end{aligned} \quad (3.1)$$

v ：代表速度

ω ：代表角速度

在 RL 中設計 Spot Agent 時，本論文參考了 Spot 的硬體限制來確定運動控

制的極限值，並依此進行速度更新計算，如(3.2)式所示。

$$v_{t_n} = \text{clip}(\max(v), \min(v), v_{t_{n-1}} + acc_{t_n}) \quad (3.2)$$

acc ：代表 Agent 的加速度

當速度達到上下限時，RL 訓練會調整加速度以保持速度在允許範圍內，使模型在訓練中修正到合適的 Action，如(3.3)式所示。

$$acc_{t_n} = v_{t_n} - v_{t_{n-1}} \quad (3.3)$$

此外，RL 中 Spot 的方向角度也會被修正到合理範圍內，確保 Agent 在計算中不會出現問題，如(3.4)式所示。

$$\theta_{t_n} = \text{clip}(0, 2\pi, \theta_{t_{n-1}} + A_{\varpi}) \quad (3.4)$$

θ ：代表 Agent 在地圖中的角度

此外，Spot 在垂直方向（z 軸）的速度控制，以及 roll 與 pitch 姿態角的調整，皆由其內部控制機制自動處理，使用者無法直接下達指令加以控制。透過上述方法，RL 環境中的動作設計得以專注於平面方向（x、y 軸與角速度）的速度控制，進而使訓練結果可順利應用於實際的 Boston Dynamics Spot 機器人操控上。

對於 Spot Action 的定義，由於 Spot 的運動控制通過加速度調整其最終速度。動作集合的定義如(3.5)式所示，包含了沿 x 軸和 y 軸的加速度以及轉動速度。

$$A_{Spot} = [A_{acc_x}, A_{acc_y}, A_{\varpi}]^T \quad (3.5)$$

A ：代表 Agent 的 Action

根據 Boston Dynamics Spot 的實驗結果，1 秒內達到最高速度 1 m/s 是穩定

的加速度，一次控制時間為 0.3s，因此在一次控制中最多能改變 0.3m/s 的速度，而轉角的改變則為 $0.075\pi \text{ rad}$ ，如(3.6)式所示。

$$\begin{aligned} \max(A_{acc_x}) &= \frac{\Delta v}{\Delta t} = \frac{0.3 \text{ m/s}}{0.3 \text{ s}} = 1 \text{ m/s}^2 \\ \max(A_{acc_y}) &= \frac{\Delta v}{\Delta t} = \frac{0.3 \text{ m/s}}{0.3 \text{ s}} = 1 \text{ m/s}^2 \\ \max(A_{\varpi}) &= \frac{\Delta \theta}{\Delta t} = \frac{0.075\pi \text{ rad}}{0.3 \text{ s}} = \frac{0.25\pi \text{ rad}}{\text{s}} \end{aligned} \quad (3.6)$$

本論文 RL 中的控制頻率為 3 fps，代表在實際執行過程中需要大於等於 3 fps（實際控制頻率可以 200 fps），就能確保至少與 RL 訓練中的反應速度一致。

Goal Agent 的 Action

在本論文中，Goal Agent 也使用「加速度控制」方式進行操控。Goal 的動作集合定義如(3.7)式所示。

$$A_{Goal} = [A_{acc_x}, A_{\varpi}]^T \quad (3.7)$$

Goal Agent 的速度控制如(3.8)式所示， v_{Goal} 代表 Goal 速度與 Spot 的比例，會根據訓練階段的不同有所變化（常態為 0.7），這個值必須要小於 1，才確保 Spot 能有效追趕 Goal。另外為確保 Goal 速度的多樣性設定 v_{skill} ，利用隨機變數讓每個 Goal Agent 上限速度不同，以構成多樣化的 Goal Agent。

$$\max(v_x) = \max(v_{Spot}) \times v_{skill} \times v_{Goal} \times 1 \text{ m/s} \quad (3.8)$$

v_{skill} ：為值域範圍 $[0.2, 1]$ 的變數

根據實驗結果，如果速度的正負對於 Reward 沒有區別，Goal Agent 的速度行動只會出現全為正值或全為負值。為了讓 Goal 方便使用，只保留了正向的速度和其前進的角度。

$$\min(v_x) = 0 \quad (3.9)$$

Goal Agent Action 的定義根據控制頻率 (fps) 計算加速度和轉動速度的最大值，如(3.10)式所示。

$$\begin{aligned} \max(A_{acc_x}) &= \frac{\Delta v}{\Delta t} = \frac{0.21 \text{ m/s}}{0.3 \text{ s}} = 0.7 \text{ m/s}^2 \\ \max(A_{\varpi}) &= \frac{\Delta \theta}{\Delta t} = \frac{0.053\pi \text{ rad}}{0.3 \text{ s}} = \frac{0.18\pi \text{ rad}}{\text{s}} \end{aligned} \quad (3.10)$$

本論文未設置 Goal Agent 的 y 軸速度，因為 Spot 不需要追逐任意方向移動的目標。

Obs Agent 的 Action

為了讓 Spot Agent 能夠應對多種障礙物環境，本論文設計了 Obs Agent 來模擬不同速度的障礙物。Obs Agent 的動作集合定義如(3.11)式所示。

$$A_{Obs} = [A_{acc_x}, A_{\varpi}]^T \quad (3.11)$$

Obs Agent 的速度控制如(3.12)式所示，為了讓 Obs Agent 具有更多的速度多樣性，依然使用 v_{Obs} 設定 Obs Agent 和 Spot 的移動速度比例，在訓練開始時期值為 0.85，最後會超過 1，讓 Spot 能夠更加適應不同速度的動態障礙物。

$$\max(v_x) = \max(v_{Spot}) \times v_{skill} \times v_{Obs} \times 1\text{m/s} \quad (3.12)$$

作為模擬障礙物，Obs Agent 僅需正向速度和方向角度的控制即可滿足所有需求，因此速度控制如(3.13)式所示。

$$\min(v_x) = 0 \quad (3.13)$$

對於 Obs Agent Action 定義，也是根據控制頻率（fps）計算加速度和轉動速度的最大值如(3.14)式所示。

$$\begin{aligned} \max(A_{acc_x}) &= \frac{\Delta v}{\Delta t} = \frac{v_{Obs} \times 0.3 \text{ m/s}}{0.3 \text{ s}} = v_{Obs} \text{ m/s}^2 \\ \max(A_{\omega}) &= \frac{\Delta \theta}{\Delta t} = \frac{v_{Obs} \times 0.075\pi \text{ rad}}{0.3 \text{ s}} = \frac{v_{Obs} \times 0.25\pi \text{ rad}}{s} \end{aligned} \quad (3.14)$$

通過這樣的設計，Obs Agent 可以以不同的速度運行，為 Spot Agent 提供多樣化的移動挑戰，從而幫助 Spot 更有效地學習和應對不同速度的障礙物。

3.3.2 RL 環境中的 Agent

本論文的 RL 環境目的是為了 Spot 機器狗提供一個安全的訓練的場域，使其能夠學習避障礙物且到達目的地的技能。

RL 環境中的 Spot Agent

計算 Reward 和 State 中需要計算大量的事件發生機率，為了便於計算 Agent 與地形障礙物之間的距離，Spot Agent 的形狀設計採用了多個圓形來模擬其頭部、身體與背部，如圖 3.5 所示。這樣的設計可以有效簡化距離計算，從而判斷事件的發生情況。

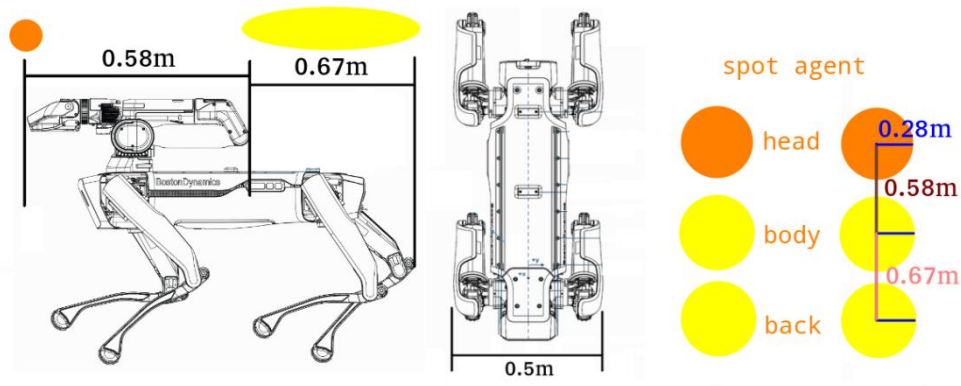


圖 3.5 Spot 在 RL 環境中的設定

當 Spot Agent 的任一部位與障礙物的距離小於指定閾值時，便認為發生了碰撞事件。事件判斷如(3.15)式所示。

$$E = Spot - Map = dis(Map, head) < 0.28 \text{ or } dis(Map, body) < 0.28 \text{ or } dis(Map, back) < 0.28 \quad (3.15)$$

Map：代表 RL 環境中的的不同靜態障礙物

E：代表事件的發生，可用於計算 Reward 和 State 資訊

Spot Agent 的位置和速度更新依據其加速度、當前速度及方向角進行計算。當前時間步長的速度 (v_{t_n}) 由上一步的速度加上加速度 ($A_{acc_{t_n}}$) 來計算，如(3.16)式所示。

$$v_{t_n} = v_{t_{n-1}} + A_{acc_{t_n}} \quad (3.16)$$

獲得當前速度後，位置更新公式考慮了移動方向與控制頻率，如(3.17)式所示。

$$\begin{pmatrix} Loc_{t_n}^x \\ Loc_{t_n}^y \end{pmatrix} = \begin{pmatrix} Loc_{t_{n-1}}^x \\ Loc_{t_{n-1}}^y \end{pmatrix} + \Delta t \cdot \begin{pmatrix} v_{t_n}^x \cdot \cos(\theta_{t_n}) - v_{t_n}^y \cdot \sin(\theta_{t_n}) \\ v_{t_n}^x \cdot \sin(\theta_{t_n}) + v_{t_n}^y \cdot \cos(\theta_{t_n}) \end{pmatrix}$$

$$\Delta t = \frac{1}{fps} \quad (3.17)$$

Loc：代表 Spot 在 RL 環境中的座標

對於位置限制，Spot Agent 的位置被限制在地圖範圍內，確保其在允許的最大與最小值範圍內，如(3.18)式所示。

$$Loc_{t_n} = clip\left(Loc_{t_n}, max(Map), min(Map)\right) \quad (3.18)$$

RL 環境中的 Goal 和 Obs Agent

在 RL 訓練中的行為規劃策略上，Spot 抵達目的地需要考量訓練中的到達機

率。與傳統演算法不同，無法將目的地設定為一個帶方向的點，這樣可能會導致 Spot 無法抵達目標。本論文設定的目的地只要 Spot Agent 的**身體**部位與目的地的距離足夠接近即可視為抵達，而無需考慮其到達的方向。足夠接近的距離設定為 0.28 公尺，這相當於 Goal Agent 的大小。因此，Spot Agent 與目標之間事件的條件如(3.19)式所示。

$$E = Spot - Goal = dis(Goal, body) < 0.28 + Size_{Goal} \quad (3.19)$$

Obs Agent 的設計目的是訓練 Spot Agent 躲避動態障礙物。Spot Agent 與障礙物之間事件的發生條件如(3.20)式所示。訓練初期，Obs Agent 的大小設為半徑 0.28 公尺，最終將大小隨機分布於 0.08 到 0.48 公尺之間。

$$E = Spot - Obs = dis(Obs, head) < 0.28 + Size_{Obs} \text{ or } dis(Obs, body) < 0.28 + Size_{Obs} \text{ or } dis(Obs, back) < 0.28 + Size_{Obs} \quad (3.20)$$

Goal 和 Obs Agent 的速度與位置更新機制與 Spot Agent 類似，但不包含 y 軸方向的移動。設計上假設它們的移動方式如同正常車輛，只能轉向並前進。因此，位置更新如(3.21)式所示。

$$\begin{pmatrix} Loc_{t_n}^x \\ Loc_{t_n}^y \end{pmatrix} = \begin{pmatrix} Loc_{t_{n-1}}^x \\ Loc_{t_{n-1}}^y \end{pmatrix} + v_{t_n}^x \cdot \Delta t \cdot \begin{pmatrix} \cos(\theta_{t_n}) \\ \sin(\theta_{t_n}) \end{pmatrix} \quad (3.21)$$

通過這些設置，Goal 和 Obs Agent 能夠在 RL 環境中以預定的方式移動，為 Spot Agent 提供一個多樣且具有挑戰性的訓練環境。此設計有助於 Spot Agent 在複雜環境中有效地達成任務目標，並提升其行為規劃的策略性。

3.4 Reward 的設計

本論文對 RL Reward(獎勵函數)的設計方式進行探討，綜合分析了實驗中所發現的評估方法。此外，本論文會闡述了在 Single-Agent RL 和 Multi-Agent RL

訓練過程中，如何精心構築 Reward。Reward 的設計對於學習成果的質量起著決定性作用，因為它指引 Agent 在學習過程中的行為選擇，確保 Agent 能夠從其互動中獲得最大利益。

3.4.1 Reward 的目的

RL 與其他 AI 訓練方法不同，RL 中的 Agent 估計 Action 好壞的方式並非基於單一時間點的結果，而是透過預測未來累積獎勵或懲罰(Reward)來判斷。具體來說 Agent 會使用當前的 Model θ 與 RL 環境進行多次互動，計算從當前時刻到未來結束時刻的訓練資料 τ ，平均能累積不同的 Reward(R)其發生的機率(p)，將此期望值 Value(V)作為評估當前 Action 好壞的指標，如(3.22)式所示。

$$V_{\theta} = \sum_{\tau} R(\tau)p_{\theta}(\tau) \quad (3.22)$$

V ：代表期望值

θ ：代表在某一時刻的 Agent

R ：代表 Reward

p ：代表機率

如(3.23)式所示，設計者需要定義 Agent 與 RL 環境互動過程中可能發生的事件(E)，並為每個事件設定對應的 Reward 值。這些 Reward 值代表了事件對 Agent 完成目標任務的影響，正向 Reward 鼓勵 Agent 重複該行為，而負向 Reward 則促使其避免該行為。透過這種方式，Agent 可以學習到最大化未來期望值(V)的策略，從而更好地完成目標任務。

$$V_{\theta} = V_{R_{E_1}} + V_{R_{E_2}} = \sum_{\tau} R_{E_1}(\tau)p_{E_{\theta}}(\tau) + \sum_{\tau} R_{E_2}(\tau)p_{E_{\theta}}(\tau) \quad (3.23)$$

R_E ：代表 Agent 在事件 E 發生時能獲得的 Reward 值。

$p_{E_{\theta}}$ ：代表 Agent θ 發生事件 E 的機率。

V_{R_E} ：代表 Agent 在 R_E 這一項 Reward 的期望值。

E 事件的產生方式為 Agent 與其他 Agent 發生接觸，或是與 RL 環境中的地形發生接觸，我們稱之為事件的發生，而 E 事件發生時常會伴隨 Reward 的設定。因此本論文將分析 Reward 的 Agent 稱為 A，被接觸的物體或另一 Agent 則稱為 B，來探討這些事件對訓練的影響，如(3.24)式所示。另外 R_{A-B} 所產生的 Reward 期望值就是 V_{A-B} 。

$$R_E = R_{A-B} \quad (3.24)$$

單純的 Reward 適合用於計算 Agent 訓練時的數值，但在開發過程中難以直接進行分析。

$$V_{\psi_E} \neq V_{R_E} \quad (3.25)$$

ψ ：代表 Agent 的某一項能力

如(3.25)式所示，如果希望開發一個具備能力 ψ 的 Agent，例如尋找目的地的能力，那麼我們會將 Agent 抵達目的地的事件設為 $E = \text{Spot} - \text{Goal}$ ，並希望透過 $V_{R_{\text{Spot}-\text{Goal}}} > 0$ 的方式來學習這項能力。然而，實際上要訓練一個 Spot 抵達目的地的能力，並非單純只需要考慮 $V_{R_{\text{Spot}-\text{Goal}}}$ ，還需要考慮是否會撞到牆體、Agent 是否有移動意圖，以及其他可能影響尋找目的地能力的 Reward，因此在分析 Agent 的 Reward 設計時，我們需要從能力的角度來考量設計方法，如(3.26)式所示。

$$V_{\psi} = V_{R_{E_1}} + V_{R_{E_2}} + V_{R_{E_3}} + \dots \quad (3.26)$$

因此，本論文會以不同能力的角度分析 Reward 的設計方法。

3.4.2 Single Agent 的分析

在進行 Multi-Agent RL 訓練之前，對單一 Agent 在靜態環境下的獎勵設計進

行分析是一項不可忽視的重要前置工作。分析單一 Agent 的行為有助於確定在其他 Agent 行為不變的情況下，如何引導 Agent 學習。只有當每個 Agent 在各自的方向上達到理想的訓練效果後，合併後的 Multi-Agent RL 才能夠成功。

3.4.2.1 Agent 訓練進步條件

為了讓 Agent 的能力逐步提升，我們需要明確其學習進步的條件。根據實驗結果，每個 Agent 至少需要具備以下幾個基本能力：

- $V_{survive} > 0$ ：存在期望值大零
- 重要能力的期望值一定要大於次要能力的期望值
- Agent 需要有改變機率的可能性

存在期望值大於零的條件非常重要，因為如果在某段時間內期望值小於零，Agent 可能會快速結束訓練，從而無法收集到任何有用的訓練數據，最終導致 Agent 迅速遺忘所學技能。

Reward 的本質在於使用不同的 Reward 影響 Agent 對於不同事件重要性的判斷，從而改變 Agent 的行為選擇。如果次要能力的期望值大於重要能力的期望值，Agent 可能會逐漸重視不重要的任務，遺忘重要技能，引起 Reward 的劇烈波動，最終會導致 Model 的損毀。

對於 Agent 來說，Reward 必須具備引導作用。如果某事件發生的機率過低，甚至該事件的發生不是由 Agent 主導的，那麼 Agent 將無法從中學習有用技能，此時的 Reward 就會變成無效的雜訊。

3.4.2.2 Spot Agent 的分析

Spot Agent 需要能夠抵達目的地、選擇最短路徑到達目的地、盡量朝前方移動、避免撞到靜態或動態障礙物、以及上下樓梯行為需符合機器人的限制。基於這些目標，本論文對於 Spot 訓練進步條件設定了以下三個關鍵原則。

Spot 存在期望值

存在期望值需要考慮 Agent 在 RL 環境中平均能獲得多少期望值，因此需要考慮 Spot 存活時會遇到的全部 Reward，如(3.27)式所示。我們需要確保總期望值大於 0，Spot 才會選擇移動，否則待在原地不動就能獲取最大的期望值。

$$V_{Survive} = V_{RStair} + V_{RMove} + V_{RSpot-Goal} + V_{RSpot-Spot} + V_{RSpot-Map} + V_{RSpot-Obs} > 0 \quad (3.27)$$

V_{RStair} ：代表 Agent 在成功上下樓梯 Reward 的獎勵期望值

V_{RMove} ：代表 Agent 在移動時的獎勵期望值

$V_{RSpot-Goal}$ ：代表 Spot 成功抵達目的地時的獎勵期望值

$V_{RSpot-Spot}$ 、 $V_{RSpot-Map}$ 、 $V_{RSpot-Obs}$ ：代表 Spot 碰撞到不同物體時的懲罰期望值

Spot 行為規劃策略中最重要的任務，是抵達目的地並且避免碰撞障礙物，因此存在期望值是最重要引導 Spot Agent 進步的項目，由於 Spot Agent 起初的策略是隨機移動，因此碰到障礙物的機率顯著高於抵達目的地的概率，如(3.28)式所示。

$$p_{Spot-Obs} + p_{Spot-Spot} + p_{Spot-Map} \gg p_{Spot-Goal} \quad (3.28)$$

$V_{Survive}$ 越大，Spot 對於抵達目的地並且避免碰撞的能力訓練速度越快，一旦 $V_{Survive}$ 小於零，Spot Agent 將選擇停止移動以降低撞到障礙物的可能性。如(3.29)式所示，為了提高**存在期望值**需要將 Spot 接觸到 Goal 單一事件的獎勵 ($R_{Spot-Goal}$) 調整大一些，才有可能有足夠大的存在期望值。

$$V_{Spot-Goal} + V_{Spot-Obs} + V_{Spot-Spot} + V_{Spot-Map} > 0 \\ \Rightarrow R_{Spot-Goal} \gg |R_{Spot-Obs}| + |R_{Spot-Spot}| + |R_{Spot-Map}| \quad (3.29)$$

(3.27)式中的 V_{RMove} 為 Spot 移動的期望值，此 Reward 的目的在於鼓勵 Spot

多移動，此 Reward 只能用於前期讓 $p_{Spot-Goal}$ 提高，後續不能讓移動的重要性高於抵達目的地，因此 V_{RMove} 不能有太大的數值。(3.27)式中的 V_{RStair} 為鼓勵 Spot 上下樓梯的，也不能有太大的數值。因此 Spot 的存在期望值，主導的核心 Reward 為 $R_{Spot-Goal}$ 。

Spot 行為期望值

行為期望值的主要目標是基於 Boston Dynamics 的 Spot 機器狗的硬體條件，訓練出一個能夠符合「正常」行為模式的 Spot Agent。具體來說，我們期望 Spot Agent 更傾向於前進而非後退，並希望其行為選擇能夠遵循，(3.30)式的優先順序。

$$V_{v_x} > V_{v_{-x}} > V_{v_y} \quad (3.30)$$

另外我們也希望 Spot 在進行方向調整時，能夠更偏好使用轉向角來達到目標位置，而非側邊移動，如(3.31)式所示。

$$V_{\omega} > V_{v_y} \quad (3.31)$$

如(3.32)式所示，生存期望值的優先級應高於所有移動速度期望值的總和，這確保 Spot 優先選擇能夠長期生存的行為，而不是盲目追求姿態控制。

$$V_{Survive} > V_{v_x} + V_{v_{-x}} + V_{v_y} \quad (3.32)$$

總結來說，我們的最終目標是通過基於期望值的獎勵控制方法，使 Spot Agent 能夠自然、靈活地執行多方向移動，並在前進、後退、轉向和生存策略之間達到平衡，最終有效實現訓練目標。對於如何讓 Spot Agent 有效控制身體姿態，詳細內容在融合能力的方式的章節中說明。

Spot 上下樓梯期望值

針對 Boston Dynamics 的 Spot 機器狗，在上下樓梯時存在一些硬體上的動作限制。例如，上樓梯時不能以屁股先行，下樓梯時則不能讓頭朝前，這使得在 RL 環境中的 Spot Agent 要學會正確的上下樓梯動作變得更加具有挑戰性。

為了確保 Spot 能夠學會上下樓梯的技能，我們需要設定樓梯上的期望值高於平地的行為期望值，這意味著在 RL 環境中，Spot 在樓梯上的行為應該獲得比在平地上更高的獎勵值，如(3.33)式所示。

$$V_{Stair} > V_{Flat} > 0 \quad (3.33)$$

在設計行為期望值時，我們期望 Spot Agent 儘量保持朝前移動。然而，當 Spot 在上下樓梯時，其行為模式會和平地上的行為有所不同。因此，當 Spot 在樓梯上行動時，它的行為決策與平地上不相同，如(3.34)式所示。

$$\underset{a}{\operatorname{argmax}} V_{Behavior}(a) \neq \underset{a}{\operatorname{argmax}} V_{Stair}(a) \quad (3.34)$$

這意味著 Spot 需要在樓梯上做出不同的動作選擇，特別是當它在向後移動時，也能夠根據周圍環境做出正確的選擇。由於樓梯空間較狹窄，碰撞風險較高，這也進一步增大了 Spot Agent 在這種環境中學習的難度。

此外，我們希望 Spot 在判斷如何到達目的地時，選擇經過樓梯，而不是基於自身行為偏好。因此，要讓 Spot 能夠學習並執行正確的上下樓梯行為，Reward 設定變得極具挑戰性，這需要利用 Multi-phase RL 才能處理，相關細節將在 Multi-phase RL 的章節中進一步討論。

3.4.2.3 Goal Agent 的分析

Goal Agent 的目標是間接引導 Spot Agent 學習如何到達目標位置以及掌握上下樓梯的技能。因此，Goal Agent 最重要的目標是盡可能走到樓梯上，其次是採

取多元策略，以便 Spot Agent 能學會高效抵達不同目的地的方法。為了提升 Spot Agent 的學習效果，需要根據以下幾個原則設計 Goal Agent。

Goal 上下樓梯期望值

因為 Spot Agent 不應獨自做出上下樓梯的決策，因此上下樓梯行為對 Goal Agent 來說至關重要。只有當目的地位於樓梯時，Spot 才應該選擇上下樓梯的行為。因此，引導 Spot Agent 學會上下樓梯成為 Goal Agent 的核心任務。

$$V_{Stair} = V_{RStair} + 0.6 \times V_{Goal-Map} + 0.5 \times V_{Goal-Spot} \quad (3.35)$$

$V_{Goal-Map}$ ：代表 Goal Agent 撞到 RL 環境中的障礙物這個事件的期望值

$V_{Goal-Spot}$ ：代表 Goal Agent 碰到 Spot Agent 這個事件的期望值

如(3.35)式所示，上下樓梯期望值除了考慮成功上下樓梯的獎勵外，還需要考慮撞牆的風險 $V_{Goal-Map}$ 和路途中可能遇到 Spot Agent 被抓住的風險 $V_{Goal-Spot}$ 。由於此期望值為核心期望值，因此 V_{RStair} 需要具備主導 Goal Agent 學習的能力。

Goal 存在期望值

存在期望值並非 Goal Agent 最重要的 Reward，上下樓梯的能力才是最核心的要求。如(3.15)式所示，Goal Agent 的存在期望值需要大於 0。由於 V_{RStair} 為主導的 Reward，理論上存在期望值必然大於 0，但實際上在訓練的早期階段，Goal 並沒有上下樓梯的能力，因此 p_{RStair} 的機率過低，無法成為有效的引導條件。

$$V_{Survive} = V_{RStair} + V_{RMove} + V_{RStep} + V_{Goal-Spot} + V_{Goal-Map} > 0 \quad (3.36)$$

常見的設計方法如論文[21]所提的 Step Reward，若 Agent 在 RL 環境中多存在一個時刻，就增加一點正向的 Reward，本論文中的 Goal Agent 也有 Step Reward(R_{Step})，確保 Goal Agent 會想要存活於 RL 環境中。也可以通過論文

[22]和[23]所提的 Move Reward，讓 Agent 在移動的過程中增加一點正向的 Reward(R_{Move})，來獎勵 Goal Agent 的移動行為，因為 Move Reward 也需要 Agent 存在於 RL 環境中才有獎勵，因此這兩種 Reward 都能促使 Goal Agent 想要存活於 RL 環境中。

Goal 移動期望值

存在期望值中使用 Step Reward 或 Move Reward 作為初期主導，可能會產生兩種不良影響：Move Reward 主導會讓 Goal Agent 想要減少碰撞機率又想要多移動，採取原地徘徊的策略，而 Step Reward 主導則會讓 Goal Agent 想要存活久一些，學會躲在地圖角落的策略。為了讓 Goal Agent 學會移動，而不是停留在原地或逃避至角落，需要確保**移動**行為的期望值高於**靜止**狀態，如(3.16)式所示。

$$V_{Move} > V_{Rstep} \quad (3.37)$$

要鼓勵 Goal Agent 探索地圖，才能讓 Goal Agent 學會走樓梯的能力，並最終引導 Spot Agent 學會上下樓梯。因此移動期望值需要考慮沿途的風險和可能遇到的獎勵，如(3.38)式所示。

$$V_{Move} = V_{Rstair} + V_{RMove} + V_{Goal-Map} \quad (3.38)$$

Move Reward 的具體設計取決於 Goal Agent 的移動速度，速度越快 Move Reward 越大，如(3.39)式所示。

$$V_{RMove} = w \times v \quad (3.39)$$

w ：代表用於平衡權重之參數

當 Goal Agent 採用更多的移動策略時，更容易接觸到樓梯並獲得大量的 Reward，這將提升 Goal Agent 接觸樓梯的機率，最終自然轉為 V_{Rstair} 主導

的存在期望值。

3.4.2.4 Obs Agent 的分析

Obs Agent(動態障礙物)的目的是模擬真實世界的動態障礙物，讓 Spot Agent 學習如何在面對不可控的障礙物時的避障策略。為達此目的，Obs Agent 需要具備多樣化的移動速度、大小和移動策略，且不常選擇會撞牆的行為。為此，設計 Obs Agent 時需考慮以下幾個原則。

Obs 移動期望值

$$V_{Move} = w_1 \times v > 0 \quad (3.40)$$

如(3.40)式所示，為鼓勵 Obs Agent 多元化移動，首先需要確保 Obs Agent 能夠持續移動。因此，採用了與 Goal Agent 類似的移動期望值計算方式，但不同之處在於，Obs Agent 的移動期望值是核心能力，無需考慮其他獎勵的影響，因此移動期望值僅依據移動速度來設定。

為了增加 Obs Agent 行為選擇的多樣性，除了通過 Loss 讓各種 Action 的 Reward 差距不大之外，還可以通過 Reward 鼓勵 Obs Agent 選擇加速度變化較大的行為，如(3.41)式所示。

$$V_{Aacc} = w_2 \times |A_{acc}| \quad (3.41)$$

此 Reward 會鼓勵 Obs Agent 增加加速度變化，藉由給予加速度變化較大的行為更高的價值，引導 Obs Agent 進行速度上的調整，使其有更多樣化的速度。

Obs 撞牆期望值

如(3.42)式所示，為 Obs Agent 碰撞到 RL 環境的期望值，要模擬真實世界的

動態障礙物，Obs Agent 應盡量避免碰撞牆壁。

$$V_{Obs-Map} < 0 \quad (3.42)$$

Obs Agent 的最核心 Reward 是移動期望值，其他獎勵則相對次要。為了讓 Obs 優先考慮移動期望值，不應過度強調撞牆的影響。

為了平衡移動期望值與撞牆期望值，本論文採用簡單的設計方式：當 Obs 撞到障礙物時，將其移動期望值設為 0，這樣可以讓 Obs 學會盡量避免碰撞牆體，同時確保移動 Reward 始終大於 0。

3.4.3 Multi Agent RL 的分析

Agent 在訓練時的相互影響是 Multi-Agent RL 的訓練的重點。由於討論的是 Agent 之間的影响，本小節會注重於在同一訓練環境中 Agent 和 Agent 所產生的事件，對於 Agent 訓練過程中的改變，和相對應的 Reward 應該如何設計。

3.4.3.1 同一種 Agent 的關係

當相同類型的 Agent A 和 Agent B 發生交互時，Agent 之間可能產生對抗、無關或合作三種關係。由於每一種 Reward 都會影響 Agent A 和 B 的互動，因此 Agent A 和 B 之間可能會表現出多種交互關係。

舉例來說，若 Spot Agent 可以接觸到其他 Spot，但 Spot 會想避免碰撞，由於兩個 Agent 目標一致，會產生合作關係；若 Spot 碰撞到牆體對於其他 Spot 沒有影響，則產生無關關係；而若多個 Spot Agent 爭奪唯一的目標（Goal），則會因為資源有限產生對抗關係。

了解這些事件（E）的影響對 Multi-Agent RL 訓練至關重要，本論文對三種關係進行了實驗。如圖 3.6 所示，將所有動態障礙物（Obs）用 Spot Agent 替代，雖然這樣可增加訓練數據，但當所有 Agent 都是 Spot 時，為了減少碰撞機率，所有 Spot 在遇到障礙物時都會統一右轉，以減少碰撞發生的可能性。

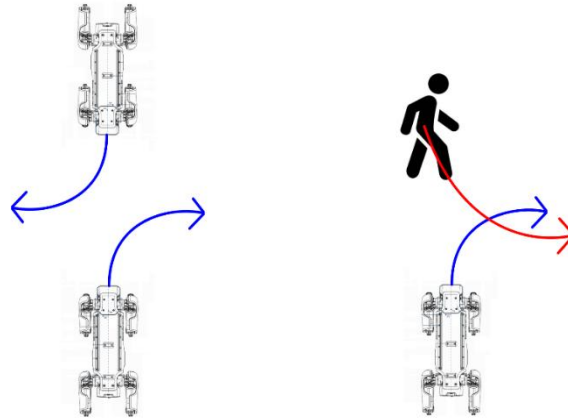


圖 3.6 Spot 只會右轉就能減少碰撞

然而，這樣的設置會使 Spot 不需有效辨識障礙物，且不需學習如何應對隨機移動障礙物，因此在真實世界中無法培養出有效的避障能力。實驗顯示，依賴同類型 Agent 之間的合作無法提供全面的訓練，如果現實中的交互對象是未知的，在訓練時就不應對同類 Agent 採用合作關係。

在同類型 Agent 的對抗實驗中，如圖 3.7 所示，將多個 Spot 置於相同的環境中，使其競爭同一目標，並設定 Spot 之間不會相互碰撞。此設置下 Spot 無法看到或接觸彼此，只能共同追逐目標，只有最快抵達目標的 Spot Agent 能得分。此方法有助於快速篩選出性能最佳的 Spot。論文[24]將同一訓練環境中同 Agent 的競爭現象，稱為「虛構自我對抗 (Prioritized Fictitious Self-Play, PFSP)」策略。

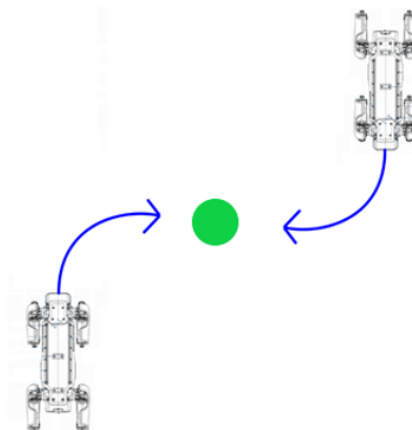


圖 3.7 Spot 對抗實驗

無關關係的測試，由於 Agent 間無相互影響，這種情況下可將每個 Agent 當作單一個體來進行訓練。

如表 3.1 所示，對於同類型 Agent 的對抗關係，能快速比較不同行為的影響，其訓練資料的效果高於無關關係。實驗顯示，同類型的 Agent 相互合作應該鄭重考慮，否則會產生意料之外的行為。此結果強調理解 Agent 互動及其對訓練過程影響的重要性，從而在不同情境下設計有效的訓練策略。

表 3.1 同一種 Agent 能否合作表

	對抗關係	無關關係	合作關係
效果	良好	可以	不行

3.4.3.2 不同 Agent 的關係

在 Multi Agent 場景中，當 Agent A 和 Agent B 交互時，若 A 與 B 涉及不同的 Agent 時，其互動關係關係也是三類：對抗關係、合作關係和無關關係。

本論文對於關係做了詳細的定義，對抗關係指的是其中一個 Agent 希望事件 E 發生的機率提高，而另一方則希望降低；合作關係則是兩方 Agent 都希望事件 E 發生的機率增加或降低；無關關係意味著其中一方 Agent 對事件的發生機率漠不關心。

本論文將 Spot Agent 和 Goal Agent 設置成多種互動關係進行實驗。對抗、合作和無關關係中，Spot Agent 的目標是避障並到達目標位置，而 Goal Agent 則根據不同關係策略做出相應行動，影響 Spot Agent 的行為表現。

如表 3.2 所示，實驗測試是 Goal Agent 在一樣的條件下執行出來的結果，為了公平起見，測試時的 Goal Agent 固定是訓練無關和對抗策略的模型，Spot Agent 是在不同關係下訓練出來的結果，在 60 個測試環境中的平均數值。

表 3.2 不同互動關係策略對 Spot 訓練效果的影響

	對抗關係	無關關係	合作關係
$\rho(\theta_{phase3})_{Spot-Goal}$	0.88	1.38	0.92
$p(\theta_{phase3})_{Spot-Obs}$	0.91	0.84	0.82

$\rho(\theta_{phase3})_{Spot-Goal}$ 類似於 $p(\theta_{phase3})_{Spot-Goal}$ ， ρ 表示在 Spot 訓練到 phase3 時，測試環境中 Spot 與 Goal Agent 相遇的平均次數，測試環境中只有一個 Goal Agent，當 Spot 遇到 Goal Agent 時會在環境中的任一地點隨機出現新的 Goal Agent，因此有大於 1 的可能性， $p(\theta_{phase3})_{Spot-Obs}$ 表示 Spot Agent 的碰撞機率，由於 Spot 碰撞後就結束了，因此最大為 1。

如表 3.2 所示，數據顯示無關關係是最優的選擇，而對抗關係則是最差的，此結論與同一種 Agent 的關係完全不一致，因此本論文將不同關係策略下 Spot 的行為進行解釋。在理想狀況下，Spot 在合作關係中可順利前往目標地點，Goal Agent 會主動靠近 Spot，增加 $p_{Spot-Goal}$ 的發生機率，如圖 3.8 所示。

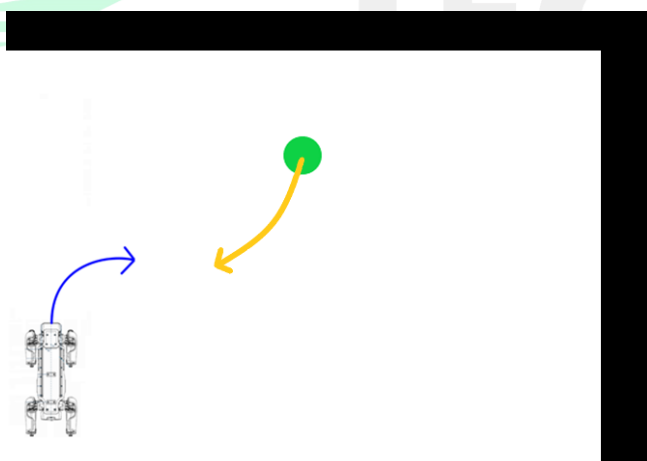


圖 3.8 期望中 Spot 在合作關係的策略

然後，實際情況中 Goal Agent 受自身 Reward 的影響，若抵達 Spot 位置也是 Goal 的核心 Reward，那 Goal 就會學習到使 $p_{Spot-Goal}$ 最大化的行為，Goal 跑到最容易發生 $E = Spot - Goal$ 事件的地點，如圖 3.9 所示。但此時 Spot 所學到的策略不再是朝向目標，而是學會朝向最容易觸發 $E = Spot - Goal$ 事件的位置的策略。

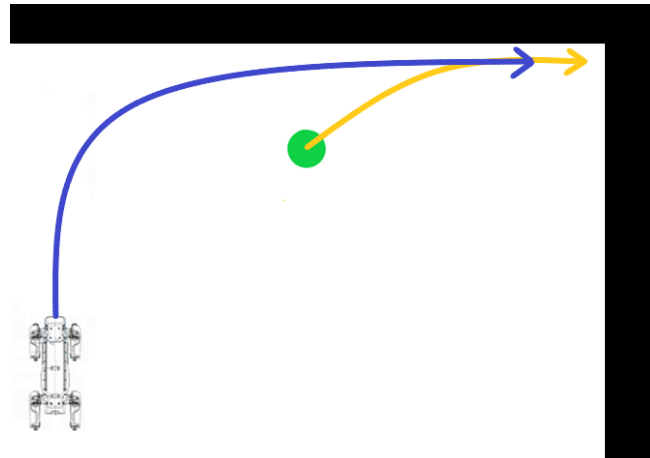


圖 3.9 實際 Spot 在合作關係的策略

在對抗關係中，Goal Agent 一旦接觸 Spot 後就不再獲得 Reward，因此 Goal Agent 將試圖遠離 Spot，減少 $p_{Spot-Goal}$ 的發生機率，如圖 3.10 所示。由於 $p_{Spot-Goal}$ 是 Spot 訓練的核心的 Reward，對抗關係降低了 Spot 成功抵達目標的機會，導致最差的訓練效果。

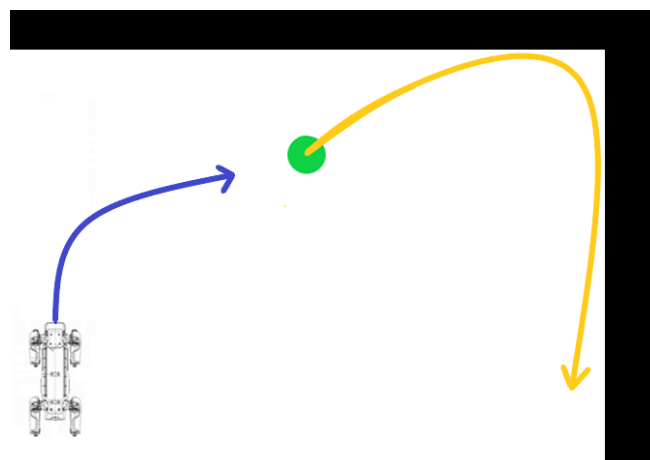


圖 3.10 實際 Spot 在對抗關係的策略

在無關關係下，Goal Agent 不會主動回應 Spot，僅專注於自己的目標，此策略效果與期望的合作策略較為接近。最終結果顯示，無關關係讓 Spot 在訓練中的表現最佳，如圖 3.11 所示。

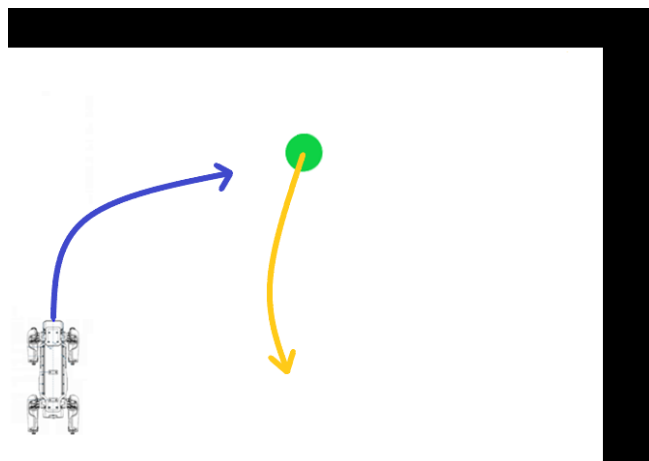


圖 3.11 實際 Spot 在無關關係的策略

論文[25]提到的使用 Agent B 攻擊 Agent A 的訓練方式，與本論文中不同 Agent 的對抗策略類似。該論文指出，使用 Agent B 攻擊 Agent A 是較為容易的，只需少量訓練量即可對 Agent A 產生有效的攻擊效果。不過，使用 Agent B 攻擊 Agent A 也具有優勢，可以改善 Agent A 的 Action 分佈不均的弱點。這表明，對抗關係雖然無法提升訓練效率，但能增加 Agent 抵禦攻擊的難度。

由此可以推導出以下結論，如表 3.3 所示。Multi Agent 中的不同 Agent 之間可以採用對抗關係，但這並非效果最好的方法，對抗關係能使模型在面對攻擊時更為穩定，但由於對抗關係會導致訓練效率降低，建議在訓練的最後階段再採用對抗關係。

表 3.3 不同 Agent 能否合作表

對抗關係	無關關係	合作關係	對抗關係
效果	可以	良好	可以

3.4.3.3 本論文 Multi Agent RL 的結論

Multi Agent 進行訓練時，最佳策略是與自身進行競爭(PFSP)，不是進行合作，因為合作可能會導致彼此掩蓋各自的弱點。由於每一個 Reward 都會產生其中的一種關係，因此主要與次要 Reward 的關係統一，對於高效訓練是有必要的。

當 Reward 會導致 Agent 之間的關係對抗時，建議在訓練初期採用無關關係進行訓練，以提高訓練效率，後面階段再進行對抗的訓練。

AI 中著名的 AlphaGo Zero[26]，或許早就發現自我對抗是提升效果的最佳方式，不使用兩 Agent 代表黑白棋，而是用同一 Agent 不同獲勝條件來代表黑白棋，這比較好解釋為何 AlphaGo Zero 可以自由的調整貼目的數量，和如此優秀的訓練效果。

3.4.3.4 實際設計

了解 Agent 之間關係對於 Reward 的設計有何影響之後，便能決定在 Single Agent 中每一個期望值(V)應該如何設計，本段落會介紹具體應該如何使用分析出來的 Agent 之間關係知識設計相關的 Reward。

Spot 事件分析

- $E = Spot - Spot$

考慮事件 $E = Spot - Spot$ 會讓 Spot 為了減少碰撞與其他 Spot Agent 進行合作，導致真實情景和訓練產生差異，為了避免讓這類事件存在。故此，我們將 $V_{Spot-Spot} = 0$ ，以排除 Spot 之間合作的可能性。

- $E = Spot - Goal$

事件 $E = Spot - Goal$ 是 Spot 的核心獎勵條件，因此其存在是必要的。

- $E = Spot - Obs$ and $E = Spot - Map$

本論文中 Map 代表 RL 訓練環境中的不同種類障礙物，因此這兩項事件均

與 Spot 碰撞相關，因此它們的存在對於學習是必須的。但需要注意，確保存在期望值保持在正數。

Goal 事件分析

- $E = Goal - Spot$

依據不同 Agent 的關係分析，要讓 Goal 與 Spot 選擇「無關」關係，以便於 Spot 達到更佳的效果。訓練後證明只有在無關關係下 Spot 才能真正學會攀爬樓梯，若採用「對抗」關係當 Goal 會攀爬樓梯時，Spot 也會逐漸學習如何攀爬。這會讓 Goal 在試圖攀爬時更易被 Spot 捕捉，從而導致 Goal 不願意攀爬樓梯，最終使兩 Agent 攀爬的效果都很差。

- $E = Goal - Goal$

相同 Agent 間的合作不應存在，不過本論文有測試合作關係的 Goal Agent。其中設定 Goal Agent 之間不會發生碰撞，也不會產生 Reward，但是能看到其他 Goal Agent 的位置資訊。測試發現，即使事件不會直接導致「碰撞」也不會產生任何獎勵，但會使 Goal Agent 群聚，增加攀爬樓梯的機會，減少 Spot 學習的機會。因此證明，即使是在只有狀態資訊中能看到其他 Agent 的情況下，相同 Agent 間的合作也會存在，所以應該斷絕一切同類型 Agent 合作的機會。

- $E = Goal - Obs$

對於 Spot Agent 來說，Goal 與 Obs 之間的關係完全不重要，現實情況下目標物也不會考慮障礙物，因此事件 $E = Goal - Obs$ 不需要存在。

- $E = Goal - Map$

Goal 和 Map 的關係與 Spot Agent 沒有直接的關聯，所以應單獨對中的「碰撞」和「獎勵」進行個別分析。

對於碰撞的考量，若 Goal 能夠穿越牆體，Spot 將學習到目標位置於不合理位置時如何應對。但這樣會導致 Spot 無法學習爬樓梯，減弱 Goal 引導 Spot 的效果，因此應考慮 $E = Goal - Map$ 中的「碰撞」。

Reward 方面的考量，若考慮 Goal 撞到 Map 時的 Reward，會使 Goal 儘量避免接觸牆壁，減少 Spot 學習識別牆壁的機會，不考慮這個 Reward 則可能使 Goal 靠近牆壁。為了兼顧二者，本論文的策略是使撞到牆壁的 Goal 被彈開到其他方向，而不設定碰撞的 Reward。

Obs 事件分析

- $E = Obs - Spot$

鑑於已經設定了 Spot 碰撞到 Obs Agent 的 Reward ($R_{Spot-Obs}$)，因此對於 $E = Obs - Spot$ 採取「無關」關係是最合適的選擇，因此設置 Obs Agent 不能看到 Spot 的資訊，也不會從事件 $E = Obs - Spot$ 獲得任何 Reward。

- $E = Obs - Goal$

這個事件對 Obs 和 Goal Agent 都不重要。然而 Goal 的資源與 Spot 息息相關，不應該減少 Spot 學習抵達目的地的機會，因此不設置 $E = Obs - Goal$ 事件。

- $E = Obs - Obs$

該事件將導致障礙物之間產生合作關係，在本論文的框架中這是不需要的，故不設置此事件。

- $E = Obs - Map$

對於與地圖 (Map) 相關的事件，我們與 Goal Agent 有相同的考量：需要考慮「碰撞」，但不需要設置與之相關的獎勵。動態障礙物主要目的是為了模擬障礙物對於 Spot 行為路徑的限制，而非訓練出真實的動態障礙物策略。

3.5 Multi RL 的分析

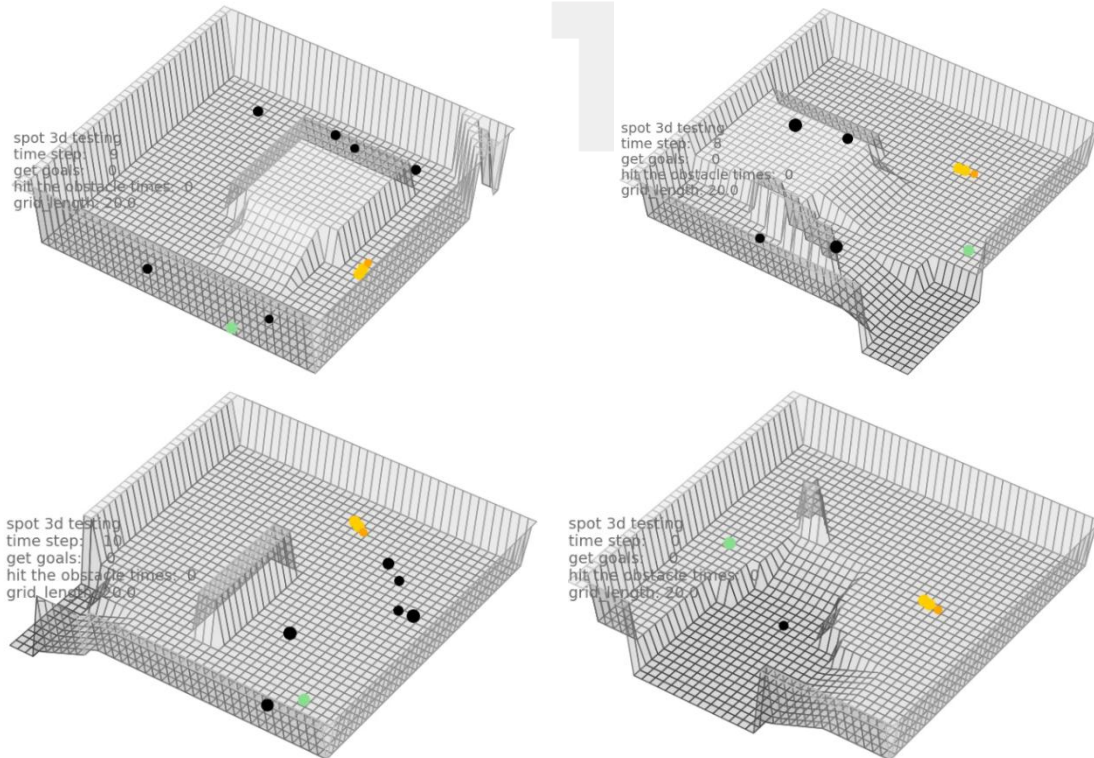
在 RL 的訓練過程中，Agent 時常會出現意想不到的行為策略，因此，除了觀察數據變化，還需要使用可視化工具觀察 Agent 在環境中的策略選擇。根據這些策略行為，開發者可以判斷策略形成的原因並評估其優劣，從而有效改進 RL

策略。

3.5.1 RL 測試環境

本論文詳細的測試環境會在 RL 開發過程將在實驗測試章節中介紹，為了敘述論文的方便性，以下概述了評估策略效果的一種方法。本論文使用統一的測試環境來評估不同訓練策略的效果，該環境包含 30 個測試場景。對每個場景數據取平均以得出 $\rho(\theta_{phase3})_{Spot-Goal}$ ，代表 Spot Agent 在訓練到 Phase3 階段時平均抵達目的地的次數，數值越大越好。數據 $p(\theta_{phase3})_{Spot-Obs}$ 代表 Spot 平均碰撞機率，因此最大為 1，且值越小越好。為了確保測試公平性，測試環境中固定了 Goal 和 Obs Agent 的權重，而 Spot Agent 採用不同的訓練策略或資訊處理，這樣能夠更客觀地評估不同訓練條件的影響。

為了模擬真實情境，測試環境的設置如圖 3.12 所示。黃色點代表 Spot Agent，橘色點代表 Spot Agent 的頭部，綠色點代表 Goal Agent，黑色點代表 Obs Agent。每個 Agent 都會根據自身的策略進行移動，以測試訓練時的行為。



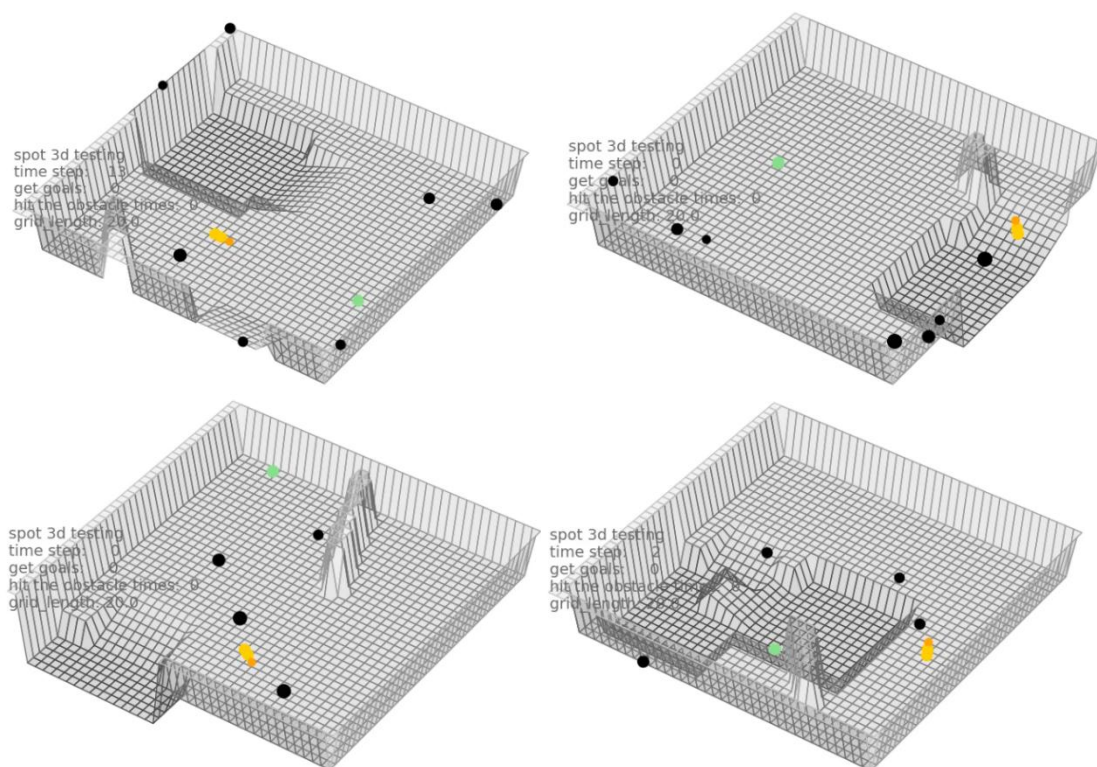


圖 3.12 其中八個 RL 測試環境

每個測試場景為 20 平方公尺的區域，內含一個 Spot 和一個 Goal。此設計旨在模擬 Boston Dynamics Spot 的實際測試場景，並通過觀察 Spot 的行為來量化其在不同訓練階段的表現，參數如表 3.4 所示。

表 3.4 測試環境的參數

	樓梯數量	牆體數量	執行時間	Spot 數量	Goal 數量
參數值	最多 2 個	1 個	60 秒	1 個	1 個

	Obs 數量	Goal 半徑大小	Obs 半徑大小	Goal 速度	Obs 速度
參數值	最多 7 個	0.28 公尺	0.23-0.33 公尺	0-0.7 m/s	0-0.9 m/s

測試環境參數依據以上設定並使用 Seed 生成。對於不同訓練策略的測試，本論文將不相關的參數固定，以確保比較的公平性。在評估訓練策略時，本論文只將 Agent 訓練到 Phase 3，主要關注抵達目的地及避障能力。

3.5.2 Multi-Environment RL

為了解決單一訓練環境中訓練速度較慢的問題，多環境強化學習（Multi-Environment Reinforcement Learning, Multi-Env RL）成為重要研究方向。

什麼是 Multi-Env RL？

Multi-Env RL 是指在多個獨立環境中同時訓練一個或多個 Agent 的強化學習方法。這些環境可以相同或有差異，其目的是模擬真實世界情境的多樣性，以提升策略的泛化能力。如論文[27]一文所述，Multi-Env RL 允許 Agent 同時接收多個環境的獨立反饋，在高並行性框架下展現學習優勢。

Multi-Env RL 有何好處

Multi-Env RL 能顯著提升 Agent 的學習效率和策略的泛化能力。由於可以並行處理多個環境的數據，模型得以更快速地積累經驗。如論文[28]提到，Multi-Env RL 不僅加快了學習速度，還使 Agent 在多樣化問題中保持穩健性，減少單一環境中的噪聲或過擬合風險。

在論文[29]一文中，作者強調大量訓練數據可快速提升效果，並提出通過最大化 Multi-Env RL 和 Multi-Agent RL 的數量來達成目標。他們建議將 RL 訓練完全轉移至 GPU 平台，以替代傳統的 CPU+GPU 模式。此框架允許用戶根據 GPU 記憶體容量，靈活選擇適當的 Agent 數量和 RL 環境數量，以提升訓練數據的使用效率。

本論文基於 WarpDrive 工具，同時運用 Multi-Agent RL 與 Multi-Env RL 進行訓練。由於 GPU 記憶體有限，為找出增加 Multi-Agent 數量或 Multi-Environment 數量的最佳方法，本論文通過調整 Spot Agent 數量和 Multi-Env 數量進行實驗，保持其他參數不變，並最大化 GPU 使用率，以判斷將 GPU 資源優先分配至 Multi-Agent 還是 Multi-Environment 能達到最佳效果。接著，通過不同的 Multi-Agent 與 Multi-Environment 組合，並保持其他參數固定，來確定最佳比例組合，如表 3.5 所示。

表 3.5 Multi Agent 和 Multi Environment 的比較

訓練策略	一個環境 Spot 數量	同時訓練 環境數量	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
一個 Spot	1	2777	0.43	1
環境較多	3	1088	0.83	0.93
Agent 數量較多	5	777	0.6	0.95

實驗結果顯示，相較於增加 Multi-Agent 數量，增加 Multi-Environment 數量能更有效地提升訓練效果。因此，在 Multi-Agent 和 Multi-Environment 的配置上，本論文建議將 GPU 資源優先用於增加 RL 環境數量，以達到最佳訓練效果。Multi-Agent 中的 Agent 數量主要用於實現虛擬自我對抗，並不適合作為提升訓練效率的手段。

3.5.3 Multi-Task RL

為了提升模型的泛化能力，本論文採用論文[30] Multi-Task RL 的方法，隨機生成不同任務來訓練機器狗。然而，隨機性的具體實現方式對訓練結果有不同的影響，接下來將探討不同隨機方法及其具體影響。隨機生成 RL 環境具體可分為兩個方向：其一為隨機化 RL 環境，另一為隨機化 Agent 的起始狀態。

隨機生成 RL 環境

為提升機器狗在不同環境中的行為策略泛化能力，本論文每訓練模型 100 次便隨機生成新地圖，並觀察更換地圖對泛化性能的影響，如表 3.6 至表 3.7 所示。

表 3.6 隨機與固定地圖的效果對比（測試 1）

訓練策略	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
隨機地圖	0.83	0.93
固定地圖	0.67	0.83

表 3.7 隨機與固定地圖的效果對比（測試 2）

訓練策略	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
隨機地圖	3.17	0.7
固定地圖	3.53	0.57

實驗結果顯示，在固定地圖中訓練的 Spot Agent 在隨機地圖中的表現優於直接在隨機地圖中訓練的 Agent，這一結果與訓練目標有關：在前三階段中，訓練重點是抵達目的地，而避障能力的訓練則安排在後期。頻繁更換 RL 環境反而減少了 Agent 對抵達目標的學習機會，未能有效增強此能力的泛化性。如圖 3.13 所示，在 Phase 1 中使用隨機地圖的策略增加了碰撞率，對抵達目標訓練產生負面影響。

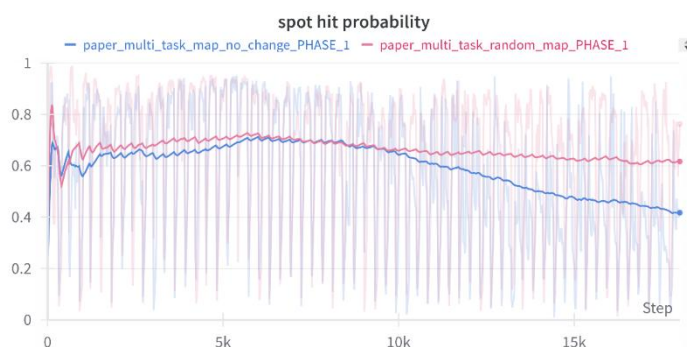


圖 3.13 隨機和固定地圖策略的碰撞率

實驗結果顯示，若追求 RL 的泛化性，訓練任務需要根據具體目標進行設計。

對於上下樓梯和避障能力的訓練，隨機切換 RL 環境能夠有效增加泛化性，但在抵達目標的能力訓練中反而有反效果。因此，RL 環境的更新頻率應視任務類型而定。

此外，RL 環境與實際情境的差異也是重要考量。RL 訓練中常認為在多數樓梯、牆體和障礙物的環境下訓練較具挑戰性，因此這些任務通常安排在後期。然而，若後期訓練環境中持續存在大量樓梯、牆體和障礙物，Agent 在實際應用中可能無法應對空曠場景，儘管這些場景在早期訓練階段已經涵蓋。

因此，為提高 Agent 模型的泛化能力，對於隨機生成的 RL 環境，應同時考慮環境難度的隨機性。本論文在全部的訓練階段，僅設定地圖中樓梯、牆體和障礙物的平均數量，而不指定每個任務的具體數量，讓同一階段的訓練能有不同的任務難度，以培養能夠適應多種情境的 Agent。

隨機 Agent 起始狀態

對於 Agent 的起始狀態而言，由於更改 Agent 的起始位置的運算成本很低，因此不需要受 RL 環境更新頻率的影響，可以更加自由地設定初始狀態。為此，本論文也針對 Agent 的起始狀態進行實驗，將 Agent 起始位置以不同頻率進行更新，以測試適合的隨機頻率。

表 3.8 隨機 Agent 起始位置測試

訓練策略	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
同一 Phase Agent 位置固定	0.47	0.9
訓練 100 次更新 Agent 位置	0.83	0.93
每個 RL 環境 Agent 位置不一致	1.23	0.8

如表 3.8 所示，隨著 Agent 的隨機頻率增加，訓練效果也隨之提升。隨機位置更新頻率可以高於地圖更新頻率，因為固定位置會導致 Spot 過度依賴特定動作來達成目標，從而降低模型的泛化能力。

需要注意的是，為了讀取數值和訓練的方便性，需在訓練的 100 次過程中固定事件的機率。因此，即使每個 RL 環境中的 Agent 位置隨機分配，相同編號的環境內的 Agent 位置依然需要保持固定。

設定 RL 環境中的 Agent 位置不一致，不僅提升訓練效果，還能增強訓練的穩定性。尤其在上下樓梯技能訓練時，需要減少 RL 環境中 Agent 的數量，這表示每一個 Agent 收集的數據重要性都會增加。當 Agent 數量減少，若 RL 環境中的 Agent 位置一致，可能導致某些任務中的 Agent 獲得大量無效的訓練數據，如圖 3.14 所示，Spot 在環境生成時可能會只接碰到障礙物。

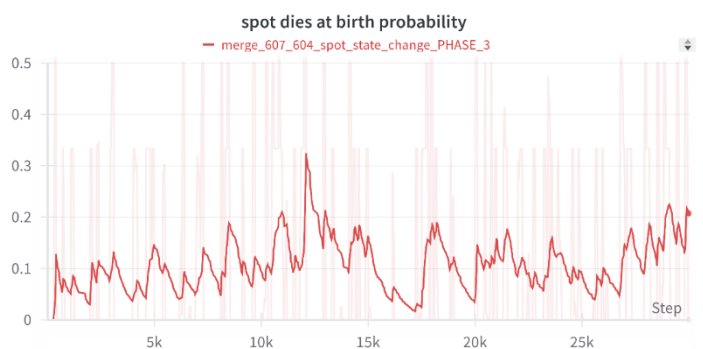


圖 3.14 Spot 一開始的碰撞機率

如果 Spot 在環境生成時直接位於危險位置，由於每個 RL 環境中的 Agent 都遭遇相似情況，Agent 可能僅學習片面行為，最終導致模型損毀，使得 Spot 在後續訓練中無法順利抵達目標，如圖 3.15 所示。



圖 3.15 Spot Model 損毀

當 RL 環境中 Agent 初始位置為隨機配置，Spot 的初始碰撞機率更趨於均勻，這有助於在 Agent 數量較少時獲得更好的梯度訓練數據，並在較嚴苛的訓練環境下保持訓練的穩定性，如圖 3.16 所示。



圖 3.16 Spot 一開始的碰撞機率

3.6 融合能力的方式

在訓練 RL 的過程中，我們經常需要讓 Agent 同時具備多種能力。以本論文中 Spot 的行為規劃策略為例，需要同時擁有抵達目的地、保持朝向目的地的方向、避開靜態或動態障礙物、並具備上下樓梯的能力。因此，本論文會介紹不同的融合訓練策略、效果、實驗結論，以及最終選用的融合方法。首先會從具備抵達目的地和良好姿態控制能力的 Spot 為例，來展示如何有效地融合多種能力。

3.6.1 修改 Action 的 Reward

傳統演算法如[31]MPC 中通常使用代價函數 (Cost function)，以不同行為的權重來計算最佳行為策略。在 RL 訓練中，本論文首先通過調整不同動作的 Reward 值來引導 Spot Agent 的行為，具體的獎勵函數如(3.43)式所示。

$$V_{move} = w_1 \times (1 - A_{\omega})^2 + w_2 \times (1 - |v_y|) + w_3 \times (v_x + b)^2 \quad (3.43)$$

這項 Reward 的設計是讓 Spot 根據目的地距離選擇行動，當目標位置在 Spot

的左右兩側，可以使用轉向角 A_{ϖ} 或水平移動的速度 v_y 抵達目的地，在目標距離較近時，自動的使用速度 v_y ，讓轉向角平方倍的 Reward 最大化。而距離目的地較遠時則使用轉向角(A_{ϖ})對準目標，讓 Agent 在行駛至目的地的期間的每次都改變一點轉向角，同時讓 A_{ϖ} 和 v_y 相加的 Reward 最大化，達到在遠距離時 $V_{\varpi} > V_{v_y}$ 的策略。

此外，參數 b 為微小正整數，用以確保機器狗向前方移動的獎勵值高於反向移動，以實現 $V_{v_x} > V_{v_{-x}}$ 的目標。這樣設定的最終目標是優先使 Spot 向前方移動以達成最優策略。

然而，實踐過程中在設定 $V_{Survive} > V_{move}$ 的時候會遇到了挑戰。當獎勵值 V_{move} 設置過低時，對 Spot 的對於姿態控制的效果不明顯；反之，當獎勵值過高時，Spot 可能更傾向於選擇獎勵高的動作，而忽視實際達成目標的需要。只有在獎勵值適當調整時，才能既促使 Spot 成功抵達目的地，又能增加符合姿態控制的目的。

測試環境中的實驗結果如圖 3.17 所示，圖中 y 軸為機器狗左右移動的速度。Spot 的速度分布顯示橢圓區域缺少了左轉的運動控制能力，表明在 RL Reward 設計中，若採用傳統算法的 Cost function，並無法得理想效果。

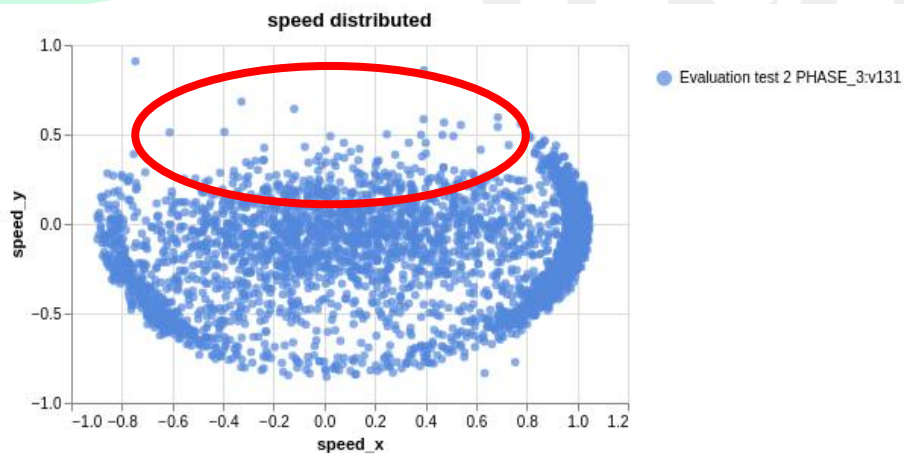


圖 3.17 修改不同 Action 的 Reward，Spot 的速度分布圖

對於左轉速度的缺失，本論文推測原因如圖 3.18 所示，此圖展示了 Spot 抵

達目的地時的移動軌跡。由於訓練開始時，抵達目的地任務難度過高，Spot 只學會了直行和右轉行為來抵達目的地，沒學會左轉抵達目的地的能力。此時右轉的 Action 獎勵值會高於左轉的 Action，導致 Spot 更不願嘗試左轉動作，最終沒有學會左轉抵達目的地的能力。

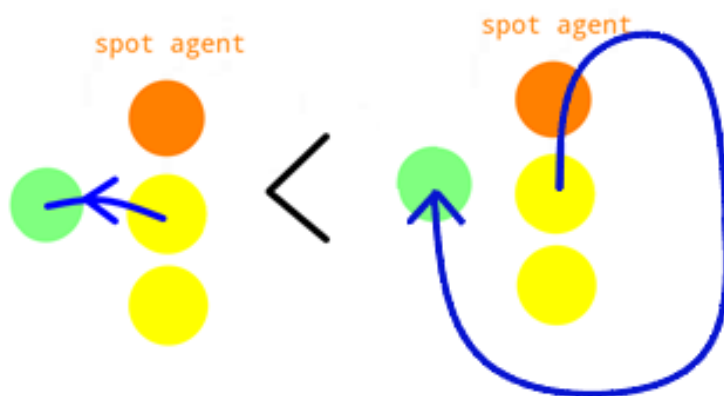


圖 3.18 Spot 的姿態控制策略

此發現表明，僅僅通過調整不同動作的獎勵值並非最佳解決方案，因為這可能導致 Spot Agent 過度依賴特定動作，忽略達成目標的多樣性和靈活性。

3.6.2 障礙物的改變姿態控制

透過障礙物來引導 Agent 的姿態控制，是為了幫助 Agent 更有效地達到 $V_{v_x} > V_{v_{-x}}$ 的目標。本論文在 Spot 與障礙物相遇時，通過調整不同碰撞位置的獎勵值來改變其行動偏好。這種設計利用了 Spot 與障礙物碰撞的概率和角度差異，從而促使 Spot 選擇符合預期的姿態控制策略，如圖 3.19 所示。

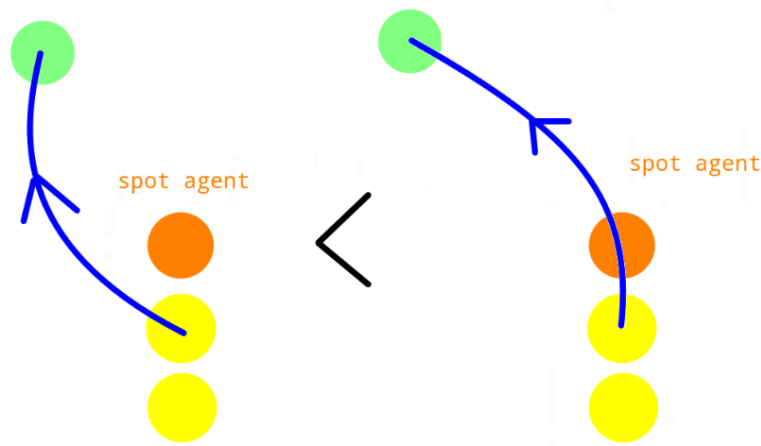


圖 3.19 改變 Agent 的行為喜好

具體來說，根據動作所帶來的碰撞位置不同，設置了不同的獎勵值，如圖 3.20 所示。當 Spot 向前移動並用「head」發生碰撞時，扣分較低，而倒退用「back」發生碰撞的扣分則更高。這一設計鼓勵 Spot 保持向目的地前進，並改變其對行動的偏好，以促使其沿著預期的方向行進。

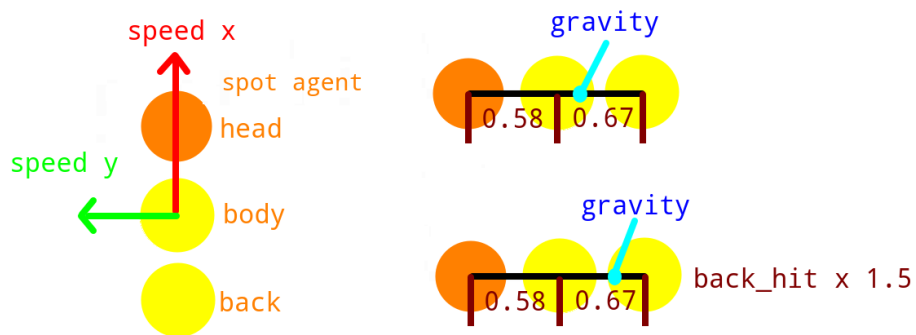


圖 3.20 計算 Spot 碰撞的 Reward

這種對不同碰撞部位的獎勵值進行差異化設計，有效地調整了 Spot 對不同動作的期望值，幫助其在面對障礙物時，做出更有效的行動選擇，並朝向目的地的方向移動，以實現(3.44)式的目標。

$$V_{v_x} > V_{v_{-x}} > V_{v_y} \quad (3.44)$$

然而，這種方法在實際應用中也遇到了挑戰，雖然在訓練中此方法能夠解決八成的姿態控制，但在運氣不好的情況下，仍會出現倒退移動的行為。這一發現提示我們，僅僅通過改變碰撞期望值來引導運動控制，仍不足以完全的實現姿態控制的目的。

3.6.3 修改 Spot 的能力上限

在第三種姿態控制策略中，本論文通過 Spot 的速度上限來引導其選擇前進、後退、左右移動和轉向的行為偏好。這種方法主要通過設置不同方向的速度上限，以引導 Spot 選擇符合預期的運動控制方式，達成(3.45)式的目的。

$$V_{Av_x} > V_{Av_{-x}} > V_{Av_y} \quad (3.45)$$

本論文中的 Goal Agent 是會移動的，要追上會移動的 Goal Agent，對於 Spot 的移動速度來說很重要，所以 Agent 在不同方向設定不同的速度上限，就能達到改變 Spot 喜好行為的目的。

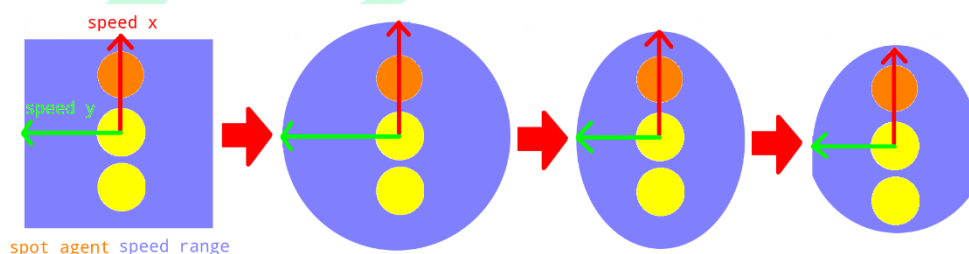


圖 3.21 改變 Spot 的能力上限

如圖 3.21 所示，最初的設計 Spot 的前後左右移動速度相等，導致 Spot 偏好斜向移動。為改善這一行為，將速度總和超過 1 的速度正規化到 1，使 Spot 會選擇任一方向移動。

在第二階段中，本論文進一步調整了轉向角度設計，將側移速度上限設為前進速度的 0.88 倍，使 Spot 更傾向於轉向而非側移，提高了運動效率，並符合

$V_w > V_{v_y}$ 和 $V_{v_x} > V_{v_y}$ 的要求。

最後，基於對 Spot 行為的深入觀察，本論文將速度上限形狀由橢圓調整為蛋形(向前速度為 1，左右移動速度為 0.88，向後移動速度為 0.9)，使向後移動的速度略低於前進，這確保了 Spot 在需要長距離移動時優先選擇前進動作，滿足 $V_{v_x} > V_{v_{-x}}$ 。

調整後的實驗結果如圖 3.22 所示。Spot 的速度分布顯示，使用運動上限控制 Spot 的移動姿態，不會強求 Spot 一定要使用哪個 Action，而是根據目前狀況自己選擇最合適的 Action，如此才能讓 Agent 最有靈活性的調整姿態控制策略。

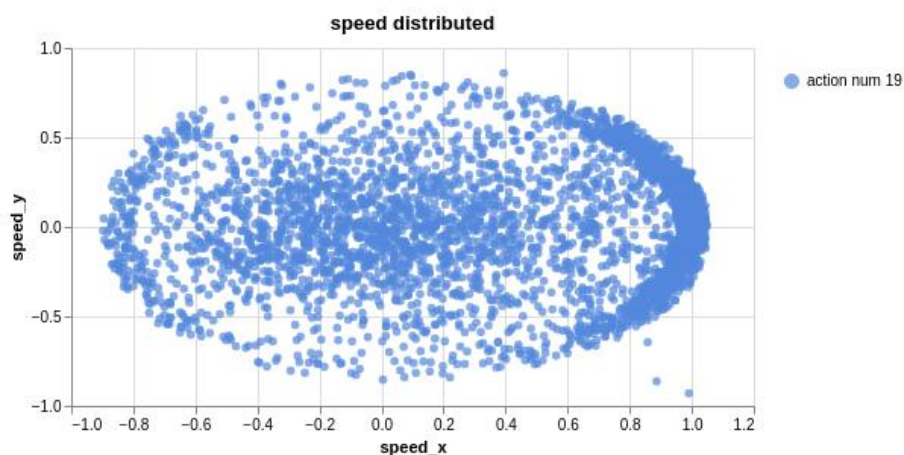


圖 3.22 修改 Agent 的能力上限，Spot 的速度分布圖

3.6.4 融合能力的結論

本論文所使用的三種不同的姿態控制設計，其本質在於結合姿態控制與抵達目的地的能力，並且對應了三種不同的 Reward 設計方法。第一種是將姿態控制能力轉化為一個獨立的 Reward，第二種是將姿態控制能力綁定在避障能力上，第三種是將姿態控制能力與抵達目的地的能力結合在一起。

第一種方法是將姿態控制能力轉化為一個獨立的 Reward，在訓練 Agent 的過程中，根據我們對各種能力的重要性程度，使用權重來決定策略，如(3.46)式所示。

$$V = w_1 \times V_{Survive} + w_2 \times V_{move} + w_3 \times V_{Stair}$$

$$V = w_1 \times V_{\psi_1} + w_2 \times V_{\psi_2} + w_3 \times V_{\psi_3} \quad (3.46)$$

如圖 3.23 所示，當遇到需要多種能力的情境，例如障礙物靠近時，抵達目的地的能力建議繼續前進，而避障能力則建議向後退。策略的選擇會如(3.47)式所示，Agent 會同時關注不同能力的最佳行為，針對不同能力給出一個策略，然而不同能力的策略是不一樣的，若採用權重的方式取最大的期望值最大的 Action 當成策略，就會因其中一個能力(核心 Reward)的期望值較大，選擇忽視其他能力，最終會繼續向前移動。

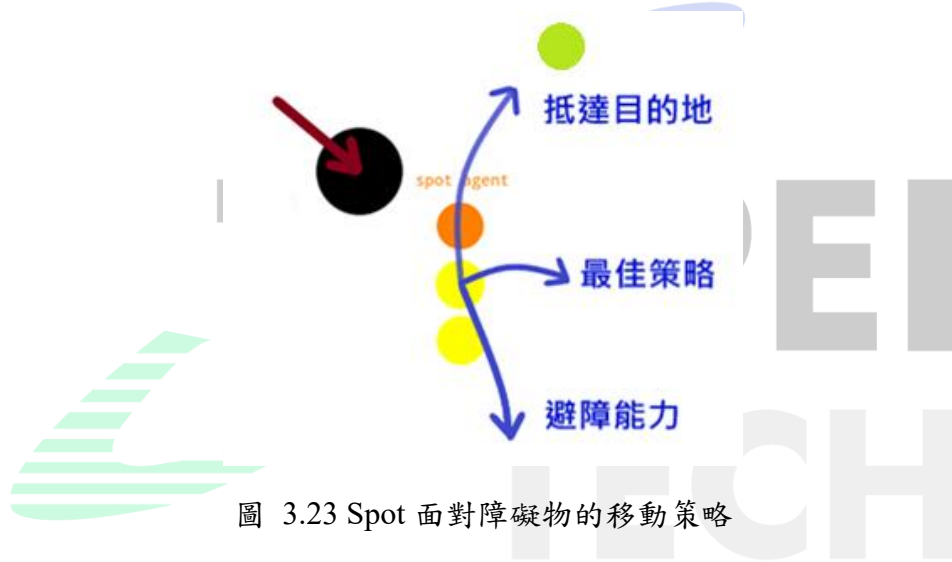


圖 3.23 Spot 面對障礙物的移動策略

$$\underset{a}{\operatorname{argmax}} V_{\psi_1}(a) \neq \underset{a}{\operatorname{argmax}} V_{\psi_2}(a) \neq \underset{a}{\operatorname{argmax}} V_{\psi_3}(a) \quad (3.47)$$

a ：代表在運動空間(A)，Agent 真實執行的 Action

然而，我們可以發現最佳策略並非前進也不是後退，而是向右上方移動。如(3.48)式所示，最佳策略應該要同時考慮避障和抵達目的需求，這證明最佳策略無法僅通過權重來融合不同的能力。

$$\underset{a}{\operatorname{argmax}} V(a) \neq \underset{a}{\operatorname{argmax}} (w_1 \times V_{\psi_1}(a) + w_2 \times V_{\psi_2}(a) + w_3 \times V_{\psi_3}(a)) \quad (3.48)$$

如圖 3.24 所示，不同能力的融合呈現三種關係圖。新增一個 Action Reward，採用權重融合兩個能力，兩能力關係圖為紅色，表示其中一個能力增強時另一個會減弱。使用障礙物改變姿態控制，關係圖為綠色，表示兩者關係不大。使用修改 Agent 不同方向的速度上限改變姿態控制，關係圖為藍色，表示兩者能力可以共同提升。

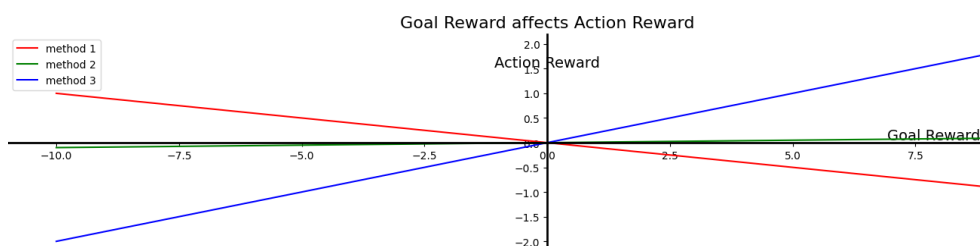


圖 3.24 抵達目的地能力對姿態控制的影響

修改 Agent 不同方向的速度上限改變姿態控制成功的關鍵在於，將姿態控制與抵達目的地的能力強制結合。這樣的設計使得 Spot 在增強核心能力（抵達目的地）的同時，亦提升了姿態控制的能力，從而使 Spot 在抵達目的地的同時也能夠有效地提升姿態控制的能力。

且強制結合的能力具有一個優點，如圖 3.25 所示。我們可以透過增強姿態控制的訓練，反過來提升抵達目的地的能力，只需注意不讓次要能力搶走核心能力的主導地位即可有效優化兩種能力。本論文中的次要能力期望值設定為主要能力的 0.1 倍。



圖 3.25 姿態控制能力對於抵達目的地能力的影響

方法二探討了將姿態控制與避障能力結合的策略。此設計中，姿態控制主要依賴避障能力的強弱，由於避障能力並非核心能力，導致姿態控制能力表現出不穩定的情況。

綜上所述，融合能力的關鍵在於初期確定 Agent 的核心 Reward，並充分利用 RL 環境中的所有條件，將全部所需能力與核心 Reward 巧妙結合，這樣才能實現 Agent 同時考慮多種目標並靈活的應用每一個能力。

3.6.5 本論文融合能力的方法

目前已介紹了抵達目的地和姿態控制能力，接下來將說明避障能力及上下樓梯能力的設計方法。

避障能力

在 Reward 設計中，本論文提出了增加 $R_{Spot-Goal}$ 以提升抵達目的地的能力，並降低 $R_{Spot-Obs}$ 以提高 Spot 存活的期望值。然而，僅透過降低避障的權重，雖能讓 Spot 具備良好的抵達目的地能力，但這並不足以賦予其合格的避障能力。

本論文指出，為了成功融合避障與抵達目的地的能力，需要建立兩者的聯繫。因此，本論文引入了**死亡**條件，將抵達目的地的目標與避障行為結合。具體方式是：每當 Spot 撞上障礙物，將無法再獲得後續所有 Reward。如此一來，不論後續抵達目的地的價值多高，Spot 都會優先考慮避開障礙，從而學得避障能力。

$$E = Spot - Obs \Rightarrow R(\tau) = R_{Spot-Obs} \quad (3.49)$$

上下樓梯能力

上下樓梯的能力是本論文設計中最具挑戰的部分。此能力也需要與抵達目的地的目標相結合，但上下樓梯並非 Spot 的必要能力，若目標位於樓梯或普通平面上，Spot 會優先選擇普通平面上的目標，因而無法學習經過樓梯以抵達目標的技能。

為了訓練出具備上下樓梯能力的 Spot，必須讓 Spot 覺得經過樓梯抵達目的地具有額外價值。本論文採用了 Multi Phase RL，先讓 Spot 掌握抵達目的地的基礎能力，再逐步減少 Goal Agent 的數量（最終僅留一個 Goal Agent），使 Spot 有學習通過樓梯達到目標的需求。此過程中還修改了 Goal Agent 的生成方式，將被 Spot 碰觸後消失的 Goal Agent 改為重新生成於地圖的其他位置。如此一來，Spot 成功通過樓梯後，除了獲得一個 $R_{Spot-Goal}$ ，還能在後續環境中擁有獲得更多 $R_{Spot-Goal}$ 的機會。

為確保 Spot 使用正確的姿態通過樓梯，依然參照避障能力的設計。當 Spot 使用錯誤的姿態通過樓梯時，亦會觸發死亡條件，以促使 Spot 在遇到樓梯時首先考慮上下樓梯的姿態是否正確。

3.7 Multi-Phase RL

Multi-Phase RL 是指將學習過程分為一系列具有特定目標和獎勵機制的階段，使學習進程能隨著任務複雜度的增加逐步適應。此方法有助於 Agent 在學習中聚焦於當前的學習目標，逐步克服每個階段的挑戰。正如論文[32]和[33]所述，這種分階段策略允許 Agent 專注於解決特定類型的問題或任務，從而在較短的時間內攻克更複雜的挑戰。

為觀察單一階段 RL 訓練的學習限制，本論文亦實作僅訓練個別能力（如抵達目標與避開障礙物）的情境，觀察其 reward 收斂曲線與訓練效率。如圖 3.26 所示，雖然模型在訓練初期 reward 呈穩定上升，但進入中後期後，reward 成長

速率明顯下降，甚至出現長時間震盪與停滯現象，顯示模型難以突破策略瓶頸。

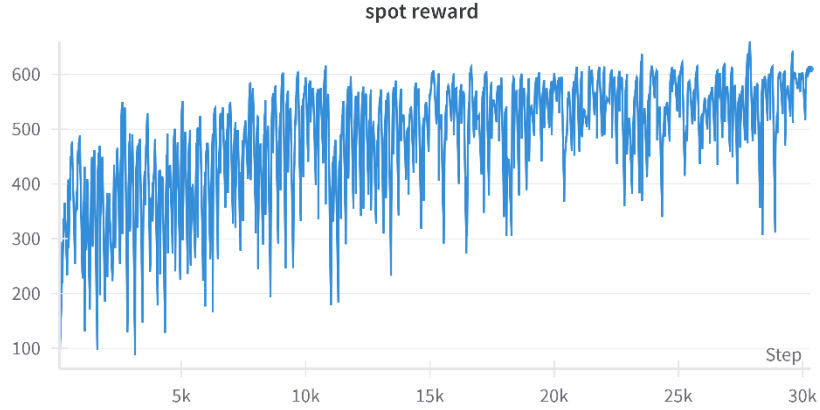


圖 3.26 單一階段 RL 訓練的學習

3.7.1 適合 Multi Phase RL 的任務

Multi Phase RL 需解決的核心問題是如何設計 RL 環境中的全部超參數 ϕ ，使得 Agent 在最後階段能達到最佳效能 V 。此效能的評估需要考慮 Agent 所應具備的多項能力 ψ ，如(3.50)式所示。

$$\max_{\phi \in \text{RL-Env}} V(\phi) = w_1 \times V_{\psi_1}(\phi) + w_2 \times V_{\psi_2}(\phi) + w_3 \times V_{\psi_3}(\phi) \quad (3.50)$$

ϕ ：代表 RL 環境中全部的超參數，包含環境大小，Agent 數量，Reward 的具體數值，環境設計方式等等。

因此，我們需要探索如何設計 RL 環境中參數 ϕ 的策略。本論文針對此問題進行了實驗，針對行為規劃中最重要「抵達目的地」和「避障」能力做測試。

已知一個有效的訓練過程會滿足(3.51)式，在訓練完一個階段後，能夠提升抵達目的地的能力，或是降低碰撞發生的機率。

$$\begin{aligned} p(\theta_{\text{phase2}})_{\text{Spot-Goal}} &> p(\theta_{\text{phase1}})_{\text{Spot-Goal}} \\ p(\theta_{\text{phase2}})_{\text{Spot-Obs}} &< p(\theta_{\text{phase1}})_{\text{Spot-Obs}} \end{aligned} \quad (3.51)$$

設計一個有效的 Multi Phase RL 策略，可以讓期望 V_ψ 保持固定調整 Reward 參數，如(3.52)式所示。還是讓期望值變動固定 Reward，能夠在最終測試效果比較好，本論文對此做了實驗。

$$R(\phi_{phase1})_{Spot-Goal} > R(\phi_{phase2})_{Spot-Goal} \quad (3.52)$$

將「抵達目的地」和「避障」問題分成三個訓練階段，一個訓練策略固定訓練環境中的 Reward，另一策略則固定期望值。Phase1 到 Phase3 的訓練 $R_{Spot-Goal} = 30$ ，只改變 $R_{Spot-Obs}$ 的設定數值。兩策略在 Phase1 都使用一樣的 $R_{Spot-Obs} = -5$ ；Phase2 固定訓練環境中的 Reward 策略 $R_{Spot-Obs} = -5$ ，固定訓練環境中的期望值策略 $R_{Spot-Obs} = -10$ ，讓 Agent 在訓練過程中包持差不多的期望值；Phase3 兩策略 $R_{Spot-Obs} = -30$ ，確保最後的公平性。這兩測試只有 Phase2 碰撞的 Reward 不同，實驗結果如表 3.9 所示。

表 3.9 Multi Phase RL 有效性測試

訓練策略	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
$V(\phi_{phase1}) = V(\phi_{phase2})$	0.83	0.93
$R(\phi_{phase1}) = R(\phi_{phase2})$	1.52	0.77

結果顯示，直接將訓練目標設定為最終難度，而非逐步提升難度，反而更為有效。實驗中逐步改變 Reward 效果不佳，而一步到位的 Reward 設計則效果更佳，相差接近一倍的提升。

這結果非常違反常識，但這並不意味 Multi Phase RL 沒有意義，如(3.53)式所示。期望值的調整不僅可以透過改變事件的 Reward，還可使用不同的 RL 環境參數，讓不同階段注重不同的能力訓練。

$$\begin{aligned}
\text{Phase 1: } & \max_{\phi_{\text{phase1}} \in \text{RL-Env}} V_{\psi_1}(\phi_{\text{phase1}}) \\
\text{Phase 2: } & \max_{\phi_{\text{phase2}} \in \text{RL-Env}} [j \cdot V_{\psi_1}(\phi_{\text{phase2}}) + q \cdot V_{\psi_2}(\phi_{\text{phase2}})] \\
\text{Phase 3: } & \max_{\phi_{\text{phase3}} \in \text{RL-Env}} [j \cdot V_{\psi_1}(\phi_{\text{phase3}}) + q \cdot V_{\psi_2}(\phi_{\text{phase3}}) + k \cdot V_{\psi_3}(\phi_{\text{phase3}})]
\end{aligned} \quad (3.53)$$

本論文依舊通過改變 $R_{\text{Spot-Obs}}$ 進行實驗，測試能力應該一起訓練還是分開訓練，將「抵達目的地」和「避障」問題拆分為三個訓練階段。能力分階段訓練策略，Phase1 使用 $R_{\text{Spot-Obs}} = -5$ ，Phase2 使用 $R_{\text{Spot-Obs}} = -10$ ，Phase3 使用 $R_{\text{Spot-Obs}} = -20$ ，讓 Phase1 主要訓練抵達目的地的能力，Phase3 才重點訓練避障能力；不同能力同時最大化策略，Phase1 使用 $R_{\text{Spot-Obs}} = -10$ ，Phase2 使用 $R_{\text{Spot-Obs}} = -15$ ，Phase3 使用 $R_{\text{Spot-Obs}} = -20$ ，盡量在每個訓練階段都找尋能夠讓「抵達目的地」和「避障」最大化效能的參數，實驗結果如表 3.10 所示。

表 3.10 Multi Phase 任務策略測試

訓練策略	$\rho(\theta_{\text{phase3}})_{\text{Spot-Goal}}$	$p(\theta_{\text{phase3}})_{\text{Spot-Obs}}$
能力分階段訓練	0.83	0.93
同時最大化策略	0.53	0.87

實驗結果如表 3.10 所示，優先訓練「抵達目的地」能力的策略，能獲得了更好的抵達目標的能力，而避障能力則相對較弱。這一結果雖然在預期之內，但對於理解 Agent 的策略選擇卻很有意義。優先訓練「抵達目的地」的 Agent 更願意嘗試較高難度的任務，而統一訓練策略的 Agent 相對較為保守，這種訓練差異最終導致了能力上的不同。嘗試高難度任務的 Agent 在持續挑戰過程中逐漸掌握所需技能；而採取保守策略的 Agent 在早期訓練表現良好，但最終進步有限。

因此，本論文認為 Reward 的設計應該著重於引導 Agent 漸進優化，而非僅作為最終的績效評估。若一開始即以最終期望值為訓練標準，Agent 難以達到最佳效果。只有在逐步提升引導的情境下，才適合引入最終的 Reward 評估，以達

到最佳訓練效果。

3.7.2 調整 RL 環境參數優化能力

在 Multi Phase RL 的實驗中，我們了解到僅調整 Reward 值無法有效改善 Agent 的效能。若要提高單一能力的重要性，需提升該能力的期望值。如(3.54)式所示，我們得知期望值是由 Reward 和機率組成，因此要優化特定能力，除了調整 Reward 還需改變該能力在 RL 環境中發生的機率。

$$V(\phi) = R_{\psi_1}(\phi) \times p_{\psi_1}(\phi) + R_{\psi_2}(\phi) \times p_{\psi_2}(\phi) \quad (3.54)$$

如(3.55)式所示，能力的機率是由 RL 環境和 Agent 的能力構成。雖然我們無法直接改變 Agent 的能力，但可以通過不同的 RL 環境設置來調整機率，以控制各能力對於 Agent 而言的重要性。

$$p_{\psi} = Env(\phi, \theta) \quad (3.55)$$

常見的方式可以修改 RL 環境的長度，即 Agent 與 RL 環境互動的次數上限。此參數除了可以利用執行時間來改變 Reward 的數值之外，還能調整 Agent 在 RL 環境中需要擁有不同能力的重要性。測試方式如表 3.11 所示，設置兩個不同的 RL 環境執行時間。在 batch size 相同的限制下，兩者的訓練量一致，測試環境也相同，結果仍顯示出顯著差異。

表 3.11 RL 環境長度訓練效果測試

調整參數 ϕ	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
RL 環境長度 90 秒	0.87	0.93
RL 環境長度 150 秒	1.93	0.73

這結果並非表示 RL 環境執行時間越長越好，而是「避障」能力的訓練中，執行時間延長使得避障的重要性提高。在較長時間下，Spot 可以等待 Goal Agent 移動到安全地點後再前往，仍能抵達目的地，從而提升了避障能力的需求機率，降低「抵達目的地」能力的影響。

本論文想表達並非只有 RL 環境長度一個參數，而是 RL 環境中的每個參數均需考量。不同參數對訓練的各能力影響不同，因此我們需根據所優化的能力調整 RL 環境參數。本論文設計中，主要使用的參數包括不同 Agent 的數量、Agent 生成方式、RL 環境長度以及樓梯和牆體的數量和大小，來控制不同能力的機率。

Spot 上下樓梯的能力訓練

為了讓 Spot 具備上下樓梯的能力，需控制其選擇上下樓梯的機率。因 Spot 需學會在目的地位於樓梯上時才選擇樓梯，且錯誤姿態會導致死亡，因此只有當 Spot 獲得大量樓梯訓練機會，且認為死亡風險小於等待不到 Goal Agent 的風險時，才會學會上下樓梯。

在訓練初期，由於 Spot 未具備理解樓梯訊息和向後移動的能力，會傾向避開樓梯，即便目的地位於樓梯上。因此，我們透過 Spot 在樓梯上成功移動的高度差異來設定較高的樓梯期望值 V_{Stair} ，以鼓勵 Spot 嘗試樓梯，如(3.56)式所示。

$$R_{Stair_{t_n}} = w \times |Spot_{t_n}^{Map_z} - Spot_{t_{n-1}}^{Map_z}| \quad (3.56)$$

此 Reward 設置並非為了讓 Spot 立即學會上下樓梯，而是鼓勵其在訓練早期學會倒退動作，以提高後續 Spot 願意嘗試走樓梯的機會。

$$V_{Spot-Goal}^{Stair} > V_{Spot-Goal}^{Flat} > 0 \quad (3.57)$$

如(3.57)式所示，最終目標是讓 Spot 在 Goal 位於樓梯上時才選擇樓梯。並

且訓練後期將 R_{Stair} 設為 0，僅透過 Goal Agent 的位置引導 Spot 上下樓梯。

為此，需提高 Goal 出現於樓梯上的機率，如(3.58)式所示。

$$p_{Spot-Goal}(Stair) > p_{Spot-Goal}(Flat) \quad (3.58)$$

本論文除了鼓勵 Goal Agent 多走樓梯外，還修改 Goal Agent 的生成方式，盡可能的減少 $p_{Spot-Goal}(Flat)$ ，將被 Spot 碰觸後消失的 Goal Agent 改為重新生成於地圖的其他位置。這樣可大幅減少 Goal Agent 的總數量，還能讓 $V_{Spot-Goal}$ 仍舊維持主導 Reward 的地位。由於一個 RL 環境中有多個 Spot Agent，因此在平面上的 Goal Agent 非常容易被其他 Spot Agent 取得，讓 $p_{Spot-Goal}(Flat)$ 降低，而較為困難 $p_{Spot-Goal}(Stair)$ 的重要性就會不斷提高，直到 Spot 能學會上下樓梯的技能，Goal Agent 也會因為(3.59)式，讓(3.58)式成立，Spot 就能一直能維持上下樓梯的能力。

$$V_{Spot-Goal}(Stair) > V_{Spot-Goal}(Flat) \quad (3.59)$$

3.7.3 訓練能力的順序

在了解不同能力的訓練方式後，可以根據能力之間的相互影響確定優先訓練的順序，如表 3.12 所示。

表 3.12 不同能力得因果關係

改變的能力 被影響的能力	提升抵達目的地能力	提升避障能力	提升上下樓梯能力	提升姿態控制能力
目的地能力的影響	好	差	差	差
避障能力的影響	無	好	好	無
上下樓梯能力的影響	少	非常差	好	差
姿態控制的影響	少	少	無	好

抵達目的地能力

由於融合了其他能力的需求，提升此能力會對 Agent 的整體表現有正面影響，應優先優化。

避障能力

避障的提升可能限制 Agent 的環境探索意願，因此需要等到其他能力都訓練完成後，再提升避障能力。

上下樓梯能力

受其他能力的影響較大，應在姿態控制訓練前完成，以確保 Action 具備多樣性。

姿態控制

不受其他能力影響，可與避障能力同時進行提升。

3.7.4 本論文實際 Reward 的設計策略

Phase 1-3: 優先提升抵達目的地的能力

Phase 1 使用 Multi Agent RL 的技術，以虛擬自我對抗提升訓練效率，在一個 RL 環境中設定較多的 Spot 和 Goal Agent (6 個 Spot 和 20 個 Goal)，以促使 $E = Spot - Goal$ 事件的頻繁發生，加速 Spot 的抵達目標能力的學習。

Phase 2 通過降低 Agent 數量 (3 個 Spot 和 16 個 Goal)，讓 Spot 擴大活動範圍，進一步增強其抵達目標的能力。

Phase 3-5: 提升上下樓梯的能力

Phase 3 調整 Goal Agent 的生成方式 (3 個 Spot 和 1 個 Goal)，降低 R_{Stair} 的數值(樓梯數量都是 2 個)，此階段同時提升抵達目的地和上下樓梯的能力。

Phase 4 將 RL 環境時常從 100 秒降為 70 秒，增加樓梯的生成機率，讓 Agent 能夠多嘗試上下樓梯的能力，此階段之後 R_{Stair} 設定為 0。

Phase 5 由於此階段開始前 Spot 已經掌握上下樓梯的能力，但 Spot 仍為少碰撞風險，有時不願意經過樓梯，將 Spot 的數量從三個改為兩個，並且再增

加梯的生成機率，促進 Spot 學習走樓梯的能力。

Phase 6:優化姿態控制

RL 環境長度改為 200 秒，動態障礙物數量從兩增加至十個，Spot Agent 數量調整為三個，目的是利用虛擬自我對抗，強化姿態控制的學習。此階段訓練前 Spot 已具備良好的抵達目標能力，但偶爾會使用較差的姿態控制。多個 Spot 增加了對速度優化的要求，進一步強化姿態控制的能力。

Phase 7 以上:優化避障能力

將 Spot Agent 的數量調整回兩個，讓 Spot 依然有較好的姿態控制，並且將避障的 Reward 調高。此階段就可以將我們期望 Spot 行為的期望值變成實際 Reward 的設置，在符合 Agent 訓練進步條件下，不斷提升避障能力的重要性。

本論文 Reward 設計策略

本論文提出的 Reward 設計策略有效解決了多種能力訓練目標之間的衝突，成功培養出一個具備多能力的行為規劃策略模型。通過單一 Agent 的分析，展示了如何將訓練目標轉化為具體的 Reward 設計，並在 Multi-Agent 模型中闡述了不同 Agent 之間的協作如何助力目標達成。最後，本論文利用 Multi-Phase RL 技術，逐步融合各種能力，形成完整的策略優化框架。

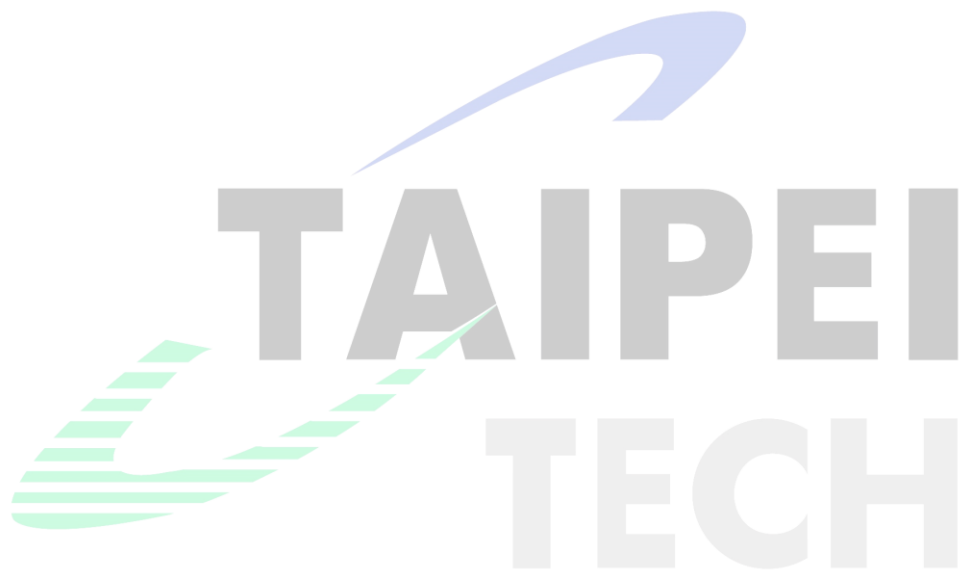
本論文認為，RL 在效能上超越其他 AI 架構的關鍵不在於更精準地預測未來，而在於其在數據收集的分佈不同。RL 的特點是 Agent 的決策會反作用於訓練數據的分佈，透過 Reward 讓 Agent 改變下一次的策略，達成上一時刻的數據每一筆重要性不一樣的目的。相比之下，其他 AI 架構將每一筆數據的重要性視為相同，因此對於重要數據的 Loss 改進較少。

然而，根據 Multi-Phase RL 實驗結果，本論文發現，針對多種能力的優化應分階段進行，並在每個階段集中訓練特定能力。這意味著 RL 在訓練中仍然將一個時刻收集到的數據的重要性視為一致，而這與人類的思維方式不同。人類在分析問題時，會根據不同資訊的重要性進行權衡，從而實現全面考慮，

而非僅僅聚焦於當前事件制定策略。

因此，本論文認為這是目前還需要使用 Multi-Phase RL 訓練中的主要原因。

未來要讓 AI 具備類似人類的思考能力，應集中研究改善 RL 演算法，使其能根據數據的重要性有效調整 Loss 定義。這將有助於合理權衡不同資訊的價值，並實現更完整且高效的策略學習。



3.8 Agent 的訓練策略

3.8.1 Agent 的折扣因子

在 RL 中折扣因子 γ 是一個重要的參數，它與訓練環境的長度及 Agent 的 fps 相關，如(3.60)式所示。 γ 值的設定用於調節未來 Reward 的影響範圍，範圍介於 0 到 1 之間。在最初的 PPO 演算法中 γ 設為 1，表示所有未來獎勵被平等考量。然而，適當調整 γ 值，可以讓模型更加關注近期獎勵，降低對遠期獎勵的依賴，進而提升訓練效率。

$$R(\tau^n) = \sum_{t=t_n}^{t_{\text{end}}} r_t \times \gamma^{t-t_n} \quad (3.60)$$

r ：Agent 在某一時刻獲得的 Reward

τ^n ：在時刻 n 獲得的訓練數據

如(3.61)式所示，可以看出未來的 Reward 是如何按當前時間點進行折扣。這種設計使訓練過程更為簡化，但也會改變 Reward 的期望值。例如在 Spot 的存在期望值中，由於目標點（Goal）比地圖（Map）稀疏， $V_{\text{Spot-Goal}}$ 事件的發生機率較 $V_{\text{Spot-Map}}$ 低，導致(3.60)式中的 t_j 通常大於 t_q 。若 γ 值過小，則即使 $R_{\text{Spot-Goal}}$ 值大，Spot 仍可能選擇不移動，因為未來獎勵對當前行為的影響減少。

$$V_{\text{Spot-Goal}} + V_{\text{Spot-Map}} + V_{\text{Spot-Obs}} > 0$$

$$V_{\text{Spot-Goal}} \times \gamma^{V_{t_j}} + V_{\text{Spot-Map}} \times \gamma^{V_{t_q}} + V_{\text{Spot-Obs}} \times \gamma^{V_{t_k}} > 0 \quad (3.61)$$

t_j ：在 RL 環境中發生 $E = \text{Spot} - \text{Goal}$ 事件的平均時長

t_q ：在 RL 環境中發生 $E = \text{Spot} - \text{Map}$ 事件的平均時長

t_k ：在 RL 環境中發生 $E = \text{Spot} - \text{Obs}$ 事件的平均時長

最終，可以將 Spot 的存在期望值化簡為(3.62)式。

$$V_{Spot-Goal} \times \gamma^{V_{t_j}-V_{t_q}} + V_{Spot-Map} + V_{Spot-Obs} > 0 \quad (3.62)$$

γ 的實驗分析：

為了驗證 γ 對訓練效果的影響，本論文進行了實驗測試，如表 3.13 和表 3.14 所示。表格中的 γ 設定基於 Spot 的 fps，折算出其在一秒鐘後對未來 Reward 的折扣因子。

表 3.13 RL 環境長度 90 秒

γ	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
0.86	0.87	0.93
0.91	0.57	0.63
0.97	0.42	0.88

表 3.14 RL 環境長度 150 秒

γ	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
0.86	1.27	0.7
0.91	2.43	0.37
0.97	0.53	0.93

結果顯示，較小的 γ 有助於模型更快收斂，但可能導致 Agent 忽略長期獎勵的重要性。當環境時間較長時，可以適當提高 γ ，平衡即時獎勵與長期獎勵。

本論文認為 γ 本質上反映了時間對效果的重要性。當 γ 設定較小時，表示「時間」越重要。在 Spot 訓練姿態能力的任務中，「移動速度」是一個關鍵要素，影響 Agent 判斷最優姿態，因此更需要較小的 γ 來識別不同姿態的

Reward 差異。

γ 設定應根據核心任務目標而定。若核心任務與完成時間無關， γ 可設為 1。

若核心任務頻率較低，可提高 γ ；若頻率較高，則適當降低 γ 。對於多階段訓練，在後期提高 γ 亦有助於提升模型的長期策略表現。

總而言之， γ 的合理設計對於強化學習模型的訓練效果及策略選擇有重大影響，應靈活調整以契合具體的訓練目標和環境需求。

3.8.2 Agent Action 的採樣

A2C 和 Q learning 的演算法中，Action 的選擇是評估每個可能動作的 Reward 期望值來進行的。最基本的動作選擇方法是選擇具有最高 Reward 期望值的動作來當成輸出行為，如(3.63)式所示。

$$P(a | s) = \underset{a}{\operatorname{argmax}} Q^{\theta}(s, a) \quad (3.63)$$

這種方法雖然追求即時最大化 Reward 的策略，但它可能導致學習過程中的探索不足，從而限制了算法發現更好策略的機會。為了克服這一問題，增加動作選擇的多樣性，本論文採用了論文[34]的 Boltzmann Exploration 技術。這種技術通過將 Q 函數值轉換，將動作選擇以概率來實現，即使是 Reward 期望值不是最高的動作也有機會被選擇，如(3.64)式所示。

$$P(a | s) = \frac{\exp(Q(s, a))}{\sum_a \exp(Q(s, a))} \quad (3.64)$$

這種基於 Boltzmann Exploration 的方法不僅增加了策略的多樣性，還提高了模型在 Multi Env RL 的學習效率。如果沒有這種探索機制，所有的 Agent 可能都會採取相同的行動，導致學習過程效率低下。

通過適當的多樣化探索，能夠更有效地訓練強化學習模型，使其在面對不確定和變化的環境時能夠做出更加智能的決策。

3.8.3 Agent 行為調整

本論文深入分析了控制 Spot 運動方式的相關結果，這些結果基於 Spot 在選擇最佳動作時的表現。然而，在實際訓練 RL 模型的過程中，需要考慮更多變數，因為 Agent 並不總是會選擇最佳的動作，如圖 3.27 所示。

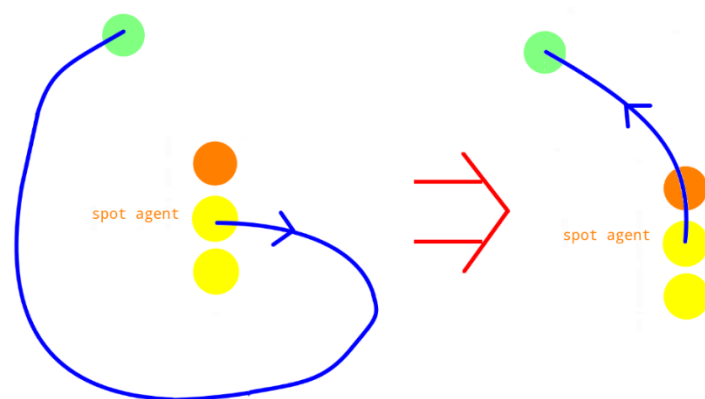


圖 3.27 Spot 的最佳路徑

Agent 動作選擇的多樣性與挑戰

如圖 3.27 所示，Agent 達成目標的方式可能多種多樣，它們會傾向於選擇最容易達到目標的方式，但這不一定是最佳或最高效的方式。例如，當目標位於 Spot 的左上方時，如果 Spot 更擅長捕捉位於右上方的目標，它可能會選擇繞一圈使目標出現在右上方再進行捕捉，這並非我們所期望的行為模式。

原因分析與解決方法

造成此問題的原因除了 Reward 的設計外，也可能是 Agent 不願意嘗試新的 Action，認為舊的 Action 就是能得到最高 Reward 的選擇，解決方法可以提高 Action entropy 讓每一種 Action 的期望值不要差太多，Sample 出其他 Action 的機率就會提高很多，就能更容易學習到最優解。

Soft Actor Critic (SAC) 策略的運用

論文[35]策略透過以下損失函數來更新策略網絡，同時限制了動作採樣的分

佈情況不要相差太多，如(3.65)式所示。

$$J(\theta) = \sum_{t=0}^T E_{(s_t, a_t) \sim \rho_\theta} [r(s_t, a_t) + \alpha H(\theta(\cdot | s_t))] \quad (3.65)$$

H ：稱 entropy，用於計算在一個 batch size 中 Action 的散度

透過實踐，本論文採用了[36] Stable baseline3 的方法，計算不同動作的 entropy 並將其納入損失函數，目的是使每個動作的概率分佈更加均衡。

如表 3.15 所示，顯示透過 SAC 的策略，可以有效地促進 Agent 對新動作的探索，從而提升學習效率和策略的多樣性。這一發現與我們對於初始狀態分佈調整的策略相呼應，均指向了提高 Agent 探索新動作的重要性。

表 3.15 SAC entropy 重要性測試

訓練策略	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
entropy=0	0.01	1
entropy=0.15	0.83	0.93

實驗結果顯示 SAC 策略透過提高熵系數來增強 Agent 的動作探索能力，在強化學習中顯示出了顯著的效果。這種策略不僅促進了對新動作的探索，也為解決 Agent 在學習過程中的探索問題提供了一個有效的途徑，對於提升強化學習模型的性能具有重要的意義。

Sample Action

RL 的訓練過程，Agent 選擇非最優解的一個主要原因是缺乏對新動作的嘗試。除了利用 Soft Actor Critic 之外，另一種有效的策略是調整初始狀態 (s_0) 的設置。這是因為 Agent 的動作選擇可能導致某些狀態 (State) 的分佈消失，從而使 Agent 僅學習到有限的、特定狀態分佈下的技能，進一步導致狀態分佈變得更加集中。結果，Agent 對於嘗試新動作的意願進一步降低。

初始狀態分佈的調整方法

以 Spot 為例，如果其初始速度都設定為 0，並且 Agent 偏好向前移動的動作，則最終導致 Agent 不會選擇向後移動的動作。為了解決這一問題，本論文通過調整初始速度的分佈來促使 Agent 探索更多元的動作選擇，具體的調整方式如(3.66)式所示。

$$v_0 = \begin{cases} 0, & \text{if } 0 \leq r < 0.2 \\ \min(v) + \frac{r-0.2}{0.8} \cdot (\max(v) - \min(v)), & \text{if } 0.2 \leq r \leq 1 \end{cases} \quad (3.66)$$

r ：隨機變數範圍 0 到 1

通過這種方式，將初始速度設定為一個基於隨機變量的函數，這樣不僅包括了靜止狀態，也涵蓋了從最小速度 $\min(v)$ 到最大速度 $\max(v)$ 的範圍。這種分佈的調整，使得 Agent 在訓練過程中有機會探索到更多的狀態，從而學習到更加多樣化的動作策略。

表 3.16 初始狀態重要性測試

訓練策略	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
$v_{init} = 0$	0.38	0.97
v_{random}	0.83	0.93

實驗結果如表 3.16 所示，將初始速度設定為隨機值對於提高 Agent 的學習效率和行為多樣性具有顯著的正面影響。這種方法不僅有助於 Agent 探索到新的動作，還能夠促使 Agent 學習到如何在不同的狀態下達成目標，從而提高其整體的性能。

3.8.4 Action 數量的選擇

由於 Action 輸出並非連續值，而離散的數值來代表，因此需要討論 Action 離散的數量對於控制 Spot 運動方式的影響。以 Spot 的加速度 Action(A_{acc_x})為例，其值域範圍 $A_{acc_x} \in [-0.333 \text{ m/s}^2, 0.333 \text{ m/s}^2]$ ，若設定 Action 離散的數量為 7，則代表最終的速度控制精度為 0.11 m/s。然而，Agent 是否能夠實現如此的控制精度，仍然取決於模型的性能。本論文將同樣條件下，將 Spot 設定不同 Action 離散輸出數量，探討 Action 精度的影響，實驗結果如表 3.17 所示。

表 3.17 Action 數量測試

Action 離散數量	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
7	0.87	0.93
11	1.27	0.67
19	1.77	0.73

實驗數據顯示動作精度越高，訓練效果越好。進一步觀察了在全部執行時刻，每一種動作執行時的機率分佈，就能得到圖 3.28 至圖 3.30，分別以紅、綠、藍色對應 Action 數量 7、11 和 19 個的分布機率，能看到不論是幾個離散數量其值域都是固定的(x 軸)。

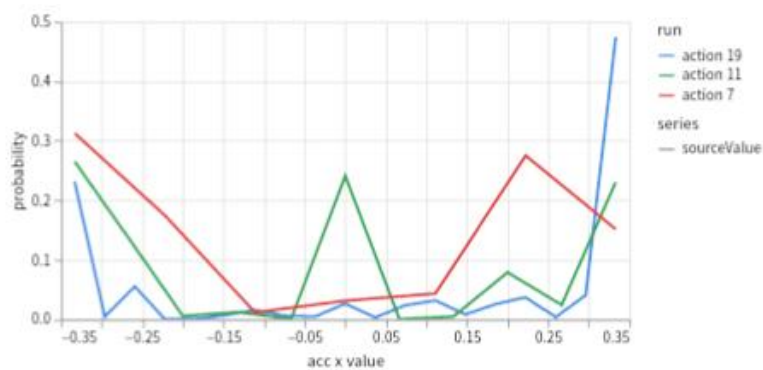


圖 3.28 A_{acc_x} 分佈機率

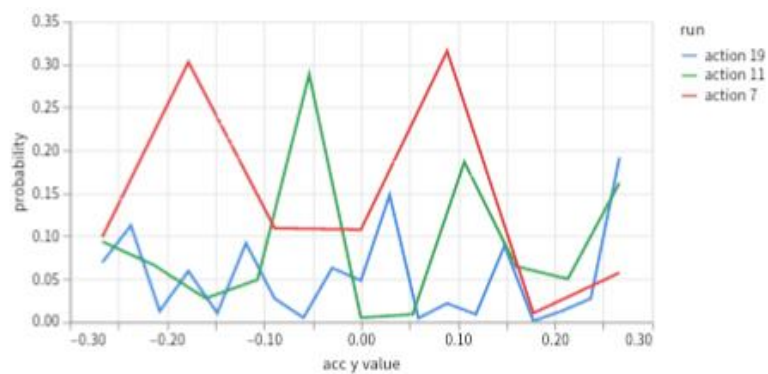


圖 3.29 A_{acc_y} 分佈機率

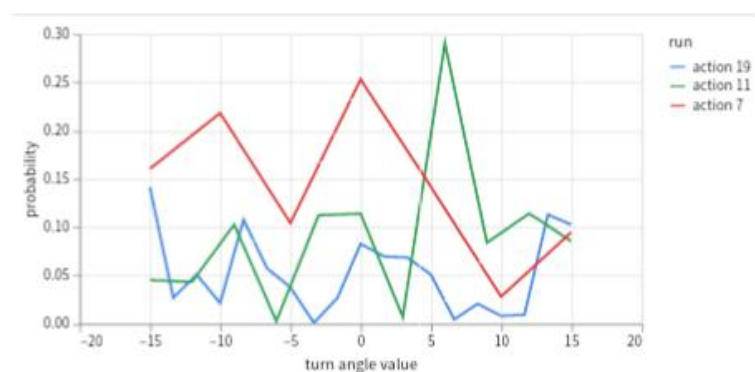


圖 3.30 A_{ω} 分佈機率

圖 3.31 至圖 3.33 所示，測試在不同離散 Action 數量，在測試環境中的速度分布，可以看出有 19 個離散值的 Action 分佈最為平均，並不會有任何一項數值出現的特別多。

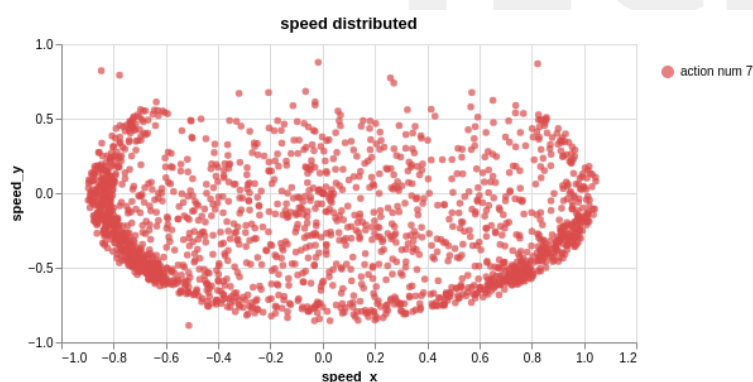


圖 3.31 7 個 Action 的速度分布圖

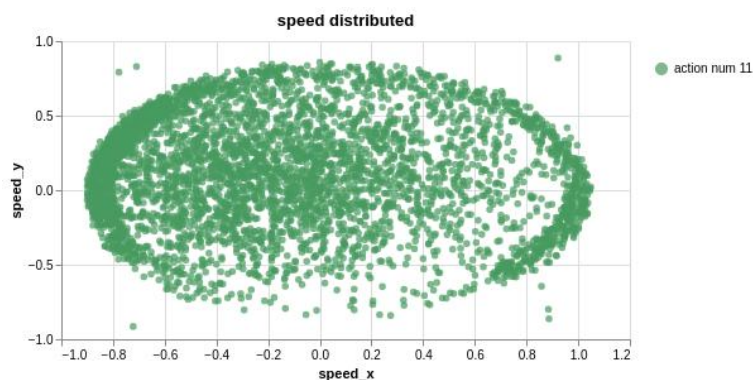


圖 3.32 11 個 Action 的速度分布圖

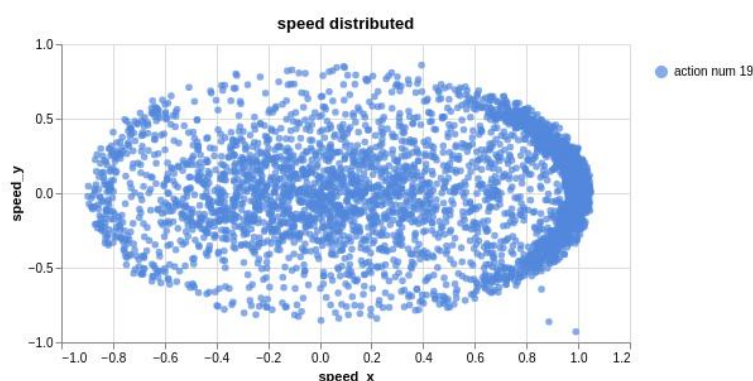


圖 3.33 19 個 Action 的速度分布圖

如圖 3.31 至圖 3.33 所示，Action 數量為 19 的速度分布效果最佳，能夠完美的實現姿態控制，然而這一結論並非固定，這主要是由 SAC 中 entropy 的影響。只有當模型願意嘗試多種動作選擇時，設定更多的動作數量才具有意義。

3.9 State 的設計

State 為 Agent 在 RL 環境中執行時所感知到的輸入資訊，這些資訊直接影響 Agent 對環境的理解及隨後的行為決策。因此 State 的設計需要精準抓取與任務目標密切相關的環境特性，以確保 Agent 能夠根據這些信息採取最佳的 Action。本小節將討論 State 撰寫的重要性、不同設計的差異，以及 State 應如何撰寫的結論。

3.9.1 State 撰寫方式的重要性

AI 訓練的本質是訓練相似性 (likelihood)，旨在盡可能找到輸入與我們需要預測結果之間的關聯性，並盡可能發掘資訊中隱藏的所有關聯。與其他 AI 訓練相比，RL 中的訓練資料分佈更加動態變化，因此探究同樣資訊內容用不同的方式撰寫 State 是否有差異成為一項重要的實驗。

本論文以 Spot Agent 為例，對其計算距離的方式進行了比較實驗：一種是利用地圖座標系結合 Spot 自身的方向角度來計算，另一種則直接使用 Spot 的機器人座標系來表示。這兩種方法在資訊內容上是相同的，但在表達形式上有所不同。具體來說，本論文定義 Spot 的目的地資訊如(3.67)式所示。

$$S_{Spot_{Goal}} = \{Goal^{Map} - Spot^{Map}, Spot^{dir}\} = \{TF_{Spot^{Map} \rightarrow body} (Goal^{Map} - Spot^{Map})\} \quad (3.67)$$

TF ：為機器人系統的坐標系轉換，代表將地圖坐標系換成機器人坐標系

在進行的實驗中，將使用地圖座標系的 State（包括 Spot 目標、障礙物和激光雷達信息）全部轉換為 Spot 的機器人座標系，並保持其他參數不變，以此來比較兩種 State 表示方法的學習效果。

表 3.18 State 撰寫方式測試

State 設計策略	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
機器人座標系	0.87	0.93
地圖座標系	0.1	0.97

如表 3.18 所示，顯示儘管資訊內容相同，機器人座標系和地圖座標系在學習成效上存在顯著差異。使用機器人座標系的 State 表示方法，模型能夠更有效地學習到避開障礙物和到達目的地的行為。相反地圖座標系的表示方法對 Spot Agent 來說過於複雜，導致其難以學習有效的行為策略。

3.9.2 State 資訊長度的影響

論文[12]指出，模擬的保真度並不直接促進學習效果，反而可能增加訓練時間。實驗表明，使用真實世界數據構建的簡化機器人運動模型，能夠有效改善學習和提升泛化效果。

為了深入理解如何設計高效的 State，本論文分析 State 輸入數量對模型訓練效果的影響，來測試 State 數量的影響。 $S_{SpotSweep}$ 表示 Spot 機器人從不同角度到 RL 環境障礙物 (Map) 的距離，我們可以根據 Spot 機器人目前的所在坐標系，推算不同角度的距離資訊，控制用多大的角度當成一個資料點，來控制 State 資訊的數量。

本論文究設定了兩種不同的角度間隔，分別為 5 度和 10 度，對應 72 個和 36 個資訊點，以代表兩種不同的 $S_{SpotSweep}$ 狀態數量。在其他參數保持不變的情況下，對這兩種設置進行訓練效果的比較。

表 3.19 State 資訊長度影響測試

$S_{SpotSweep}$ 資訊數量	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
72 input	0.87	0.93
36 input	1.53	0.8

如表 3.19 所示，當 State 資訊數量減少時，與到達目標地點相關的性能提升，而避開障礙物的能力也有所改善。這表明在不影響資訊完整性的前提下，使用較少的輸入表示 State 可以達到更好的訓練效果。

3.9.3 State 加噪音

在設計 RL 環境時，Agent 的 State 資訊有時代表連續數值，例如速度。然而，由於 RL 環境中 Action 的輸出值為離散的，State 也可能表現為離散形式。這種設計可能導致 Agent 在實際應用中，由於從未見過某些中間值而採取錯誤的行

動。為解決此問題，本論文對離散的 State 資訊加入噪音，將原本離散的速度數值轉變為連續數值，以提升模型的泛化能力。

表 3.20 State 資訊抗噪測試

State 設計策略	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
加噪音	0.87	0.93
沒加噪音	0.7	0.63

如表 3.20 所示，本論文分析了加噪音前後訓練效果的差異。結果顯示，未加噪音的情況下碰撞機率較低，這是由於測試環境中因加噪音讓 Agent 誤判導致的。然而，更為重要的發現是，加入噪音提升了 Agent 的到達目標能力。這一實驗證明，對離散 State 加入噪音能使 Agent 更好地適應數值變化，進而提升模型在實際環境中的表現。

3.9.4 State 設計結論

通過上述三個實驗，本論文認為 Spot 座標系與 Map 座標系之間的差異在於 State 是否有複合資訊，這種差異對訓練效果有著巨大影響。Spot 座標系要求 Agent 理解單一角度的距離資訊，以便在距離過近時，制定不往某方向移動的策略；相比之下，Map 座標系則需要 Agent 同時處理距離向量 $Goal^{Map} - Spot^{Map}$ 與 Spot 角度方向 $Spot^{dir}$ 的資訊，並將其轉換至 Spot 座標系來制定行動策略。困難之處在於此轉換關係需要通過 Reward 的差異來計算，增加了從 State 中提取有用資訊的難度。

由於 RL 訓練資料的分布變化極快且不穩定，Agent 在快速理解 State 的效果有限，難以有效整合有用資訊。因此，State 的設計對 Agent 的學習成效具有顯著影響。在設計 State 時，應仔細考量資訊表達方式，以便清晰揭示輸入與輸出之間的關係，從而提升模型的學習效率與性能。

在 State 資訊長度和 State 加噪音的實驗中，本質上是考慮實際執行與 RL 訓練時的落差，因此對訓練效果影響較小，但實際測試就會看出巨大的差別。RL 訓練出的 Agent 對 State 的變化極為敏感，因此需盡量減少真實執行與訓練的落差。無論是壓縮 State 資訊長度，還是通過加噪音將離散數值變得連續，這些方法都能有助於 Agent 提升泛化性。

此外 State 作為 Agent 的輸入資訊，將被評估網路和策略網路使用，分別評估 State 的期望值和制定 Agent 的策略。這兩個神經網路對 Agent 同等重要，因此 State 還需包含可推斷未來 Reward 的資訊。在設計完成後，若能依據 State 判斷未來 Reward 和適當的策略，才表明 State 的設計已基本完善。

State 設計的三大原則

1. 清晰的資訊表達

選擇能直觀反映輸入與輸出關係的 State 表達方式，便於模型更好地理解環境。

2. 簡化複合資訊

在保留關鍵資訊的前提下，盡量簡化 State 中的複合資訊，以減少模型處理的複雜度。

3. 完整的資訊

設計 State 時應考量是否能根據 State 判斷未來 Reward 和適當策略，從而提高學習效率和決策質量。

3.9.5 Spot Agent 的 State

$$S_{spot} = \{Speed, Goal, Time, Died, Obs, Sweep\} \quad (3.68)$$

根據 State 設計的結論，本論文為 Spot Agent 的 State 包含當前運動狀況的速度 (Speed)、目標位置 (Goal)、動態障礙物資訊 (Obs)、以及地形資訊 (Sweep)。

此外，還包含評估網路所需的時間（Time）和死亡狀態（Died）資訊。

Speed 資訊

$$S_{Spot_{Speed}} = [v_x, v_y]^T \quad (3.69)$$

Speed 表示 Spot 在上一時刻的運動速度，分別包含前向和左右方向速度。由於在現實中無法精確確認速度值，且因 Action 的離散性問題，因此，在 RL 訓練中增加了大小為 0.1 m/s 的隨機噪音。

Goal 資訊

$$S_{Spot_{Goal}} = [Goal_x, Goal_y]^T \quad (3.70)$$

Goal 資訊包括目標相對於 Spot 的 x 和 y 方向距離。由於 Spot 僅在地面移動，RL 訓練中又可以依靠 $Goal_z$ 信息得知是否上下樓梯，而不直接依賴 Sweep 資訊；因此移除 $Goal_z$ 資訊，避免在真實場景中出錯。

Died 資訊

$$S_{Spot_{Died}} = \begin{cases} 0, & \text{agent 死亡} \\ 1, & \text{agent 存在} \end{cases} \quad (3.71)$$

Died 用於判斷 Agent 是否仍然存在於 RL 環境中。當 Spot 撞上障礙物後，會從 RL 環境中移除。Spot 雖無法再獲得 Reward，但仍會出現在訓練集中，為了估算合理的期望值需讓 Agent 了解是否還能取得 Reward。

Time 資訊

$$S_{Spot_{Time}} = \frac{T-t_n}{T} \quad (3.72)$$

T ：RL 環境中最多能執行的時間長度

t_n ：收集資料時 RL 環境的執行的時間長度

Time 是用來計算 RL 環境剩餘時間，用於評估網路預測未來可獲得的期望值。在實際應用中沒有執行時間限制，可以將此值設為 0.5 作為固定輸入。

Obs 資訊

$$S_{Spot_{Obs}} = [Obs_x, Obs_y, Obs_{dir}, Obs_v, Obs_{size}]^T \quad (3.73)$$

Obs 提供了 Spot 能感知到的至多十個動態障礙物的資訊，包括相對位置、方向、速度及大小，為 Spot 的動態避障策略提供有效的資訊。

Sweep 資訊

$$S_{Spot_{Sweep}} = \{Sweep_{angle}, Sweep_{Map-gradient}\} \quad (3.74)$$

Sweep 提供 Spot 可感知的地形信息。因 Spot 需在三維空間導航，Sweep 包含 Spot 需知的三維信息，且此資訊可以直接使用真實感測器接簡單轉換取得，使系統反應更快速。由於三維信息量龐大，本論文將 Sweep 切分為二維可行區域 $Sweep_{angle}$ 資訊及地表起伏 $Sweep_{Map-gradient}$ 資訊。

如圖 3.34 所示，為 Spot 在 RL 環境中的 $Sweep_{angle}$ 資訊。包含 36 個不同角度從 Spot 的身體看出去的距離，在真實情況由 Boston Dynamics 的 Spot 上的 3D 雷達轉換而成，轉換方式是藉由判斷點雲在地表的起伏，是否大於 Spot 能行駛的高度來決定 Spot 的可行駛範圍。

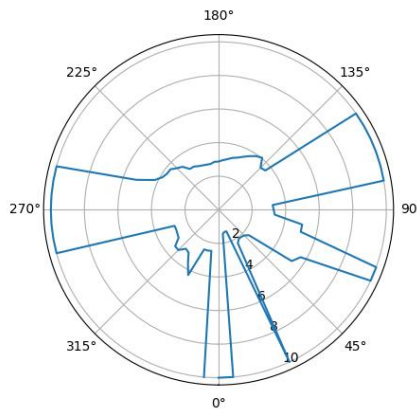


圖 3.34 Spot 的 $Sweep_{angle}$

$Sweep_{Map-gradient}$ (地表起伏資訊)用於調整上下樓梯姿態的規劃，補足 2D 可行區域內無法判別的地表高度信息。因為本論文策略為行為規劃而非馬達控制，因此只需能夠判斷策略的地表高度資訊即可，如圖 3.35 所示，本論文選取八個重要點。最前和最後的點用於提前轉向，左右點用於防撞和姿態調整，近點則判斷近距地表起伏。

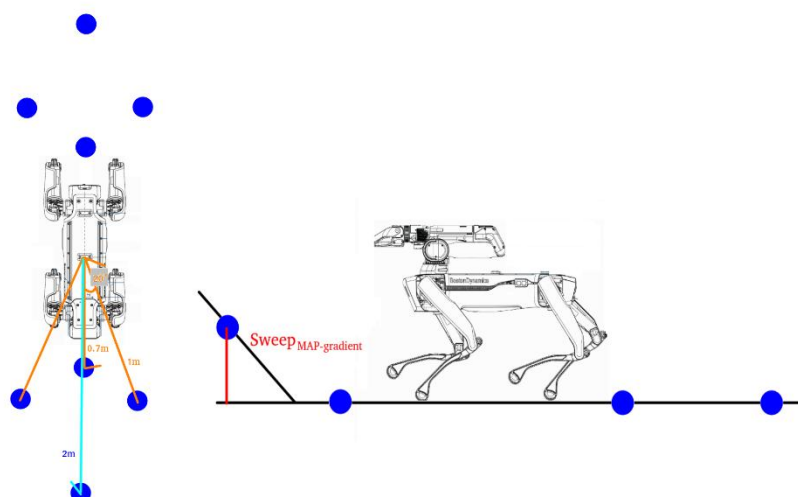


圖 3.35 Spot 的 $Sweep_{Map-gradient}$

實際取得 $Sweep_{Map-gradient}$ 的方式，以 3D 雷達、深度相機及 IMU 結合獲取周圍點雲資訊。3D 雷達和深度相機都是用來獲取機器狗周圍的點雲資訊，由於感測器會受到機械手臂影響，導致資訊被遮擋視角，因此使用 3D 雷達和深度相機的資訊合併資訊確保完整視野，如圖 3.36 所示。

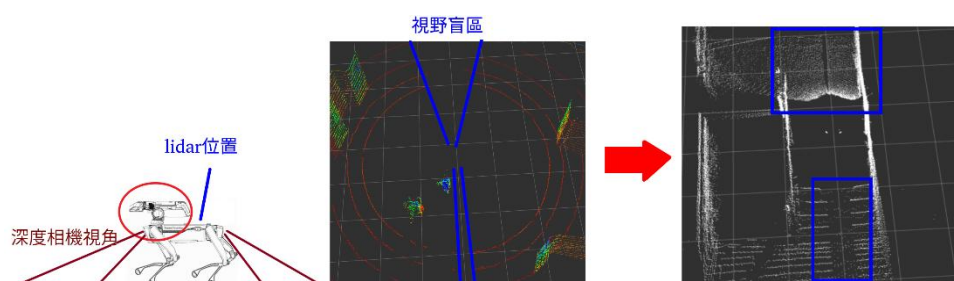


圖 3.36 Boston Dynamics Spot 的感測器視角

在實際情況本論文將點雲資訊轉換至 $Sweep_{angle}$ 和 $Sweep_{Map-gradient}$ ， $Sweep_{Map-gradient}$ 用來計算地面梯度，由於點雲資訊是基於 Spot 座標系，需通過 IMU 確認姿態再回歸到正確角度，最終將地表高度信息依據 Spot 位置轉換為相對地表高度的 8 點資訊，如圖 3.37 所示。

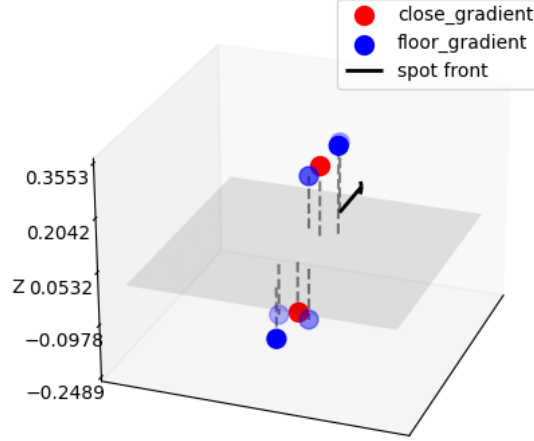


圖 3.37 實際的 $Sweep_{Map-gradient}$

3.9.6 Goal Agent 的 State

$$S_{Goal} = \{Speed, Time, Died, Spot, Sweep\} \quad (3.75)$$

Goal Agent 的輸入資訊包含 Goal 自身的速度(Speed)、需要關注的 Spot 資訊(Spot)，以及用於識別地形的資訊(Sweep)，同時還有評估網路所需的時間(Time)和死亡狀態(Died)資訊。

Speed 資訊

$$S_{Goal_{Speed}} = v_x \quad (3.76)$$

Goal 的移動速度，由於 Goal 只能沿 x 方向運動，因此此處僅考慮向前移動的速度。

Spot 資訊

$$S_{GoalSpot} = [Spot_x, Spot_y, Spot_{dir}, Spot_{v_x}, Spot_{v_y}]^T \quad (3.77)$$

為了測試 Multi-Agent RL 中 Agent 的互動，Goal Agent 需要獲取 Spot 的資訊。這包括從 Goal Agent 視角看到 Spot 在 x 和 y 方向的距離、角度差異，以及 Spot 的移動速度。

Time 和 Died 資訊

Time 和 Died 資訊與 Spot 的資訊相同，Time 用來計算剩餘時間，而 Died 則用於表示 Goal 在 RL 環境中的存活狀態，幫助評估網路評估未來期望獎勵。

Sweep 資訊

$$S_{GoalSweep} = \{Sweep_{angle}, Sweep_{Map-gradient}\} \quad (3.78)$$

關於地表的 Sweep 資訊，由於 Goal 與 Spot 的目標不同，因此需要不同的資訊來做出行為決策。Spot 的 Sweep 資訊用於避免以錯誤姿態上下樓梯，而 Goal Agent 則需要確認樓梯的位置。因此，Goal Agent 和 Spot 一樣需要 $Sweep_{angle}$ 來了解可行走區域，並且額外需要更廣的 $Sweep_{Map-gradient}$ 來尋找樓梯的位置。

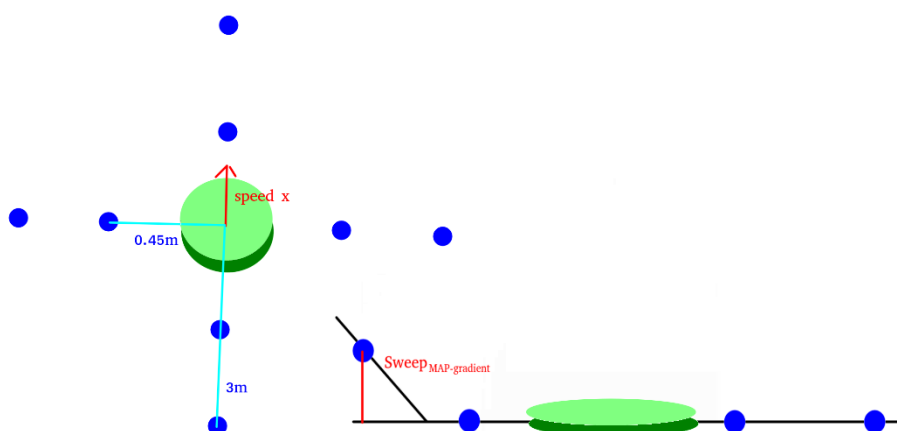


圖 3.38 Goal Agent 的 $Sweep_{Map-gradient}$

如圖 3.38 所示， $Sweep_{Map-gradient}$ 除了前後方的資訊外，Goal Agent 還增加了左右兩側的資訊，以便於從各個方向尋找樓梯的位置。

3.9.7 Obs Agent 的 State

$$S_{Obs} = \{Speed, Time, Sweep\} \quad (3.79)$$

Obs Agent 的狀態相對簡單，因為其主要目標是隨機移動並盡量避免撞到牆壁。因此，Obs 不需要考慮其他 Agent 的資訊，而僅依賴基本的行為決策資訊以及用於估算未來獎勵期望值的資訊。

Speed 資訊

代表 Obs 當前的運動速度，與 Goal Agent 的速度狀態一致。

Time 資訊

Time 資訊與 Spot 的時間資訊一致，用於計算剩餘時間。由於 Obs Agent 不會因任何情況被移除出 RL 環境，因此不需要 Died 資訊。

Sweep 資訊

提供了 Obs 對於周圍環境的感知能力，與 Spot Agent 的 Sweep 資訊相同。

這樣的資訊能讓 Obs 執行基本功能，在 RL 環境中盡可能快速移動，同時避免撞到牆壁，從而為訓練 Spot Agent 提供必要的挑戰。

第四章 RL Model 的分析

4.1 Model 的分析

A2C 演算法的模型包含策略網絡 (Policy Network) 與評估網路 (Critic Network)，通常使用多層感知機 (MLP) 結構進行設計。本論文探討了不同模型架構對訓練效果的影響，旨在分析並改進模型結構，提升行為規劃的效能。

如論文[37]和[38]所示，A2C 算法的模型通常採用圖 4.1 所示的標準架構。

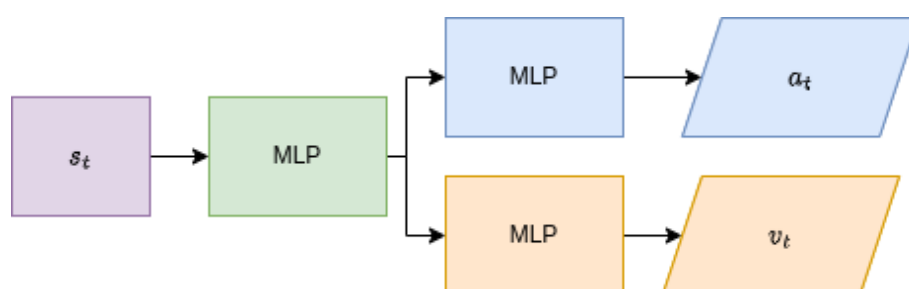


圖 4.1 常見的 A2C Model

s_t : Agent 的 State 資訊

a_t : Agent 的 Action 輸出

v_t : Agent 評估 State 的期望值

A2C 架構介紹

綠色部分為策略與評價網絡的共用層，細節為三層全連接層[512, 512, 512]。

藍色部分為策略網絡獨有層，細節為四層全連接層[512, 256, 128, Action]。

橘色部分為評價網絡獨有層，細節為四層全連接層[512, 256, 64, Value]，最後輸出資訊寬度為一，用於評估狀態。

A2C 架構測試

要測試不同架構對效果的影響，本論文將不同模塊的網絡組合進行實驗。如圖 4.1 至圖 4.4 所示，測試中將 A2C 最常見的模型稱為 **Normal Model** 作為基本的 A2C 比較模型，將評價網絡後面的全連接層移除稱為 **Policy Large**

Model，將策略網絡後面的全連接層移除稱為 **Critic Large Model**，將兩個網絡後面的全連接層都移除稱為 **Small Model**，這也是 **stable-baselines3** 採用的方法，特點是沒有獨立的分支。

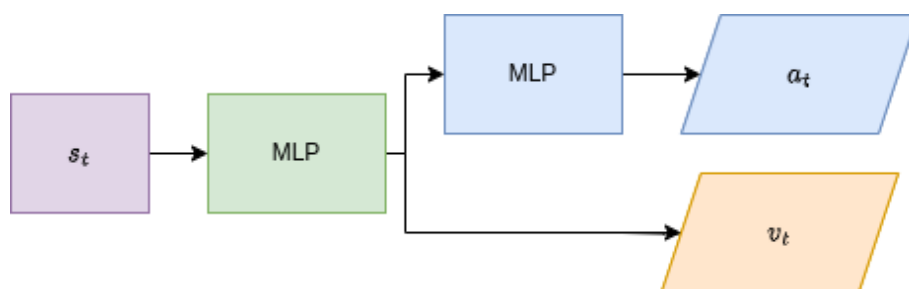


圖 4.2 Policy Large Model

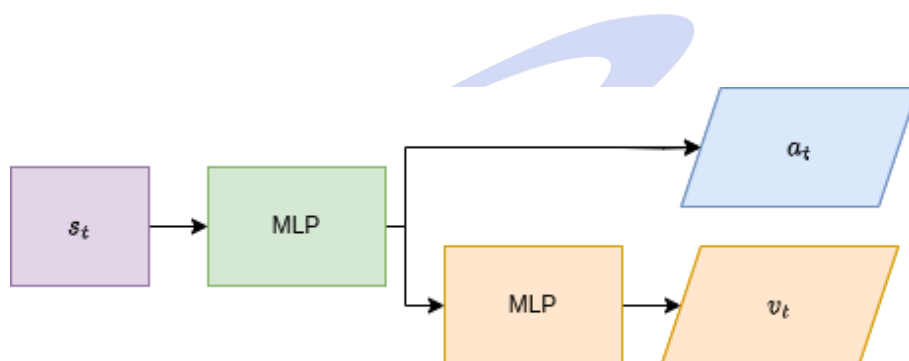


圖 4.3 Critic Large Model

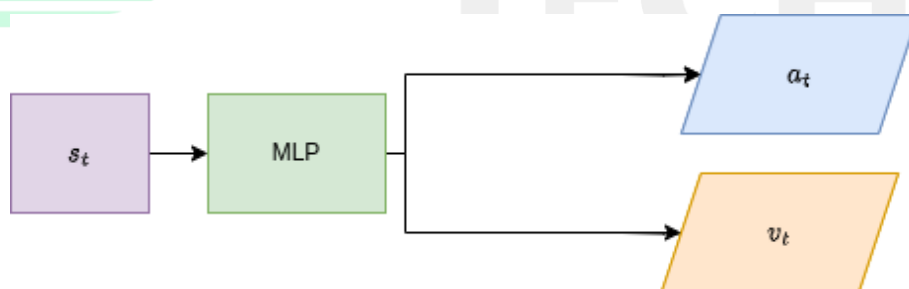


圖 4.4 Small Model

固定 Goal Agent 的訓練數據，僅調整 Spot Agent 模型架構。如表 4.1 所示，Normal Model 效果最佳，Policy Large Model 次之，Small Model 效果最差。此結果表明策略網絡需要更深的網絡結構以提升效能，而過於簡化的模型會導致效能顯著下降。

表 4.1 不同網路測試效果

架構測試	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
Normal Model	2.26	0.87
Policy Large Model	1.53	0.9
Critic Large Model	0.73	0.77
Small Model	0.13	0.83

鑑於 Small Model 效能不佳，本論文進一步測試將策略網路和評價網路合併為單一支的架構，並稱此架構為 **Merge Mode**，此架構在紫色的模塊採用 MLP 架構，分別為全連接層[1024, 512, 192]，如圖 4.5 所示。

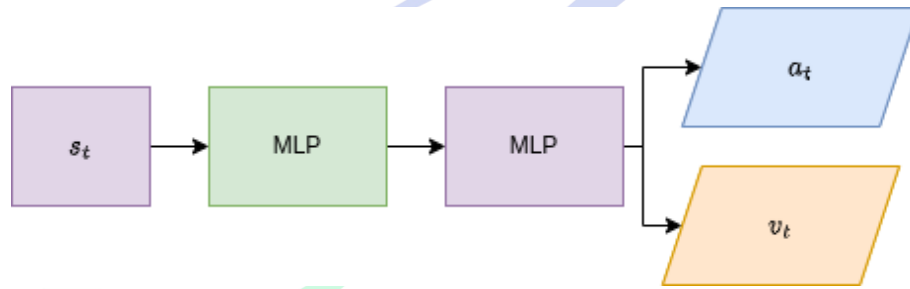


圖 4.5 Merge Model

Merge Model 參數會比 Normal Model 更多，但實際的訓練效果顯示 Merge Model 才是效果最差的模型，如表 4.2 所示。由此推斷策略網路和評價網路不應該連接在一起，它們的更新過程會相互干擾。

表 4.2 Merge Model 效果測試

架構測試	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
Normal Model	2.26	0.87
Small Model	0.13	0.83
Merge Model	0.07	1

本論文架構

為了進一步驗證獨立分支的優勢，本論文修改 Normal Model，將策略網絡和評價網絡的共用架構拆除，改為神經原本總數量一致的策略網絡和評價網絡，細節為策略網絡的全連接層[256, 256, 256, 512, 256, 128, Action]和策略網絡的全連接層[256, 256, 256, 512, 256, 64, Value]，如圖 4.6 所示。

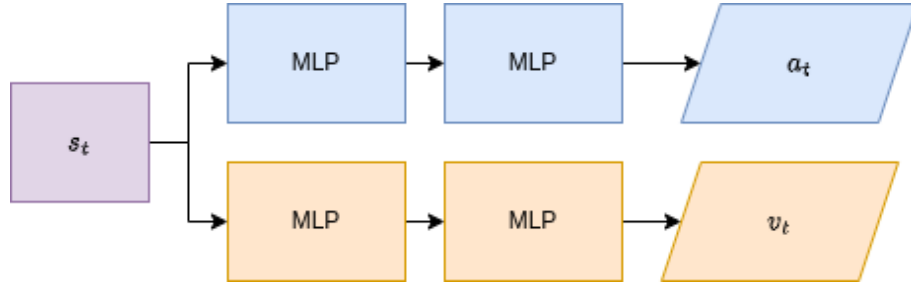


圖 4.6 Split Model

如表 4.3 所示，Split Model 強化了策略網絡和評價網絡的獨立性。證明 Split Model 效能最佳，佐證了策略網絡和評價網絡獨立分支的有效性。

表 4.3 分裂架構效果測試

架構測試	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
Normal Model	2.26	0.87
Split Model	2.63	0.67

此外，本論文也測試網絡寬度對模型效能的影響。將 Normal Model 每層神經元數量減半，架構不變，如圖 4.7 所示。

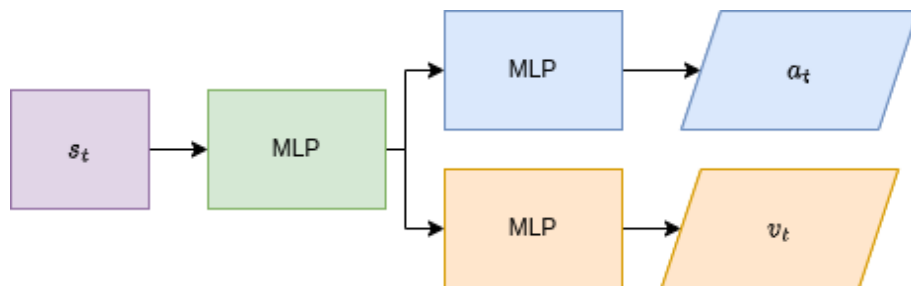


圖 4.7 Thin Model

策略與評價網絡的共用層將三層全連接層從[512, 512, 512]改為[256, 256, 256]。

策略網絡獨有層將四層全連接層[512, 256, 128, Action]改為[256, 128, 64, Action]。

評價網絡獨有層將四層全連接層[512, 256, 64, Value]改為[256, 128, 32, Value]。

如表 4.4 所示，網絡寬度較低模型的效能下降，但差別較小，表明網絡深度對模型效能的影響遠大於網絡寬度。

表 4.4 網絡寬度測試

架構測試	$\rho(\theta_{phase3})_{Spot-Goal}$	$p(\theta_{phase3})_{Spot-Obs}$
Normal Model	2.26	0.87
Thin Model	1.77	0.77

4.2 演算法的修改

4.2.1 PPO 演算法

根據論文[39]的描述，RL 的目標是訓練出一個 Model，使得在給定的狀態下可直接生成最佳行為。該模型稱為策略網路，亦即 Model θ 。在 RL 環境中，需計算不同 Action 的機率分佈 p_{θ} ，如(4.1)式所示。

$$p_{\theta} = (a|s) \quad (4.1)$$

為了訓練策略網路，需利用 RL 的 Reward 計算不同 Action 對應的 Reward 期望值，如(4.2)式所示。

$$\nabla R_{\theta} = \sum_{\tau} R(\tau) \nabla p_{\theta}(\tau) = \sum_{\tau} R(\tau) p_{\theta}(\tau) \nabla \log p_{\theta}(\tau) \quad (4.2)$$

Reward 最大化過程中，通過微分 Reward 期望值可更新模型參數 θ 。其中， $R(\tau)$ 為設計的 Reward 值，因此微分只需對 p_{θ} 項目進行操作，最終更新模型 θ ，(4.3)式所示。

$$\nabla R_{\theta} = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} R(\tau^n) \log p_{\theta}(a_t^n | s_t^n) \quad (4.3)$$

訓練過程中，通過不斷迭代更新 Model θ ，策略網路便能學習到更高 Reward 的行為策略，如(4.4)式所示。

$$\theta_{t_{n+1}} = \theta_{t_n} + \mu \nabla R_{\theta} \quad (4.4)$$

μ ：代表學習率

4.2.2 A2C 演算法

根據論文[40]的描述，A2C 對 PPO 進行改進，使得 Reward 期望值估計更準確。其改進方法是僅估測到下一個時刻的 Reward 差值，減少未來不確定性帶來的誤差，如(4.5)式所示。

$$\nabla R_{\theta} = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} R(\tau^n) \log p_{\theta}(a_t^n | s_t^n) \Rightarrow \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} (r_t^n + V^{\pi}(s_{t+1}^n) - V^{\pi}(s_t^n)) \nabla \log p_{\theta}(a_t^n | s_t^n) \quad (4.5)$$

r ：代表 Agent 得到的 Reward

A2C 估計 Reward 的方法為訓練一個 Model π ，判斷在當前 State 下 Reward 期望值 V ，如(4.6)式所示。

$$V^{\pi}(s) = Q^{\pi}(s, \pi(s)) \quad (4.6)$$

如(4.7)式所示，訓練方法參考 Q-learning，只是 Q-learning 沒有策略網路，所以評估的期望值是用 π 決定後面的 Action 的期望值，而 A2C 演算法會計算使用策略網路決定後面的 Action 的期望值。

$$Q(s_t, a_t) = E[r_t^n + V^\pi(s_{t+1}^n)] \quad (4.7)$$

E ：在此表示期望值

4.2.3 演算法改進方法

本論文以 A2C 演算法為主，並進行了一些修改。由於本論文採用了 Multi-Env RL 與 Multi-Agent RL 的訓練方法，因此訓練資料量遠超一般 RL 訓練，batch size 在三萬筆數據以上進行訓練。因此，常見的演算法改進方法在本研究中的效果並不明顯，需要針對大數據量進行專門的優化調整。

Target Network

論文[41]針對 Q function 的訓練提出了 Target Network 的更新策略。該方法透過先固定評估 Reward 期望值的評估網路，並對策略網路進行多次更新後再替代評估網路。Target Network 的主要目的是降低 Q function 在更新時的模型震盪。然而，儘管本論文在訓練過程中也觀察到一些震盪現象，但由於更新評估網路的頻率減少會使訓練速度下降，因此本論文未採用此方法。

Replay Buffer

論文[42]中 Replay Buffer 的技術將評估網路的訓練數據進行存儲，並在訓練時從中隨機選擇樣本用於訓練評估網路。這種方法不僅能減少數據收集時間，還能增強樣本的多樣性。本論文也應用了此技術，然而受限於 GPU 記憶體容量，儲存的訓練數據量相對較小，只比基礎數據稍多。本質上與 Replay Buffer 所期望的藉由大量資訊來增強評估網路訓練的目的是一致的，但在數據量上有所限制。

Double DQN

論文[43]提出了 Double DQN 構架，引入了兩個獨立的 Q 值估計器：一個用於選擇最佳動作，另一個則負責評估所選動作的價值。這樣可以有效減少估計過程中出現的過度樂觀估計，從而提高算法的穩定性與性能。本研究則已經採用了 Boltzmann 探索機制，使行動選擇不僅僅基於最大化估計回報，而是考慮多種可能的 Action 概率分佈，從而避免過度樂觀估計的問題。因此，本論文並未額外引入 Double DQN 架構，以免增加算法的複雜度，同時保持訓練過程的穩定性。

本論文演算法

基於以上的分析，本論文發現 RL 訓練在面臨大量訓練資料的情況下，與傳統 RL 訓練的問題不同。由於數據量大，本論文並不過於關注 Q 函數的更新精度，因為大量數據能緩解 Q 函數估算不準的問題。真正需要解決的挑戰在於，Multi-Task RL 訓練有時會生成不適合訓練的場景，進而導致模型損壞，因此本論文對算法進行了以下幾項修改。

參考論文[44] PPO2 問題的解決方式，此演算法需要解決的問題是在 Off-policy 的訓練條件下，如何讓真實收集資料時 Action 不要和訓練 RL 時 Action 的機率相差太多，如(4.8)式所示。

$$J_{PPO2}(\theta) = E_{(s_t, a_t) \sim \pi_\theta} [\min(r_t(\theta) \times A_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \times A_t)] \quad (4.8)$$

ε ：代表一次能接受的修改率

A_t ：Advantage function 表示當下動作 a_t 相對於平均策略的好壞程度

本論文為解決 Action 策略更新前後的偏差過大，本論文參考了 PPO2 方法以解決 Off-policy 訓練中 Action 不穩定的問題。為避免模型在更新後和更新前的策略出現過大偏差，本論文限制了 Loss function 的更新上限，如(4.9)式所示。該限制並非直接約束 Action 機率，而是控制策略網路更新的幅度，以

達到相近的穩定效果，確保模型更新過程不至於偏離訓練策略。

$$\theta_{t_{n+1}} = \theta_n + clip(-k, \mu_1 \nabla R_\theta, k) \quad (4.9)$$

$k=8$ ：代表 policy network 一次能接受的梯度修改

對於評估網路，本論文遇到的主要問題是 Multi-Task RL 訓練有時生成不合理的場景（例如，樓梯重疊），導致評估網路出現極大的 Loss，這樣的情況下進行大量更新並不合適。然而，並非所有的高 Loss 都表示訓練環境問題，因為當新的策略成功達成目標時也會產生較大 Loss，直接將較大的 Loss 降低也會影響訓練效率。

為了平衡更新，本論文僅限制評估網路的最大更新幅度，並根據策略網路的更新量動態調整評估網路的更新上限，以防止因評估網路的錯誤更新而影響策略網路策略，如(4.10)式所示。

$$\pi_{t_{n+1}} = \pi_{t_n} + \begin{cases} \mu_2 \nabla V^\pi, & \text{if } |\pi_{loss}| < q \\ \min(\mu_2 \nabla V^\pi, q, \theta_{loss}), & \text{if } |\pi_{loss}| > q \end{cases} \quad (4.10)$$

$q=30$ ：代表 critic network 一次能接受的梯度修改

訓練策略網路和評估網路，有時會遇到神經網路訓練有問題，所以為了觀察策略網路和評估網路訓練狀況，本論文設計了一個觀察用數據 module entropy，如(4.11)式所示。

$$module\ entropy = \log_{10}(|\mu_1 \nabla R_\theta|) - \log_{10}(|\mu_2 \nabla V^\pi|) \quad (4.11)$$

此參數能直觀反映兩個網路的訓練狀況，不同範圍的 module entropy 值代表了以下意義：

module entropy > 1：

表示策略網路更新過大，而評估網路更新不足。這種情況若在訓練開始時出現，表示策略網路學習率過高，應適當降低策略網路的學習率；若

出現在訓練後期，代表評估網路可能到達學習瓶頸，繼續訓練難以有效提升。

module entropy ≈ 0 :

此值顯示策略網路和評估網路均在有效訓練，表示最佳訓練狀態。

module entropy ≈ -2 :

表示策略網路訓練已達上限，而評估網路仍有學習空間。當 A2C 訓練中評估網路的估計不夠準確時，策略網路亦難以達到最佳效果，此時需要調整 SAC 中的 entropy，讓策略網路更具行動分佈的多樣性，以提供更有效的訓練數據。

module entropy < -3 :

此情況下僅評估網路在訓練，而策略網路無有效梯度，這意味著 RL 環境設置有誤，Agent 訓練無法有效進行。

4.3 資訊延遲

訓練 RL 的策略，State 選擇最準確且清楚的輸入資訊，可以讓 RL 的策略的效能最佳化，但這並不意味整體系統也達到最佳化。本論文動態障礙物的資訊表示為 Spot 座標系中的位置、速度與大小，這些資訊在 RL 訓練過程中易於取得，但在現實應用中需經過額外的系統運算才能生成，這導致其他系統中產生的資訊延遲會影響了 RL 策略的執行速度與行為判斷。因此，本小節會介紹本論文如何根據系統優化 RL 行為規劃策略，增加整體效能。

4.3.1 資訊延遲的影響

在實際測試過程中，資訊延遲對 RL 行為規劃策略的影響尤為關鍵。特別是在處理動態障礙物時，系統需整合多源感測資訊（如影像、雷達或 MOT 模組）以預測障礙物位置，而此預測過程通常需耗時約 0.5 秒。在此期間內，障礙物實際位置可能已經發生明顯變化，造成系統所獲得的資訊無法即時反映現況。

如圖 4.8 所示，障礙物的擬合位置與實際位置出現偏差。這樣的誤差不僅無法提供行為規劃系統有效輔助，反而可能導致策略做出錯誤決策，進而增加碰撞風險或導致路徑選擇失誤。

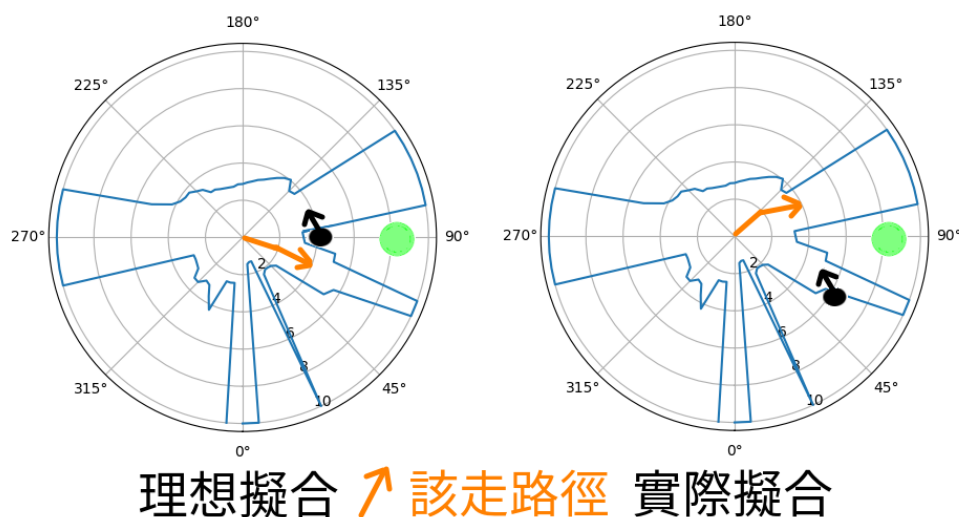


圖 4.8 障礙物擬合示意圖

雖然現代追蹤系統的運算效能已大幅提升，但在多任務並行處理與複雜環境下，延遲仍難以完全消除。而即便延遲僅為 0.3–0.5 秒，對於需即時決策的強化學習策略而言，擬合資訊時間的不同，也會對 RL 行為規劃策略產生重大影響。此外，目前尚無延遲足夠低、整合效果穩定的動態障礙物追蹤系統可直接應用於本論文實作場景中。

為解決這一問題，本論文移除動態障礙物預測的系統，僅使用點雲資訊轉換得到的可行使範圍資訊作為輸入，統一交由 RL 策略處理。此方法避免了因延遲導致的資訊錯位問題，並簡化了系統流程。如圖 4.9 所示，經過優化的系統流程更加高效，減少了不必要的計算環節。

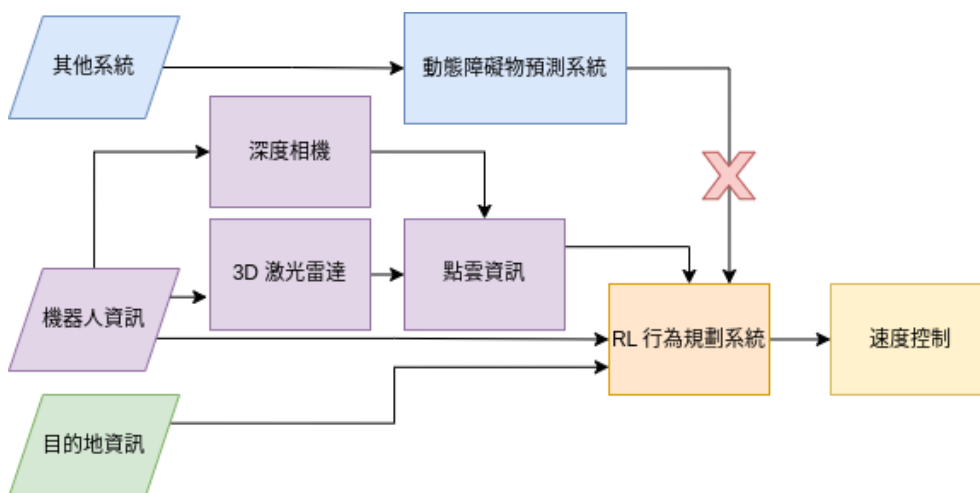


圖 4.9 系統流程圖的變化

要訓練出具備避開動態障礙物能力的 RL 行為規劃策略，依然需要在 RL 訓練中，加入動態障礙物的避障訓練。如圖 4.10 所示，本論文將 S_{Obs} 的資訊轉換成的 $S_{Sweepangle}$ 來表示障礙物的資訊，這雖然無法區分靜態與動態障礙物，但實驗結果顯示，Spot 依然能夠根據單一時序的 $S_{Sweepangle}$ 資訊，訓練出能穩定的避障策略。

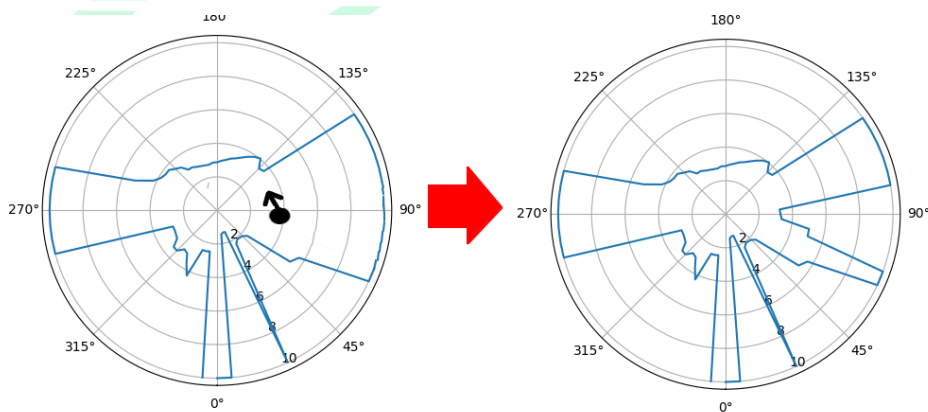


圖 4.10 S_{Obs} 轉 $S_{Sweepangle}$ 示意圖

4.3.2 RNN Model 的嘗試

儘管簡化輸入能夠提高系統穩定性，本論文仍進一步嘗試將 RNN 模型融入 RL 行為規劃系統，希望透過歷史資訊，改進 Spot 對於動態障礙物位置與 Spot 行

走路徑等資訊的判斷。在架構設計上，將 RNN 的輸出作為 RL 模型的輔助輸入，
如圖 4.11 所示。

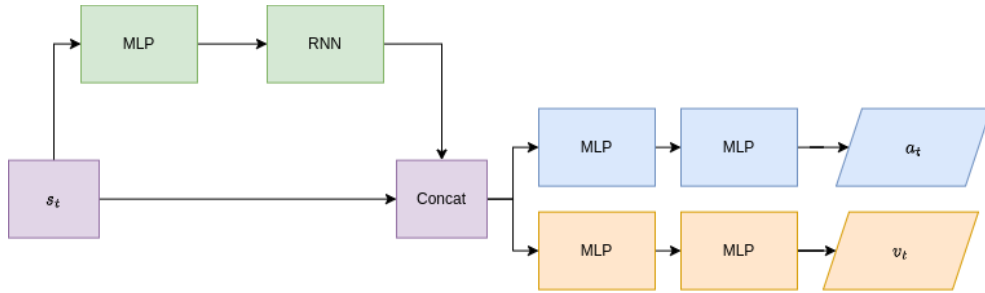


圖 4.11 RNN RL Model

如圖 4.12 至圖 4.13 所示，實驗數據將 RNN RL Model 與原本 RL Model 的最優策略進行比較。紫色曲線表示 RNN RL Model 的數據，紅色曲線則代表本論文在 RL Model 的最優策略數據。實驗條件保持完全一致，並以 Spot 取得環境中所有 Goal Agent 的機率作為衡量標準，數值越小代表策略性能越好。

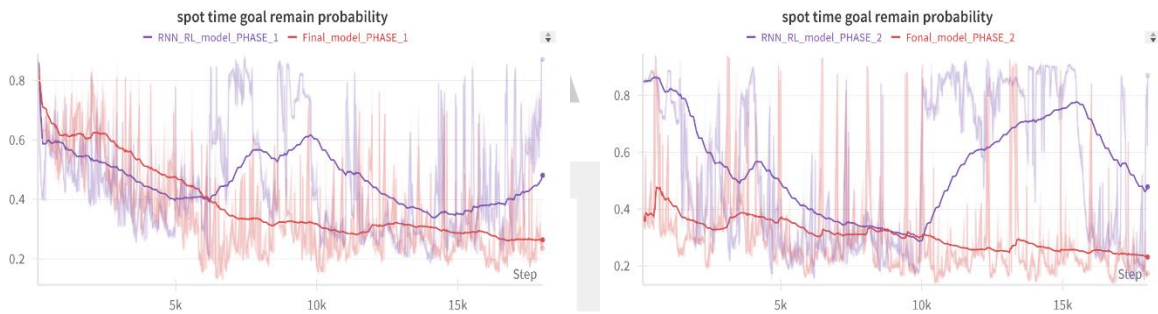


圖 4.12 第一和第二階段訓練數據

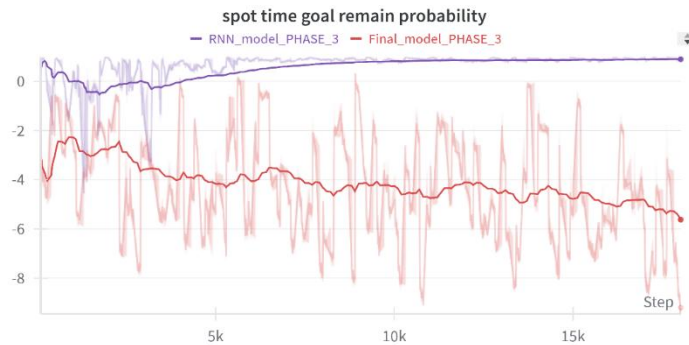


圖 4.13 第三階段訓練數據

經過數月對於 RNN RL Model 細節的嘗試，數據結果都非常相似。結果顯示，在 Spot 訓練抵達目的地的初期階段（Phase 1 - Phase 2），RNN RL Model 經常出現突發性技能退化現象（如圖 4.12 中的劇烈震盪），即 Spot 無法穩定完成抵達目的地的任務，且這種退化現象並非總能恢復。在進入更高階段（Phase 3）的上下樓梯能力訓練時，當 Goal Agent 減少至一個時，RNN RL Model 的行為規劃策略徹底失效。

對模型的詳細分析表明，RNN 的隱藏狀態 h_n 未能有效記錄相關歷史資訊，而輸出 r_{out} 的數據波動則明顯干擾了 RL Model 的穩定學習。與原本的 RL Model 相比，RNN 的引入不僅未提升性能，反而導致訓練過程出現更高的不穩定性。

4.3.3 模塊化神經網路

發現 RL 演算法難以直接反推出 RNN Model 所需的訓練數據後，本論文採取了模塊化設計的策略，以 RNN 模型訓練出 RL Model 所需的 State 作為核心目標，探索模塊化神經網路的效果。如圖 4.14 所示，將動態障礙物的預測獨立為一個模塊（綠色的網路），專門負責生成下一時刻的動態障礙物資訊。

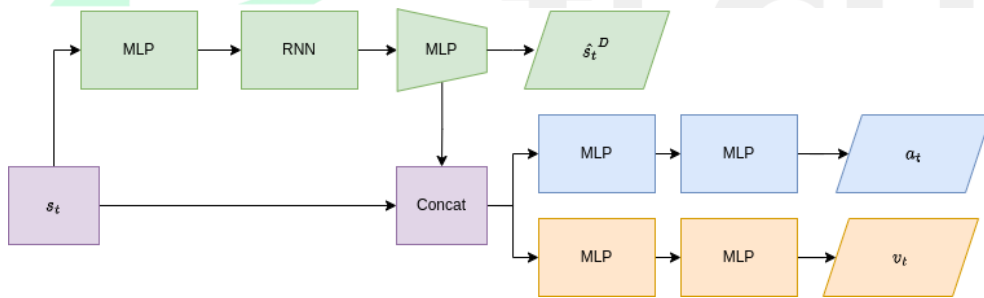


圖 4.14 Module RNN Model

RNN 模塊輸出 $\hat{s}_t^D$ ，通過輸入的靜態障礙物資訊，預測最近的兩個動態障礙物在下一時刻的位置與距離，如(4.12)式所示。

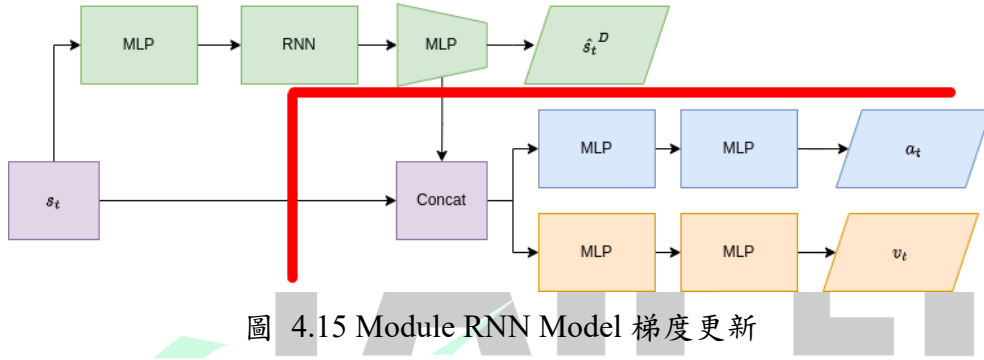
$$(\hat{s}_t^D, h_t) = f_{\xi}(s_t, h_{t-1}) \quad (4.12)$$

ξ ：代表 RNN 的架構

在訓練過程中，動態障礙物的真實數據作為訓練目標，其包含相對 Spot 當前位置的下一時刻資訊，如(4.13)式所示。

$$s_t^D = \{Obs_x^{t+1}, Obs_y^{t+1}, Obs_{distance}^{t+1}\} \quad (4.13)$$

RNN 模塊的梯度回歸，如圖 4.15 所示。在訓練動態障礙物預測模塊時，其它網路的更新率被設為零，以避免互相干擾。同樣地，在更新策略模塊時，動態障礙物模塊的參數保持不變。這種設計確保了模塊之間的功能獨立性。在實現方式上，策略模塊僅取動態障礙物模塊輸出的前一層結果作為輸入，兩者互不直接干涉。



RNN 模塊的損失函數設計考慮到障礙物的相對重要性，由於靜態障礙物資訊可能不包含所有動態障礙物的數據，且距離較近的障礙物影響更大，因此損失函數如(4.14)式所示。

$$\mathcal{L}_{dyn}(\xi) = -\ln \left| f_{\xi}(\hat{s}_t^D | s_t, h_{t-1}) - s_t^D \right| \quad (4.14)$$

如表 4.5 所示，儘管 RNN 模塊在動態障礙物的數據上與 RL Model 的表現無明顯差異，但在抵達目的地能力上有顯著不同。測試顯示，當面對平面上的動態障礙物時，兩種行為規劃策略均能有效應對，但包含 RNN 模塊的網路對障礙物位置的判斷更加精確，因此行為更加簡潔；而不含 RNN 模塊的網路由於策略更為保守，面對樓梯或擁擠場域時，表現不如包含 RNN 模塊的網路。

表 4.5 RNN 模塊測試

測試架構	$\rho(\theta_{phase7})_{Spot-Goal}$	$p(\theta_{phase7})_{Spot-Obs}$
Module RNN Model	4.1	0.2
Split Model	3.6	0.26

4.3.4 RL Model 的設計結論

從 State 和 RNN 實驗中可以得知，RL Model 本身在理解輸入資訊的能力不佳。RL 演算法要求 Agent 從不同動作中選擇最優策略，然而此過程會產生樣本差異極大的訓練數據。讓 Agent 僅依賴 Reward 差異分析輸入數據並推導策略，這並不是 RL 演算法的特長，因此需要額外的神經網路來解析輸入資訊，然後再由 RL 計算策略以達到更佳效果，如圖 4.16 所示。

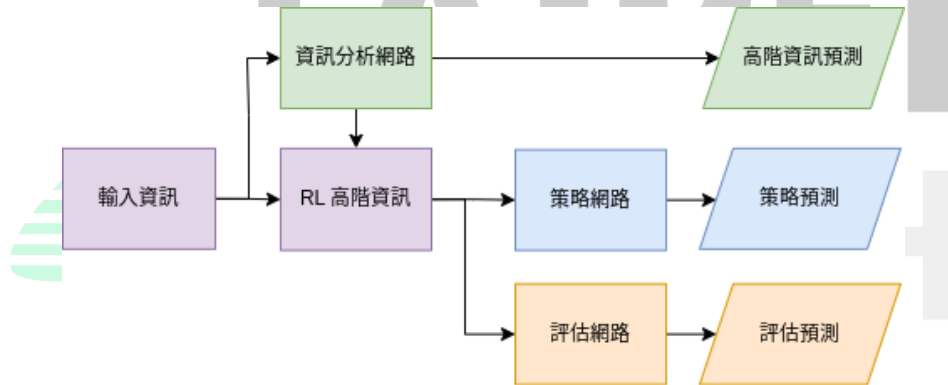
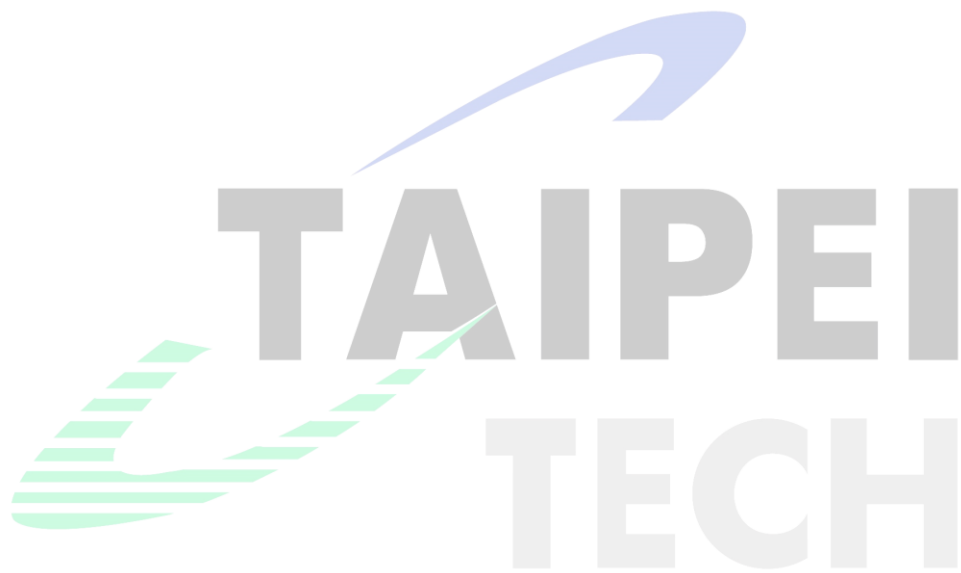


圖 4.16 本論文 RL Model 任務分工

本論文認為原始 A2C 模型考慮了不同資訊處理方式，將前端網路用於提取低階資訊的關鍵特徵，例如 CNN 用於圖像處理。然而，本論文的輸入資訊已高度精煉，屬於高階資訊，因此無需額外的特徵提取步驟，所以在網路架構分析中能得出將策略網路和評估網路拆開的結論。在模塊化網路設計，若將本論文中的高階資訊預測移除，就能還原成最原始 A2C 模型設計。

資訊預測模塊的設計意圖在於彌補 RL Model 無法準確推理資訊的能力，優

點是能穩定訓練過程，但缺點是資訊利用的上限受限於預測結果，無法挖掘更多價值。因此，本論文認為根據任務需求選擇合適的模型架構，才是提升系統表現的關鍵所在。



第五章 實驗測試

在進行實際效能測試之前，本章節將介紹本論文的開發流程，包括從修改 RL 環境中的任意參數到最終在機器狗上執行的完整過程。由於 RL 訓練從開始到結束需要耗費約一周的時間，而 RL 的設計過程又極為複雜，因此如何在最短時間內找出可優化的項目是至關重要的。

如圖 5.1 所示，本論文的系統開發流程主要可分為三個部分。首先在 RL 訓練階段，透過強化學習方法訓練出具備良好行為規劃能力的策略。接著在系統測試階段，將訓練出的規劃策略整合至完整系統，並於模擬環境中進行測試，藉此降低實際測試過程中可能出現的風險，並確保整個執行過程的安全性。最後才進入實際測試階段，將系統部署至機器狗上，進行最終測試以驗證整體系統的性能表現。

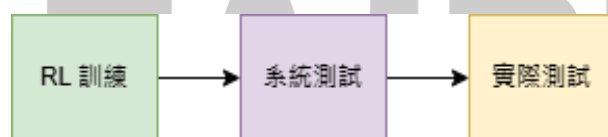


圖 5.1 RL 系統研發流程

為了縮短開發時間，本論文的開發流程如圖 5.2 所示。由於 RL 訓練過程耗時最長，因此需要在訓練不同階段測試各種問題，期望能夠在最短時間內找出問題，並解決問題。本章節內容將詳細說明各個時間節點中可觀察到的資訊，如何針對行為規劃策略進行優化，以及最終的測試結果。

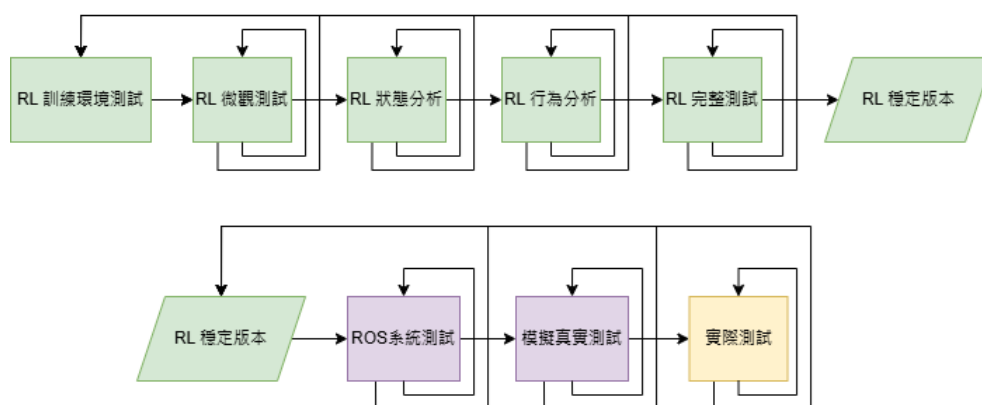


圖 5.2 實際研發流程

5.1 RL 優化方式

在 Multi-Phase RL 中為了訓練 Agent 的特定能力，需要調整 RL 環境中的參數 ϕ ，以提升 Agent 在此能力上的需求性，如(5.1)式所示。特定能力的機率 p_ψ 可通過 RL 環境計算而得，儘管在優化策略模型時具體 p_ψ 大小並不重要，但選擇合適的 RL 環境參數設定 ϕ 對於提升 p_ψ 則至關重要。

$$p_\psi = Env(\phi, \theta) \quad (5.1)$$

Env ：代表 RL 環境

為了優化 p_ψ ，需要分析如何修改 ϕ 能夠改善特定能力。這一過程因受 RL 環境整體設定及模型設計的影響，計算難度較非常高。然而在相同的 RL 環境設定下，若 Agent 訓練條件相似，則 p_ψ 也會非常相似。如(5.2)式所示，本論文通過對不同設計方案的參數取梯度，可以量化並比較不同設計方案的優劣。

$$p_\psi + dp_\psi = Env(\phi + d\phi, \theta) \quad (5.2)$$

此外，RL 訓練的初期，因為缺乏參考設計，第一個有效模型的訓練難度通常較高。在此階段，應降低訓練難度，以成功構建穩定的測試環境為首要目標，為後續優化奠定基礎。

本論文使用 **Wandb** [45]來管理整體工作流程和訓練數據。此工具能夠記錄 AI 訓練過程中的所有參數 ϕ 及初始 Model，藉此能達到優化 RL 的目的。此外，為了便於開發，Wandb 也能記錄不同版本的訓練程式及各階段的訓練數據變化，展示 Model 在不同訓練階段的執行結果，使我們能夠輕鬆比較不同訓練數據，並從以往的訓練成果中汲取經驗。

本論文中所有的機率 p 與期望值 V ，皆為訓練過程中由系統自動統計與可視化產出的資料點。為確保資料具統計代表性，每個數據點皆為約六萬筆 Agent 行為紀錄的平均值（即一個 batch）。橫軸的每一單位代表一次 iteration 的訓練，

因此在單一訓練階段內共進行了 30,000 次 iteration，最終累積約 30,000 筆訓練數據點。

訓練曲線中可觀察到的數據波動，主要來自於不同任務訓練環境的切換所造成的策略適應性挑戰。Agent 必須學會在多變的環境條件下維持穩定表現，才能使各項能力在訓練數據中呈現出趨勢性變化，並逐步學習出具泛化能力的策略。

5.1.1 Spot Agent 訓練數據

對於 Spot 來說，存在期望值($V_{Survive}$)、行為期望值(V_{Move})以及上下樓梯期望值(V_{Stair})是訓練中的三個重要指標。如圖 5.3 至圖 5.4 所示，本論文分別分析了不同情況下的 Reward 數據，以便更清楚地了解各參數對不同行為的訓練效果。



圖 5.3 存在期望值和行為期望值

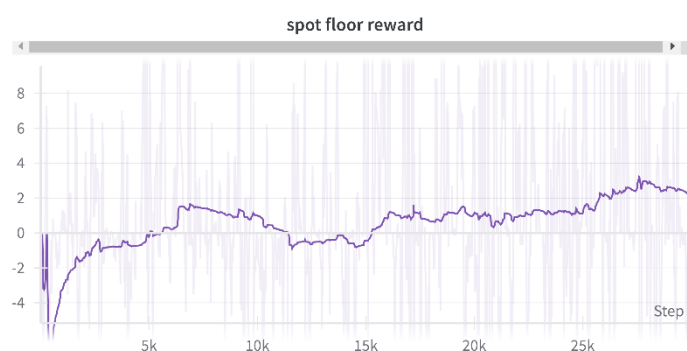


圖 5.4 上樓梯的期望值

如圖 5.5 至圖 5.7 所示，在 Spot 的訓練數據中，進一步分析了各類數據的

期望值，例如碰撞率($p_{Spot-Obs} + p_{Spot-Map}$)、靜態障礙($V_{R_{Spot-Map}}$)與動態障礙($V_{R_{Spot-Obs}}$)碰撞的比例等，以便評估 Spot 策略具體的造成原因。

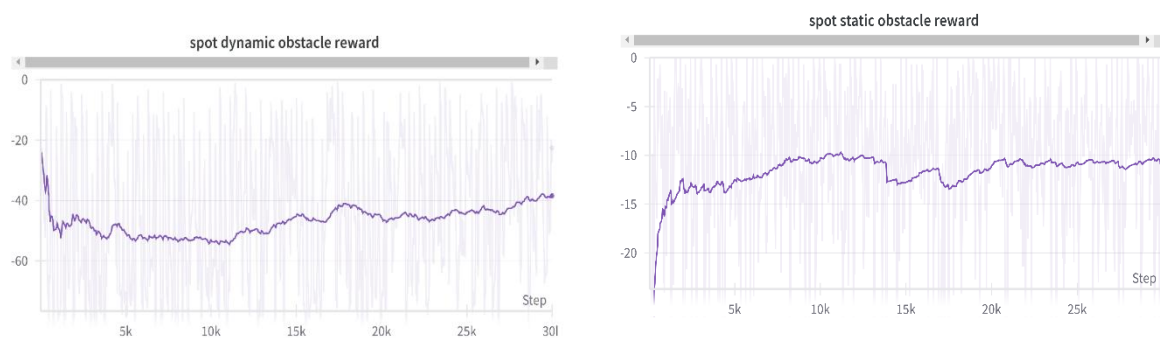


圖 5.5 Spot 碰撞動態和靜態障礙物的期望值

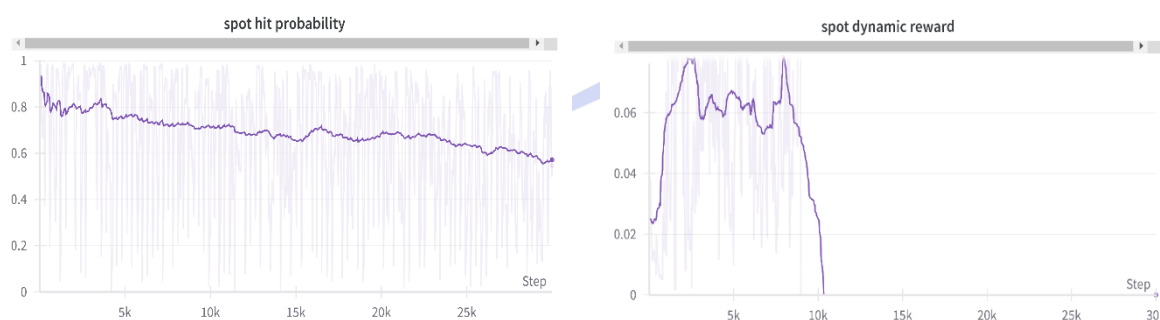


圖 5.6 Spot 執行完的存活率和 Action Reward 的期望值

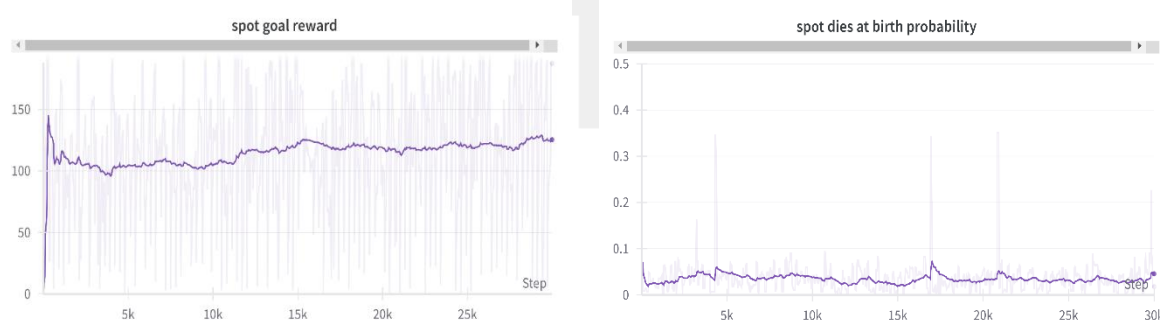


圖 5.7 Spot 的 $V_{Spot-Goal}$ 和環境生成時 Spot 的死亡率。

在訓練過程中，各 Agent 的剩餘存活率受到訓練環境參數 ϕ 的影響。由於 GPU 記憶體的限制，Spot Agent 與 Goal Agent 並非同時訓練，而是交替在不同環境中進行訓練，以此增加訓練 Agent 的數據量，並且還能對不同的 Agent 設定不同的訓練任務。如圖 5.8 至圖 5.9 所示，都是 RL 環境執行結束時，Goal Agent

剩餘的機率($p_{Spot-Goal}$)，數據差異就是訓練 Spot 和 Goal Agent 使用的環境參數 ϕ 不一樣導致的。

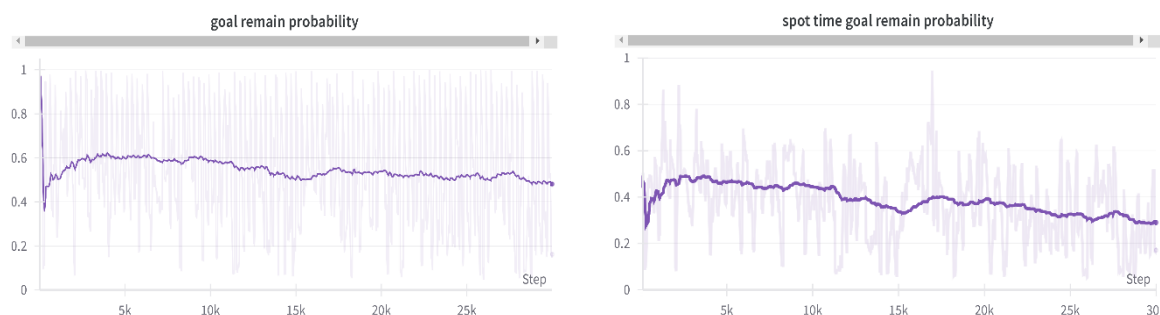


圖 5.8 全部訓練和 Spot 訓練時 Goal Agent 剩餘的機率

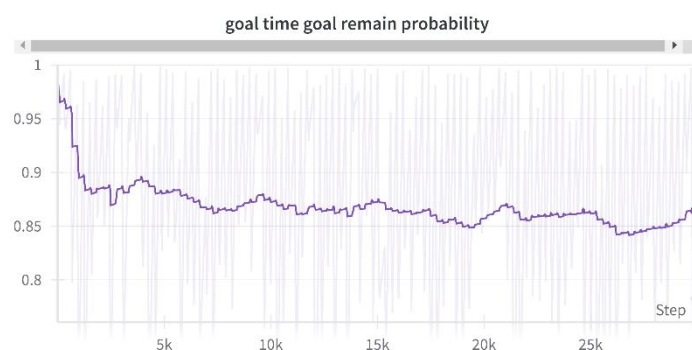


圖 5.9 在 Goal 訓練時 Goal Agent 剩餘的機率

如圖 5.10 所示，是訓練 Spot 的 Reward。由於環境參數 ϕ 的不同，Reward 並不一定在 Phase 2 大於 Phase 1，但在同一訓練階段中，Reward 的期望值是需要不斷增加(V_{Spot})。當調整環境參數 ϕ 能使期望值上升，並且在訓練階段中 Reward 有所提升時，即表示該訓練參數有助於增強特定能力的訓練，例如： $V_{Survive}$ 的提升代表抵達目的地能力的提升。

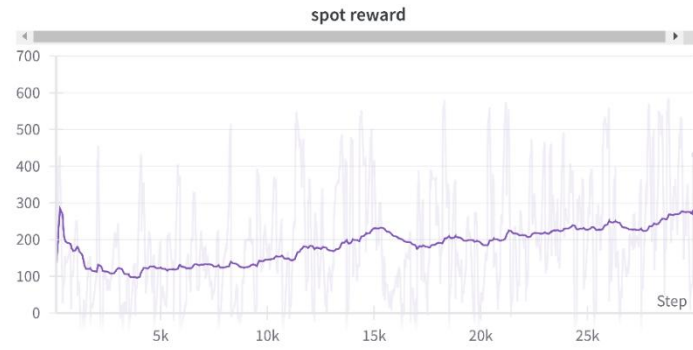


圖 5.10 總體的 Spot Reward

如圖 5.11 所示，RL 訓練中的 Loss 與其他 AI 訓練有所不同，Loss 並不一定隨著訓練時間縮小，有時甚至會隨訓練增大，這是正常現象。這是因為訓練數據分布不同導致的，RL 訓練資料由 Agent 遇到的 State 組成，策略網路在輸出更佳策略的同時會改變 State 的分布，因此策略網路和評估網路的 Loss 並不會持續下降。

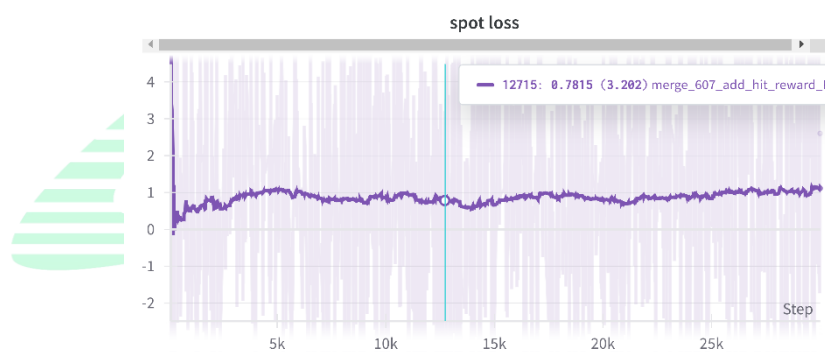


圖 5.11 Spot Loss

如圖 5.12 所示，訓練狀況無法透過 Loss 來判斷，因此本論文開發的 Spot module entropy 數據，能更細緻地分析策略網路和評估網路的訓練情況。



圖 5.12 Spot module entropy

5.1.2 Goal Agent 訓練數據

Goal Agent 的訓練重點包括上下樓梯(V_{Stair})、存活($V_{Survive}$)和移動等行為(V_{Move})期望值，這些期望值反映了其在不同情況下的行為表現和適應能力。圖 5.13 至圖 5.16 顯示了不同期望值的記錄情況，能幫助進一步分析和優化 Goal Agent 的訓練效果。

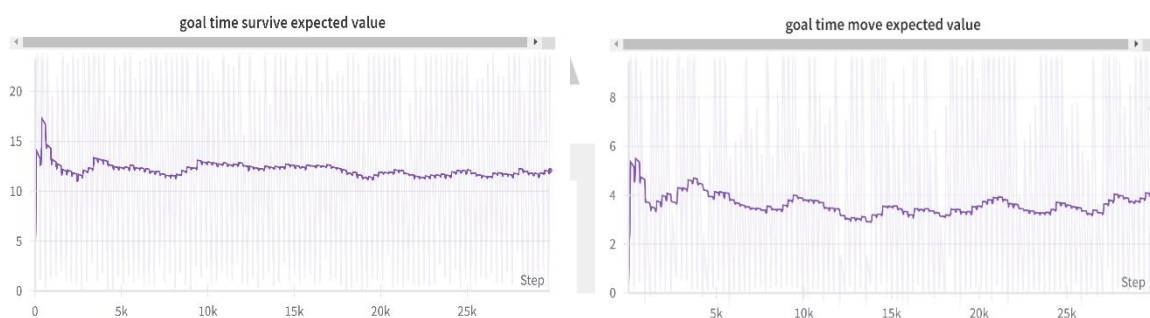


圖 5.13 Goal 存在期望值和移動行為的期望值

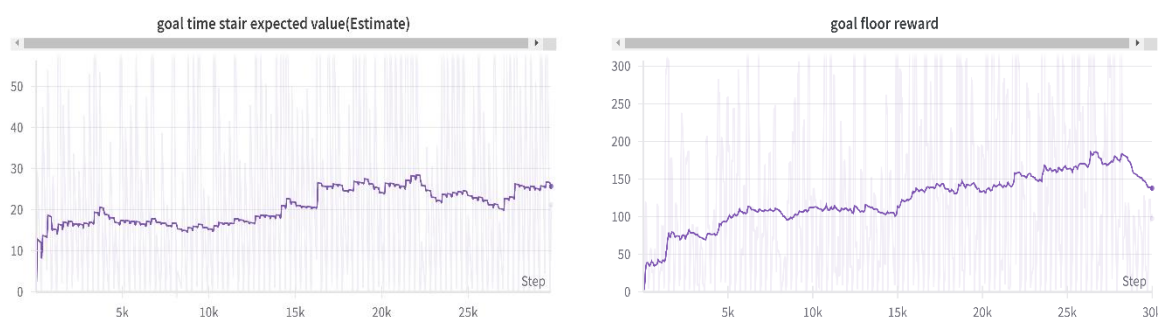


圖 5.14 Goal 的 V_{Stair} 和 V_{RStair}

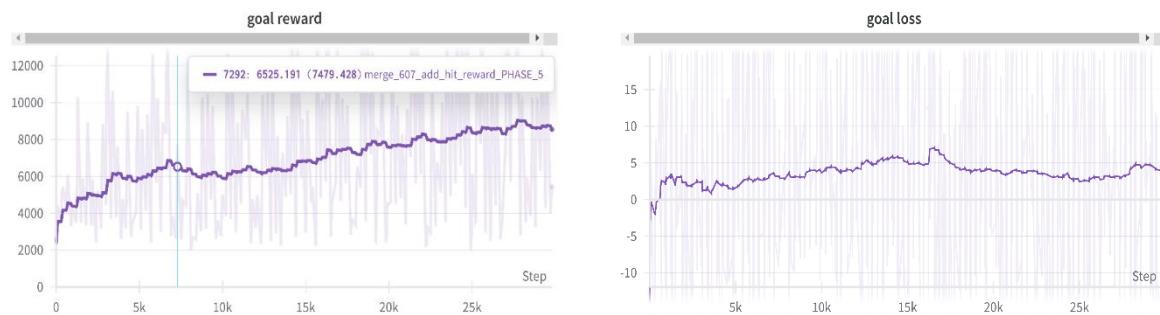


圖 5.15 Goal 的 Reward 和 Loss



圖 5.16 Goal module entropy

5.1.3 Obs Agent 訓練數據

動態障礙物 Obs 在訓練過程中完全不需考慮其他 Agent 的情況，主要集中於其自身的移動期望值。因此，通過觀察 Obs 的總體 Reward、Loss 和 module entropy 即可充分評估訓練效果，如圖 5.17 至圖 5.18 所示。

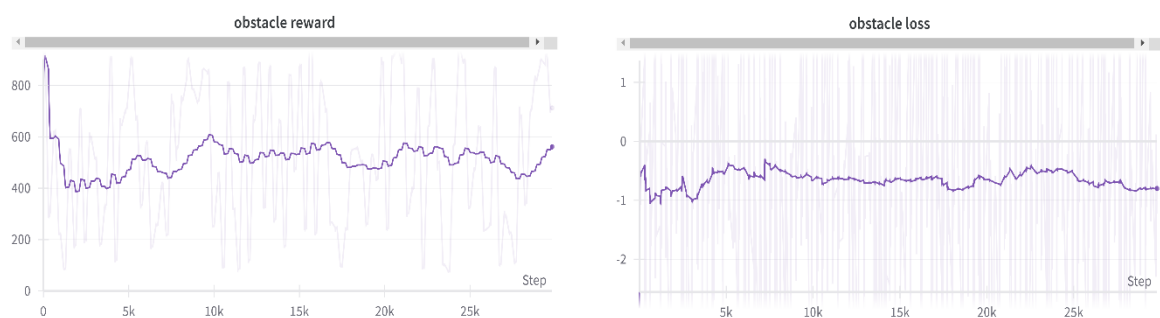


圖 5.17 Obs 的 Reward 和 Loss

如圖 5.18 所示，Obs 的 module entropy 相對於 Spot 和 Goal 更高，這意味著

Obs 的評估網路早已達到收斂，而策略網路的更新也僅是為了增加動作選擇的多樣性。

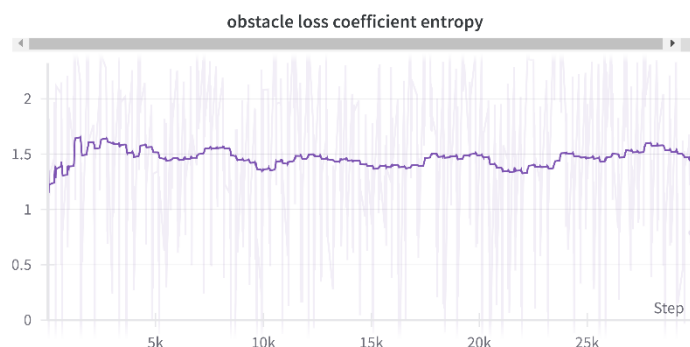


圖 5.18 Obs module entropy

5.2 RL 微觀測試

在數據分析過程中，考量到單一階段的訓練時間可能持續超過一天，因此對於不同的測試都等待整個訓練階段結束後再進行數據檢視並不可行。由於每個階段包含多達 300 個不同的 RL 環境，本論文在完成兩個 RL 環境訓練後便進行初步分析，此時 Spot 和 Goal Agent 都已經訓練過一輪完整的 RL 環境訓練。

通過觀察 Agent 的 Reward 變化和訓練效果，若獎勵數值顯示出正向提升，即可初步確認訓練狀況正常，如圖 5.19 至圖 5.20 所示。在這一階段，不僅可以發現一些系統常見的錯誤，也能及時進行調整。

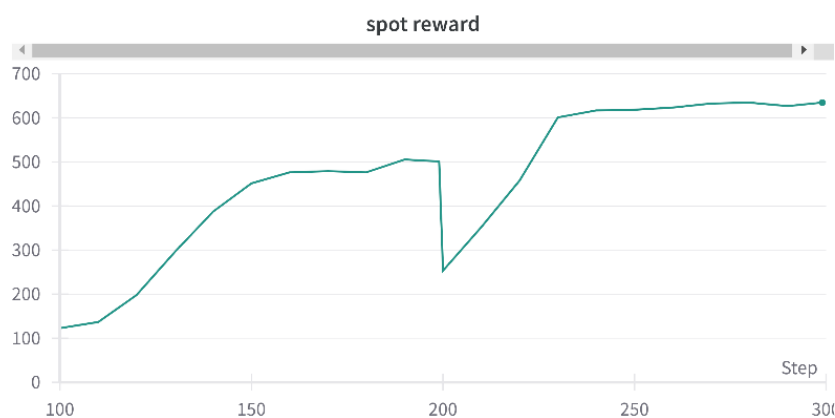


圖 5.19 Spot Agent 在短期訓練階段的效率

如圖 5.20 所示，通過觀察 Spot 到達目標地點的機率，可以看到單一訓練環境中的數據提升速度非常快。然而，這種改變並不能直接在真實測試中反映出效果，數據的變化通常是因為 Agent 策略更適應當前環境設置。因此，本論文才需要採用 Multi-Task RL 訓練策略，不斷更新訓練環境，使 Agent 能夠在更豐富的环境中學習。

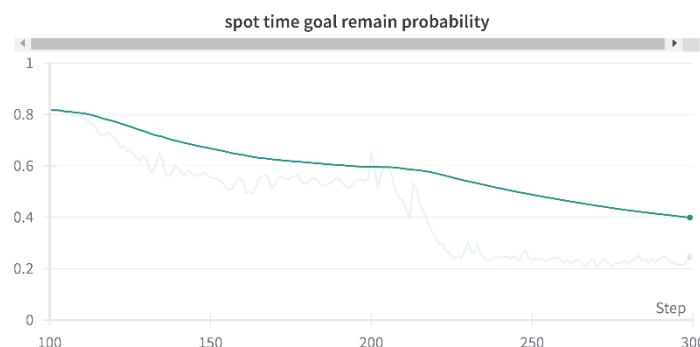


圖 5.20 Spot 訓練期間 Goal 剩餘比例的變化

5.3 RL 狀態分析

在 Multi-Phase RL 的訓練過程中，切換不同的訓練階段時，由於參數的改變，Agent 會對不同能地的重要性有新的判斷，常能在訓練開始時觀察到數據的變化。此時 Reward 的波動不一定反映實際效能的提升，而是顯示 Agent 的策略更適應新環境參數的設置，如圖 5.21 所示。

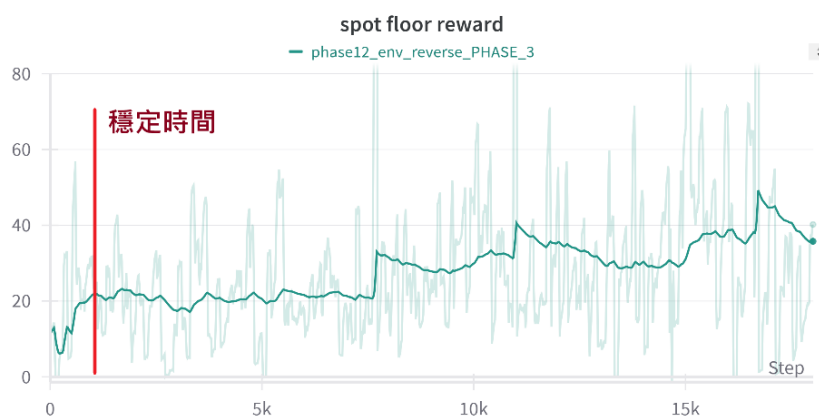


圖 5.21 Spot Phase 3 樓梯 Reward

為分析新階段訓練效果，採用訓練時間的前 10%作為初步分析點，這一階段數據已基本趨向穩定，便於觀察訓練參數的調整對目標能力優化的影響。若數據顯示出正向的影響，則說明後續訓練會繼續優化此能力；若影響有限，則後續訓練成效預計也將相對較小。在這一訓練階段中，僅通過數據分析有時可能受不同 RL 環境影響導致數據偏差，因此需要輔以有效的方式評估 Agent 的訓練狀態。

如圖 5.22 所示，為了快速的 RL 狀態分析，本論文會自動生成影片，其中每個 Agent 都會按訓練狀態進行移動，展示 Agent 實際的移動狀況，這不僅便於追蹤訓練進度，還能直觀呈現參數調整對 Agent 策略的影響。

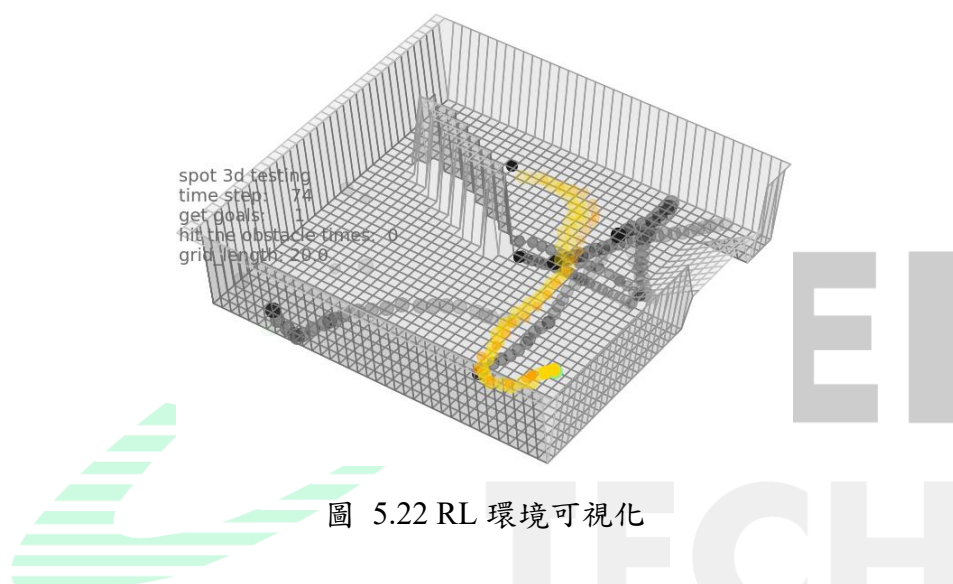


圖 5.22 RL 環境可視化

5.4 RL 行為分析

每組參數訓練一段時間後，模型通常會達到一個性能平衡點，此後進步速度會逐漸放緩。此時便需要透過切換至新的環境參數組(Multi-Phase 切換)，以調整 Agent 在不同環境中的使用不同能力的機率。本論文設定每訓練 300 或 180 個 RL 環境地圖後，便尋找下一組適合當前訓練階段的參數。

如圖 5.23 所示，每個階段訓練完成後，全面評估所改變的參數尤為重要。首先，本論文會根據 RL 優化過程中的各項數據進行分析，以辨識 $p_{\psi}(d\phi)$ 的差異，並根據參數調整分析數據變動的原因。

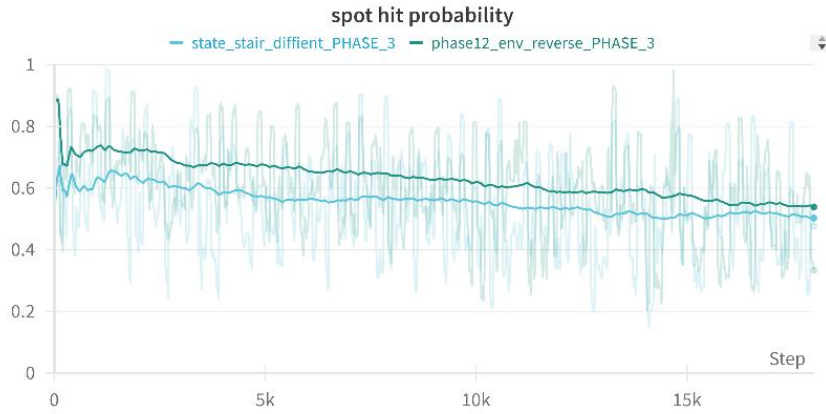


圖 5.23 $p_{\psi}(d\phi)$ 的差別

如圖 5.24 所示，在進行訓練階段效果分析時，還需要藉由視覺化工具分析。本論文將訓練過程分為十個階段，為每段生成影片，以直觀地呈現訓練成果，並顯示抵達目的地的機率、碰撞機率等。這樣可更清晰地觀察 Agent 的能力隨著訓練的演變過程。

	state	video	weight	Get Goals Probability
1	1231		merge_607_add_hit_re	0.875
2	1261		merge_607_add_hit_re	0.0625
3	1291		merge_607_add_hit_re	0.1875
4	1321		merge_607_add_hit_re	0.375
5	1351		merge_607_add_hit_re	0.875
6	1381		merge_607_add_hit_re	0.375

圖 5.24 可視化訓練階段的成果

如圖 5.25 所示，由於 Multi-Phase RL 的修改成功率並不高，能夠重現以往的訓練成果至關重要。本論文使用 Wandb 平台的版本管理功能，儲存每次訓練的參數、數據、程式碼與模型權重，便於選取任何版本作為新訓練的初始值，以比較不同修改的效果。這一方法有助於更精確地追蹤訓練進展，並根據即時情況調整訓練參數，確保模型性能在長期訓練中持續提升。

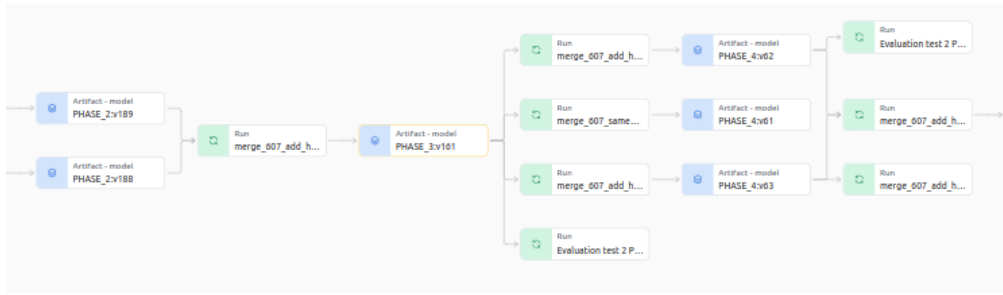


圖 5.25 Wandb 版本管理

5.5 RL 完整測試

訓練完 RL 的行為規劃策略，需對訓練結果進行全面測試。測試場景涵蓋：空曠地形、狹窄地形、障礙物密集地形、上下樓梯，以及目標不可達的情況。測試中每種情況選取四個地形進行測試，並對每個地形執行十次測試，取平均值作為數據依據，以直觀呈現策略在不同情境下的表現。

Goal Agent 移動測試

在測試過程中 Goal Agent 會持續移動，Spot 需要追蹤並抵達 Goal Agent 的位置。為了呈現行為規劃策略的執行過程，本論文使用軌跡圖來記錄 Spot 的移動路徑。起始位置以顏色最淺的區域標示，執行結束的地方則顯示為顏色最深，這種視覺化方式直觀地展示了 Spot 的移動軌跡，如圖 5.26 所示。

在軌跡圖中，不同 Agent 以顏色和標記區分：黃色點代表 Spot 在特定時間的位置，橘色點標示其頭部方向，綠色點表示 Goal Agent 的位置，黑色點則顯示動態障礙物的位置。如果 Spot 在運行過程中與障礙物發生碰撞，軌跡將立即中斷，並顯示此次測試結束。這樣的展示方式既能反映策略執行的效率，也便於觀察各種測試條件下 Spot 的行為反應。

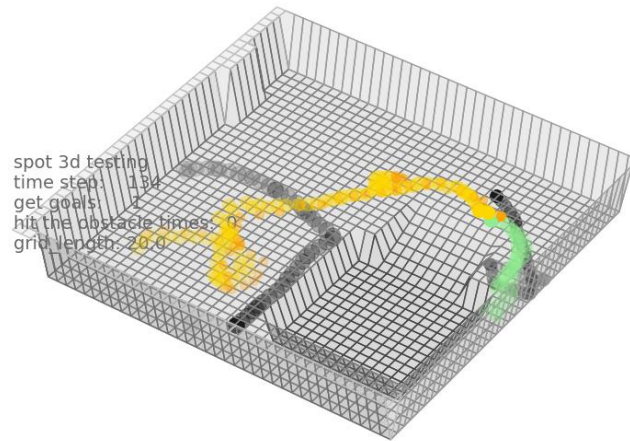


圖 5.26 RL 測試環境

然而，當 Goal Agent 處於移動狀態時（訓練時是移動的），Spot 在不同次的測試中需要啟用的能力會不一樣，這會使得行為規劃策略的數據分析變得更加複雜。如圖 5.26 至圖 5.28 所示，顯示同一地圖中 Spot 有時需要上下樓梯的能力，有時只需要抵達目的地的能力，為了簡化後續的測試並方便觀察 Spot 的行為策略，本論文將測試環境中的 Goal Agent 速度設定為零，使測試結果更具可比性。

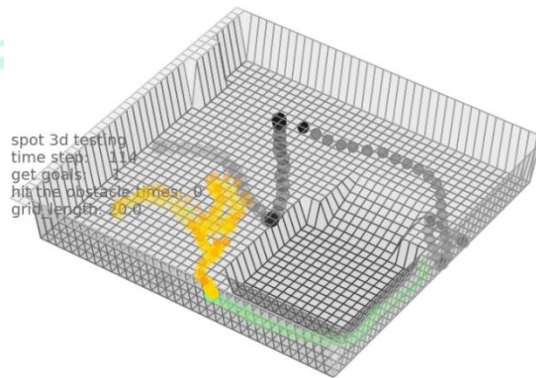


圖 5.27 Goal Agent 移動測試，需要抵達目的地的能力

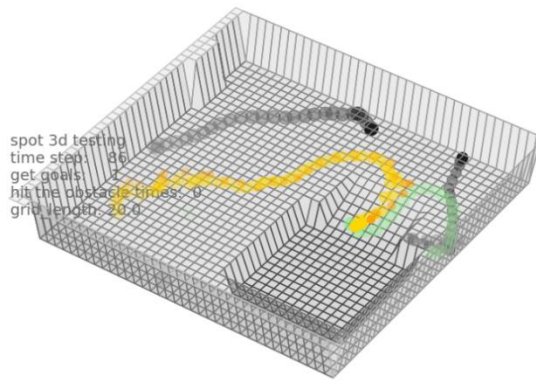


圖 5.28 Goal Agent 移動測試，需要上下樓梯的能力

在 Goal Agent 移動的情況下，Spot 也能夠順利抵達目的地，這一測試驗證了行為規劃策略在動態場景中的可靠性，但靜態測試更適合進行數據比對和效果展示。

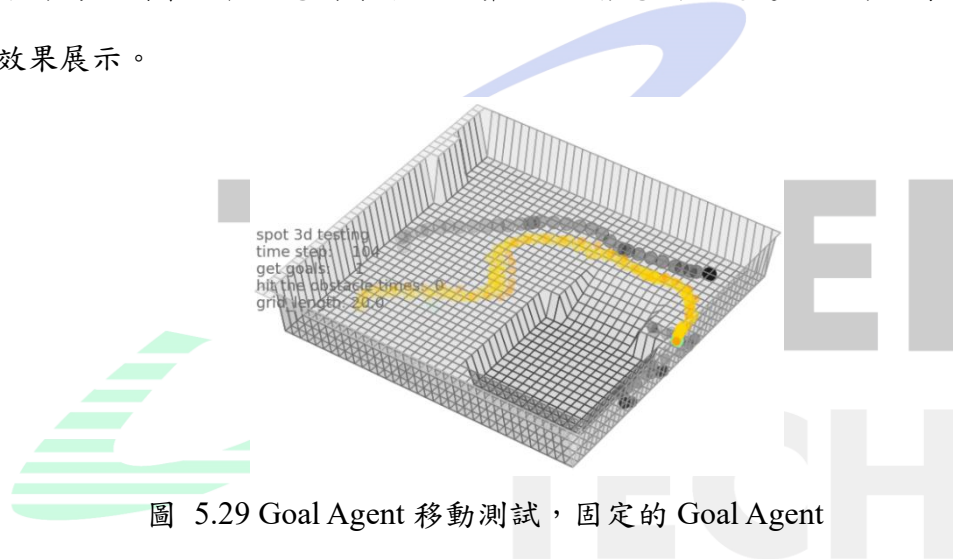


圖 5.29 Goal Agent 移動測試，固定的 Goal Agent

空曠的地形測試

評估 Spot 在空曠地形中的行為規劃能力，測試針對 Spot 抵達目的地的精確性和姿態控制進行驗證。測試場景中生成不同的地形，設置不同距離的目的地，如圖 5.30 至圖 5.31 所示。

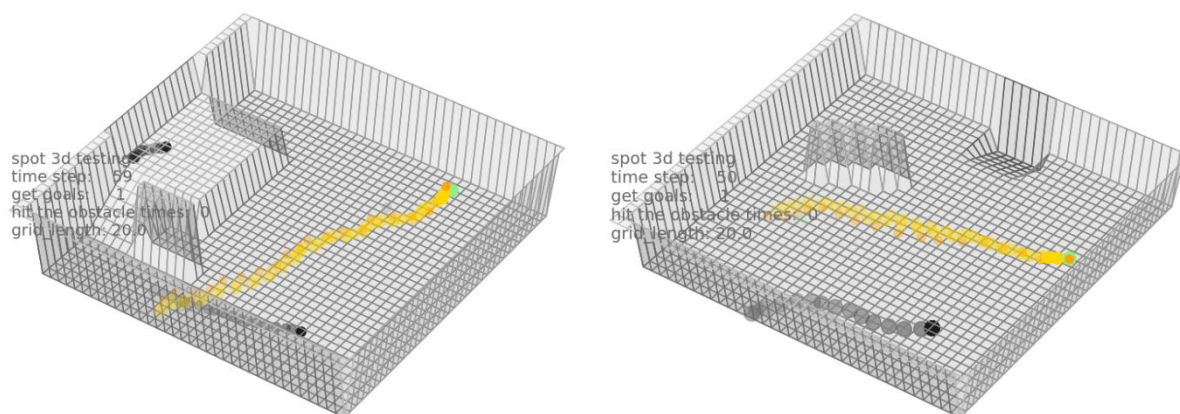


圖 5.30 空曠的地形測試 Case 1 和 Case 2

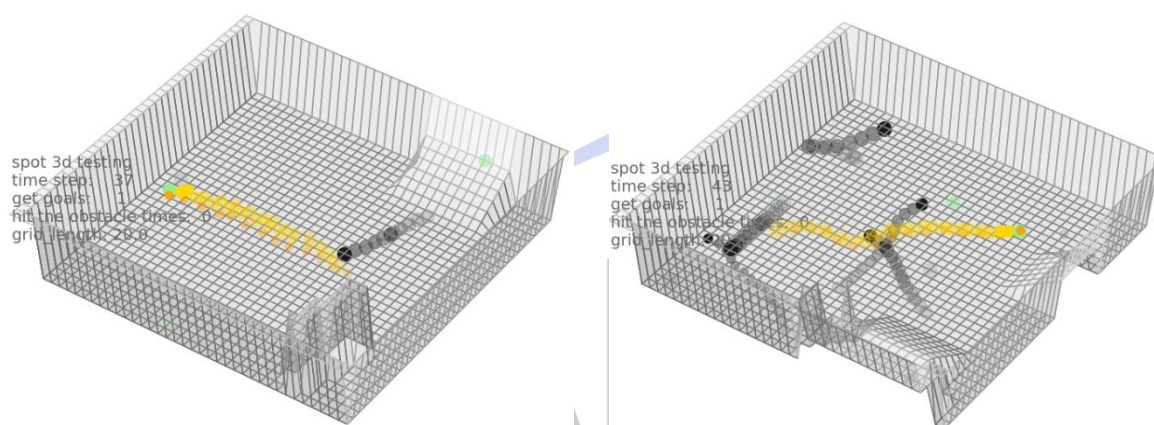


圖 5.31 空曠的地形測試 Case 3 和 Case 4

測試中 Spot 都能夠根據場景條件主動控制姿態，多以頭部朝向目標的方式移動，並在需要時才使用左右的速度控制，成功展示其規劃能力與姿態控制效果。如表 5.1 所示，Spot 每一次都能抵達目的地，且多次測試的平均耗時相近，顯示 RL 行為規劃策略所選擇路線接近最優。這些結果表明，Spot 在空曠地形中的策略執行穩健且具備良好的適應性與可靠性。

表 5.1 空曠的地形能力測試

	抵達成功率	碰撞率	平均時間	最小時間	最大時間
Case 1	100%	0%	19.8 秒	19.0 秒	21.0 秒
Case 2	100%	0%	17.6 秒	16.6 秒	19.0 秒
Case 3	100%	0%	13.1 秒	12.0 秒	14.0 秒
Case 4	100%	0%	15.5 秒	13.0 秒	23.0 秒

狹窄的地形測試

為了全面評估 Spot 在狹窄地形中的行為規劃能力，測試設置了需要穿越狹窄通道才能抵達目的地的場景，其中還加入了動態障礙物以增加挑戰難度。這種測試不僅檢驗了 Spot 對障礙物資訊的理解能力，也觀察了其策略傾向，能夠判斷 Spot 是否會採取較為保守的避讓行為，還是嘗試更具挑戰的行動，如圖 5.32 至圖 5.33 所示。

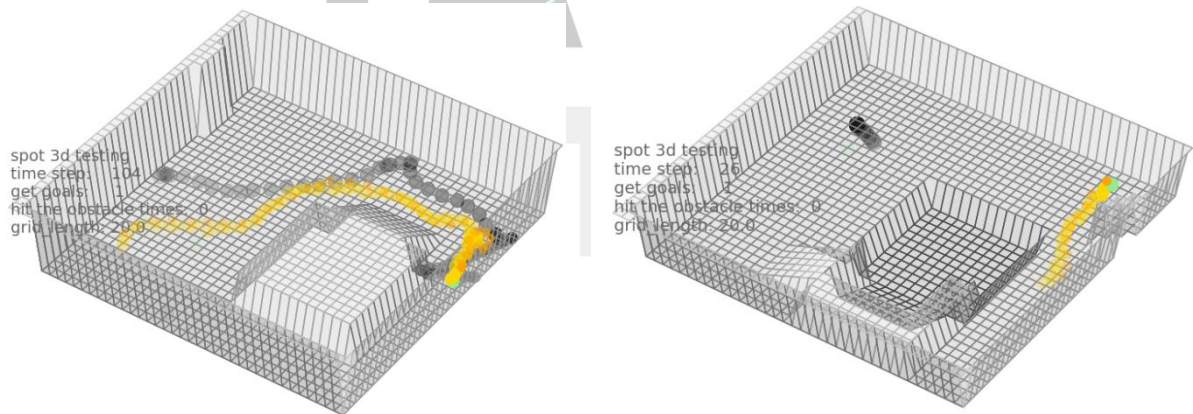


圖 5.32 狹窄的地形測試 Case 1 和 Case 2

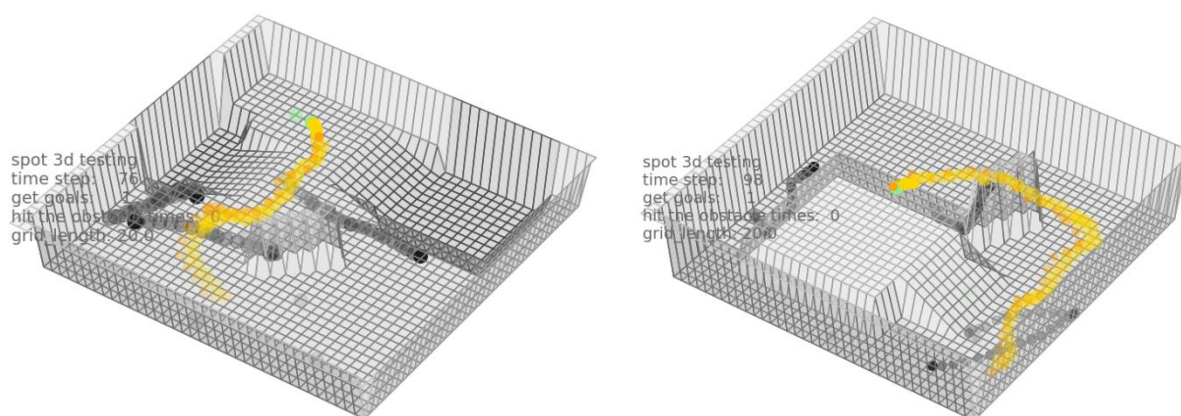


圖 5.33 狹窄的地形測試 Case 3 和 Case 4

如表 5.2 所示，測試結果顯示 Spot 在穿越狹窄地形時，能準確解析 $Sweep_{angle}$ 資訊並有效避開靜態障礙物，展現出穩健且可靠的行為規劃能力。在 Case 1 的測試中，十次試驗僅發生一次碰撞，原因是 Spot 在閃避動態障礙物時過於貼近樓梯，導致其上下樓梯的姿態控制出現問題。

表 5.2 狹窄的地形能力測試

	抵達成功率	碰撞率	平均時間	最小時間	最大時間
Case 1	60%	10%	50.4 秒	34.6 秒	94.0 秒
Case 2	100%	0%	9.1 秒	8.6 秒	9.6 秒
Case 3	100%	0%	33.8 秒	18.6 秒	64.3 秒
Case 4	50%	0%	37.6 秒	31.3 秒	44.0 秒

對於 Case 4 的測試，Spot 有 50% 的成功抵達率，儘管無碰撞事件發生，也會發生失敗案例。這是因 Spot 在部分測試中誤選通往死路的路徑(如圖 5.34 所示)，導致無法順利抵達目的地。值得注意的是，在此類情境中，Spot 能辨識牆體阻隔並選擇停在牆前，說明其具備基本環境理解與策略判斷能力，進一步驗證行為規劃策略在狹窄環境下的可靠性。此外，表中所列之平均時間、最小時間與最大時間，僅針對成功抵達目的地之測試結果進行統計分析。

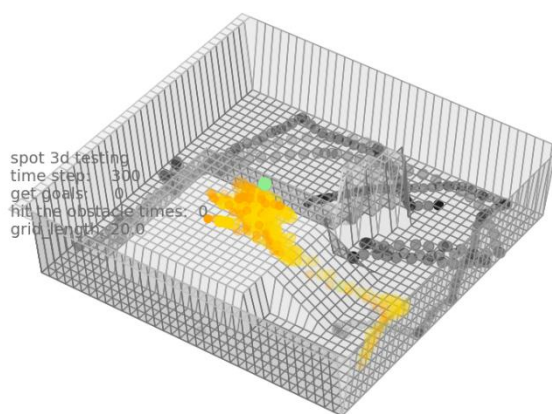


圖 5.34 狹窄的地形測試，Case 4 測試失敗

動態障礙物測試

此測試的目標是驗證 Spot 能否根據動態障礙物的運動特性靈活調整策略，並成功避開障礙到達目標地點。測試場地為一個面積 20 平方公尺的複雜環境，場景包含多個隨機移動的障礙物，其速度分布範圍廣泛，其中部分障礙物的速度甚至超過了 Spot 的最大移動速度。在這種極端條件下，Spot 需要展現出強大的行為規劃與即時反應能力，才能做出有效的策略，如圖 5.35 至圖 5.36 所示。

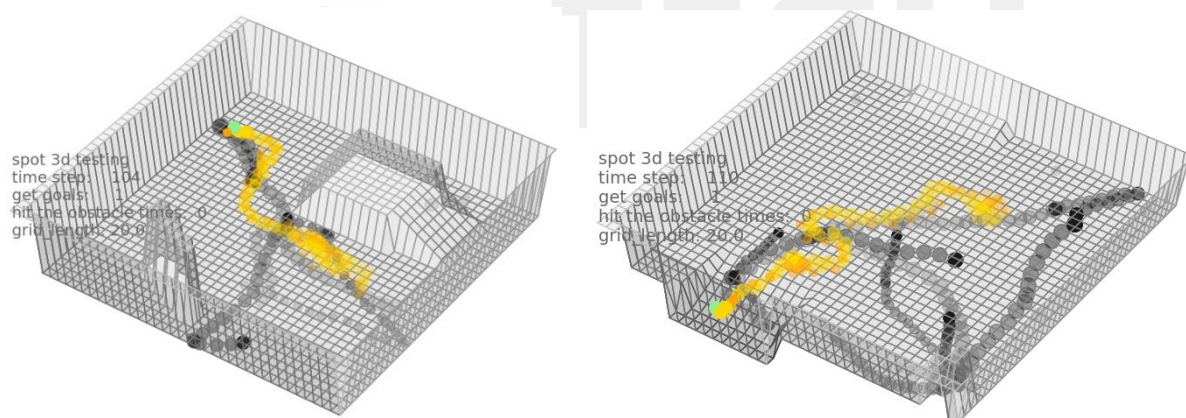


圖 5.35 動態障礙物測試 Case 1 和 Case 2

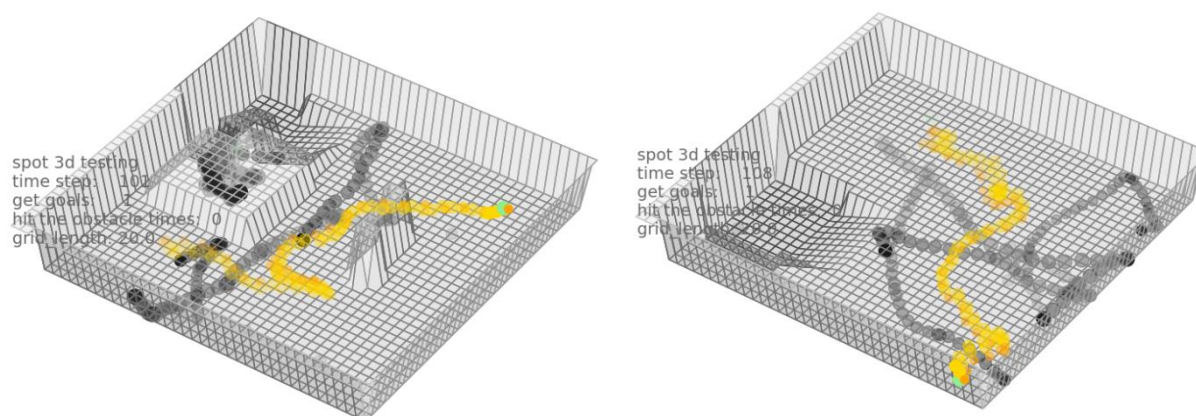


圖 5.36 動態障礙物測試 Case 3 和 Case 4

如表 5.3 所示，RL 訓練出的行為規劃策略在絕大多數動態障礙物情境下都能表現出色，即使是在高密度且高速移動的障礙物環境中，策略仍能快速判斷並作出正確的行動。由於動態障礙物完全不考慮其他 Agent 的動態交互，在這樣的測試場景中，即使由人類操控也難以完全避免碰撞。然而，RL 策略展現了穩定的適應能力，成功避開了大部分障礙物，進一步證明 RL 在多變環境中的應用潛力與可靠性。

表 5.3 動態障礙物多的地方能力測試

	抵達成功率	碰撞率	平均時間	最小時間	最大時間
Case 1	90%	10%	20.4 秒	17.6 秒	34.6 秒
Case 2	100%	0%	31.8 秒	21.3 秒	64.0 秒
Case 3	90%	10%	34.29 秒	23.0 秒	64.6 秒
Case 4	100%	0%	28.5 秒	24.3 秒	36.0 秒

靜態障礙物測試

靜態障礙物測試旨在評估 Spot 是否能繞過大型靜態障礙物並順利抵達目標位置。與狹窄地形測試著重避障能力不同，此測試關注 Spot 在面對固定且複雜障礙物（如樓梯或牆體）時的導航性能，如圖 5.37 至圖 5.38 所示。

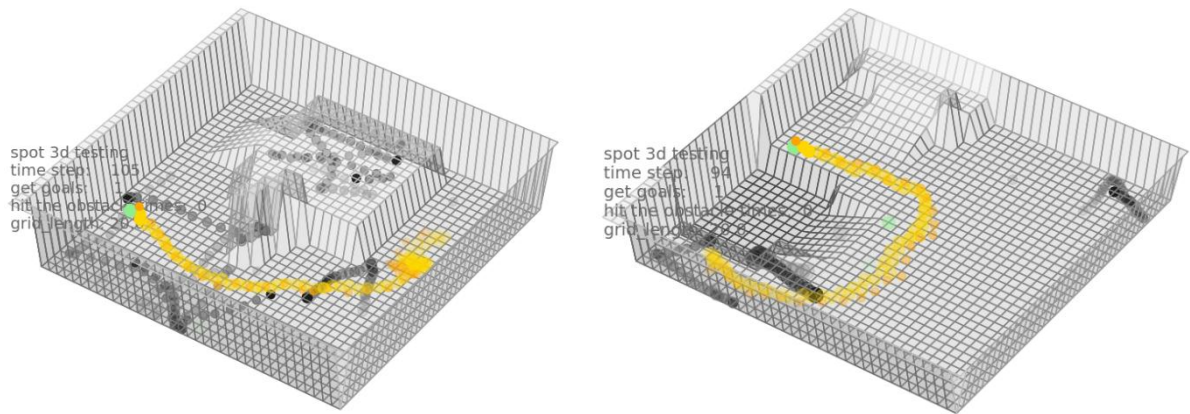


圖 5.37 靜態障礙物測試 Case 1 和 Case 2

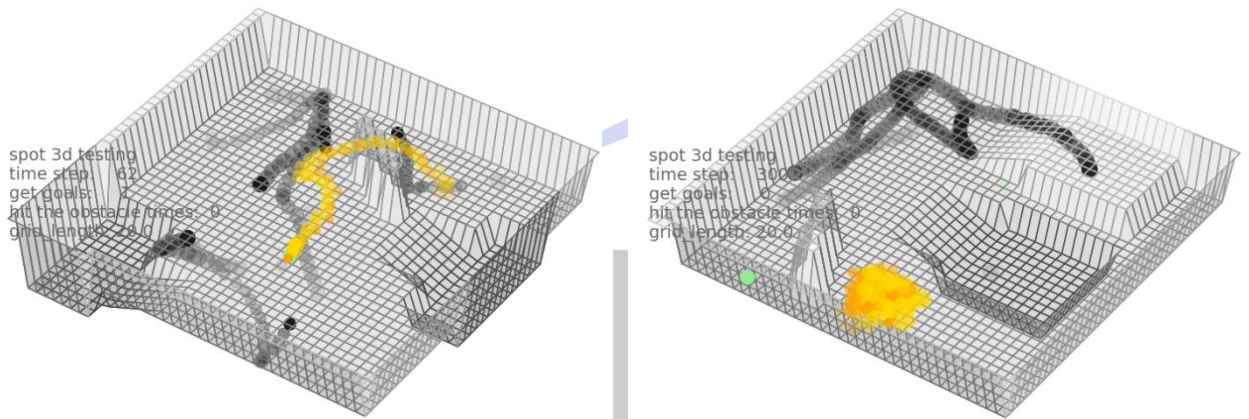


圖 5.38 靜態障礙物測試 Case 3 和 Case 4

如表 5.4 所示，Spot 能夠在目標距離較遠的情況下有效規劃路徑並順利抵達。然而，若同時存在多個前進方向，Spot 傾向於選擇距離目標最近的路徑，而無法判斷該路徑是否通暢或已經行走過。這是由於 Spot 在策略模塊中缺乏記憶功能，僅能基於當前資訊做出行為選擇，無法記得過往的嘗試或避免死路。

表 5.4 靜態障礙物多的地方能力測試

	抵達成功率	碰撞率	平均時間	最小時間	最大時間
Case 1	100%	0%	39.1 秒	29.0 秒	69.0 秒
Case 2	90%	10%	40.7 秒	31.3 秒	54.6 秒
Case 3	90%	10%	32.0 秒	18.0 秒	51.0 秒
Case 4	20%	10%	71.3 秒	69.3 秒	73.3 秒

為了應對上述限制，本論文在系統中設計了一種基於建圖的路徑規劃方法。在建圖完成後，系統會設定可行走的路徑，並通過定位功能在執行過程中自動計算中繼點的位置。中繼點之間的路徑已排除不可通行的區域，因此即使面對複雜迷宮地形，Spot 也能避免陷入死路，此改進方法能夠改善 Spot 在複雜環境中的導航能力。

上樓梯能力測試

上下樓梯測試旨在評估 Spot 是否能有效使用 *SweepMap-gradient* 資訊，實現姿態調整並完成上樓梯的任務。實驗中考慮了多種起始條件和環境變化，逐步提高測試難度，以全面檢測 Spot 的適應性與靈活性，如圖 5.39 至圖 5.40 所示。

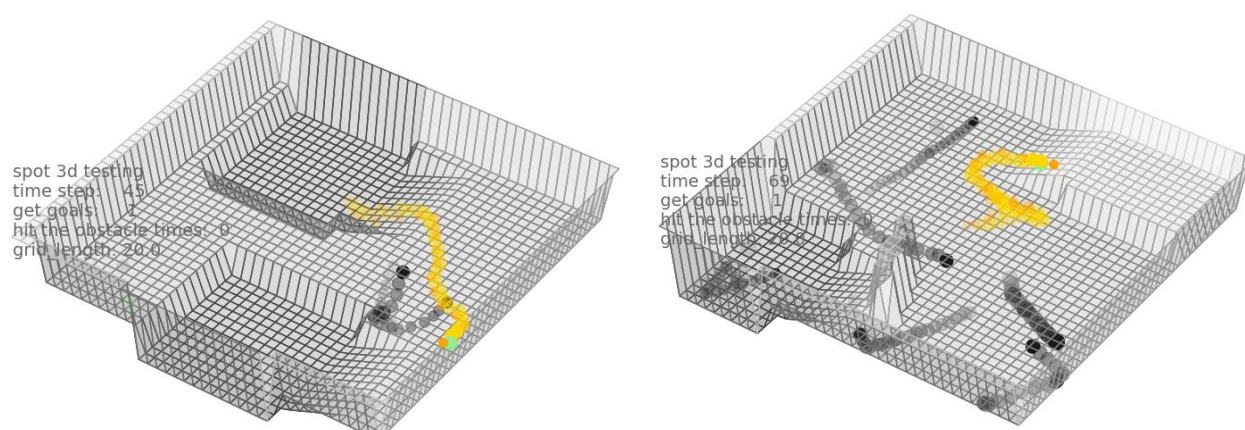


圖 5.39 上樓梯能力測試 Case 1 和 Case 2

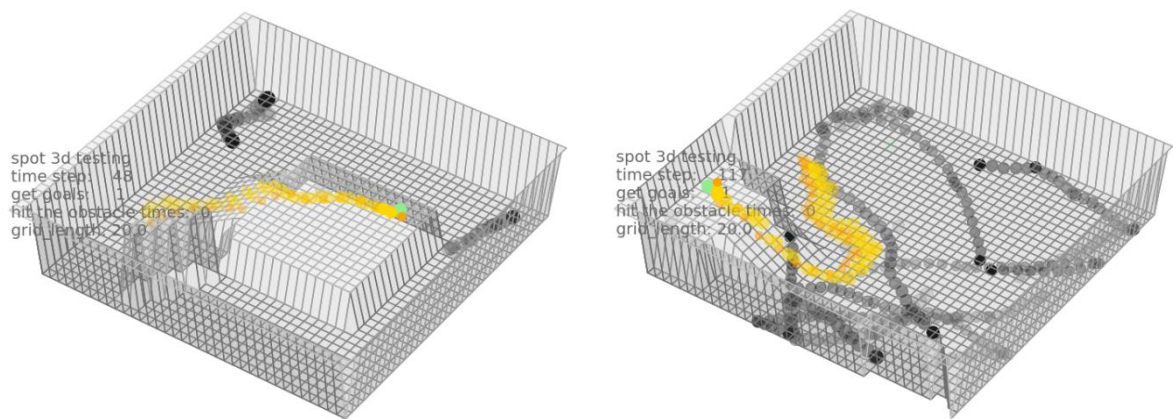


圖 5.40 上樓梯能力測試 Case 3 和 Case 4

如表 5.5 所示，無論 Spot 在任何角度面向樓梯，大多能正確調整姿態完成上樓梯任務。在包含動態障礙物的場景中，Spot 展現出靈活的避障策略，能選擇在樓梯上進行避讓或暫時放棄上樓，等待障礙物移動後再繼續執行任務。這表明 RL 行為規劃策略在應對多變環境方面具有較高的穩定性和適應性。

表 5.5 上樓梯的能力測試

	抵達成功率	碰撞率	平均時間	最小時間	最大時間
Case 1	100%	0%	15.2 秒	14.0 秒	17.6 秒
Case 2	100%	0%	10.4 秒	9.3 秒	11.6 秒
Case 3	100%	0%	16.0 秒	15.0 秒	17.3 秒
Case 4	70%	10%	47.7 秒	18.3 秒	47.0 秒

下樓梯能力測試

下樓梯測試重點評估 Spot 是否能從不同的起始狀態下正確完成下樓梯任務，如圖 5.41 至圖 5.42 所示。

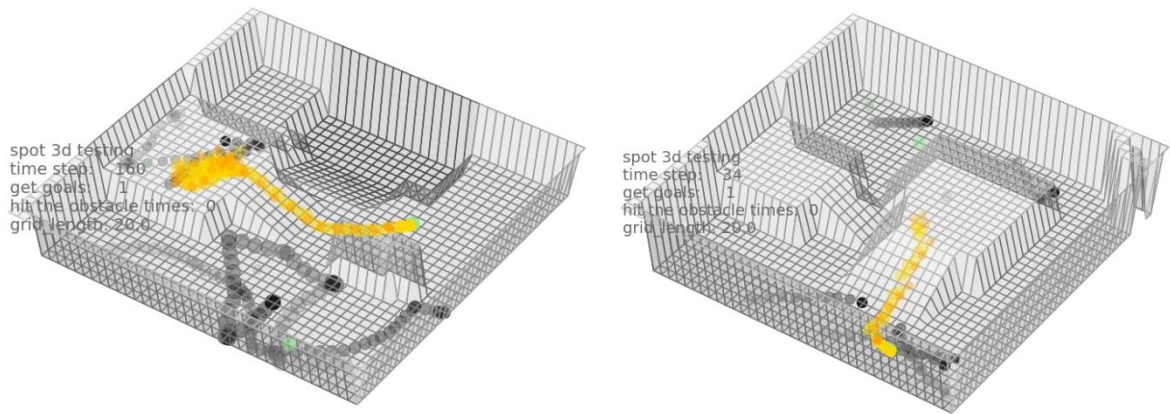


圖 5.41 下樓梯能力測試 Case 1 和 Case 2

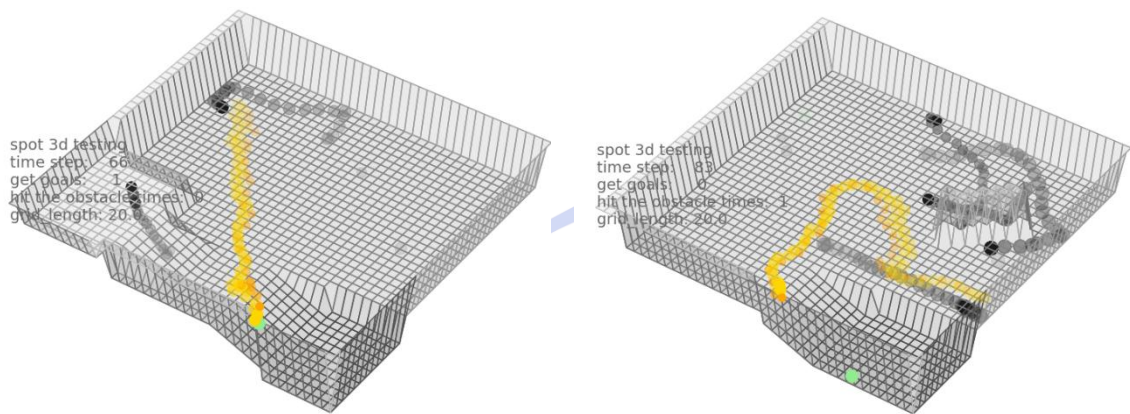


圖 5.42 下樓梯能力測試 Case 3 和 Case 4

如表 5.6 所示，不同場景下 Spot 的下樓梯成功率差異顯著。某些場景的成功率高達 90%，而另一些場景僅為 30%，即使樓梯較寬且沒有障礙物。這種現象的主要原因在於 Spot 的地形資訊感知集中在正前方和後方，當樓梯位於側面時，無法及時發現樓梯的存在，從而錯誤地將該地形視為平面並作出不正確的行動選擇。

表 5.6 下樓梯能力測試

	抵達成功率	碰撞率	平均時間	最小時間	最大時間
Case 1	70%	0%	39.6 秒	14.6 秒	57.3 秒
Case 2	90%	10%	12.0 秒	9.3 秒	16.0 秒
Case 3	70%	30%	32.4 秒	21.6 秒	65.3 秒
Case 4	30%	30%	40.5 秒	34.3 秒	46.6 秒

在實際應用中，為提升下樓梯能力，本論文提出了兩種保險機制。第一，提高執行過程中的 fps，相比於訓練過程高出 10 倍以上，為行為規劃策略提供了更多的反應時間。第二，針對配備定位功能的系統，在下樓梯前於樓梯口設置帶有方向的中繼點，幫助 Spot 預先調整至適合下樓梯的姿態，確保其能以正確的行動完成下樓梯任務，這些改善方式能夠讓 Spot 在下樓梯時更佳可靠。

不能碰撞測試

不能碰撞測試旨在檢驗 Spot 的行為規劃策略在特殊情況下的穩定性。當定位系統出現偏差或目的地被設置在牆體內時，Spot 是否能夠避免碰撞並停留在距離目標最近的安全位置，是測試的重點。本測試故意將目的地設置於不可達的靜態障礙物後，並在 100 秒的時間內，讓 Spot 在包含動態障礙物的環境中執行操作。Spot 需要盡可能接近目標的安全位置停留，而不能試圖穿越障礙物，如圖 5.43 至圖 5.44 所示。

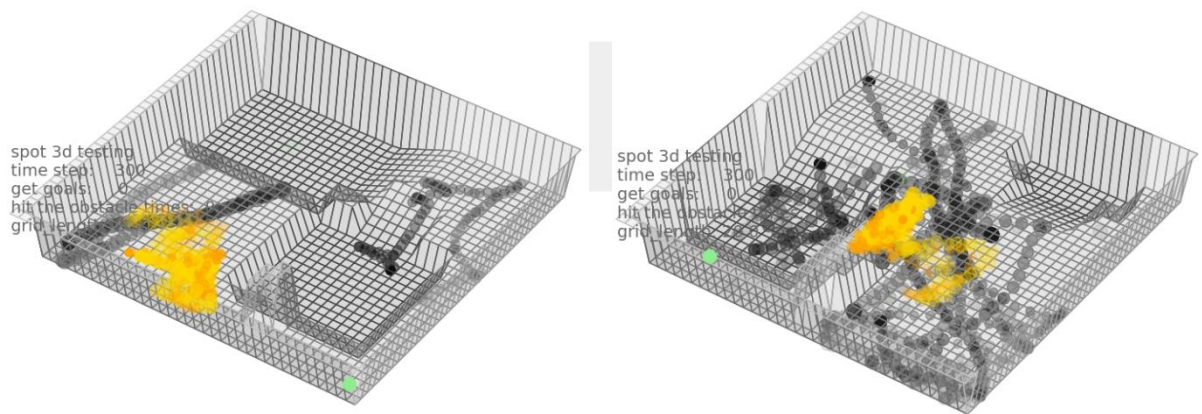


圖 5.43 不能碰撞測試 Case 1 和 Case 2

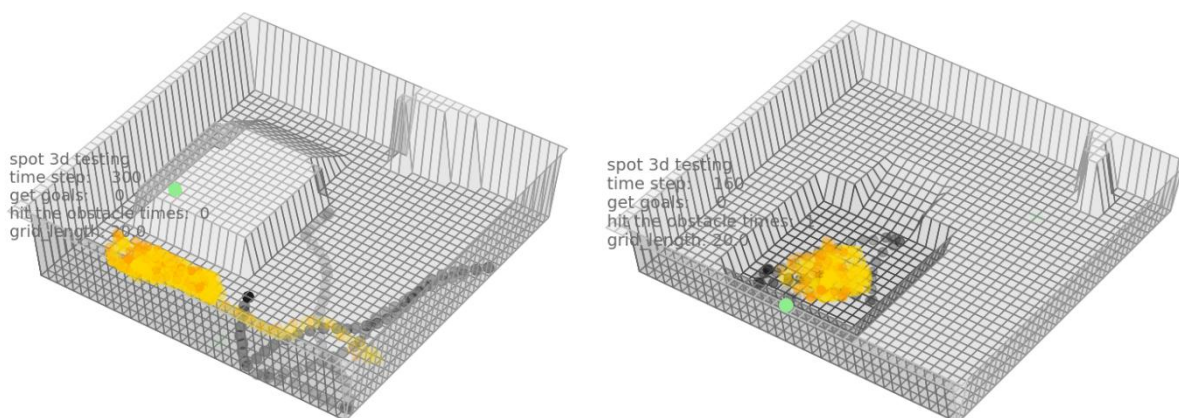


圖 5.44 不能碰撞測試 Case 3 和 Case 4

如表 5.7 所示，測試結果表明，Spot 能夠準確判斷不可跨越的障礙物，並在距目標最近的安全位置停下。絕大多數情況下，Spot 並未發生碰撞，少數碰撞事件則是由動態障礙物的不可預測性導致。這表明，雖然目標設定在不可達成的位置，但行為規劃策略在處理此類情況時依然表現出良好的穩定性和可靠性。

表 5.7 不能碰撞測試

距離	Case 1	Case 2	Case 3	Case 4
碰撞率	10%	30%	0%	10%

RL 測試結論

本論文通過設計多樣化的測試場景，全面驗證了 RL 行為規劃策略的穩定性與適應性。測試涵蓋了空曠地形、狹窄地形、上下樓梯、動態障礙物和不可達的目標等情境，模擬現實應用中可能面臨的多數挑戰，充分評估了 RL 策略的實用性。

1. 空曠地形

Spot 在空曠地形的測試中表現極為穩定。RL 策略能快速計算最優路徑，成功率接近 100%，且移動軌跡平滑高效。Spot 基於目標位置進行直接導航，無冗餘動作，顯示出良好的路徑規劃能力。

2. 動態障礙物

面對動態障礙物的多變環境，Spot 能準確感知障礙物的運動特性並採取合理避障行為。測試結果顯示，絕大多數情境中，Spot 能避開快速移動的障礙物，但在高密度或高速障礙物環境中偶爾發生碰撞，此類情境對人類操控也具有挑戰性。

3. 狹窄地形與靜態障礙物

Spot 展現了對於 $Sweep_{angle}$ 資訊的解析能力，能有效避開靜態障礙物並完成目標導航。然而，因缺乏記憶能力，Spot 偶爾會在死路中被困。為此，系統中引入基於建圖的中繼點設置，有效解決此問題並顯著提升導航性能。

4. 上下樓梯場景

上樓梯測試中，Spot 準確使用 $Sweep_{Map-gradient}$ 資訊進行姿態調整，即使面對動態障礙物，也能靈活選擇策略完成任務。下樓梯測試中，當 Spot 初始方向有誤時，姿態修正偶爾失敗，但改進後的樓梯入口中繼點設置成功降低了失誤發生的概率。

5. 目標不可達情境

在目標設置於牆體或不可達區域的極端條件下，Spot 能判斷障礙物不可跨越，選擇停留在距目標最近的安全位置，測試中未出現不必要的碰撞行為。此表現顯示了 RL 策略對極端情境的適應性與穩定性。

在多數場景下，Spot 的 RL 行為規劃策略能穩定完成目標定位與路徑規劃，對環境變化的適應性良好。這些結果表明，RL 策略訓練具有很高的實用價值。

5.6 ROS 系統測試

本論文基於 ROS[46]架設完整系統，為了適應 RL 訓練快速更新的需求，設計了一種高效轉移 RL 行為規劃策略至 ROS 平台的方法，並確保策略執行效果的一致性。

5.6.1 測試流程

在 ROS 系統中成功應用 RL 訓練結果，需要確保真實執行環境中的數據分佈與 RL 訓練數據的一致性。本論文利用 Wandb 平台記錄 RL 完整測試過程中每一個輸入數據在不同時刻的分佈，並將 ROS 系統測試結果同步輸出至 Wandb，通過二者數據差異的比對，快速定位並解決因環境不一致導致的問題，如圖 5.45 所示。

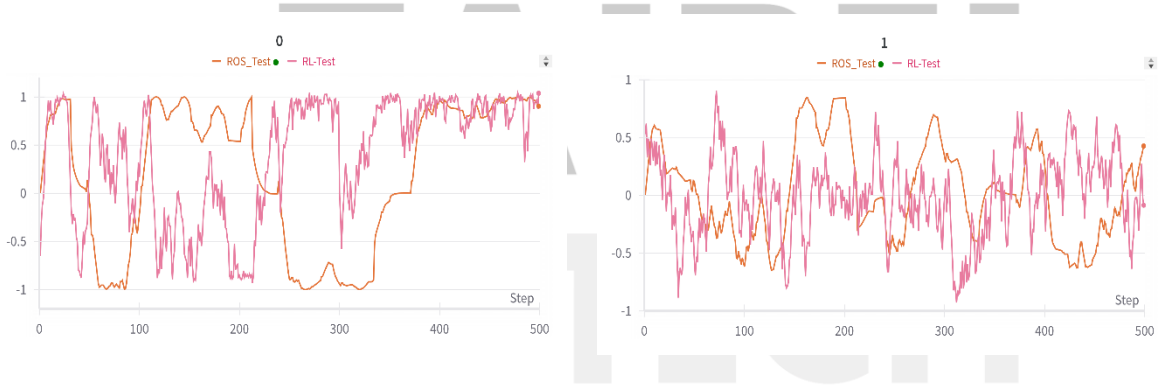


圖 5.45 數據值域比對

為簡化頻繁更新 RL 策略對 ROS 系統的影響，本論文採用 Wandb 儲存所有與 RL 相關的參數、策略權重、網絡結構和程式，當 ROS 系統需要更新時，可直接從 Wandb 下載任一版本的模型，避免頻繁修改系統內部結構影響整體穩定性。為驗證行為規劃策略是否已成功轉移至 ROS 系統，本論文針對 Spot 的每項能力進行逐步測試，確保系統各部分功能正常運作，測試數據如(5.3)式所示。

$$S_{Spot} = \{Speed, Goal, Time, Died, Obs, Sweep_{angle}, Sweep_{Map-gradient}\} \quad (5.3)$$

抵達目的地測試

確認 *Speed*、*Goal*、*Time* 和 *Died* 的前處理與模型連接是否正確。

避障能力測試

驗證 *Obs* 和 *Sweep_{angle}* 的數據處理與行為規劃是否準確。

上下樓梯測試：

評估 *Sweep_{Map-gradient}* 的功能，確保在複雜地形中的有效性。

通過這些逐步測試，本論文成功將 RL 的行為規劃效果轉移至 ROS 系統，實現了從策略訓練到真實環境執行的完整鏈路驗證。

5.6.2 ROS 抵達目的地測試

行為規劃系統的測試中，確認 Spot 能否準確抵達目標位置是最重要的核心目標。最基礎的測試需要依賴於兩項關鍵資訊：*Speed* 和 *Goal*。其中，*Speed* 可以由 *Odometry* 資料計算得到，由機器人提供 Spot 的當前移動速度，而 *Goal* 是基於其他系統計算出的相對於 Spot *Odometry* 坐標系的目標位置，並在輸入 RL 系統前轉換為 Spot 坐標系。

測試過程中，本論文利用 ROS 的 **Rviz** 工具，在 *Odometry* 坐標系上直接設置目標點進行驗證。目標點以粉紅色箭頭標示設定，設定後顯示為紅色箭頭，而 Spot 的實際移動軌跡則以綠色線條呈現，其中一格網格大小為一公尺。這樣的測試設置提供了直觀的視覺反饋，能準確檢測行為規劃策略的執行情況，如圖 5.46 所示。



圖 5.46 設定 Spot 的目的地

如圖 5.47 所示，Spot 的移動軌跡穩定，能夠精確抵達目標，證實 *Model, Speed, Goal, Time* 資訊在 ROS 系統中的整合效果良好。

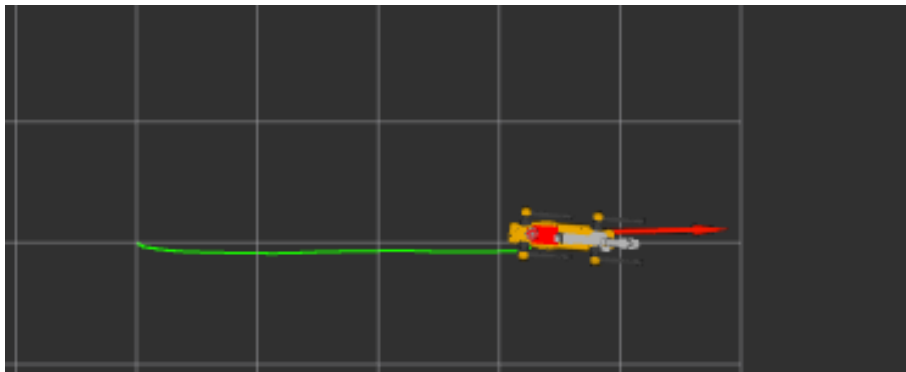


圖 5.47 測試 Spot 抵達前方的目的地

如圖 5.48 至圖 5.50 所示，測試將目標設置於 Spot 的不同方向，包括左側、右側及後方，結果表明 Spot 無論目標方向如何變化，均能順利抵達目的地。

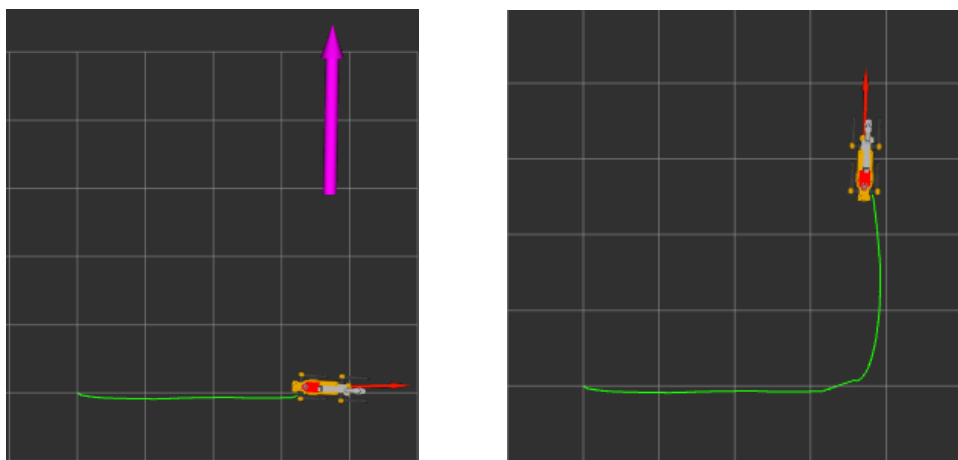


圖 5.48 測試 Spot 抵達左邊的目的地

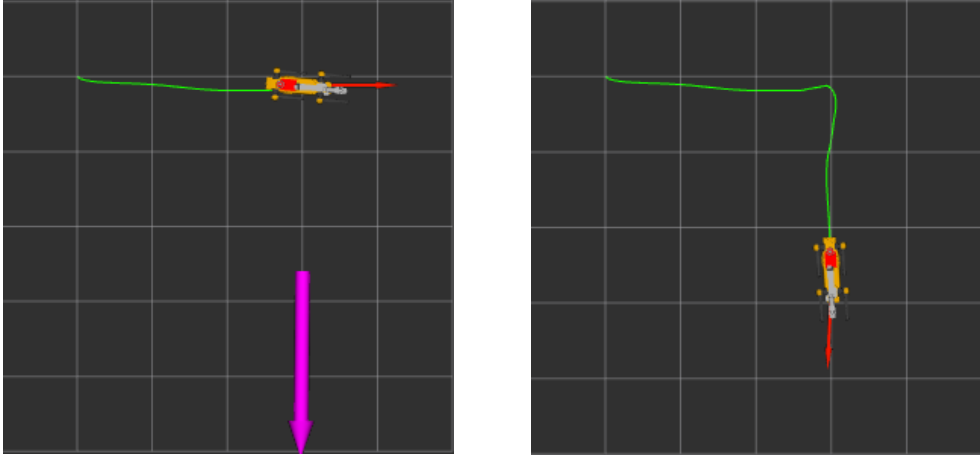


圖 5.49 測試 Spot 抵達右邊的目的地

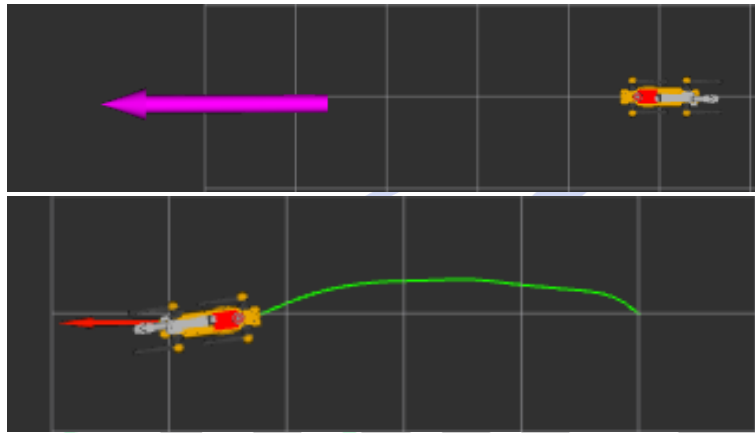


圖 5.50 測試 Spot 抵達後面的目的地

值得一提的是，在 RL 訓練中 Spot 的目標成功判定標準是距離目的地小於 0.28 公尺，且對抵達時的角度未作要求。然而，直接應用於 ROS 系統時，這一標準可能導致 Spot 在靠近目標時，因未能正確調整方向或出現如原地轉圈等不理想行為。為了解決這一問題，本論文採用了比例控制的**對齊控制機制**，系統會基於 Spot 的當前位置與目標點的線性距離與角度誤差，計算出所需的調整的加速度，如(5.4)式所示。

$$\begin{aligned}
 acc_x &= 1 \times (goal_x - spot_x) \\
 acc_y &= 1 \times (goal_y - spot_y) \\
 acc_{\omega} &= 1.5 \times (goal_{\omega} - spot_{\omega})
 \end{aligned} \tag{5.4}$$

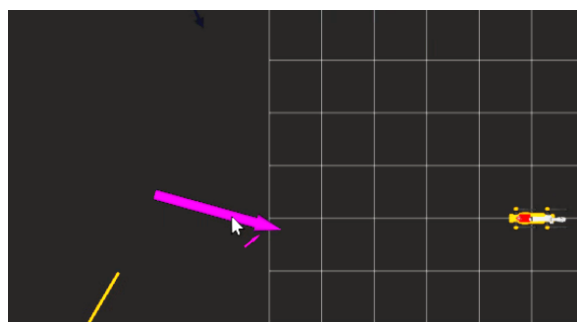
具體設計中，當 Spot 滿足以下條件時，系統會觸發對齊控制機制：首先，Spot 必須與周圍障礙物保持至少 1 公尺的安全距離；其次需要與目的地距離小於 0.5 公尺；還須確保 Spot 並非位於斜坡上；最後該目標需為最終目的地，而非中繼點。

該機制通過計算目標與當前位置之間的線性和角度誤差，動態調整 Spot 的速度和方向，使其穩定朝向目標並完成精確定位。這一機制充分利用了比例控制簡單高效的特性，為解決目標對齊問題提供了一種實用的方案。

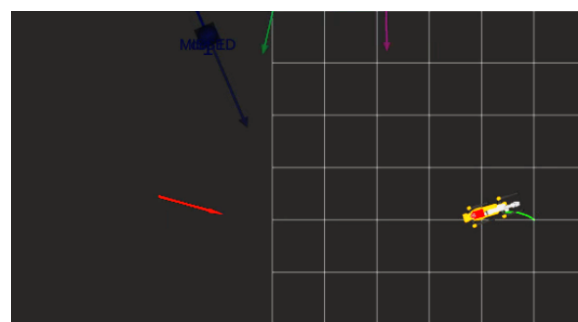
5.6.3 ROS 避障能力測試

為了驗證 ROS 系統的避障能力及在真實場景中的姿態控制表現，本論文設計了一系列測試來觀察 Spot 面對障礙物時的行為反應。由於 RL 行為規劃策略中基於資訊延遲的考量，Spot 並未直接使用 *Obs* 資訊，而是全部採用 *Sweep_{angle}* 來表達障礙物資訊。

本論文使用了 `pedsim_ros`[47] 移動障礙物模擬系統來生成動態場景。該系統模擬了行人的移動行為，支持設置障礙物的大小、移動速度和起始位置，並提供了高度靈活的參數化場景。考慮到動態障礙物與 Spot 的實時交互無法直接透過運動軌跡展現，本階段以連續畫面記錄 Spot 的具體動作。這樣的設計既能清晰呈現 Spot 的策略應用，也便於觀察策略細節。



(a)



(b)



(c)



(d)



(e)



(f)



(g)



(h)



(i)

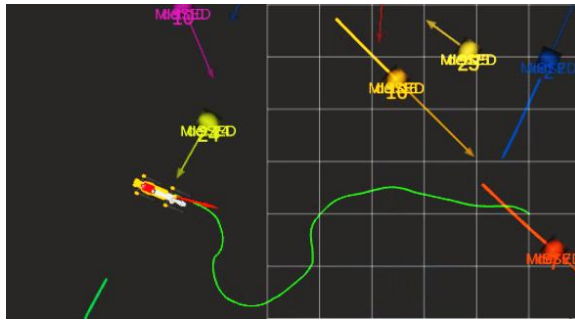


(j)

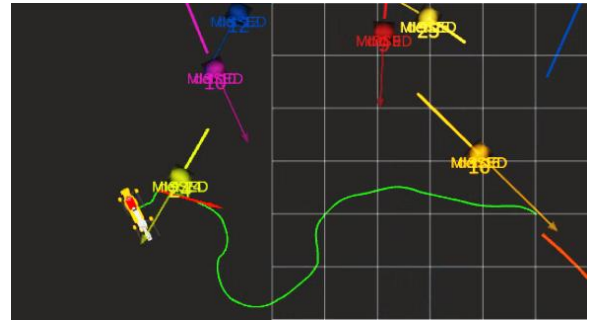
圖 5.51 Spot 穿越眾多障礙物，抵達目的地

如圖 5.51 所示，Spot 展示了在多個動態障礙物環境中的高效避障能力。起

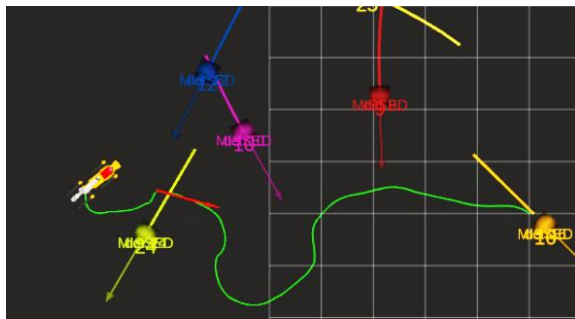
初，Spot 通過轉向來避免與障礙物直接相撞，並調整行進方向往目標靠攏。在人
群密集區，Spot 選擇繞行障礙物，最終成功到達目的地。



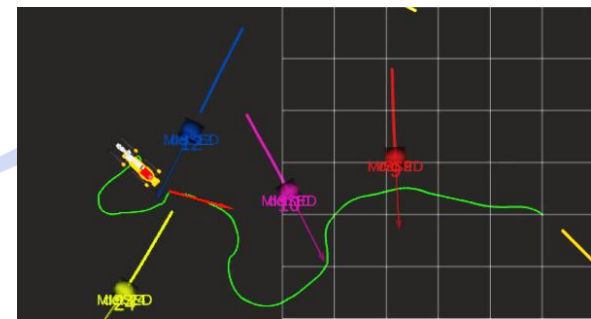
(a)



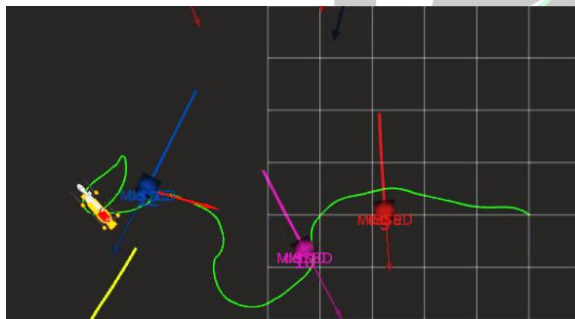
(b)



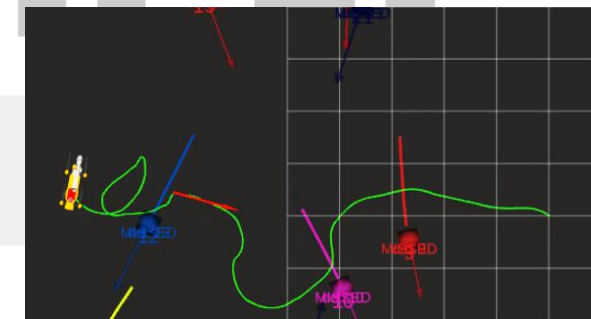
(c)



(d)



(e)



(f)



(g)



(h)

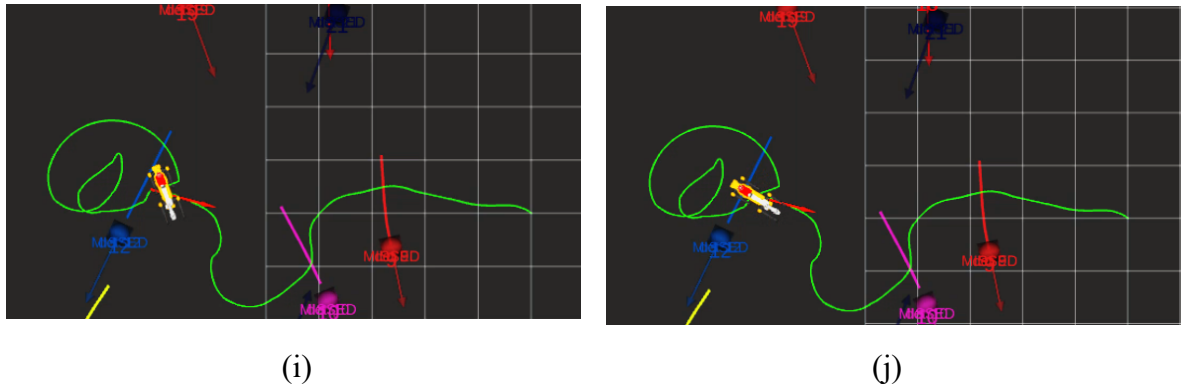


圖 5.52 Spot 抵達目的地後，仍舊可以躲避障礙物

如圖 5.52 所示，當 Spot 已經到達目標位置，行為規劃系統依然保持啟動狀態。Spot 在停下後，若有動態障礙物靠近，仍會做出遠離障礙物的策略。這表明，即使在靜止狀態，Spot 的行為規劃系統仍能實時的做出正確的策略。

ROS 系統測試證明了避障能力和姿態控制在複雜動態場景中的適應性與可靠性。無論是行進過程中還是靜止狀態下，Spot 均能展現穩定的避障能力，並對周圍環境的變化作出即時反應。此外，為方便讀者更直觀地了解測試過程與結果，本論文將 ROS 系統的測試影片上傳至 YouTube。讀者可透過以下連結 (<https://www.youtube.com/@ken4600>) 觀看更多測試影片，了解 Spot 在不同場景中的表現與策略運用。

5.6.4 ROS 模擬數據測試

為了驗證 ROS 感測器數據轉換至 RL 狀態資訊的準確性，本論文使用 Gazebo[48]模擬環境進行測試。測試目標為確認基於感測器數據的行為規劃策略是否能正確生成控制訊號，並驗證整體系統的穩定性與即時性。RL 狀態資訊完全來自 Gazebo 模擬的機器狗背上的 3D 激光雷達數據經過數據處理後生成，如圖 5.53 所示。

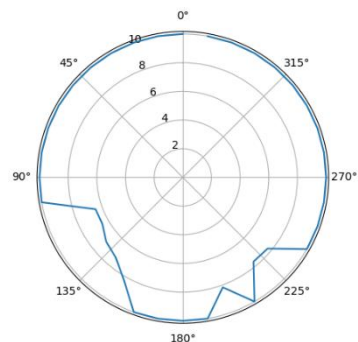
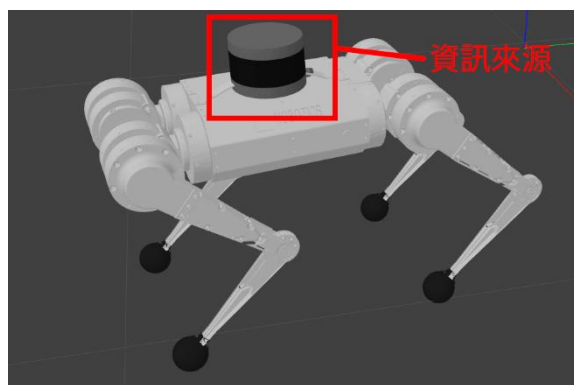


圖 5.53 Gazebo 的 Cheetah 機器狗

測試中本論文設置了三種類型的環境：一般地形、15 度樓梯以及 45 度樓梯，每一個測試機器狗都沒有事先看過地圖，全部根據當下的感測器資訊來決定控制策略。一般地形包含眾多分散的靜態障礙物，用於測試系統的感測能力及即時避障效果，如圖 5.54 所示。

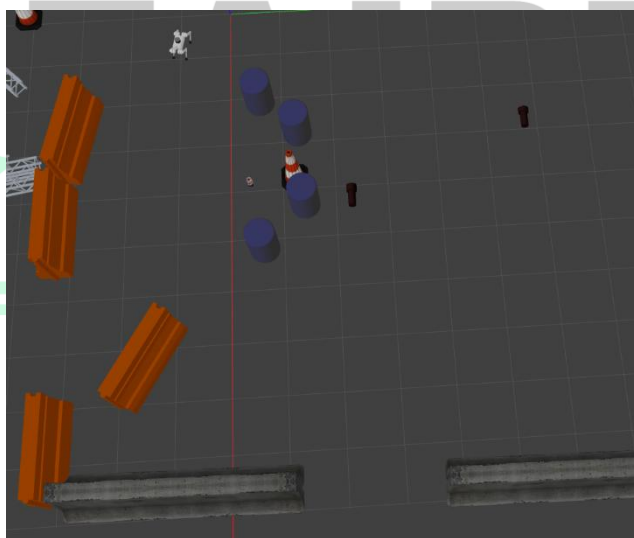
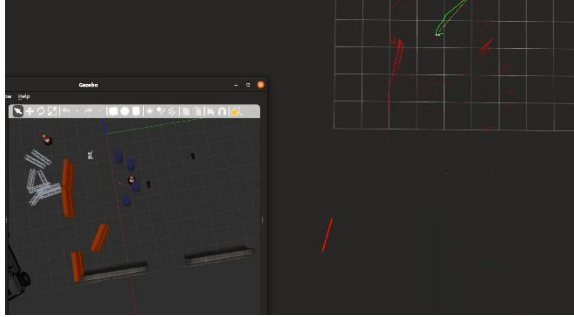
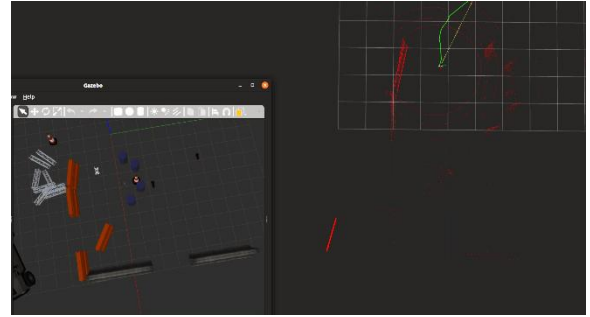


圖 5.54 一般地形測試環境

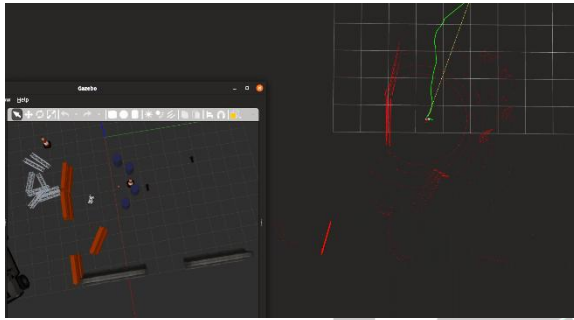
如圖 5.55 所示，圖(a)(j)(o)使用 rviz 設定目的地，並在地圖上顯示 Gazebo 算出的紅色點雲 3D 激光雷達資訊，讓機器狗會基於感測器轉換成需要的資訊，及時運算 RL 行為控制策略，成功避開障礙物並抵達目標位置。



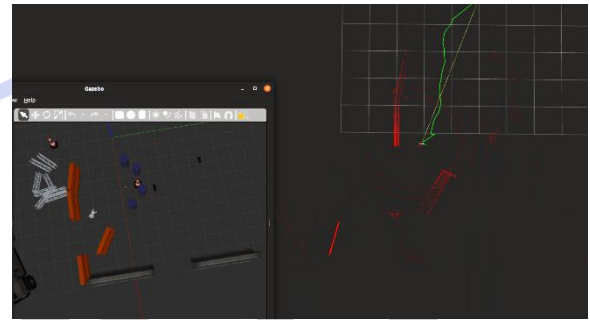
(a)



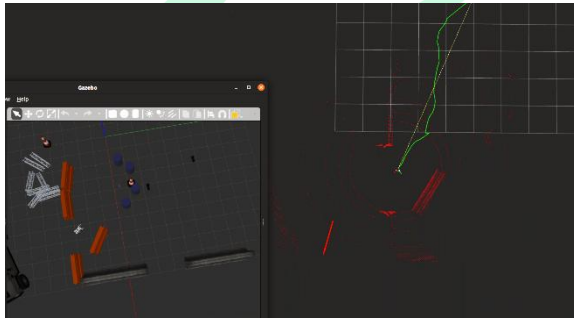
(b)



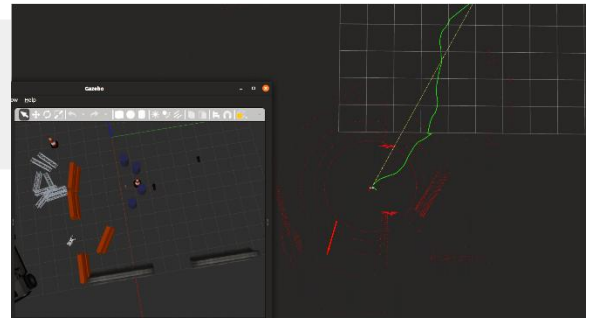
(c)



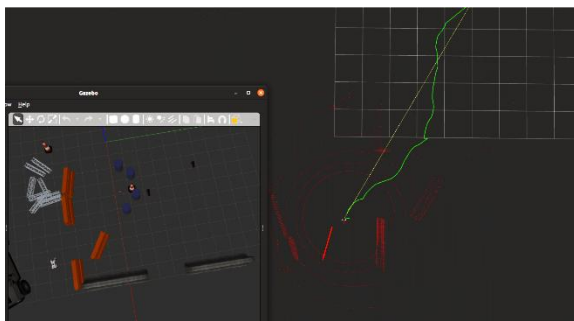
(d)



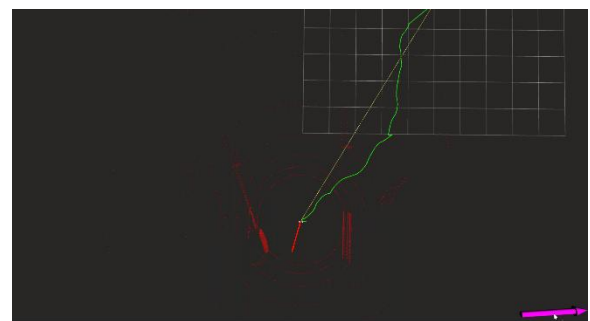
(e)



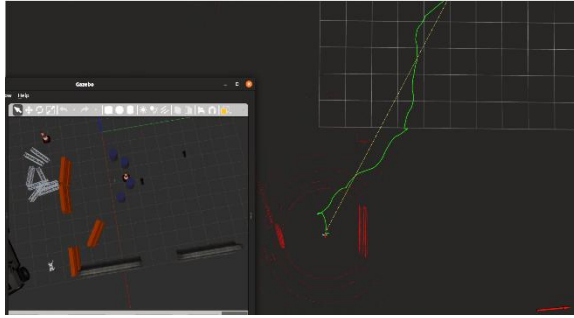
(f)



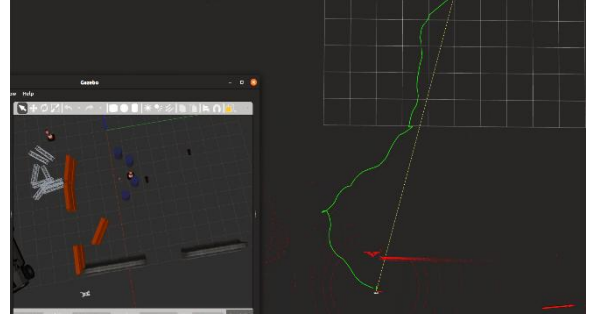
(i)



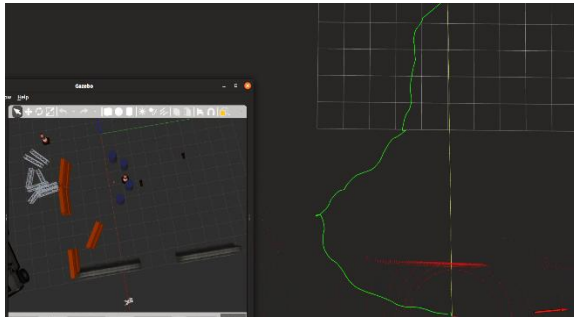
(j)



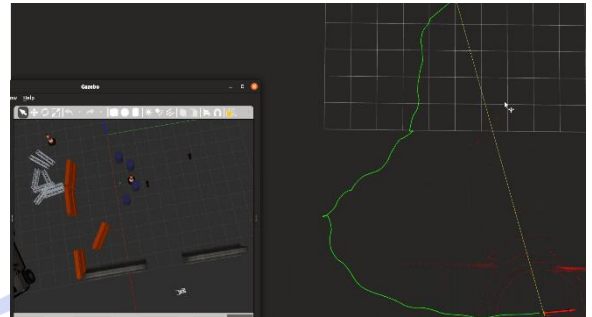
(k)



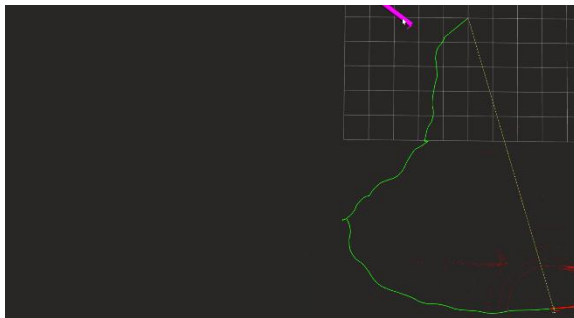
(l)



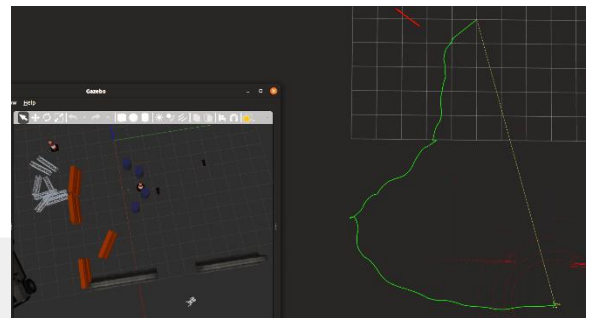
(m)



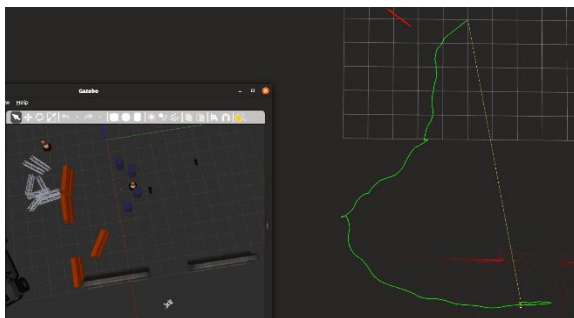
(n)



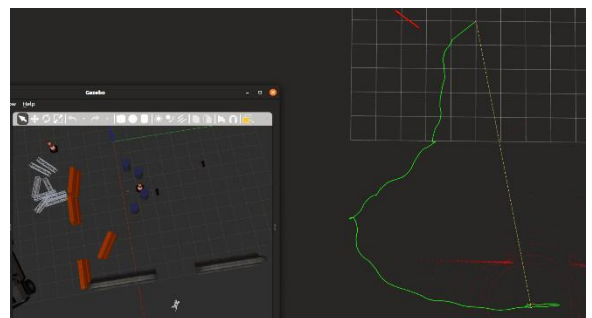
(o)



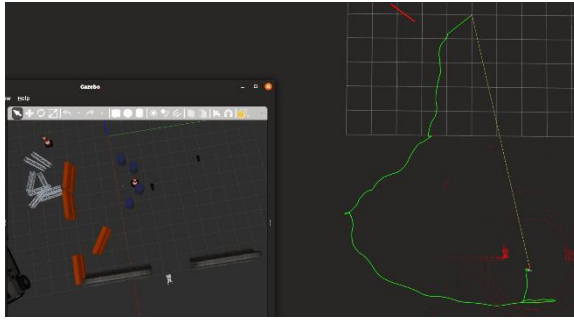
(p)



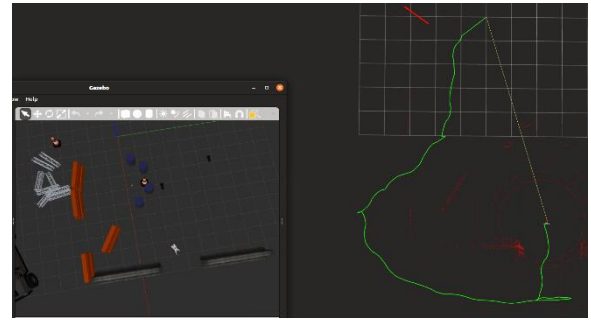
(q)



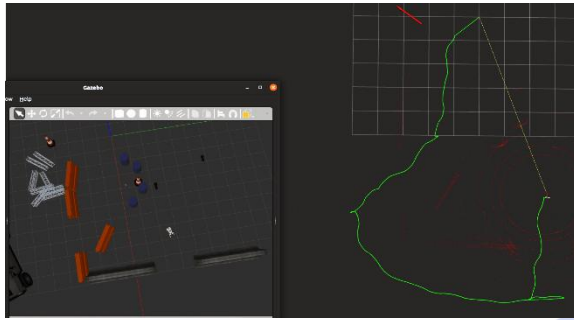
(r)



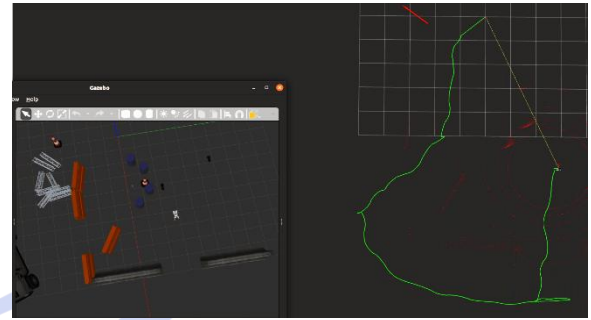
(s)



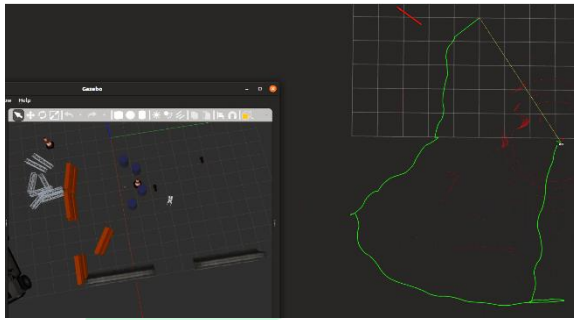
(t)



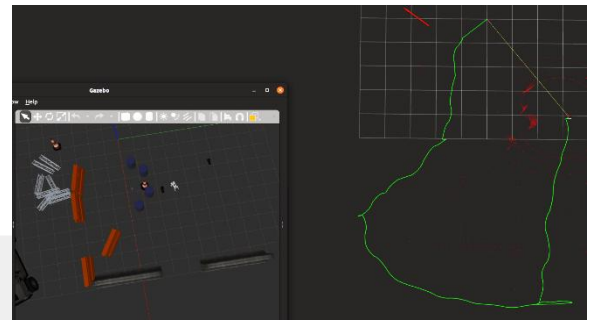
(u)



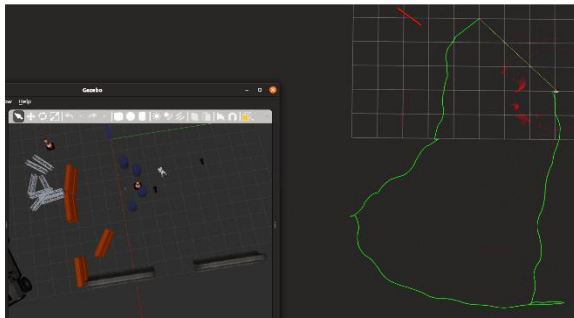
(v)



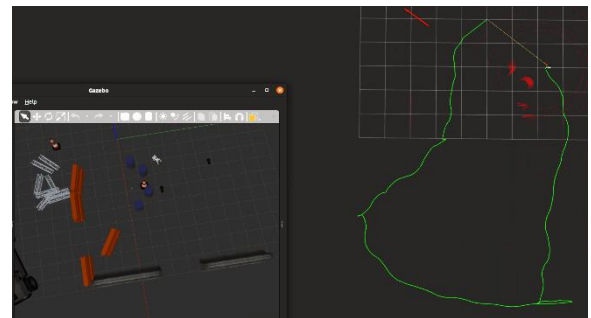
(w)



(x)



(y)



(z)

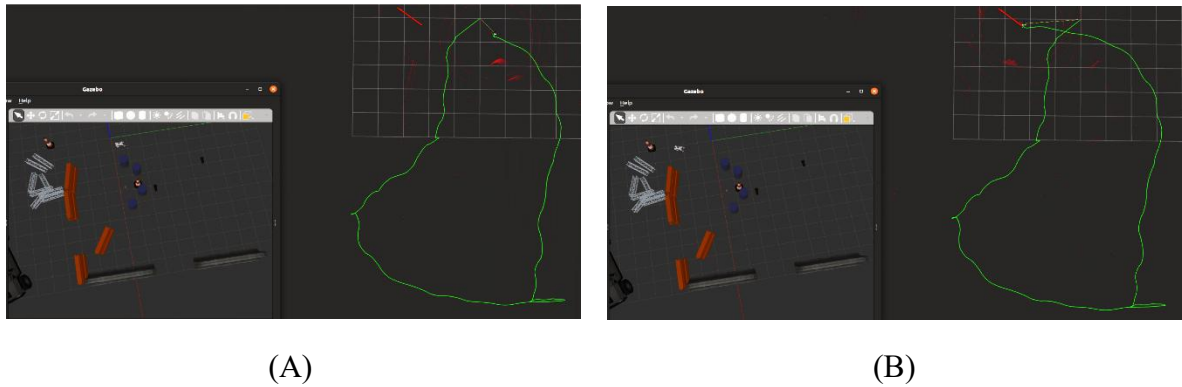


圖 5.55 一般地形測試

在 15 度樓梯環境中，模擬了低斜度樓梯的攀爬情境，用以驗證基於激光雷達資訊生成的攀爬策略是否能夠產生正確的控制訊號。45 度樓梯環境中，模擬了超出機器狗能力範圍的攀爬情境，測試系統是否能正確識別該樓梯為不可通過的障礙物，進而選擇繞行策略，避免攀爬失敗的風險，如圖 5.56 所示。

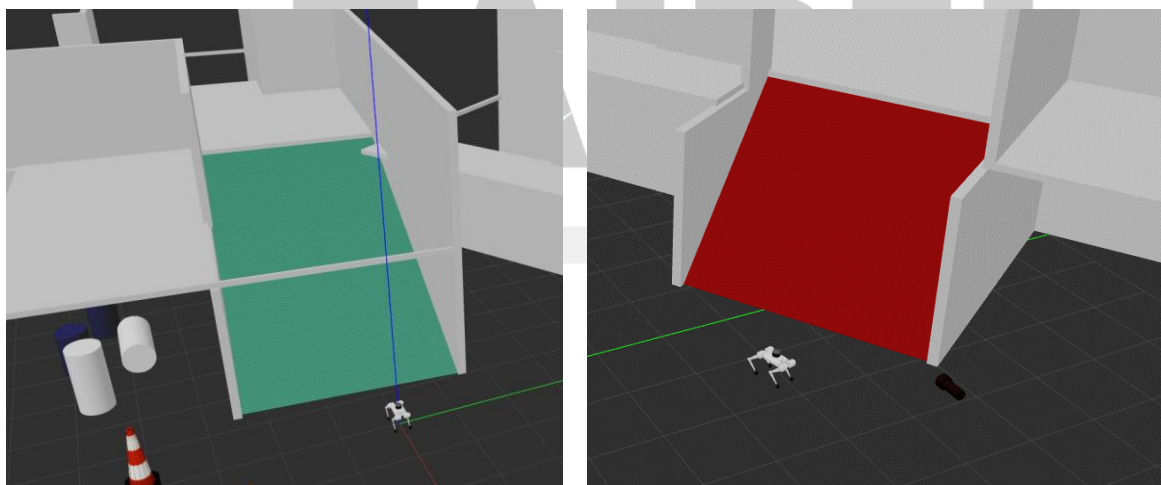
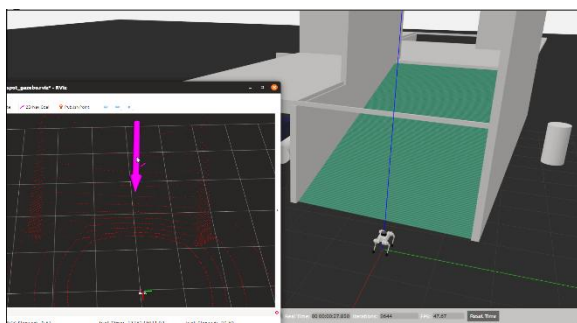
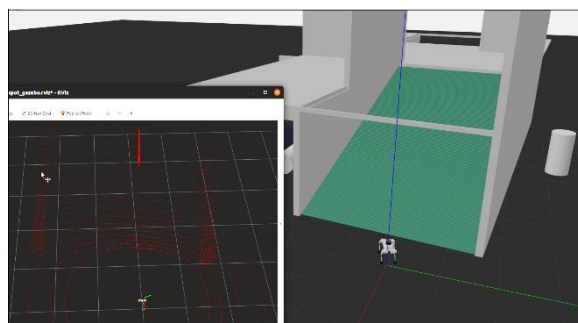


圖 5.56 樓梯測試環境

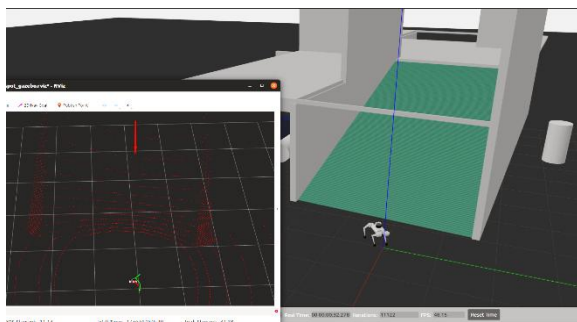
如圖 5.57 所示，圖(a)設定一個上樓的目的地，並在圖(j)設定下樓，面對 15 度樓梯時，行為規劃策略能生成合理的攀爬動作計劃。策略首先會調整機器狗的姿態，使其適合進行攀爬，再開始上樓梯，並能在抵達目的地位置時，考慮樓梯的角度避免錯誤的旋轉，做出適合的策略。



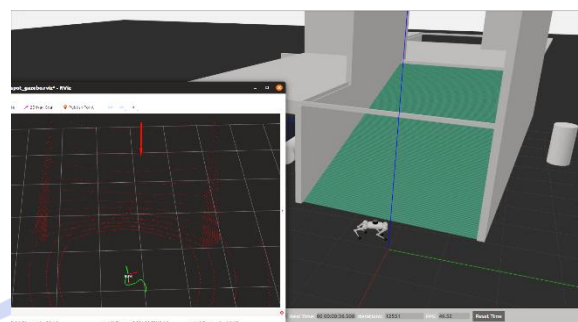
(a)



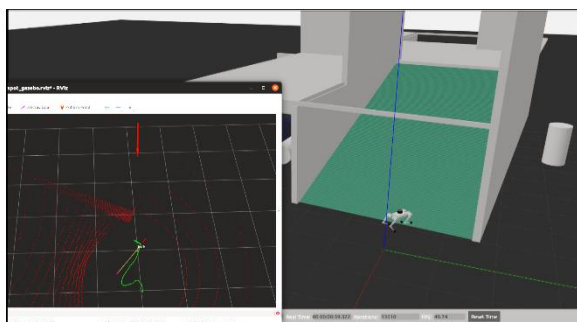
(b)



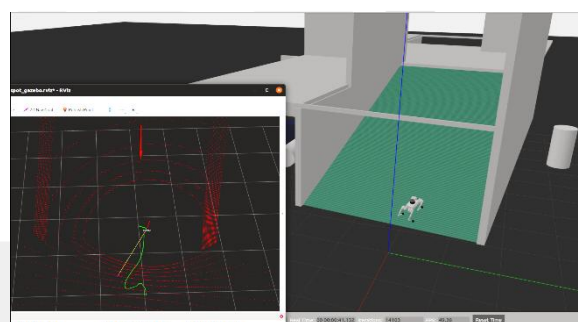
(c)



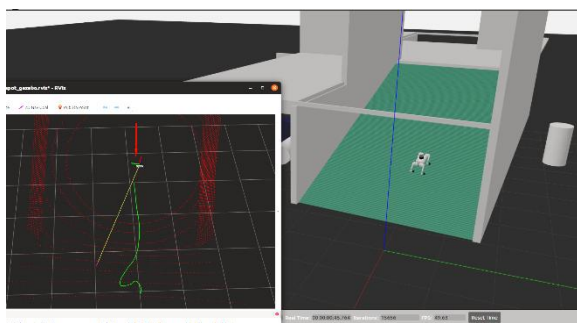
(d)



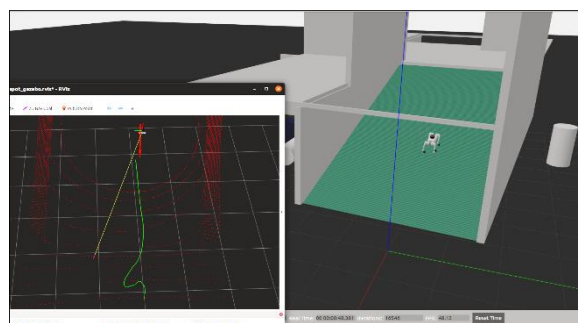
(e)



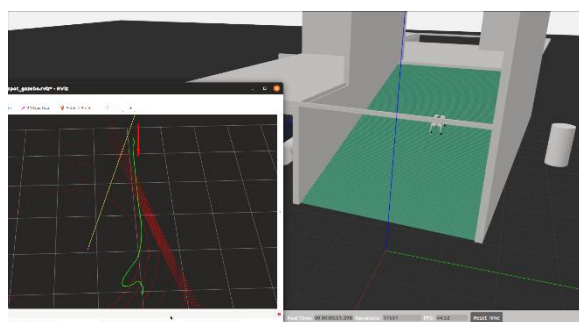
(f)



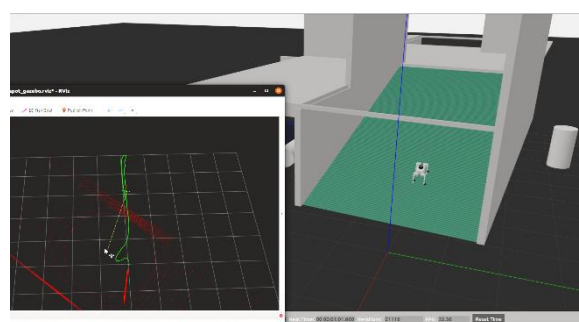
(g)



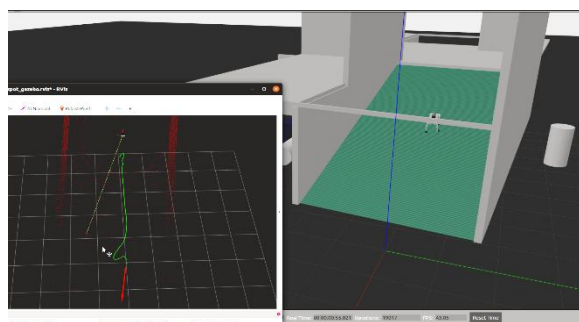
(h)



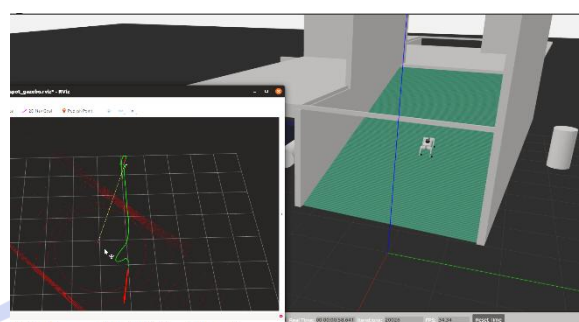
(i)



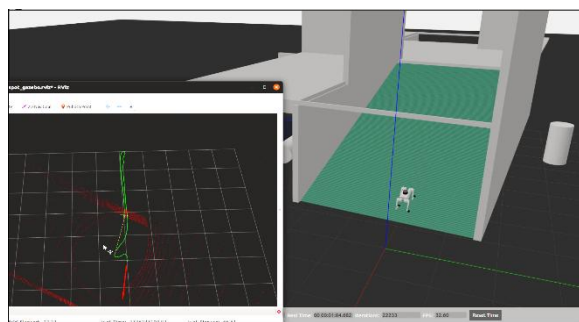
(j)



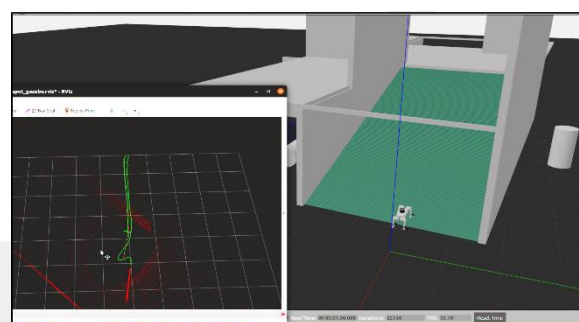
(k)



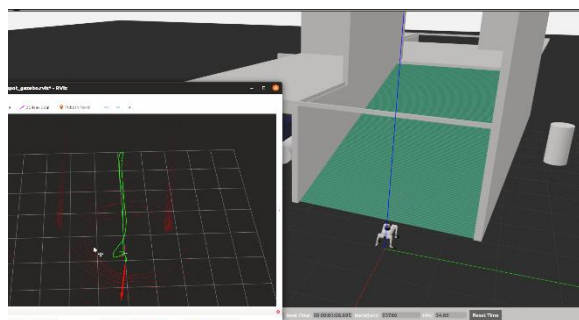
(l)



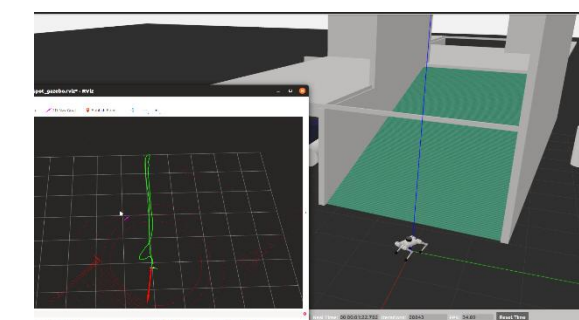
(m)



(n)



(o)



(p)

圖 5.57 15 度樓梯測試

如圖 5.58 所示，45 度樓梯測試中，行為規劃策略能正確地將該樓梯識別為

不可通過的障礙物，避免嘗試攀爬，停在了樓梯口前，展現了系統良好的決策能力。

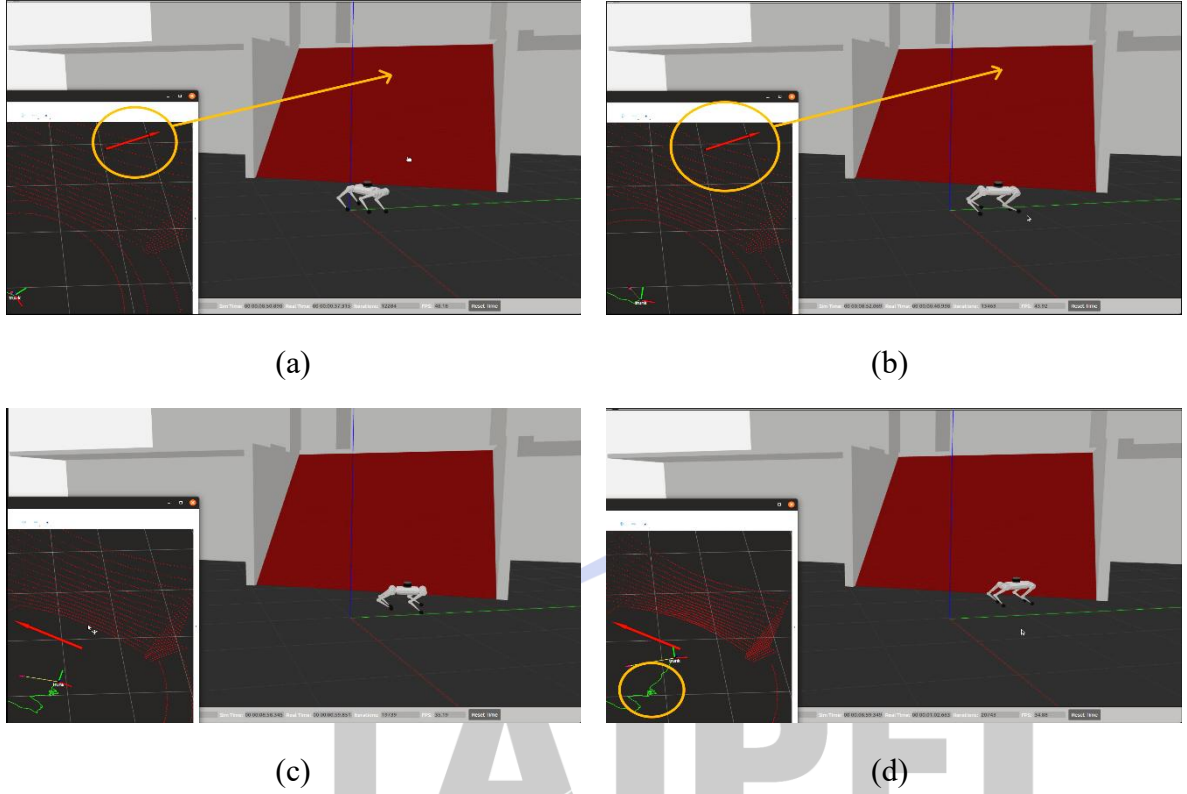


圖 5.58 45 度樓梯測試

值得一提的是，整個系統僅使用 CPU 進行模擬與 Gazebo 的渲染，並計算行為規劃策略。在此條件下，Gazebo 依然能穩定維持 30 fps 以上的運行速度，同時 CPU 使用率極低，充分展現了系統的高效性與穩定性。

5.6.5 系統更新頻率

RL 訓練的執行頻率為每秒 3 次，而在實際 ROS 系統中，執行頻率可以輕鬆的超過 1000 fps。雖然直接輸出 RL 訓練結果不會影響系統功能，但這樣的方式無法充分利用機器狗的運算性能。因此，本論文將多餘的運算資源用於提升控制精度。具體而言，利用更高的執行頻率來細化控制過程，實現速度的穩定調整。

$$v_{t_{n+1}} = v_{t_n} + A_{acc} \times dt/0.33 \quad (5.5)$$

如(5.5)式所示， A_{acc} 為 RL 行為規劃系統輸出的加速度， dt 表示 ROS 系統兩次執行間隔時間。通過調整公式中的執行時間比例，系統能夠在保持最大加速度不變的情況下，減少單次加速度的變化幅度，從而提升機器狗速度控制的穩定性。通過更高的執行頻率進行精細化控制，使得每一次加速度的變化更小，從而有效提升最終速度曲線的平滑度。

如圖 5.59 所示，粉紅色數據表示 RL 訓練期間的加速度控制，而橘色數據表示 ROS 系統中的執行結果。對比顯示，ROS 系統中每一次控制的加速度更加精準，並能在一樣的時間內穩定達到目標速度。



圖 5.59 速度控制曲線 (y 軸單位：公尺/秒，x 軸： dt)

5.7 實際測試

5.7.1 測試環境與設備

為了驗證 RL 行為規劃策略在實際環境中的效能，本論文使用 Boston Dynamics Spot 機器狗作為測試平台，並搭配高效能運算設備進行模型推論與數據分析。本節將介紹實驗所使用的訓練設備、機器人平台、感測器配置與邊緣計算設備，以確保測試條件的可重現性與結果的可靠性。

訓練配置

強化學習的訓練過程高度依賴 GPU 的計算能力，以加速策略學習與模型更新。本研究採用兩張 NVIDIA GeForce RTX 3060[49]顯示卡進行長達兩年的訓練，並額外使用 NVIDIA GeForce RTX 4080[50]進行三個月的數據收集與測試。在整個訓練過程中，累積了 16,331 小時的有效訓練數據，涵蓋多種環境、障礙物類型與機器人行為模式。這些長期累積的數據為本研究的實驗結果提供了堅實的理論與實證基礎。

機器人平台

本研究使用 Boston Dynamics Spot 作為測試平台，這是一款高機動性的四足機器人，能夠適應多種地形，如平地、狹窄通道及樓梯環境。Spot 內建的原廠避障系統可根據障礙物距離緊急停止，確保機器人在未知環境中的安全性。然而，由於原廠避障系統的執行策略為黑盒，且存在約一秒的控制延遲，因此本論文在系統設計預處理補償此問題，以提升決策的靈活性與適應性。此外，Spot 可透過 ROS 通訊架構接收外部控制指令，並回傳機器狀態資訊，使 RL 策略能夠調整速度控制，確保在現實環境的穩定性與適應性。

感測器配置

為了提升機器狗對環境的感知能力，本研究配置了多種感測器，以獲取不同類型的環境資訊。機器人搭載了一個 3D 激光雷達，提供精準的空間深度資訊，並搭配五個深度相機與一個 RGB 相機，以捕捉即時影像與距離資訊。此外，內建的 IMU 與測速計可用於機器人的姿態估計與移動速度監測，使行為規劃策略能夠根據感測數據進行動態調整。

邊緣運算設備

本研究選擇 NVIDIA Jetson AGX Orin[51]作為機器狗的運算平台。Orin 是一款專為邊緣運算設計的高效能 AI 計算模組，能夠支援機器狗的感測數據處理、行為決策與控制系統。其強大的運算能力允許即時執行 RL 模型，使機器狗能夠根據當前環境資訊快速調整行動策略。



圖 5.60 NVIDIA Jetson AGX Orin

最終完成的 RL 模型大小僅 1.5M 個 MLP 參數，相較於常見的 AI 模型，CNN 在參數需要重複使用的情況下，yolov4-tiny 還有 6M 個參數，本論文的模型顯示出更高的計算效率，適合大多數邊緣設備部署。此外，在 CPU 環境下，yolov4-tiny 可運行至 10 fps，而本研究的 RL 模型則可輕鬆超過 1000 fps。然而，由於 Spot 存在一秒的控制延遲，過高的控制頻率無法提供額外的優勢，因此最終決定以 20 fps 作為控制頻率，以平衡運算效率與實際應用需求。

測試環境

實際測試環境設置於大學校園內，涵蓋多種複雜場景，以實驗機器狗在真實環境中的應用潛力。為了驗證 RL 行為規劃策略的適應能力，測試場景包括有多個靜態障礙物、人流密集的走廊、狹窄通道及樓梯環境。透過上述環境測試，本研究能夠全面驗證 RL 行為規劃策略在現實世界中的適應性，並確保所提出的方法能夠應對多種複雜場景，提升機器狗在真實應用中的可靠性。

5.7.2 實際測試

為驗證 RL 行為規劃策略在不同場景下的實用性與適應能力，本論文設計多種真實環境模擬場景，並與 Spot 原廠避障系統進行性能比較。在所有測試中，機器狗均需在預設的環境中完成指定任務，過程中透過內建感測器記錄其移動軌跡、避障策略與任務執行情形。

測試內容涵蓋四種類型，包括空曠地形、靜態障礙物、動態障礙物與狹窄通道及樓梯。每個場景皆模擬實際應用中可能遇到的複雜情境，並透過相同控制策略與延遲補償條件進行對照測試。測試方式主要讓機器狗分別以 RL 行為規劃策略與 Spot 原廠避障系統（加上 PID 控制器）進行導航行為執行，並記錄其在一秒延遲補償下的移動軌跡、避障反應與任務完成時間，以評估兩種策略的差異。

空曠地形測試

為進一步量化不同策略於一般環境下的差異，本論文設計一組空曠地形測試，並針對環境中障礙物配置進行多種變化。如圖 5.61 所示，測試任務為機器狗從 p6 點出發，先抵達 r6 點停留 10 秒，再前往終點 e1。測試情境包含無障礙物、多個靜態障礙物與多個動態障礙物，藉此分析策略於不同情境下的應對能力。

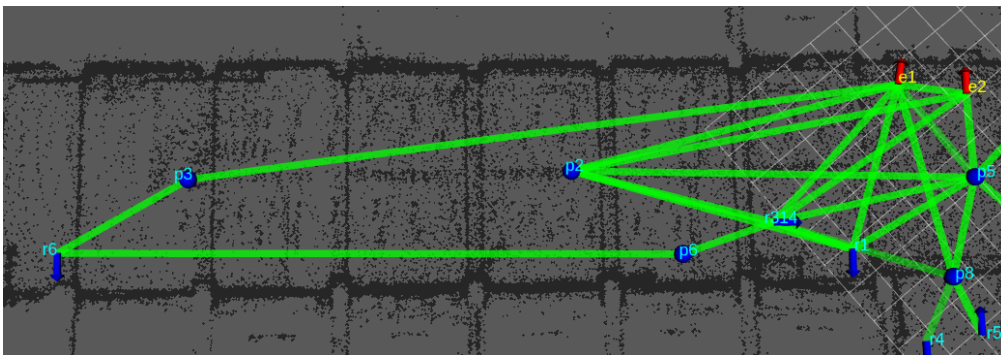


圖 5.61 測試場域點位



圖 5.62 測試場域

機器狗分別以 RL 行為規劃策略與 Spot 原廠避障系統搭配 PID 控制進行測試，如圖 5.63 所示，呈現了測試中多次執行的移動軌跡，其中橘色軌跡代表 RL 策略，綠色軌跡則為 Spot 原廠系統路徑，並以藍色漸層表示路徑上所花費的時間（時間越久顏色越深），每一種策略都執行十三次，顯示於的地圖上。

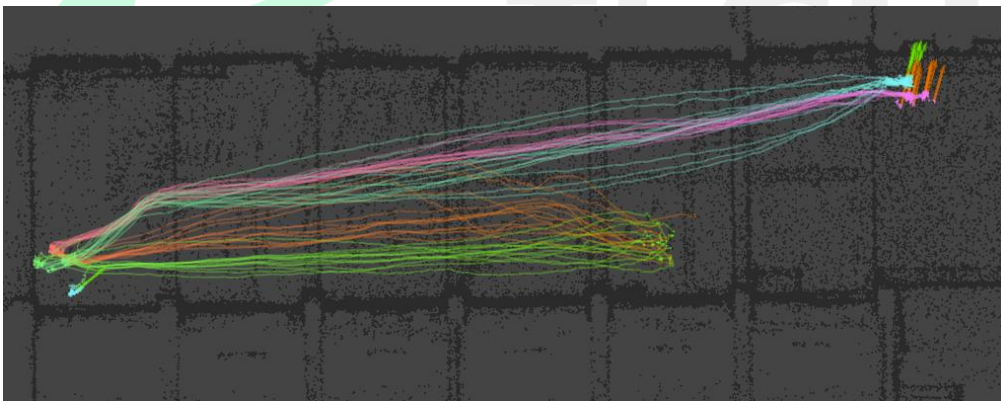


圖 5.63 空曠地形測試軌跡

如表 5.8 所示，RL 行為規劃策略能同時進行旋轉與移動，移動效率略高於原廠系統。另外在接近牆面區域，RL 策略傾向選擇更遠離牆面的路線，以減少未來潛在碰撞風險，而 Spot 系統則較傾向直線行進。在離牆遠一些的

地形，也可以看到 RL 行為規劃策略的路徑選擇反而更為集中。

表 5.8 空曠地形測試數據

移動策略	移動距離	通行時間	抵達成功率	減速次數
RL 行為規劃策略	41m	58.8s	100%	0
Spot 原廠避障系統	41m	61.6s	92.3%	0

靜態障礙物測試

靜態障礙物測試場景中設置多個固定物體，以觀察機器狗的繞行能力與避障效率。起點與終點與空曠場景一致，但中間路徑被數個障礙物佔據，使得繞行策略成為評估重點。



圖 5.64 靜態障礙物測試場域

如圖 5.65 所示，RL 行為規劃策略（橘）會優先選擇最短且順暢的可行路徑。相較之下，Spot 原廠避障系統在面對障礙物（綠），依然會選擇距離較短的路徑，反而可能會被障礙物卡住，或是需要額外的等待時間，影響整體通行效率。

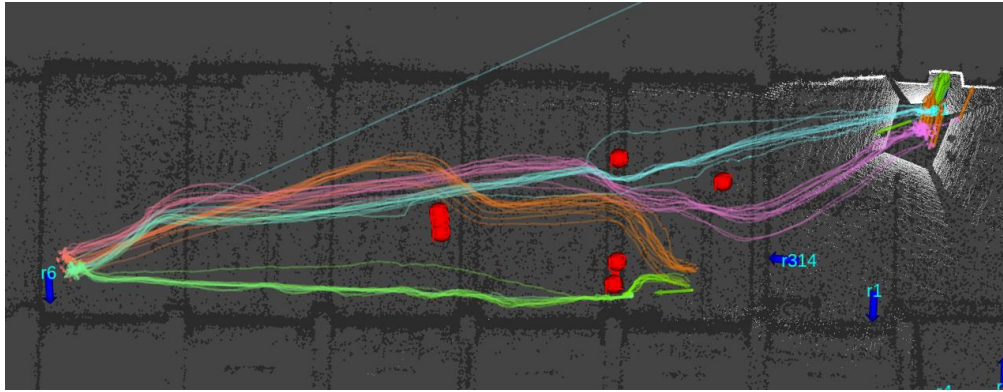


圖 5.65 靜態障礙物測試軌跡

如表 5.9 所示，RL 行為規劃策略能夠靈活適應環境，選擇更有效率的繞行路徑，並且在更早的時間進行避障，面對不同大小的障礙物還會選擇不同的避障距離，作到更快速抵達目的地的效果。

表 5.9 靜態障礙物測試數據

移動策略	通行時間	避障距離	減速次數	抵達成功率
RL 行為規劃策略	62.7s	35cm	0.0	100%
Spot 原廠避障系統	71.3s	6cm	2.0	84.6%

動態障礙物測試

為模擬真實世界中行人穿越的情境，本論文在靜態障礙物場景的基礎上，額外加入一名行人作為動態障礙物。當機器狗自 p6 點出發時，行人會自地圖左側向右行走；而當機器狗從 r6 前往 e1 點時，行人則從右側返回左側，如圖 5.66 所示。此設計強迫機器狗與行人產生動態交會，進一步驗證其面對移動障礙物時的避障能力。

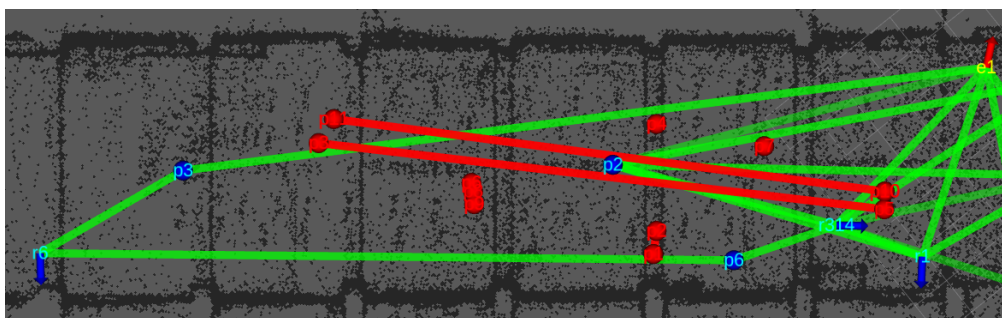


圖 5.66 動態障礙物測試場域

由於 Spot 原廠避障系統並不具備應對動態障礙物的能力，故本測試主要聚焦於比較 RL 行為規劃策略中，不同 RNN 預測時間對動態避障效果的影響。如圖 5.67 所示，橘色與黃色路徑分別對應於 RNN 預測一秒與二秒的策略結果。

在由 p6 前往 r6 的任務中可觀察到，RL 策略在偵測到行人後，會主動避讓、等待行人通過後再繞過靜態障礙物繼續前進。從 r6 前往 e1 的任務中亦可發現，策略原先偏好走中間路徑，改為選擇靠邊緣繞行，顯示其對於動態環境具備自我調整的能力。

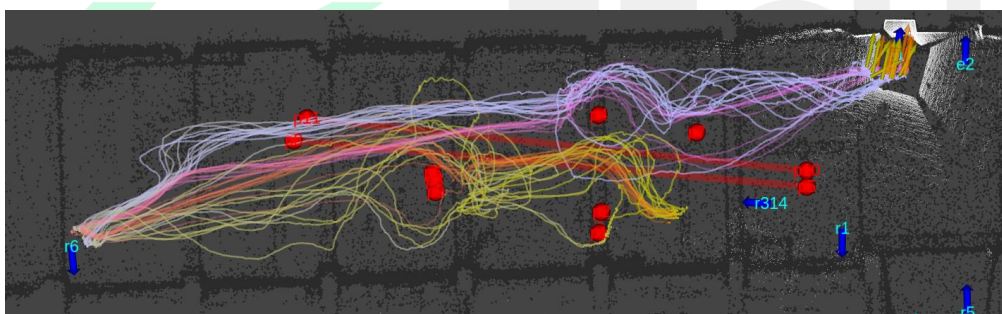


圖 5.67 動態障礙物測試軌跡

與靜態障礙物測試相比，RL 策略在動態場景下展現出更明顯的路徑調整行為。特別是在採用二秒預測的情況下，路徑變化幅度更大，能夠更早預測障礙物動向，並提前做出避障決策。

如表 5.10 所示，所有策略均成功避開行人，驗證其動態避障能力之有效性。

在避障距離方面，二秒預測策略能更早做出反應，提升避障安全性；但由於其行為相對保守，導致通行時間略長。綜合避障效果與通行效率後，本論文最終選定 RNN 預測一秒之 RL 策略作為標準版本。

表 5.10 動態障礙物測試數據

測試方式	通行時間	避障距離	抵達成功率	碰撞率
RL RNN 預測 1 秒	64.36s	12cm	100%	0%
RL RNN 預測 2 秒	79.66s	42cm	100%	0%

狹窄通道測試

狹窄通道測試選擇了辦公室作為實驗場域，走道寬度分別為 70 公分與 91 公分。機器狗需從 70 公分寬的走道出發，穿越環境後原路返回起點，如圖 5.69 所示。由於 Spot 本身寬度為 56 公分，在最窄的通道中，兩側僅剩 7 公分的間隙，對行動規劃策略的準確性與靈活度提出嚴峻考驗，如圖 5.69 所示。

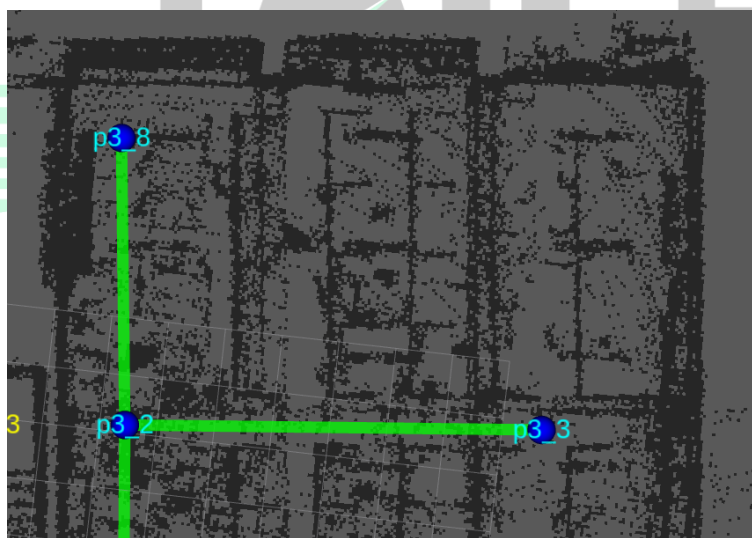


圖 5.68 狹窄通道測試場域點位



圖 5.69 狹窄通道測試場域

如圖 5.70 所示，在進入狹窄區域需要往回走時，RL 行為規劃策略（橘）會直接選擇後退通過，避免不必要的旋轉操作，大幅降低卡住的風險。相比之下，Spot 原廠避障系統結合 PID 控制器，傾向於先轉向再移動，導致在狹窄環境中容易因旋轉空間不足而發生卡頓。

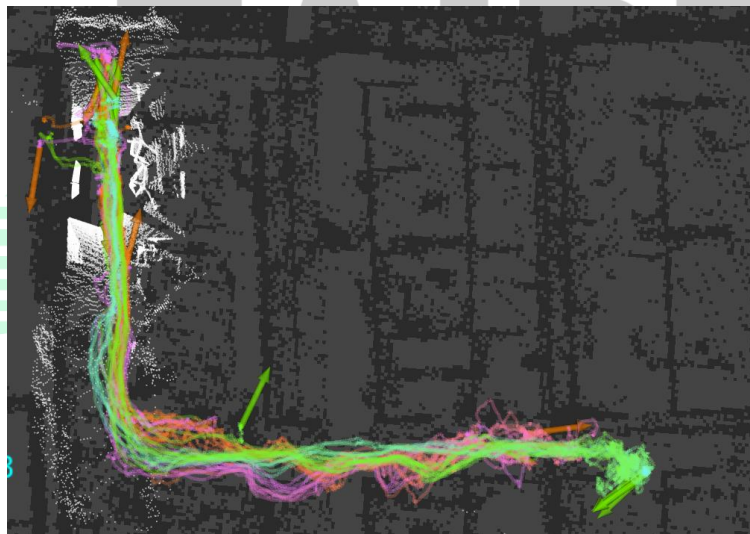


圖 5.70 狹窄通道測試軌跡

此外，RL 策略還具備動態速度調整能力。在狹窄區域中，其會自動減速至極低速度，評估風險後才安全通行；反之，Spot 原廠系統只有在距離障礙物極近時才執行避障行為，並使用高速移動，碰撞風險相對較高。

如表 5.11 所示，系統測試中還發現一項關鍵問題，當 Spot 距離障礙物過近

時，即使更改目標位置，也可能觸發其內建的安全保護機制，導致任務中斷，無法持續控制，這也成為 RL 策略某些失敗案例的主因。

表 5.11 狹窄通道測試結果

移動策略	移動距離	通行時間	碰撞率	抵達成功率
RL 行為規劃策略	15.9m	84.8s	0%	84.6%
Spot 原廠避障系統	15.9m	59.2s	15.3%	61.5%

樓梯測試

為評估機器狗於高度變化場景中的表現，原擬設計不同台階高度的樓梯場景，測試其在上下樓梯時的穩定性。然而，在實際應用中發現，由於 RL 行為規劃策略依賴即時感測資訊（如 IMU 與 LiDAR）進行坡度估算與步態調整，一秒的延遲效應導致系統無法即時掌握樓梯幾何資訊與機器狗當前姿態，進而無法產生穩定且可信的輸入特徵。

儘管系統具備融合 IMU 與 LiDAR 進行樓梯坡度分析的能力，但在延遲情況下，IMU 的即時性不足以及 LiDAR 幾何結構辨識的瞬時性需求難以配合，導致推算結果不穩定。基於安全性考量，本研究並未實際進行樓梯場景之實體測試，僅保留於模擬環境中進行初步分析。

多動態障礙物測試

多動態障礙物測試數據較難量化，附上實際測試效果。



(a)



(b)



(c)



(d)



(e)



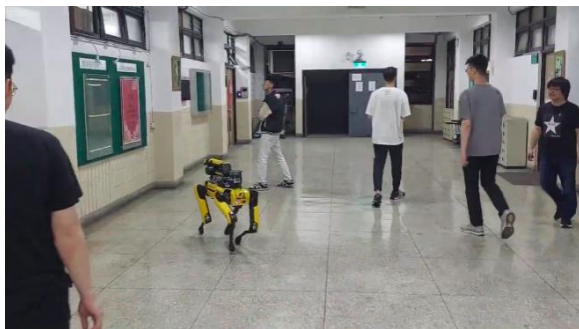
(f)



(g)



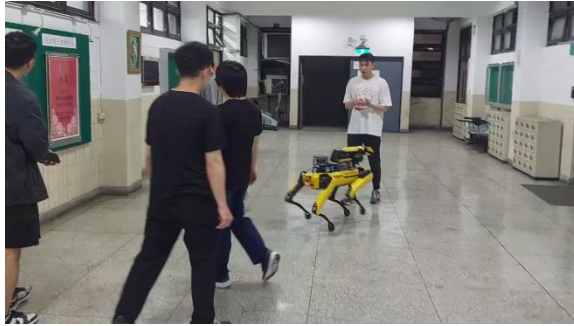
(h)



(i)



(j)



(k)



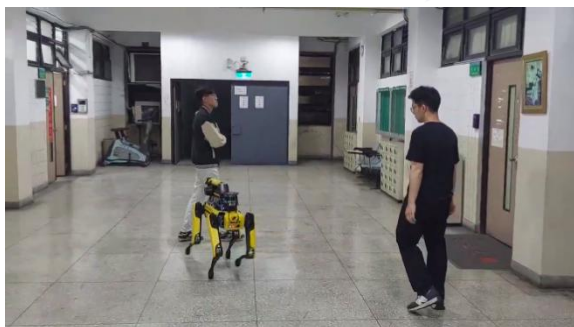
(l)



(m)



(n)



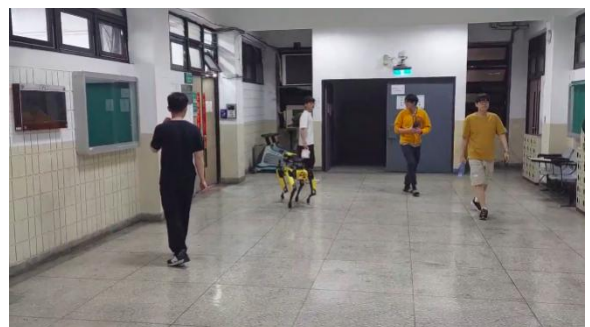
(o)



(p)



(q)



(r)



(s)



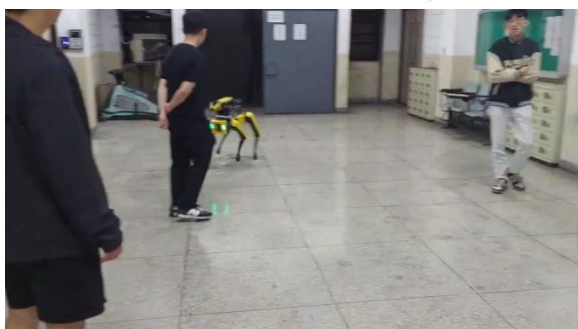
(t)



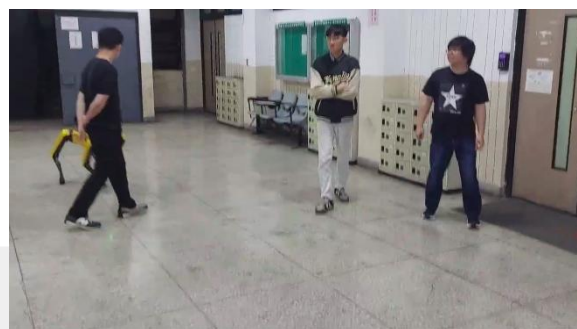
(u)



(v)



(w)



(x)



(y)



(z)



(A)



(B)



(C)



(D)



(E)



(F)



(G)



(H)



(I)



(J)



(K)



(L)



(M)



(N)



(O)



(P)



(Q)



(R)



(S)



(T)

圖 5.71 多動態障礙物測試

圖 5.71 顯示實際測試中機器狗的行為表現，在動態障礙物（如行人）頻繁出現的環境中，仍能有效避障並穩定抵達目標位置，展現出良好的環境適應能力與規劃靈活性。圖 5.72 則呈現於相同場景下的機器狗實際移動軌跡，可觀察其在多個移動障礙物干擾下，依然維持平穩且合理的規劃路徑。

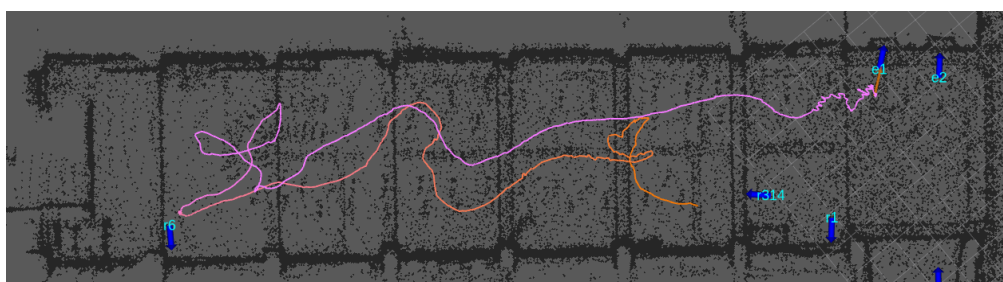


圖 5.72 多動態障礙物測試軌跡

5.7.3 Sim2Real 挑戰與延遲補償方法

雖然在強化學習的訓練過程中已經對 Sim2Real 做了很多努力，但實際應用中這依然是不可避免的挑戰。由於模擬環境通常假設決策執行為即時響應，而現實環境中則受到感測器延遲、硬體響應時間以及底層控制機制的影響，導致策略遷移時出現性能下降，如論文[52]所示。

Sim2Real 中的延遲問題

在模擬環境中，機器狗的決策輸出可以即時執行，系統不會有額外的反應時間。然而，在真實環境中，當外部系統發送控制指令給 Spot 時，指令會先經過 Spot 原廠避障系統，然後再執行對應的動作。這導致從指令發出到實際執行之間約有一秒的延遲，而這一秒的時間不僅包含通訊傳輸，還包含 Spot 原廠避障的計算與處理時間，這使得直接應用模擬訓練的 RL 模型時會面臨以下問題。

速度估測失準：

系統若將目前的速度當成加速度控制的基本速度，會導致加速度無法正確的影響速度，除了會發生嚴重的震盪外，延遲的誤差也會被回環多次。

目的地估測失準：

實際控制的動作是一秒前的指令，因此目的地相對於 Spot 的位置和方向已經發生改變。

靜態障礙物避障失準：

機器狗可能在接近障礙物後才開始繞行，導致原本的避障策略失效，甚至產生不必要的停滯。

動態障礙物錯誤判斷：

在高人流環境中，機器狗的反應滯後會導致錯誤預測人員的移動方向，影響避障成功率。

樓梯資訊計算誤差：

樓梯環境需要 IMU 和 LiDAR 數據進行坡度計算，但感測器數據受到機器人移動的影響本身雜訊較大，加上一秒延遲，使得未來資訊推估變得困難，影響系統穩定性。

上述問題說明，控制延遲是一個關鍵的 Sim2Real 挑戰，若不進行補償，RL 策略的實際執行效果將遠不如模擬結果。

延遲補償方法

為了解決控制延遲所導致的狀態誤差，本論文並非單純等待感測器回饋再進行修正，而是參考模型預測控制的概念，在每次規劃時即刻預測一秒後的機器人狀態（包含速度、位置、朝向），並將其作為輸入送入 PID 控制器或 RL 策略中運算，如此即使存在一秒的系統延遲，所產生的動作仍能對應一秒後的真實情境，達成延遲補償。

值得注意的是，本論文使用的 RNN 模型則專注於預測未來障礙物的位置變化，用來處理動態環境中的避障判斷。因此，雖然兩者都涉及時間上的預測，但應用面上並不重疊。延遲補償是針對機器狗自身控制與狀態預測，RNN 模型是針對外部動態環境的時序推論。

要確認延遲的方法需要紀錄真實機器狗中的實際數據，如圖 5.73 所示。右側的 y 軸為真實的時間資訊，x 軸是紀錄的數據點，其中的每一張圖的 x 軸都是在同一時間點上，左側資訊為 x 方向的速度控制，上圖為速度控制的輸入，下圖為實際機器狗的速度，由圖 5.73 可以得知，這一次的控制延遲為 0.72 秒，而計算平均的延遲約為一秒。



圖 5.73 延遲的計算方式

未來速度計算

Spot 的速度控制受一秒延遲影響，這表示當前時間點 t_n 下的補償速度 $v_{t_n}^{cmd}$ 並不會立即生效，而是對應於一秒後的真實速度 $v_{t_{n+1}}^{real}$ 。這一特性導致如果不進行適當的補償，機器狗的速度控制將無法準確反映預期的運動行為，如(5.6)式所示。

$$v_{t_n}^{cmd} = v_{t_{n+1}}^{real} \quad (5.6)$$

為了準確估計一秒後的速度，應當基於上一時刻的控制速 $v_{t_n}^{cmd}$ 作為基準，並考慮當前時刻的加速度輸出來修正速度，而非直接使用 $v_{t_{n+1}}^{real}$ 當成速度的基準，修正後的未來速度計算方式，如(5.7)式所示。

$$v_{t_{n+dt}}^{cmd} = v_{t_n}^{cmd} + A_{acc} \times dt \quad (5.7)$$

未來位置計算

要計算 Spot 未來位置相對於現在位置的計算方式，未來一秒的速度會從 $v_{t_n}^{real}$ 變化到 $v_{t_n}^{cmd}$ ，速度變化函數如(5.8)式所示，速度從當前值逐漸變化到補償後的控制值。

$$f_v(t) = (1 - t) \cdot v_{t_n}^{real} + t \cdot v_{t_n}^{cmd} \quad (5.8)$$

轉向角變化函數如(5.9)式所示，假設轉向角為 ϖ ，其變化是等速旋轉，則可以表示為。

$$f_{\varpi}(t) = t \times \varpi \quad (5.9)$$

考慮到速度和角度的變化，相對於原本機器狗的坐標系的距離，使用積分來計算，可得(5.10)式和(5.11)式。實際運算方式 dt 設定為 0.05 秒，以確保準確程度運算效率。

$$\Delta x = \int_0^1 f_v^x(t) \times \cos(f_{\varpi}(t)) \times dt - f_v^y(t) \times \sin(f_{\varpi}(t)) \times dt \quad (5.10)$$

$$\Delta y = \int_0^1 f_v^x(t) \times \sin(f_{\varpi}(t)) \times dt + f_v^y(t) \times \cos(f_{\varpi}(t)) \times dt \quad (5.11)$$

最後再從 Spot 坐標系轉換回實際的 Odom 坐標系的位置和角度，如此就可以取得下一秒 Spot 的坐標系，如(5.12)式和(5.13)式所示。

$$loc_{t_n+dt}^{cmd} = loc_{t_n}^{real} + R(yaw_{t_n}^{real}) \cdot \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \quad (5.12)$$

$$yaw_{t_n+dt}^{cmd} = yaw_{t_n}^{real} + \varpi_{t_n}^{cmd} \times 1.0 \quad (5.13)$$

loc 和 yaw ：為 Odom 坐標系的位置和角度

未來目的地位置計算

當 Spot 以一秒延遲執行動作時，目標位置相對於機身座標系的位置會發生變化，因此需要進行修正，如(5.14)式所示。

$$Goal_{loc} = Goal_{t_n}^{real} - R(yaw_{t_n}^{real}) \cdot \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \quad (5.14)$$

然後再轉換到相對於未來一秒機器人坐標系下的目的地位置資訊，如(5.15)式所示。

$$Goal_{t_n+dt}^{cmd} = \begin{bmatrix} \cos(\varpi_{t_n}^{cmd}) \times Goal_{loc}^x - \sin(\varpi_{t_n}^{cmd}) \times Goal_{loc}^y \\ \sin(\varpi_{t_n}^{cmd}) \times Goal_{loc}^x + \cos(\varpi_{t_n}^{cmd}) \times Goal_{loc}^y \end{bmatrix} \quad (5.15)$$

未來障礙物位置計算

類似地，靜態與動態障礙物的相對位置也會發生變化，需要進行補償，如(5.16)式和(5.17)式所示。

$$Obs_{loc} = Obs_{t_n}^{real} - R(yaw_{t_n}^{real}) \cdot \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \quad (5.16)$$

$$Obs_{t_n+dt}^{cmd} = \begin{bmatrix} \cos(\varpi_{t_n}^{cmd}) \times Obs_{loc}^x - \sin(\varpi_{t_n}^{cmd}) \times Obs_{loc}^y \\ \sin(\varpi_{t_n}^{cmd}) \times Obs_{loc}^x + \cos(\varpi_{t_n}^{cmd}) \times Obs_{loc}^y \end{bmatrix} \quad (5.17)$$

未來動態障礙物位置計算

延遲的影響對於動態和靜態障礙物有一些差別，主要在於這一秒的時間動態障礙物也會移動，會影響 Spot 對於障礙物位置的判斷。直覺上，若能預測更遠的未來（如二秒後的障礙物位置），除了能降低延遲的影響，機器狗也可以更早做出避障決策，理論上能提升安全性。然而，在實際測試中，發現估計時間過長會導致以下問題。

取得 *Sweepangle* 數據的過程本身存在雜訊，當預測時間超過一秒，這些誤差會累積並影響決策準確性，特別是高動態環境中或是 Spot 行走產生的晃動，可能導致不必要的避障行為。

當使用 RNN Model 預測兩秒後的環境時，並且用此訓練 RL 行為規劃策略，策略決策可能會偏向更保守的繞行，導致在行走的行為不穩定。例如，在通

道內行進時，機器狗可能會提前選擇過度閃避，並且遠離全部的障礙物，如此反而影響通行效率。

基於這些發現，本論文最終選擇估計一秒、延遲一秒的控制策略，這能使 RL 策略能夠在兼顧決策準確性與穩定性的前提下，適應 Spot 的執行延遲，並在動態環境中保持流暢的行為規劃。

未來樓梯資訊估計

樓梯相關的幾何資訊，必須先將點雲資料轉換至水平的座標系。此座標系需仰賴 IMU 所提供的重力加速度方向進行建立，並透過 Odom 資訊推算機器人當下的加速度資訊。由於重力加速度的大小遠大於機器人運動所產生的加速度，因此可簡化假設將 IMU 所量測到的主加速度視為重力方向，而機器人運動所造成的加速度則視為在水平面內的小量，近似為零。透過此方法，即可推算出目前點雲資料相對於水平面的座標系資訊，並將其轉換為代表地形梯度特徵 $Sweep_{Map-gradient}$ 訊號。

需要特別注意的是，IMU 的座標系位置與機器人本體的座標系並不一致，因此當機器人發生旋轉時，IMU 還會受到角速度所引發的額外加速度影響。這種旋轉造成的加速度可藉由座標系間的位置差與角速度資訊轉換並補償，以確保重力方向的估算精準。

考量到系統本身存在約一秒的延遲，因此在獲得當下樓梯資訊後，仍需進一步推論一秒後機器人實際觀測到的樓梯幾何。由於樓梯點雲已經轉換至水平座標系，重力方向所對應的 z 軸資訊不會隨時間改變，因此只需針對 x 與 y 平面內的相對位移進行補償，即可獲得未來一秒的預測資訊，補償方式如 (5.18) 式和 (5.19) 式所示。

$$Sweep_{loc} = Sweep_{t_n}^{real} - R(yaw_{t_n}^{real}) \cdot \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \quad (5.18)$$

$$Sweep_{t_n+dt}^{cmd} = \begin{bmatrix} \cos(\varpi_{t_n}^{cmd}) \times Sweep_{loc}^x - \sin(\varpi_{t_n}^{cmd}) \times Sweep_{loc}^y \\ \sin(\varpi_{t_n}^{cmd}) \times Sweep_{loc}^x + \cos(\varpi_{t_n}^{cmd}) \times Sweep_{loc}^y \end{bmatrix} \quad (5.19)$$

5.7.4 實際測試結果討論

本節綜合各項測試結果，深入探討 RL 行為規劃策略在不同測試場景中的表現，並與 Spot 原廠避障系統進行對比分析。強化學習策略在多種環境下展示出更高的靈活性與適應性，尤其在處理動態與不確定環境時，相比於傳統的避障系統，RL 顯示了顯著的優勢。

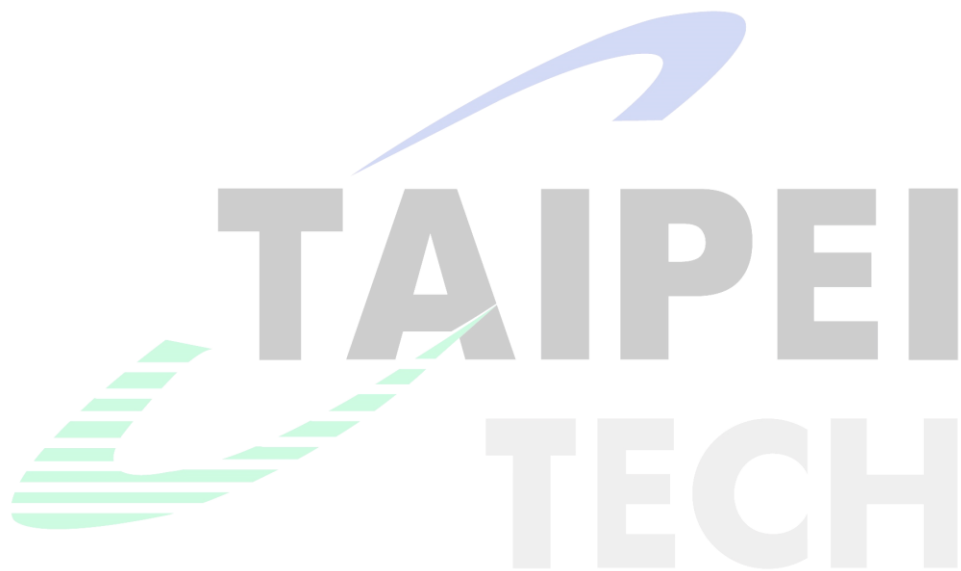
在所有測試中，無論是靜態障礙物、動態障礙物還是狹窄通道的挑戰，RL 行為規劃策略能夠根據環境的變化靈活調整行為，實現更高效的避障與路徑規劃。尤其是在動態環境中，RL 策略透過預測與提前決策，能夠更好地避免突發狀況，顯示出其優越性。然而，樓梯測試中的表現受到感測器雜訊與延遲的影響，雖然仍有進步空間，但 RL 策略在這一挑戰性場景中，仍顯示出相當程度的學習與應變能力。

進一步的測試結果表明，強化學習策略的自適應能力極強，能夠靈活調整以應對不同的環境挑戰。當面對靜態障礙物時，RL 策略可以迅速規劃出最短且最安全的路徑，而 Spot 原廠避障系統則往往因為策略不足，導致系統停滯。對於動態障礙物，RL 策略能提前預測並作出有效閃避，避免了急停或過晚的反應。當進入狹窄通道時，RL 策略能準確評估風險並做出合理的進退決策，從而避免不必要的擁擠與停滯，這是 Spot 系統較難實現的。樓梯環境中，RL 策略受到 IMU 和 LiDAR 數據雜訊的影響，表現相對較弱。儘管如此，RL 展現了其強大的靈活性與學習能力，未來隨著算法的進步，這些問題有望得到改善。

總結來看，本論文的測試結果顯示，RL 行為規劃策略在多變的環境中具有顯著的優勢，尤其在避障靈活性、適應性以及決策穩定性方面，表現優於 Spot 原廠避障系統。RL 策略能夠根據環境的變化進行學習並做出合理的行為規劃，而

延遲補償技術則有效減少了控制延遲的影響，進一步促進了 RL 策略在真實環境中的成功應用。

如需觀看更多的測試內容與影片，請參考以下 YouTube 測試視頻連結：
<https://www.youtube.com/@ken4600>。



第六章 結論與未來展望

6.1 結論

本論文通過 RL 技術的應用，對比了其與其他 AI 架構的核心差異。傳統 AI 架構依賴從訓練數據中尋找策略，而 RL 的特點在於先訓練出有效策略，再將現有資訊轉換為適合 RL 判斷的形式。這一特性使 RL 能夠減少多餘的認知和辨識需求，縮減模型規模，同時降低因辨識錯誤導致的策略偏差。

研究表明，雖然 RL 能夠訓練出性能極高的策略，但要讓訓練出來的策略表現出真正聰明的行為，訓練過程十分複雜且具挑戰性。尤其是在新增能力時，必須考量其與現有能力的相互影響，這進一步增大了訓練的難度和工作量。

因此，本論文認為 RL 技術適合用於尋找最優的行為策略，而非資料分析。若任務需要進行資料分析的策略，應由其他 AI 架構先完成資訊提取，將關鍵資料整理後再交由 RL 進行策略判斷與執行。而對於完全不涉及策略決策的任務，則不應使用 RL，因為其高運算成本與複雜性會導致資源浪費且效率低下。

在系統開發過程中，儘管對行為規劃策略進行了多次測試，但在系統移植至機器狗的實驗中仍然遇到了諸多挑戰。在開發過程中的反覆試錯與調整，與 RL 訓練中 Agent 尋找最優策略的過程如出一轍。Agent 專注於行為規劃策略的優化，而我則專注於修改設計以實現系統整體的最佳化。

這種相似性啟發了我在策略優化中的理解。RL 的訓練過程不僅加深了我對策略設計的認識，也使我在實踐中收穫了寶貴的經驗。因此，本論文將 RL 訓練中的 Agent 策略討論整合於章節 **RL 行為規劃策略**，並將系統開發與測試中的學習與經驗整理於章節 **實驗測試**。

在初入實驗室時，我選擇專注於強化學習技術。經過兩年的努力，雖然結果尚有改進空間，但目前的成果已是全力以赴的展現。最後，特別感謝姚立德教授提供充分的開發週期，讓我能在反覆試錯中找到解決方案，並完成這項研究。

6.2 未來展望

隨著技術的發展，將大型語言模型(LLM)應用於 Multi-Agent 系統已成為一個具有潛力的研究方向。RL 的核心瓶頸是如何獲取準確且高效的輸入資料，而 LLM 擅長處理非結構化數據並提取關鍵資訊，可有效彌補這一短板。透過 LLM 將複雜數據轉化為適合 RL 使用的結構化資訊，可能可以提升訓練效率，並拓展 RL 的應用場景。

LLM 和 RL 的結合，預期能夠提升 Multi-Agent 系統的整體效能。在 Multi-Agent 協作中，LLM 提供語義理解與資訊整合，RL 專注於策略學習與動態優化，兩者相輔相成，不僅增強了系統的適應性，也能提高了協作效率。

例如，LLM 可分析環境資訊，協助 Agent 理解任務需求，而 RL 負責行為決策與即時調整，確保 Agent 能快速適應環境變化。雖然這一技術融合的落地仍面臨挑戰，但其應用潛力值得期待，不僅能推動技術創新，也為智能系統的實際應用帶來新的可能。

參考文獻

- [1] Autoware Foundation. (n.d.). *Autoware*. Available at <https://autoware.org>
- [2] Tesla. (2020, June 1). *Tesla News*. Available at <https://www.ddcar.com.tw/article/4253>
- [3] Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). *CARLA: An Open Urban Driving Simulator*. Available at <https://carla.org>
- [4] Richard S. Sutton and Andrew G. Barto (2014). *Reinforcement Learning: An Introduction*. Available at: <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>
- [5] DeepMind. (2017, May 27). *AlphaGo*. Available at <https://deepmind.google/technologies/alphago/>
- [6] OpenAI. (2024, May 14). *ChatGPT*. Available at <https://chatgpt.com/>
- [7] Ng, A. (2023, July 26). *Andrew Ng: Opportunities in AI - 2023*. Retrieved from YouTube: <https://www.youtube.com/watch?v=5p248yao3oE>
- [8] T 客邦 (Techbang). (2024, May 17). 為什麼特斯拉 AI「看不出」路中央橫躺的大貨車？如果貨車車主放了三角錐或許結果會不同. Available at: <https://www.techbang.com/posts/78917>
- [9] Phan, T., Driscoll, J., Romberg, J., & Koenig, S. (2024, January 11). *Confidence-Based Curriculum Learning for Multi-Agent Path Finding*. *arXiv preprint*, arXiv:2401.05860. Available at <https://arxiv.org/abs/2401.05860>
- [10] Thorne, D., & Lopez, B. T. (2023, October 9). *Multi-Robot Task Assignment and Path Finding for Time-Sensitive Missions with Online Task Generation*. *arXiv preprint*, arXiv:2310.06153. Available at <https://arxiv.org/abs/2310.06153>
- [11] Chung, J., Fayyad, J., Al Younes, Y., & Najjaran, H. (2023, August 11). *Learning*

- Team-Based Navigation: A Review of Deep Reinforcement Learning Techniques for Multi-Agent Pathfinding*. *arXiv preprint*, arXiv:2308.05893.
Available at <https://arxiv.org/abs/2308.05893>
- [12] Truong, J., Rudolph, M., Yokoyama, N., Chernova, S., Batra, D., & Rai, A. (2022, November 25). *Rethinking Sim2Real: Lower Fidelity Simulation Leads to Higher Sim2Real Transfer in Navigation*. *arXiv preprint*, arXiv:2207.10821.
Available at <https://arxiv.org/abs/2207.10821>
- [13] fast.ai. (2023, February 16). *bdd_simple_1k [Dataset]*. Available at Weights & Biases: https://wandb.ai/fastai/bdd_simple_1k
- [14] Google. (2016). *Cartographer ROS*. Available at <https://google-cartographer-ros.readthedocs.io>
- [15] Z Zhu, X., Lyu, S., Wang, X., & Zhao, Q. (2021, August 26). *TPH-YOLOv5: Improved YOLOv5 Based on Transformer Prediction Head for Object Detection on Drone-captured Scenarios*. *arXiv preprint*, arXiv:2108.11539.
Available at <https://arxiv.org/abs/2108.11539>
- [16] Sodhani, S., Zhang, A., & Pineau, J. (2021, June 10). *Multi-Task Reinforcement Learning with Context-based Representations*. *arXiv preprint*, arXiv:2102.06177. Available at <https://arxiv.org/abs/2102.06177>
- [17] Driess, D., Xia, F., Sajjadi, M. S. M., Lynch, C., Chowdhery, A., Ichter, B., Wahid, A., Tompson, J., Vuong, Q., Yu, T., Huang, W., Chebotar, Y., Sermanet, P., Duckworth, D., Levine, S., Vanhoucke, V., Hausman, K., Toussaint, M., Greff, K., Zeng, A., Mordatch, I., & Florence, P. (2023, March 6). *PaLM-E: An Embodied Multimodal Language Model*. *arXiv preprint*, arXiv:2303.03378.
Available at <https://arxiv.org/abs/2303.03378>
- [18] Yang, R., Xu, H., Wu, Y., & Wang, X. (2020, December 7). *Multi-Task Reinforcement Learning with Soft Modularization*. *arXiv preprint*,

- arXiv:2003.13661. Available at <https://arxiv.org/abs/2003.13661>
- [19] Kipf, T., van der Pol, E., & Welling, M. (2020, January 5). *Contrastive Learning of Structured World Models*. *arXiv preprint*, arXiv:1911.12247. Available at <https://arxiv.org/abs/1911.12247>
- [20] Shah, D., Sridhar, A., Bhorkar, A., Hirose, N., & Levine, S. (2023, May 22). *GNM: A General Navigation Model to Drive Any Robot*. *arXiv preprint*, arXiv:2210.03370. Available at <https://arxiv.org/abs/2210.03370>
- [21] Zhu, Y., Mottaghi, R., Kolve, E., Lim, J. J., Gupta, A., Fei-Fei, L., & Farhadi, A. (2016, September 16). Target-driven Visual Navigation in Indoor Scenes using Deep Reinforcement Learning. *arXiv preprint*, arXiv:1609.05143. Available at: <https://arxiv.org/abs/1609.05143>
- [22] Tai, L., Paolo, G., & Liu, M. (2017, March 1). Virtual-to-Real Deep Reinforcement Learning: Continuous Control of Mobile Robots for Mapless Navigation. *arXiv preprint*, arXiv:1703.00420. Available at: <https://arxiv.org/abs/1703.00420>
- [23] Mirowski, P., Pascanu, R., Viola, F., Soyer, H., Ballard, A. J., Banino, A., Denil, M., Goroshin, R., Sifre, L., Kavukcuoglu, K., Kumaran, D., & Hadsell, R. (2016, November 11). Learning to Navigate in Complex Environments. *arXiv preprint*, arXiv:1611.03673. Available at: <https://arxiv.org/abs/1611.03673>
- [24] Zhang, R., Xu, Z., Ma, C., Yu, C., Tu, W.-W., Huang, S., Ye, D., Ding, W., Yang, Y., & Wang, Y. (2024, August 2). *A Survey on Self-play Methods in Reinforcement Learning*. *arXiv preprint*, arXiv:2408.01072. Available at <https://arxiv.org/abs/2408.01072>
- [25] Gleave, A., Dennis, M., Wild, C., Kant, N., Levine, S., & Russell, S. (2019 May 25). *Adversarial Policies: Attacking Deep Reinforcement Learning*. *arXiv preprint*, arXiv:1905.10615. Available at <https://arxiv.org/abs/1905.10615>

- [26] Silver, D., & Hassabis, D. (2017, October 18). *AlphaGo Zero*. Available at <https://deepmind.google/discover/blog/alphago-zero-starting-from-scratch>
- [27] Yu, H., Yang, Z., & Tan, X. (2022 Jun 21). *EnvPool: A Highly Parallel Reinforcement Learning Environment Execution Engine*. *arXiv preprint*, arXiv:2206.10558. Available at <https://arxiv.org/abs/2206.10558>
- [28] Nair, A., Srinivasan, P., Blackwell, S., Alcicek, C., Fearon, R., De Maria, A., ... & Legg, S. (2015 Jul 15). *Massively Parallel Methods for Deep Reinforcement Learning*. *arXiv preprint*, arXiv:1507.04296. Available at <https://arxiv.org/abs/1507.04296>
- [29] Lan, T., Srinivasa, S., Wang, H., & Zheng, S. (2021, October 8). *WarpDrive: Extremely Fast End-to-End Deep Multi-Agent Reinforcement Learning on a GPU*. *arXiv preprint*, arXiv:2108.13976. Available at <https://arxiv.org/abs/2108.13976>
- [30] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017 Mar 20). *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*. *arXiv preprint*, arXiv:1703.06907. Available at <https://arxiv.org/abs/1703.06907>
- [31] Qin, S. J., & Badgwell, T. A. (2003 Jan 1). A Survey of Industrial Model Predictive Control Technology. *Control Engineering Practice*, 11(7), 733–764. Available at [https://doi.org/10.1016/S0967-0661\(02\)00186-7](https://doi.org/10.1016/S0967-0661(02)00186-7)
- [32] Barreto, A., Hou, S., Borsa, D., & Precup, D. (2020, August 17). *Fast Reinforcement Learning with Generalized Policy Updates*. *Proceedings of the National Academy of Sciences*. Available at <https://www.pnas.org/doi/10.1073/pnas.1907370117>
- [33] Chen, X., Qi, Y., Yin, Y., Chen, Y., Liu, L., & Chen, H. (2023). *A Multi-Stage Deep Reinforcement Learning with Search-Based Optimization for Air–Ground*

- Unmanned System Navigation. Applied Sciences*, 13(4), 2244. Available at <https://www.mdpi.com/2076-3417/13/4/2244>
- [34] Cesa-Bianchi, N., Gentile, C., Lugosi, G., & Neu, G. (2017, May 29). *Boltzmann Exploration Done Right. arXiv preprint*, arXiv:2108.13976. Available at <https://arxiv.org/abs/1705.10257>
- [35] Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. arXiv preprint*, arXiv:1801.01290. Available at <https://arxiv.org/abs/1801.01290>
- [36] Driess, D., & Worm, R. (2022). *Stable Baselines3*. Retrieved June 27, 2024, from <https://stable-baselines3.readthedocs.io/>
- [37] Su, P.-H., Budzianowski, P., Ultes, S., Gasic, M., & Young, S. (2017, July 5). *Sample-efficient Actor-Critic Reinforcement Learning with Supervised Data for Dialogue Management. arXiv preprint*, arXiv:1707.00130. Available at <https://arxiv.org/abs/1707.00130>
- [38] Su, P.-H., Budzianowski, P., Ultes, S., Gasic, M., & Young, S. (n.d.). *Reciprocal n-Body Collision Avoidance*. Available at https://link.springer.com/chapter/10.1007/978-3-642-19457-3_1
- [39] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017, July 20). *Proximal Policy Optimization Algorithms. arXiv preprint*, arXiv:1707.06347. Available at <https://arxiv.org/abs/1707.06347>
- [40] Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., ... & Kavukcuoglu, K. (2016 Feb 4). *Asynchronous Methods for Deep Reinforcement Learning. arXiv preprint*, arXiv:1602.01783. Available at <https://arxiv.org/abs/1602.01783>
- [41] Kobayashi, T., & Ilboudo, W. E. L. (2020, December 25). *t-Soft Update of Target*

- Network for Deep Reinforcement Learning*. *arXiv preprint*, arXiv:2008.10861. Available at <https://arxiv.org/abs/2008.10861>
- [42] Eysenbach, B., Salakhutdinov, R., & Levine, S. (2019, June 12). *Search on the Replay Buffer: Bridging Planning and Reinforcement Learning*. *arXiv preprint*, arXiv:1906.05253. Available at <https://arxiv.org/abs/1906.05253>.
- [43] Van Hasselt, H., Guez, A., & Silver, D. (2015 Sep 22). *Deep Reinforcement Learning with Double Q-learning*. *arXiv preprint*, arXiv:1509.06461. Available at <https://arxiv.org/abs/1509.06461>
- [44] OpenAI. (2017, July 20). *Proximal Policy Optimization Algorithms*. *arXiv preprint*, arXiv:1707.06347. Available at <https://arxiv.org/abs/1707.06347>
- [45] Weights & Biases. (n.d.) Retrieved from <https://wandb.ai/>.
- [46] ROS. (2020, May). Retrieved from <https://wiki.ros.org/ROS/Installation>
- [47] Freiburg, S. R. L. (2018). *pedsim_ros*. Available at https://github.com/srl-freiburg/pedsim_ros
- [48] Gazebo. (2020, May 29). Retrieved from <https://gazebo.org/home>
- [49] NVIDIA. (2021, May). GeForce RTX 3060 & 3060 Ti. Retrieved from <https://www.nvidia.com/zh-tw/geforce/graphics-cards/30-series/rtx-3060-3060ti/>
- [50] NVIDIA. (2022). *GeForce RTX 4080*. Retrieved from <https://www.nvidia.com/zh-tw/geforce/graphics-cards/40-series/rtx-4080-family/>
- [51] NVIDIA. (n.d.). *Jetson Orin*. Retrieved from <https://www.nvidia.com/zh-tw/autonomous-machines/embedded-systems/jetson-orin/>
- [52] Chahadeh, M., Boiko, I., & Zweiri, Y. (2022 Sep 30). *The Role of Time Delay in Sim2real Transfer of Reinforcement Learning for Cyber-Physical Systems*. *arXiv preprint*, arXiv:2209.15216. Available at

<https://arxiv.org/abs/2209.15216>

